

TECHNISCHE UNIVERSITÄT DRESDEN

FACULTY OF COMPUTER SCIENCE
INSTITUTE OF SYSTEMS ARCHITECTURE
CHAIR OF DATABASES
PROF. DR.-ING. WOLFGANG LEHNER
PROF. APL. DR.-ING. DIRK HABICH

Diploma Thesis

submitted for the academic degree
Diplom-Informatiker

Active Cache-Aware Hardware Adaptation: Cache Engine for Trie-Based Index Structures

Benjamin-Elias Probst
(Born 11 April 1996 in Potsdam)

Professor: apl. Prof. Dr.-Ing. Dirk Habich
Tutor: apl. Prof. Dr.-Ing. Dirk Habich, Dr.-Ing. Alexander Krause

Dresden, September 25, 2026

Thesis Assignment

*Active Cache-Aware Hardware Adaptation:
A Cache Engine for Trie-Based Index Structures*

Context and motivation

Search algorithms and index structures are a core building block of modern in-memory database systems. The Chair of Databases at TU Dresden addresses the hardware-conscious optimization of such systems. Since the introduction of the Adaptive Radix Tree, numerous cache-aware techniques and related algorithm families have been proposed — from node layout through prefetching and succinct representations to allocation —, yet almost every publication uses its own data structures, different load profiles, and inconsistent measurement methodology, which makes results hard to compare across publications. Finally, the weightings across improvements of individual algorithm components are currently not separable.

Problem statement

Search methods — from balanced search trees (B-/B⁺-trees) through tries and radix trees to specialized, custom-built constructs — span a large yet conceptually uniform *design space*: even a two- or three-component data structure admits astronomically many designs, and a concrete algorithm is a single point in this space, described by many individual design decisions — how nodes are represented, traversed, laid out in memory, and allocated [IZA⁺18, IZH⁺18]. Cache-line-aware behaviour arises not at a single one of these places but at *many different structural places* of an algorithm or of its memory management, and from the interplay of multiple conceptual layers of an algorithm. The state of the art comprises numerous mature methods, yet each optimizes only individual ones of these places and demonstrates its benefit in its own measurement environment, which makes progress hard to compare across methods. A common, generic concept and **principle** is missing that can measure and evaluate cache-line-aware behaviour, alongside other measurement focal points, at the different places of a search algorithm — interchangeably, systematically, and fairly.

Objective

The aim of the thesis is to find, design, and prototype — as an experimentation framework — such a principle (a framework) which, with its focal point on a dynamic composition of the distributed algorithm concepts: (i) decomposes cache-aware search algorithms into interchangeable design components, (ii) reconstructs state-of-the-art algorithms as explicit configurations (points in the design space), (iii) measures and evaluates cache-line-aware behaviour both at the individual directly and indirectly affected places and for the whole algorithm on a common, CPU-only basis, and (iv) on the basis of the fine-grained measurement results, delivers the best overall algorithm as a standalone binary behind a unified search-algorithm interface. As the candidate under test for the trial integration of a new technology into the state of the art, the thesis contributes PRT-ART (a Probst Redirect Tree / ART hybrid), which is studied as a conceptually novel search algorithm, yet contributes only a few new aspects within the classification of search algorithms and not a complete algorithm.

The separability of the individual algorithm components demanded here is to be established constructively and made measurable in this thesis.

Sub-tasks

1. Systematically capture the general design components of search algorithms — in particular the cache-aware ones — and identify the places at which cache-line-aware and search-algorithm-specific decisions take effect. This includes the systematic subdivision of the design components into design levels of a generalized algorithm categorization.
2. Reconstruct state-of-the-art designs — where possible from the original code of the respective publication and original compiler binary linking — (ART, HOT, Masstree, CoCo-trie, START, B²-tree, Wormhole, SuRF, and further methods) as explicit, comparable configurations.

-
3. Design and implement PRT-ART as the candidate under test with several design components of its own, competing against the captured state of the art.
 4. Extend the overall process into a consistently declaratively driven chain: this chain starts from an XML description of the experiment, from which an automated, platform-specific system build is derived for the system in question, which in turn produces every conceivable binary permutation of the design components. Their measurement results are evaluated towards the best measured system binary and system-binary recombination; a final measurement of that one binary or of hybrid binary recombinations produces an evaluation via CSV/xlsx files, which is finally translated into LaTeX and diagrams. That declarative configuration descriptions drive the automated construction of modular system variants is an established construct [CE00, ABKS13].
 5. Build a reproducible measurement pipeline (binary $\rightarrow$ CSV $\rightarrow$ LaTeX $\rightarrow$ diagram) with profile-aware workload routing of all workload frameworks found in the state of the art over *all* available workloads and hardware-counter collection.
 6. Measure systematically and as completely as possible across the configurations in three mandatory measurement series — (A) candidate under test vs. the state of the art, (B) systematic variation of the design decisions, (C) merge/regression of old vs. new variants —, with each series collected at three granularities: *micro-benchmarking* (each individual design component is measured — within the search-algorithm hull as well as the algorithm-category hull — in each common interface function of its category, in isolation against load profiles; the algorithms of the category are evaluated comparatively against one another per interface function, in compliance with latency-aware best practices in database research), *macro-benchmarking* (the individual interface operations of the search algorithm as a whole — e.g. insert, point lookup, delete — are measured through the interface functions of the algorithm category against sample loads or test subranges of the algorithm components), and *whole-system benchmarking* (established YCSB-like load profiles and comparable frameworks over every recombination of the design components).
 7. Evaluate at which places and in which combinations cache-line-aware decisions are beneficial or detrimental for which data-type-specific load patterns — derived from the ranking across the three benchmarking granularities and presented as analysable diagrams and tables.
 8. Automation of system detection and binary selection of the most optimal cache-line-aware algorithm, based on empirically substantiated measurements, together with a measurement-driven, dynamically controlled cache-line-aware adaptation of the algorithm behaviour as a static heuristic system policy for the algorithm behaviour under varying loads and operations. This so-called extraction is to be stored as an XML load-profile result for the testing environment of an overall algorithm, under reference to the various applied load profiles, and is reusable for each architecture focus in order to generate heuristics automatically.

Method and evaluation

The evaluation covers, besides the CPU-only core, SIMD and concurrent multi-threading paths, and compares the configurations against established baselines, including a non-tree-like hash table (`flat_hash_map`) as a cross-check within the same family. The data basis consists of real string datasets of differing characteristics — e.g. `url`, `dna`, `protein`, `xml`, `tpcds-id`, and `trec-terms` — under varied data volumes, key lengths, prefix overlap, local density, skew, hit/miss ratios, and value sizes. Measured quantities include exact/prefix lookup, prefix enumeration, insert/update/delete under all variants of the C++ standard interfaces of the well-known `std::map` and `std::vector` container hulls, p50/p95/p99 latency, throughput, bytes per key, and hardware counters (cycles, instructions, L1/L2/LLC and dTLB misses, branch mispredictions). Single-thread behaviour is studied first, followed by read-parallel multi-thread scenarios.

Fair comparability is ensured by fixed rules: a common minimal mode (common-denominator) is separated from the PRT-ART native mode, all methods receive identical key spaces and seeds, missing capabilities are reported as N/A rather than covertly emulated, and the compiler, flags, ISA path, allocator, and commit hash of each baseline are logged. For reproducibility, every overall-algorithm configuration is measured in several separate runs whose spread remains part of the statement (percentiles via HDR histograms rather than averaging); each dataset carries a record of source, checksum, and seed rule in order to attest the original code beyond doubt.

Scope

The core contribution is CPU-focused with SIMD extensions and multi-threading, but restricted to read-parallel methods with serialized writing (multi-read, single-write). The GPU is for now out of scope (extendable in the future); full write parallelism and an engine/optimiser integration are optional extensions. All architecture extensions are themselves dynamic extensions against the CPU baseline run and lead to deeper measurement permutations with multiple runtimes. The system-optimized binary can be built optionally with or without the measurement observer — as a debug, measurement, or release variant.

Statement of Authorship

I hereby declare that I, Benjamin-Elias Probst (Mat.-No. 4510512, born on 11 April 1996 in Potsdam), have written the diploma thesis submitted on this day to the examination board of the Faculty of Computer Science on the topic:

*Active Cache-Aware Hardware Adaptation:
Cache Engine for Trie-Based Index Structures*

entirely on my own, using no sources or aids other than those indicated, and that I have marked all passages taken from other works, verbatim or in substance, as such.

Dresden, September 25, 2026

Benjamin-Elias Probst
Mat.-No. 4510512

Abstract

Search algorithms and index structures are central to modern in-memory databases. Since the Adaptive Radix Tree, numerous cache-aware optimizations—spanning node layout, prefetching, succinct representations, and allocation—have been proposed, yet they have never been combined and compared within a single, fair measurement framework: each publication uses its own data structures, workloads, and methodology, which makes results hard to compare across publications. This thesis presents a cache-engine framework whose modules are separated at the binary level by a single, ABI-stable C interface (exactly one living being exported per module); behind it, a three-layer abstraction within the generated tier binary (CacheEngine $\rightarrow$ ExecutionEngine $\rightarrow$ SearchEngine) provides the search API and is kept distinct from the anatomy model (family, living being, organs). The framework decomposes cache-aware search algorithms, by means of a planner and a CacheEngineBuilder, into interchangeable, distributed (scattered across the code) design decisions of a large design space [IZA⁺18, IZH⁺18], and reassembles them through dynamic composition by compile-time permutation (not runtime adaptation). Structurally different methods—from tree-like indexes (tries, B⁺-trees) to non-tree-like hash tables—are expressed as points of the same design space behind a single interface; non-tree-like methods occupy a restricted subset of the design decisions. State-of-the-art algorithms are reconstructed as explicit configurations (points in the design space) and measured at three granularities—individual design decisions, individual interface operations, and whole algorithms—under known workload-framework load profiles (YCSB and further frameworks integrated by this single framework), each module being buildable optionally with or without a measurement observer for debug, measurement, or release. As the candidate under test, this thesis contributes PRT-ART, a Probst Redirect Tree / ART hybrid with eight building-block layers of its own; a measurement driver with profile-aware workload routing evaluates it, under three mandatory measurement series, against thirty reconstructed state-of-the-art methods (eight established reference designs with building blocks, and further ones); the long-term goal is to deliver the best recombination as a standalone binary.

Contents

1	Introduction	5
1.1	Motivation	5
1.2	Problem Statement	6
1.3	Research Questions	6
1.4	Objective and Contributions	9
1.5	Structure of the Thesis	12
2	Search Trees and Fundamentals	13
2.1	Concept and Overview of the State of the Art	13
2.2	Hardware: Historical Adaptation to Search Performance	16
2.2.1	Cache Organization: Associativity, Miss Classes, and Parallelism	16
2.2.2	Cache-Line-Oriented Layout Research	17
2.2.3	Prefetching	17
2.2.4	Main Memory and Address Translation: DRAM, NUMA, and TLB	18
2.2.5	Write Path, Replacement, and Coherence	19
2.2.6	Cache-Aware and Cache-Oblivious Data Structures	19
2.3	Workloads and Benchmark Conventions	20
2.4	Software Means and Synthesis into the Abstraction	21
2.5	Heuristics as Decision Rules	22
3	Concepts of a Cache-Aware Measurement System	25
3.1	Problem Statement and Requirements for a Measurement System	25
3.2	State of the Art as Axis Instances	27
3.3	Solution Concept: the Anatomy Model	37
3.3.1	Axis Composition and the Living-Being Metaphor	37
3.3.2	The Three-Level Model and the Four Families	39
3.3.3	Design Patterns and Compile-Time Patterns	43
3.3.4	Five Interface Layers: E4 down to E0	46
3.4	The Measurement System	47
3.4.1	The Triad: Measurement, System, and Organ Axes	49
3.4.2	Complex Axes: the Target ISA as a Fixed Recombination	52
3.4.3	Hardware Detection as a Two-Way-Split Measurement Axis	53
3.4.4	Identity Stamps: One Line per Realm	54
3.4.5	The Store: Stocks under Stamp Keys	55
3.4.6	Subsystems and the Chain from the XML to the Tier Binary	55
3.5	PRT-ART as Demonstrator	59

3.6	Sharpening through Heuristics	60
3.6.1	Heuristic Loop and Learned Load Profiles	61
3.6.2	The Four Operating Modes of the Builder	62
3.6.3	Measurement-Curve Type System, Filter Chain, and Measurement-Error Detection	64
4	Implementation and Concrete Algorithm Components	67
4.1	Four-Repository Architecture	67
4.2	PRT-ART in Detail	70
4.3	Axis Implementation and Measurement-System Levels	71
4.3.1	The One Architecture in Code	74
4.3.2	From Profile to Tier Binary: Planner, Builder and Measurement Pipeline	77
4.4	Quality Assurance: Contract Tests per Layer	82
4.5	Realm Roots in Code: System and Measurement Axes	84
4.5.1	Pattern Register, Hybrid Stage and Store	91
5	Evaluation	95
5.1	Methodology	96
5.1.1	Hypotheses and Measured Quantities	96
5.1.2	Fully Automatic Experiment Evaluation from the XML	97
5.1.3	Workloads and Datasets	98
5.1.4	Platforms, Vendor Lanes, and Regimes	98
5.1.5	Measurement Levels, Collectors, and Counters	99
5.1.6	Build Scope and Optimization Levels	101
5.1.7	Fairness and Reproducibility	101
5.1.8	The Heuristic Counter-Proof as Fourth Dimension	104
5.2	Results and Analysis	106
5.2.1	Identity of the Results: Stamp, Store, Workbook	106
5.2.2	Data Flow from the Measurement to the Manuscript	106
5.2.3	Measurement Series and Merge Strategies	108
5.2.4	Available Measurements	108
5.2.5	Axis Sensitivity Analysis	109
5.2.6	Validity and Limits	109
5.2.7	Summary of the Evaluation	110
6	Conclusion and Outlook	111
6.1	Answering the Research Questions	111
6.2	Limitations	119
6.3	Outlook	121
	Bibliography	125
	List of Figures	147

List of Tables	155
A Complete Measurement Series	159
A.1 Cartesian Smoke Series (43 Permutations, DLL Measurement)	161
A.2 Bias-Break Matrix	162
A.3 Latency Surfaces per Interface Function	164
A.4 Three-Dimensional Raw-Data View	165
A.5 Working-Set Sweep	165
A.6 Per-Axis Latency Attribution	168
A.6.1 Per-Axis Observer Detail	168
A.7 Latency Distribution	171
A.8 Reference-Relative Evaluation	172
A.8.1 Ratio Matrices	172
A.8.2 Normalized Bars	173
A.9 Axis Exchangeability	175
A.10 Limitations	175
B Code Structure	179
C Glossary	193
D Building-Block Matrix	207
D.1 T0 Search Algorithm Axis T0 (Search Structures and Search Methods)	208
D.2 T1 Cache Traversal (Fanout Lookup)	210
D.3 T2 Slot-to-Offset Mapping (Traversal Sublayer)	211
D.4 T3 Path Compression	212
D.5 T4 Node Types (Node-Type Axis)	213
D.6 T5 Memory Layout	215
D.7 T6 Allocator Axis (T6)	216
D.8 T7 Prefetch Strategy	219
D.9 T8 Concurrency Synchronization Axis (Concurrency Axis T8)	220
D.10 T9 Serialization (Encoding and Compression)	221
D.11 T10 Value Handle (Value-Handle)	222
D.12 T11 Index Organization	223
D.13 T12 I/O Dispatch Axis	224
D.14 T13 Migration Strategy Axis	225
D.15 T14 Filter Axis	227
D.16 T15 Buffering Q1 (Operations Buffer)	228
D.17 T16 Buffering Q2 (Flush Policy)	231
D.18 T17 Persistence Target (Persistence-Target)	232
D.19 Organ Meta-Meta Axis: Disk I/O (<code>disk_io</code>)	232
D.20 System Sphere: Target ISA (<code>target_isa</code>)	233
D.21 System Sphere: Operating System (<code>operating_system</code>)	236

D.22 System Sphere: Extension Hardware (<code>external_utils</code>)	237
D.23 System Sphere: Outer Complex Bracket (<code>build_target_complex</code>) and Build Toolchain (<code>build_toolchain</code>)	239
D.24 Measurement Sphere: Measurement Categories (<code>measurement_category</code>)	241
D.25 Measurement Sphere: Measurement Tooling/Collector (<code>collector</code>)	241
D.26 Measurement Sphere: Telemetry Building Blocks (Conserved Catalogue)	242
D.27 Measurement Sphere: Load Framework (<code>load_framework</code>) — Meta-Meta Axis . .	243
D.28 A-Korpus Allocator Corpus (A01–A23 and A24)	244
D.29 Axis Inventory from the Generated Registries	247
D.30 Conserved Designs	247
D.30.1 Build-SIMD: Vector/Accelerator Extension Axis	248
D.30.2 Build-PG Page Type Axis (PAGE-TYPE)	249
D.30.3 Build-HW — Platform Hardware Strategy	250
E Architecture Decisions	253
F Comparison Interfaces: <code>std::map</code> and <code>std::vector</code>	263
F.1 <code>std::map</code> (Ordered Associative)	264
F.2 <code>std::vector</code> (Sequence)	267
F.3 Composite Functions and the Conformance Subset	269

1 Introduction

1.1 Motivation

Search trees and index structures form the foundation of modern data-intensive systems — from in-memory databases through file systems to language processing. At the same time, processors are dominated by the memory hierarchy: not the number of operations but their cache behaviour determines the run time [HP19, Dre07, WM95]. From this tension arises the guiding question of this thesis. At its centre stand trie-based indexes and cache-line-optimized processing. Since the Adaptive Radix Tree (ART) [LKN13] of 2013, research has diversified: trie-height optimization (HOT [BZP⁺18]), hybrid B⁺ structures (Masstree [MKM12], B²-tree [SSL⁺21]), hash/trie/B⁺ hybrid ordered indexes (Wormhole [WNJ19]), succinct representations (CoCo-Trie [BFTV24], SuRF [ZLL⁺18]), and self-tuning approaches (START [FJKN20]).

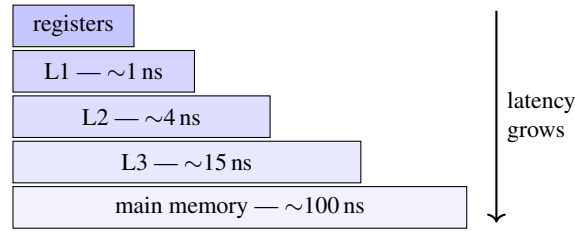


Figure 1.1: The “cache wall”: typical orders of magnitude of the access latency per level of the memory hierarchy (literature values after [HP19, Dre07]); across the hierarchy the latency grows by about two orders of magnitude.

In parallel, a research corpus of its own on cache awareness, prefetching, and allocation strategies has emerged. Heuristics such as Cache-Sensitive Search Trees [RR99, RR00], cache-oblivious layouts [FLPR99], prefetching B⁺-trees [CGM01], and hierarchical prefetching [ZGH⁺25] each address a separately considered aspect of the search-algorithm architecture, with the focus mostly resting on empirically found improvements and measurements. Because so much rests on cache-line-optimized behaviour, a segmentation of the search-algorithm constituents into *axes* suggests itself — a backward-directed abstraction of known building blocks into reusable modules across the most important works of the state of the art.

This thesis carries out this segmentation with a framework whose building blocks can be understood as interchangeable design dimensions of a large *design space* [IZA⁺18, IZH⁺18]. Structurally different search methods — from tree-like indexes to non-tree-like hash tables, including special properties such as cache-line awareness — thereby become comparable as points of the same design space; non-tree-like methods occupy a restricted *subset* of the dimensions over the same search-algorithm interface, as if they formed the sole branch beneath the root. PRT-ART, a Probst Redirect Tree / ART hybrid and an abstract, incomplete collection of novel search-algorithm constituents, serves as the candidate under test — the new search algorithm to be integrated on trial — for state-of-the-art membership.

1.2 Problem Statement

In today’s search structures, cache awareness is either hard-wired or ignored entirely; above all, however, one cannot *separate* which individual design decision benefits which load pattern — the *separability problem*, the central thread of this thesis.

Existing cache-aware index designs optimize one or a few aspects in isolation and report results that are hard to compare across publications, because each work uses its own data structures, workloads, and measurement methodology. The weightings of the improvements of individual algorithm constituents remain inseparable: a reported performance gain cannot be unambiguously attributed to a single constituent (page type, layout, value placement, prefetching), because the constituents stay intertwined in the original designs. There is no common framework that (i) decomposes such designs into interchangeable axes, (ii) reconstructs published algorithms as explicit configurations, and (iii) can measure both individual axes and whole algorithms on a uniform, fair basis.

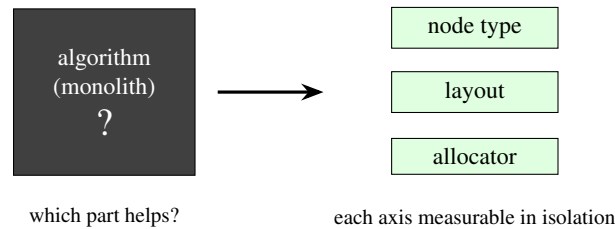


Figure 1.2: The separability problem: as a monolith it remains unclear which design decision takes effect; only the decomposition into orthogonal axes makes each contribution measurable in isolation.

1.3 Research Questions

The task description and the goal of cache-line optimization give rise — owing to the ubiquity of the influence of the cache-line settings — to one main question (RQ0) and four measurable research questions (RQ1–RQ4); complementary aspects are assigned to them as deepening *sub-questions*. Across all five questions lies a categorization question that this thesis carries along as a coinage of terms: the task description already distinguishes what is measured (the design constituents of the algorithm), the platform on which building and measuring take place, and the measuring itself. Whether the axes of the design space can be cleanly separated along these three concerns into *organ axes* (the interchangeable design constituents of the algorithm), *system axes* (the concerns of the build and target platform), and *measurement axes* (the concerns of measuring itself) — and how such a separation is to be accomplished if a design does not yet provide it — is laid out as a question in this thesis; the sub-questions are therefore named along these three *axis categories*. The current state here is unambiguous: the thesis lays out the triad as three separate axis registries¹ — an organ-axis registry, a system-axis registry, and a measurement-axis registry — and tests their viability; the answers are developed by Chapters 3, 4, and 5. Each question is thereby answered at exactly one place: RQ1 in Section 3.4.1 and Chapter 4, RQ2 in Section 3.4 and Chapter 5 (RQ2.d in Section 2.3 and Section 5.1.3), RQ3 in Section 3.5 and Chapter 5, RQ4 in Section 4.5 and Section 4.3.2 (RQ4.a); Chapter 6 draws the balance per main question and sub-question. For this, the

¹An axis registry is the directory of the axes of one category together with the building blocks offered per axis (in code a Registry).

sub-questions carry the identifiers RQ0.E, RQ1.a, RQ1.b, RQ2.a, RQ2.b, RQ2.c, RQ2.d, RQ3.a, RQ3.b, RQ4.a, and RQ4.E.

RQ0 (Main question) By how much do *active, measurement-driven*² cache-aware decisions — page type, memory layout, value placement, and prefetching — improve the performance of trie-based search structures over *passive, statically dimensioned* variants on different CPU platforms (hybrid CPUs, Sapphire Rapids)?³ *Extension RQ0.E (axis categories)*: Can a measured gain be attributed unambiguously at all — to the design decision itself (organ), to the build and target platform (system), or to the measurement setup (measurement) —, and which separation of the axis categories is necessary for this?

RQ1 (Composition) How can a search-algorithm design be canonically decomposed into orthogonal axes and systematized in a permutation matrix⁴ such that the state-of-the-art methods become reconstructable bias-free as profiles — as far as possible from their original code — and arbitrarily permutable? *Sub-question RQ1.a (category cut)*: Does the canonical decomposition itself divide along the three axis categories — must platform and measurement concerns be kept out of the permutation matrix so that the decomposition of the design constituents remains orthogonal —, and how is such a cut to be accomplished if it is not yet laid out in the design? *Sub-question RQ1.b (organ couplings and system variation)*: Which mutual dependencies between axes (node type $\times$ SIMD $\times$ path compression)⁵ limit the isolation of individual axes, and how does the same organ assignment behave in its cache-update and cache-line fill behaviour (how the cache is updated and how completely the loaded cache lines are used) when the ISA/SIMD axis alone (SSE2/AVX2/AVX-512, NEON/SVE2, RVV) is varied — as far as the respective target platform is released for building (Section 3.4.1)?

RQ2 (Measurement methodology) How must a reproducible measurement pipeline with hardware-counter collection⁶ be designed so that every axis permutation is measured fairly (all else being equal, that is, on an identical compiler and flag basis) in three measurement series — A (candidate under test vs. state of the art), B (systematic variation of the design decisions), and C (merge/regression old vs. new) — over three granularities (micro-, macro-, and overall benchmarking: individual constituents, interface operations, whole algorithms), and under which platform-calibrated criteria (thresholds that are determined per machine signature from measurement runs on the respective

²A decision is called measurement-driven when it is grounded in collected measured values and not in fixed model assumptions; a variant is statically dimensioned when its decision is fixed in advance at design time.

³The measured fleet of this thesis consists of the three registered machine signatures: prod1 (AMD Zen 5, Ryzen 9 9950X3D), prod2 (Intel Core i9-12900K, Alder Lake — a hybrid CPU with P and E cores), and an ODR0ID board with Intel Gracemont cores. The server platform Sapphire Rapids (ZIH node Barnard, Xeon 8470) is provided for in the platform detection but is not part of the measurement series (n/a).

⁴The decomposition is called canonical when every axis has a fixed place in a binding order. The permutation matrix is the table of these axes with their admissible assignments; each choice of exactly one assignment per axis is an axis permutation, and a profile is the declarative description (as XML) of a design by its assignment of these axes. Bias-free means: without distortion by a re-implementation or by unequal measurement conditions.

⁵SIMD (*single instruction, multiple data*) denotes instructions that apply one operation to several data elements at once; path compression merges, in a trie, node sequences without branching into one node; the ISA (*instruction set architecture*) is the instruction set of the target CPU, and SSE2/AVX2/AVX-512, NEON/SVE2, and RVV are the SIMD extensions of the instruction sets x86, ARM, and RISC-V.

⁶The measurement pipeline is the chain of building the binaries, measurement run, and evaluation; hardware counters are the performance counters of the CPU (*performance monitoring counters*, PMC), which count events such as cache misses per run.

platform) does a platform count as successfully verified? *Sub-question RQ2.a (measurement axes)*: Must the measurement instruments themselves — timing, counter collection, load generation — be carried as axes of their own that are always present without ever changing what is built and measured, and how is this separation to be accomplished if it does not yet exist? *Sub-question RQ2.b (organ axes)*: How can contradictory findings in the literature on the optimal node/cache-line size (one cache line for CSS/CSB⁺ [RR99, RR00] versus sixteen and more for Hankins/Patel [HP03], there nodes of 512 bytes and larger at 32-byte cache lines, as well as several cache lines per node loaded jointly via prefetch in the prefetching B⁺-trees of Chen, Gibbons, and Mowry [CGM01]) be re-measured bias-free through axis-isolated measurement permuted with all else being equal, instead of being adopted from incompatible original setups? *Sub-question RQ2.c (extensibility)*: How extensible is such a measurement pipeline — how do new candidates under test and their design constituents integrate, how are they abstracted, and do they join existing families⁷, found new families or a hybrid form; at which conceptual level should this cut run? Central here is: what constitutes an extension of the system in the first place, and how does adding or cutting away individual design constituents — such as those originating from a single publication — affect the attainable degree of cache-line awareness of the overall system? Does this require an operating mode that offers the design constituents of a selectable reference design from the literature in isolation as a closed axis set, in order to measure exactly this one publication — with measurement and system axes remaining free-standing — faithfully to the original? *Sub-question RQ2.d (workload state of the art)*: Which workloads belong to the state of the art specifically for search algorithms?

RQ3 (Candidate comparison) Does the candidate under test PRT-ART win against the eight Rank-1 designs of the state of the art (SOTA for short; ART, HOT, Masstree, CoCo-Trie, START, B²-tree, Wormhole, SuRF)⁸ and the Rank-2/3 SOTA *not* through asymptotic novelty but through better microarchitectural properties⁹ — higher cache-line utilization, fewer LLC and dTLB misses, lower branch cost, smaller hot search paths (the frequently traversed node sequences) — for exact lookup, prefix lookup (search for a key prefix), and prefix enumeration (enumeration of all keys for a prefix), measured under the three mandatory measurement series A (candidate under test vs. state of the art), B (systematic variation of the design decisions), and C (merge/regression old vs. new)? *Sub-question RQ3.a (organ axes)*: Which local page representation (8-/16-bit array-indexed page — the direct-index pages Array₂₅₆ and Array₆₅₅₃₅ — versus the vector page — the vector pages Vector⟨u8,u8⟩ and Vector⟨u16,u16⟩) is cheapest per branch point (per node at which the search path branches), and by which platform-calibrated switching rules — including the order Array₂₅₆ → Array₆₅₅₃₅ → Vector⟨u8,u8⟩ → Vector⟨u16,u16⟩ by occupancy density and access type — should switching occur adaptively? The candidate under test currently switches by fixed density

⁷In this thesis, family denotes the topmost division of the algorithms by their externally visible interface (such as Map or Container); each family carries its own subclasses with a fixed axis set.

⁸The *rank*s order the state-of-the-art publications by centrality for this thesis: Rank-1 comprises the ten most important core works — among them the eight standalone search designs as well as ARTSync (P08) [LABN16] and LOUDS (P09) [Jac89], which enter as organs —, Rank-2 the cache-aware layout and B⁺ precursors, and Rank-3 the prefetching, synchronization, and TU-Dresden hardware works.

⁹Cache-line utilization is the share of the bytes of a loaded cache line that are used; LLC is the last-level cache, the last cache level before main memory; the dTLB (*data translation lookaside buffer*) buffers address translations for data accesses; a miss is a failed access that has to resort to the next, slower level; branch cost is the cost of jump instructions, above all of mispredicted branches.

thresholds of 25, 50, and 75 percent¹⁰ (Section 3.5). *Sub-question RQ3.b (compile-time force)*: Do latency-critical measurements force decisions that are in principle selectable at run time — such as those adaptive switching rules — to be fixed as requirements already at compile time, so that every axis divides into a compile-time-fixed main axis and run-time-variable sub-axes — a sharpening that RQ4 takes up?

RQ4 (Compile time vs. run time) Which axis properties must be fixed at compile time in order to produce platform-optimized, provenance-verifiable¹¹ binaries per permutation, and which can or must be selected at run time (bit-encoded binary identity per permutation, run-time-adaptive layout selection) without violating scientific comparability (identical compiler and flag basis)? Does the measurement make it possible to generate ISA+OS-optimized binaries for a search-algorithm recombination whose heuristic optimization is perfectly tailored to its runtime environment? *Sub-question RQ4.a (declarative chain)*: Can the entire process — from the platform-specific build of all binary permutations of the design constituents through the measurement run and the selection of the best measured binary or binary recombination to the evaluation in CSV/xlsx files, LaTeX, and diagrams — be generated throughout from one XML description of the experiment, such that an experiment is completely and reproducibly determined by its declaration? *Extension RQ4.E (system axes)*: Does the boundary between compile time and run time run along the axis categories — do the platform properties in particular (ISA, operating system, extension hardware such as GPU, FPGA, or NPU accelerators) belong, as system concerns, to the compile-time identity of the binary — that is, to its fingerprint, without changing the permutation identity —, while measurement choices (the assignments of the measurement axes) may remain run-time-variable?

1.4 Objective and Contributions

The objective follows the task description in four steps: decompose cache-aware search algorithms into interchangeable design constituents, reconstruct state-of-the-art designs as explicit configurations, measure and evaluate cache-line-aware behaviour both per constituent and for the whole algorithm on a common CPU-only basis, and deliver the best overall algorithm — following an autonomous heuristic-profile evaluation and heuristic-function modelling — as a standalone binary. The code of the diploma thesis is here the *formal consumer and driver* of the cache engine: it loads the candidate under test PRT-ART, compiles it via metaprogramming against the core constituents of the cache engine, delivers ABI-stable *living beings* (search-algorithm instances), and drives them to measurement via the probe dock — the measurement interface of the family (Chapter 3). The underlying architecture model — the three levels (*family* as the outer algorithm interface, *genus* as the family subclass with a fixed axis set, in code the

¹⁰Array-indexed page and vector page are the two page forms of the PRT-ART B⁺ node: the array-indexed page addresses entries directly via an array with an 8- or 16-bit index, the vector page holds sorted key-value vectors. The density thresholds are the candidate's switching rules between the page representations; the 50 percent boundary named in the glossary is the page-type boundary between array-indexed and vector page, not a further switching rule.

¹¹Provenance-verifiable means: the origin of a binary — which axis assignment, which platform, and which toolchain brought it forth — is verifiable on the binary itself. For this, every binary carries two identities: the binary identity per permutation, that is, its assignment of the organ axes (in code the `binary_id`, a path of one segment per axis with a bijective numeric encoding), and the compile-time identity, which additionally comprises build platform, toolchain, build control quantities, and source state and is compiled into the binary as a fingerprint — a hash over all build-determining members —; the first is also called the permutation identity.

tier subclass¹² `AnatomyGenus`, and the axis organs), the four subsystems, and the ABI-stable delivery of the search-algorithm *living beings* — is developed by Chapter 3 and Chapter 4. Meant here are the four carrier subsystems measurement driver, `CacheEngineBuilder`, `CacheEngine`, and candidate under test; to be distinguished from them are the twelve stages of the recommendation pipeline in code. The concept of the axis architecture itself is adopted from prior work by the author — the axis concept of this predecessor implementation `UltiHash` (on its origin see Section 3.3.3) —; the contribution of this thesis is its transfer to cache-aware search structures and its measurement-oriented realization. The central contributions are:

- An ABI-stable module interface: a uniform C module boundary exports exactly one living being per permutation (`comdare_create_anatomy() → IAnatomyBase`); behind it lies a three-layer abstraction chain — a mediating facade (`ICacheEngine`) above the runtime ABI layer (`IAnatomyBase → IExecutionEngine`) above the axis organs. The delimitation of the search-engine module is conceptually fixed but not implemented as a module of its own; its provider is expressly deferred, and the search-engine module is realized as a topic namespace within the cache engine. The candidate’s search core specializes its hybrid API variadically by parameter count (1 parameter $\Rightarrow$ `std::vector` API; 2 parameters $\Rightarrow$ `std::map` API; $N > 2 \Rightarrow$ `std::map<K, std::tuple<V1...VN>>`), as the task description demands across all variants of the well-known `std::map` and `std::vector` container hulls; for this, the cache engine provides the family arities of its genera.
- An axis permutation matrix in a three-part order: eighteen organ main axes in fixed slot order (T00–T17) form the binary permutation; three system main axes — target ISA, operating system, and extension hardware — describe the build and target platform, whereby their complex bracket `build_target_complex` is expressly not a fourth main axis; and a separate measurement-axis realm keeps the measurement concerns apart from both, for neither system nor measurement axes ever enter the permutation identity of the binary. An optional nineteenth organ axis for disk input/output, which does not describe an assignment but describes how other axes are described, is added as soon as disk input/output is integrated into the algorithm design; it stamps in the bracket appendage of the organ line and remains outside the permutation identity. The matrix is realized with `Boost.MP11` compile-time type-list building blocks — runtime tags and `std::variant` are excluded in the tier binaries as a principle of this thesis (`[[no-runtime-switch]]`) — together with a compile-time fallback between the candidate stack and the state-of-the-art stack, which takes over sixteen of the eighteen axes from the cache-engine frame and replaces two by candidate slots.
- Thirty-three state-of-the-art profiles as persisted XML configurations (P01–P33): 30 full profiles of standalone search designs — 8 Rank-1 and 22 Rank-2/3 — as well as 3 abstract profiles (OLC, LOUDS, VAMPIR), which enter only dissected, as organs or as measurement technology. The `ExperimentDriver` transforms them automatically into pre-compiled C++23 modules; the profile set thereby covers all 33 analysed publications. By means of the cache engine, the XML design genuinely maps the individual algorithm constituents at compile time and is thereby a

¹²*Tier* is the German word for *animal*: in the metaphor canon of this thesis it names the fully composed module — the living being with all its organs (see *Tier binary* in the glossary) —, not a level or layer.

compiler-compiler build recipe within a *model-driven* optimization stack [Sch06]; that declarative configuration descriptions drive the automated construction of modular system variants is an established construct [CE00, ABKS13].

- PRT-ART as a candidate under test with eight building-block layers of its own (4+1+2 pool allocator, optimistic lock coupling with reserved blocks, MultiLevel layout, path-oriented prefetch, DensityTracker, hypothesis metrics H1/H2/H3, Inline/External/ChainRef value handles, signalling-bit serialization). In the measurement apparatus the candidate under test registers four candidate slots — page type (`page_type`, in the cache engine the axis `node_type`), prefetch, telemetry, and the ChainRef value handle — as well as a merge organ for path compression, which contains the cache engine’s own Patricia strategy; in the codegen-capable measurement path two of them take effect (prefetch and value handle), while MultiLevel layout, pool allocator, and serialization run via the cache-engine default (Section 3.5). In the cache engine, telemetry is a system axis and not a composition slot; the candidate slot of the same name is a remaining holdover. The OLC concurrency is a building-block layer of the candidate under test, not a registered slot. Note that the slot numbers of the candidate under test do not follow the slot order T00–T17 of the cache engine. The deployment of the candidate under test demonstrates the extensibility of the system by example — towards the efficient advancement of novel approaches in dealing with cache-aware algorithms and accelerated search.
- A measurement driver with a defined-/full-mode distinction (supplemented by a deterministically sampled full variant), profile-aware workload routing — by default the full permutation over all registered workloads, whose implementation status Chapter 5 reports — and three granularities: micro-, macro-, and overall benchmarking (individual constituents, interface operations, whole algorithms).
- A build and measurement chain behind this driver that is declaratively controlled throughout: an experiment *planner* resolves the XML description of the experiment against the axis offerings of the three axis registries, and the `CacheEngineBuilder` builds and measures from it the binary permutations per platform — up to the evaluation via the `xlsx` output (the workbook is the output and the standard; the CSV form is not a single child document but an ordered tree of CSV files — one overall folder per workbook, beneath it one folder per sheet, therein the CSV files of the individual sub-axis sweeps —, which is always generated from the workbook, as a strategy of the same factory, and by XML lies at the same target instead of the workbook or in addition to it) and the LaTeX tables and TikZ diagrams generated from it.
- A bias-free comparison matrix that runs *all* registered load profiles against *all* reconstructed living beings — the original-design binaries built from the profile XML — and thereby counters the self-favouring through author-owned measurement setups in individual publications (pattern term: *home-field advantage*) [vdKHA⁺19].
- A generic platform probe that determines the cache properties per target platform — line size and the sizes per hierarchy level from `sysfs` and `CPUID` — and lets them flow as fixed quantities into a system-optimized binary. The core topology (P-/E-core separation, pinning) is envisaged as a platform concern of the same probe and is mandatory for the cache-line measurement on hybrid

CPUs, but its capture by the probe is not implemented — the implementation status is stated in Chapter 5. The measurement plan provides for two operating-system regimes on the two production machines (immutable Talos and root Linux with full hardware-counter access); measured is the root-Linux regime, not the Talos regime.

- *As objective and outlook:* the delivery and substantiated evaluation of the best overall algorithm — automatically selected from the fine-grained measurement results under the measurement aspect of the best cache-aware behaviour and of the acceleration through cache-line adaptation, in comparison of all measurable parameters against the binary competitors — as a standalone, ABI-stable binary behind the uniform search-algorithm interface. The measurement pipeline already delivers the ranking required for this, and a first increment of the automatic selection and of the binary delivery is implemented as a standalone tool (`best_binary_selector`); the heuristics-driven full automation remains outlook.

1.5 Structure of the Thesis

The structure follows the path from the general to the specific: Chapter 2 lays — conceptually and visually — the foundations from established knowledge: the landscape of search structures, the hardware properties of the memory hierarchy, the workload conventions of the comparison literature, the software means of abstraction, and the notion of a heuristic. Chapter 3 develops from them the concepts of a cache-aware measurement system: the problem statement and its requirements, the state of the art as axis instances, the anatomy model with its layers, the measurement system itself, the candidate under test PRT-ART as demonstrator, and the heuristic evaluation with its operating modes. Chapter 4 realizes this concept as a software architecture — repositories, the candidate under test in code, the axis architecture and compilation pipeline, layer contracts with contract tests, and the system axes in code. Chapter 5 evaluates along the research questions, including the heuristic counter-evidence; Chapter 6 closes with conclusion and outlook. The appendices deepen alongside: the measurement-series documentation (Appendix A), the code structure (Appendix B), the glossary (Appendix C), the complete building-block matrix (Appendix D), the architecture decisions (Appendix E), and the comparison interfaces (Appendix F).

2 Search Trees and Fundamentals

This chapter leads from the familiar object — search trees — via the hardware that shapes their performance and the workloads under which they are compared, to the software means and the notion of heuristics by which their design decisions can be abstracted and selected. Everything stays with *known* terms, concepts, and publications; the systematic catalogues and the thesis’s own conceptual framework follow in Chapter 3.

2.1 Concept and Overview of the State of the Art

We first map the landscape of search structures — comparison-based trees, digital and prefix-based methods, hashing, spatial and flat structures — and classify the known representatives with their core ideas and supporting references. The literature refers to the entirety of the best published methods at any given time as the *state of the art* (SOTA); in this thesis the term always means the published search-structure designs against which a new design must measure itself. Here it already becomes apparent that every algorithm can be located in a shared design space; how this design space is systematically spanned is developed in Chapter 3. Search structures can be classified by the mechanism with which they map

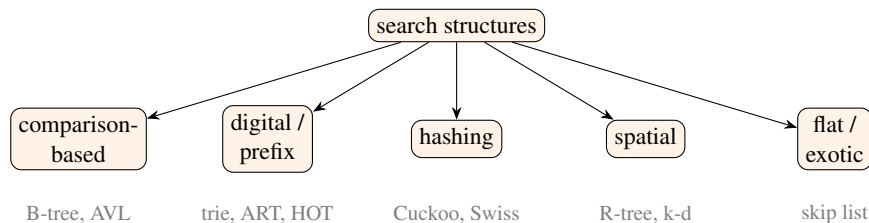


Figure 2.1: The landscape of search structures: five classes with typical representatives.

a key to a record [Knu98]. *Comparison-based* structures locate the key via order comparisons and presuppose a total order — from the self-balancing AVL trees [AVL62], whose sibling subtrees differ in height by at most one, and the red-black trees [Bay72, GS78], which secure the same logarithmic height via a node colouring with looser invariants that are cheaper to maintain, through the block- and external-memory-oriented B/B⁺ trees [BM72, Com79], whose nodes typically correspond to a cache or memory block size [GL01, HP03] and which in the B⁺ variant keep all payload data in the leaves and reduce inner nodes to signposts, to the main-memory-oriented T-tree [LC86], which combines node-local value ranges with AVL balancing and thereby reduces the number of pointer indirections per search step. *Digital* structures, by contrast, branch on the bytes of the key rather than by comparison, so that the search-path length depends on the key length and not on the number of keys: the *trie* [dlB59, Fre60] as the basic form, in which every root path spells out a key and common prefixes are shared, the path-compressed

radix/PATRICIA trees [Mor68]¹, which collapse sparsely populated one-child chains, and the *Adaptive Radix Tree* (ART) [LKN13], which adapts the node representation to the actual fanout. ART serves this thesis as the reference design: its design decisions — the node classes Node4, Node16, Node48, and Node256 graded by fanout, as well as the linear search within small nodes — recur in Chapter 3 as individually measurable partial decisions, whereas the ART tree as a whole appears as a candidate under test and not as a structure measured alongside throughout.

Beyond the tree-like methods, *hashing* maps keys directly via a hash function and has no inherent order, which makes range queries expensive [Knu98]; cuckoo hashing [PR04], which unlike open addressing and chaining guarantees constant worst-case lookups, and tables such as SwissTable [Goo18], which achieve high practical density thanks to SIMD group probing, form the non-tree-like comparison class of the literature. *Spatial* structures (k-d tree [Ben75], quadtree [FB74], R-tree [Gut84]) partition a multidimensional key space and lie outside the scope, as do the theoretical “exotics” — the van Emde Boas tree [vEB75] and the y-fast trie [Wil83] with $O(\log \log u)$ over a universe of size u , or the Fusion Tree [FW93] that undercuts the comparison bound. The flat class comprises the randomized skip list [Pug90], the flat, sorted array with binary or interpolation search [Knu98] — strictly speaking the single leaf of a tree — and the suffix array, which generalizes this layout on string keys into a space-saving full-text index and whose lightweight, memory-budgeted construction counts as a textbook example of algorithm engineering [MF02].

The scope of this thesis is structured along this landscape in three stages. The objects of investigation are, first, the tree-like structures of the comparison- and digital-based classes (Figure 2.1) and with them the elementary search methods on sorted arrays, of which tree nodes consist: binary and k-ary search [Knu98, SGL09], interpolation search [PIA78], the Eytzinger layout [KM17]² and the linear search within small nodes [LKN13]; the flat, sorted array thus does not lie outside the scope but is, as the node interior, its smallest building block. Second, approximate membership filters belong to the scope as upstream building blocks — the Bloom filter [Blo70], the cuckoo filter [FAKM14], the xor filter [GL20], and SuRF [ZLL⁺18] as a range-capable filter; cuckoo hashing thus appears in this thesis not as a hash table but in its filter form. Third, further representatives — the skip list, hash search, the binary search tree, the B-tree, the whole trees ART [LKN13], HOT [BZP⁺18], START [FJKN20], Wormhole [WNJ19], and SuRF [ZLL⁺18], as well as a scalar replica of the SwissTable — are available as switchable building blocks that are not activated in the reference run; the SwissTable replica permits no measurement statement about the SIMD-based original. Outside the scope remain the spatial structures, the theoretical exotics, and the suffix array. As outwardly different as the classes appear, they share recurring, mutually separable partial decisions; which building blocks are actually measured in the reference run is stated in Chapter 3 together with implementation status and source.

Orthogonal to this search-algorithm view stands the established taxonomy of the *containers* of the C++ standard library [ISO24]: sequence containers (`std::vector`) order by insertion position, ordered-associative ones (`std::map`, `std::set`) by key order, unordered ones (`std::unordered_map`) map via a hash function. These standardized interface contracts form the common basis against which

¹PATRICIA (*Practical Algorithm To Retrieve Information Coded in Alphanumeric*): Morrison’s path-compressed binary trie whose inner nodes record only the bit position at which the keys of the respective subtree first differ [Mor68].

²Eytzinger layout: the implicit, pointer-free arrangement of a sorted array in the level order of a complete binary tree — the children of the element at position i lie at positions $2i$ and $2i + 1$ —, so that the search path is computable without pointers and the memory accesses of the search become predictable for prefetching [KM17].

structurally different methods can be measured uniformly. This thesis rests on two of them as contract hulls: on the ordered map as the common denominator of comparison- and digital-based search structures alike and on the `std::vector` as the sequence view; the hash table remains, as the unordered counter-check, the conceptual counterpole of the literature, but it is no contract hull of its own in this thesis — its building blocks, where present, are subordinate to the ordered map hull and merely switchable (preceding paragraph). The placement of both hulls in the conceptual framework of this thesis is developed in Chapter 3; their complete elaboration in the implementation is shown in Chapter 4.

All methods named so far are *statically designed* structures: their inner organization is fixed before the first query arrives. An established line of research, by contrast, moves the design into run time. *Database cracking* [IKM07] builds the index adaptively as a by-product of query processing — each query partitions the data a little further —, and *stochastic cracking* [HIKY12] makes this approach robust against unfavourable query sequences; *learned indexes* [KBC⁺18] replace the search structure with a model of the key distribution that approximately predicts the position of a key. More recent work continues this line in an actively tuning fashion: VEGA readjusts a learned index at run time with group-wise learning granularity [LCL⁺25], and AirIndex designs the hierarchy of an index automatically from the data and storage-I/O profile of the target system [CLP23]. These approaches show a common trend: away from the hard-wired single structure, towards configurable, self-adapting designs. For the present thesis they are the closest related known approaches, since they demonstrate that the design decisions of a search structure can be kept variable; how such decisions can be decomposed systematically and measured individually is developed in Chapter 3.

For the cache-related properties of these designs, the literature has developed a graded terminological taxonomy that this thesis uses throughout. Its starting point is the *I/O model* (external-memory model) of Aggarwal and Vitter: a two-level memory whose levels exchange data only in blocks of fixed size; the cost measure of an algorithm is solely the number of block transfers [AV88]. Transferred to the cache hierarchy, a design is called *cache-aware* when it knows and explicitly uses the concrete parameters of the target platform — block size, capacity; *cache-conscious* has, since Chilimbi, Hill, and Larus, denoted the same attitude from the data structure’s point of view: its layout is deliberately adapted to cache blocks through clustering, colouring, and compression [CHL99]. *Cache-oblivious* is the counter-position of Frigo, Leiserson, Prokop, and Ramachandran: the algorithm knows no parameters at all; analysis takes place in the *ideal-cache model* — two levels, capacity Z , line length L , full associativity, optimal replacement —, and an optimal cache-oblivious design is then nearly optimal on *every* level of a multi-level hierarchy at once [FLPR99]. *Cache-sensitive*, finally, is the wording of Rao and Ross for their search trees tailored to the cache line (Section 2.2) and is used largely synonymously with cache-aware [RR99, RR00]; as the umbrella term for the parameter-knowing variants, this thesis uses *cache-aware*.

Orthogonal to this parameter question lies the question of timing: *when* is the cache-related design decision made? A *static* design fixes it ahead of the first query — be it cache-aware on determined parameters or cache-oblivious without parameters —, whereas the *active or adaptive* approaches of the previous paragraph move the decision into run time and readjust it from observed behaviour. Both distinctions — parameter knowledge and decision timing — are independent of each other; the notion of heuristics that later connects them follows in Section 2.5.

2.2 Hardware: Historical Adaptation to Search Performance

The performance of search structures has always been driven by hardware. We trace the historical advent of cache-near hardware — cache hierarchy, cache line, address translation, coherence — focusing on the *static and compile-time-relevant* properties that later support the abstraction in Section 2.4. Modern

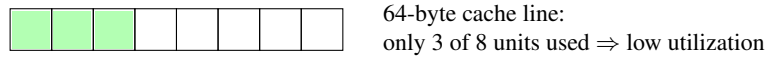


Figure 2.2: Cache-line utilization: a cache line is always loaded in full; if only a few of its bytes are used, effective bandwidth drops — cache-aware layout raises utilization.

processors interpose several cache levels (L1, L2, and — not on every platform — L3) between the registers and main memory [HP19]; an access that finds its datum nearby is cheap, whereas a miss down to main memory costs several hundred cycles [Dre07], on multi-socket and chiplet systems additionally depending on the owning NUMA node. The transfer unit between the levels is the *cache line* (Figure 2.2); its size is no universal constant but hard-wired per microarchitecture — 64 bytes on x86-64 and common ARM cores (such as Cortex-A76 or Neoverse V2), 128 bytes on Apple Silicon L2 and system-level cache [App24], narrower 32 bytes on older or embedded cores [HP19, Int24a, Adv23]. Programs can *read* it at runtime — via `CPUID` (x86), `CTR_EL0` (ARM), or `riscv_hwprobe` (RISC-V; restricted, among other things CBO block sizes) [Arm24, Int24b, Lin26a]³ —, but not change it; the depth of the hierarchy also varies (simple RISC-V cores such as the SiFive U74 end at L2, some AMD processors bring a stacked 3D V-Cache L3). For a cache-line-aware layout, the line size determined per target platform is therefore an *input parameter of compilation*, not a fixed assumption; how this input parameter is bound in this thesis is shown in Chapter 3. Moreover, the transfer of a line does not proceed strictly block-at-once: *critical-word-first* delivers the requested word first, and DDR5 splits the memory channel into two independent 32-bit sub-channels [KAS21]; non-temporal or write-combining stores additionally bypass the normal write-allocate path instead of pulling the affected cache line into the cache hierarchy [Int24b].

2.2.1 Cache Organization: Associativity, Miss Classes, and Parallelism

Where a line may reside in the cache is determined by *associativity*: the cache is organized into *sets*, each holding n *ways* (line slots); the memory address decomposes into tag, set index, and line offset, and each address may reside only in the n slots of *its* set — direct-mapped ($n = 1$) and fully associative (a single set) are the limiting cases, and common data caches lie in between at 4- to 16-way [HP19, Int24a, Adv23]. By their cause, Hill and Smith classify misses into three classes (the *3-C model*): *compulsory* (first access — the line was never in the cache), *capacity* (the working set exceeds the capacity), and *conflict* (more hot lines fall onto one set than it has ways) [HS89]; the textbook presentation adds *coherence* misses as a fourth class in multi-core systems [HP19]. The classification guides action, since it separates the countermeasures: compulsory misses are addressed by prefetching (Section 2.2.3), capacity misses by more compact layouts, and conflict misses arise precisely with the regular address patterns of large node arrays — if hot elements lie apart at power-of-two distances, they compete for the same sets [HP19, Dre07].

³`CTR_EL0` is the *Cache Type Register* of the ARM architecture, a system register that encodes, among other things, the smallest line size of the instruction and the data caches [Arm24]; `riscv_hwprobe` is the Linux system call through which a RISC-V program queries properties of the running core, among them the block sizes of the cache-block operations (CBO) for zeroing, managing, and prefetching cache blocks [Lin26a].

Moreover, a miss does not block a modern cache: since Kroft’s *lookup-free* organization, the cache keeps one status register per outstanding miss (*miss status holding register*, MSHR) and serves further accesses in the meantime [Kro81]. The small, fixed number of these registers — realized at Intel as *line fill buffers* between L1 and L2 — limits how many memory accesses can be in flight at once: the *memory-level parallelism* (MLP) [HP19, Int24a]. Whether a search structure can offer this parallelism at all is a design question — the search path of a tree is a chain of mutually dependent accesses and does not exploit it as a matter of course (Section 2.2.3).

Two further properties act at the line boundaries: *alignment* — an object spanning a line boundary costs two transfers instead of one, which is why cache-aware layouts align nodes to line boundaries — and *false sharing*: if two cores write to *disjoint* fields of the same line, the line migrates back and forth between the cores despite logical independence, because coherence (Section 2.2.5) always manages whole lines [Dre07, Int24a].

2.2.2 Cache-Line-Oriented Layout Research

This very cache-line-oriented layout is the historical guiding idea of the relevant research: the CSS tree [RR99] introduces the cache-line node, the CSB⁺ tree [RR00] the contiguous sibling cluster, Hankins and Patel [HP03] measure the effect of node size, and Samuel et al. [SPB05] make the node processor-aware; the Adaptive Radix Tree [LKN13] grades node size cache-line-orientedly with Node4/16/48/256, and Schmidt et al. [SKK⁺25] measure stride versus sequential accesses on high-bandwidth memory. As concrete layout options, the literature knows the dense and the cache-line-aligned array-of-structs, the column-oriented struct-of-arrays, the bit-packed, succinct layout as used by LOUDS-based tries [Jac89] and SuRF [ZLL⁺18], and the SIMD-tiled AoSoA⁴. The multi-level hierarchy itself is moreover the subject of software/hardware-cooperative prefetching methods such as the hierarchical instruction prefetcher for server applications of Zhang et al. [ZGH⁺25], which enters this thesis as a prefetching instance and not as a data-structure-specific tree data prefetch.

2.2.3 Prefetching

Prefetching brings data into the cache *before* their use and is the primary countermeasure against compulsory misses and unhidden memory latency (Section 2.2.1). The hardware prefetchers of modern cores form a few documented classes: next-/adjacent-line prefetchers fetch the neighbouring line of every missed line, stream prefetchers detect ascending or descending access sequences and run ahead of them, stride prefetchers learn constant distances per load instruction (recognized by instruction pointer), and more complex spatially or temporally correlating methods are the subject of research; the survey is given by Mittal [Mit16], and the concrete prefetchers of the L1/L2 stages are documented in the vendor manuals [Int24a, Adv23]. A prefetcher is judged by three quantities: *accuracy* (how many fetched lines were used), *coverage* (how many misses were avoided), and *timeliness* — prefetches that come too late hide the latency only partially, those that come too early evict lines still needed [Mit16].

In addition, the architectures provide *software* prefetch instructions (x86: the PREFETCHh family including a non-temporal variant; ARM: PRFM) with which the program itself announces an address

⁴AoSoA (*Array of Structures of Arrays*): a hybrid layout that groups the records into tiles of fixed width and stores them field-wise within each tile, so that a SIMD register loads exactly one tile column; it combines the access locality of the array-of-structs with the vectorizability of the struct-of-arrays.

together with a target level [Int24b, Arm24]. Their central tuning knob is the *prefetch distance* — how far ahead the announcement is made: large enough to hide the memory latency, small enough not to lose the fetched line again before use; it depends on latency and processing speed and is thus a quantity to be calibrated per platform [Mit16]. In index research, Chen et al. justify wider B⁺ nodes precisely for this reason: if all lines of a multi-line node are announced in parallel, the effective node load time drops, and the optimal node width grows beyond the single cache line [CGM01]; fractal prefetching transfers the same principle across levels to disk pages [CGM02].

Prefetching finds its limit at *pointer chasing*, the core of every tree search: the address of the next node is known only once the line of the current node has arrived. Such a dependent load chain offers no pattern to either a stride or a stream prefetcher, and it serializes the memory accesses — the available memory-level parallelism (Section 2.2.1) remains unused [Mit16]. The known countermeasures therefore start not at prediction but at design: more useful bytes per dependent step (wider, line-aligned nodes [CGM01]) or several *independent* searches whose accesses are interleaved so that the load chain of one fills the waiting time of the others — design answers that recur as parameters in Section 2.2.6.

2.2.4 Main Memory and Address Translation: DRAM, NUMA, and TLB

Behind the last cache lies the DRAM main memory, whose characteristics for the DDR5 generation are fixed by the JEDEC standard JESD79-5 [JED20]. Two quantities are frequently conflated: the *data rate* in megatransfers per second (MT/s; two transfers per clock) measures the bandwidth of the interface, while the *CAS latency* (CL) counts clock cycles until the first response of an opened row — since the CL cycle count grows with the nominal rate, rising data rates hardly lower the absolute access latency [HP19, KAS21]. Parallelism is supplied by the organization: several memory channels per system, the already-mentioned DDR5 split of each channel into independent 32-bit sub-channels, and bank groups within each chip whose interleaved operation covers row activation times [JED20, KAS21]. Stored on the module are the standard’s *JEDEC nominal profiles*; alongside them exist vendor-specific overclocking profiles (Intel XMP, AMD EXPO) beyond the standard rates — which rate a machine actually runs is therefore a property of the target platform that must be documented, not a constant readable from the module.

In multi-socket and chiplet systems, topology is added: with *NUMA* (non-uniform memory access), each node owns local memory, and accesses to remote nodes cost noticeably more than local ones. By default the operating system places pages according to the *first-touch* policy — a page is placed on the node of the thread that first writes it [Lam13]. For measurements on search structures this is a silent precondition: whoever builds the structure on a different node than the one measuring it measures the interconnect latency as well.

Besides the data-cache hierarchy there exists a second locality boundary: the *translation lookaside buffer* (TLB) caches virtual-to-physical mappings, and a TLB miss triggers an expensive page walk [HP19]. Its *reach* — entry count times page size — limits the memory region addressable without a page walk. For locality reasons processors separate the instruction TLB from the data TLB (dTLB) — since instruction accesses are considerably more local than data accesses — and lay it out in multiple stages (L1 dTLB plus a shared L2 STLB) [Int24b, Dre07]; for search structures the dTLB is the second, independent boundary alongside the cache line, since large or widely scattered nodes raise dTLB pressure independently of cache utilization. In the state of the art the dTLB appears mainly as a *measurement target*;

algorithmically it is addressed by large pages (huge pages, 2 MiB/1 GiB), which enlarge the TLB reach, are supported by several allocators — mimalloc [LZdM19], jemalloc [Eva06], TCMalloc [HKT⁺21], and smlalloc [LDPB19] — and which Schmidt et al. [SKK⁺25] use empirically with 2-MiB pages; these four allocators are available in this thesis as interchangeable allocator building blocks, and which allocator building blocks are activated in the reference run is stated in Chapter 3. A standalone dTLB optimization nevertheless remains underrepresented in the search-algorithm publications, an observation to which the research gap in Chapter 3 (Section 3.1) returns.

2.2.5 Write Path, Replacement, and Coherence

The write and update behaviour of the caches, too, is largely microarchitecturally fixed. Writes follow *write-through*, which writes immediately through to the next level, or *write-back*, which at first merely marks the change in the cache line (*dirty*) and writes it back only upon its eviction — which saves bus traffic and lowers write latency; orthogonally to this the allocation policy (write-allocate, typical with write-back, or no-write-allocate, typical with write-through) determines the handling of a write miss; ARMv8 makes this choice explicit via page attributes — write-through, write-back, or non-cacheable together with read- or read-write-allocate hints — [Arm24], while x86 normally uses write-back with write-allocate [Int24b]. Which line is evicted is decided by a heuristic replacement strategy from LRU/pseudo-LRU to the two-bit-frugal, scan- and thrash-resistant *re-reference interval prediction* (dynamic variant DRRIP) [JTJE10], which builds on the adaptive insertion policy [QJP⁺07]; how many levels hold a line at once is governed by the inclusion policy — an *inclusive* cache holds every line of the higher levels redundantly, an *exclusive* one (victim cache) excludes them, a *non-inclusive* one allows both —, and in multi-core systems an invalidation-based coherence protocol secures consistency: the widespread MESI keeps every line in one of the states *Modified*, *Exclusive*, *Shared*, or *Invalid*, extended by vendors with variants such as MOESI (AMD [Adv23]) or MESIF (Intel [Int24a]) [HP19]. As a concrete instance of these fixed properties, the L3 inclusion policy varies between generations — inclusive on earlier Intel client generations (such as Nehalem), non-inclusive on Intel servers from Skylake-SP/Xeon Scalable, and predominantly victim-like on AMD Zen [Int24a, Adv23]. Since for the most part this is not freely selectable at runtime, these properties are given per target platform and accessible only through measurement; their effect — in particular the *cache-line ping-pong* under competing writes to the same line — must be made visible explicitly by a fair measurement procedure (Chapter 3).

2.2.6 Cache-Aware and Cache-Oblivious Data Structures

With the terminological taxonomy from Section 2.1 and the parameters now introduced, the state of the art of cache-near data structures can be ordered. The empirical impulse of the cache-aware line was given by Ailamaki et al., whose time breakdown of commercial database systems showed that around half of the execution time lies in memory stall cycles — above all in L2 data and L1 instruction misses [ADHW99]. The answers of the following years each bind part of the parameters from the previous sections explicitly into the design: the cache-line size (CSS/CSB⁺ tree [RR99, RR00]), the prefetch distance together with multi-line nodes (pB⁺ tree [CGM01], across levels fractal prefetching [CGM02]), page size and SIMD width at once (FAST, an architecture-sensitive tree layout with interlocked page, line, and SIMD blocking [KCS⁺10]), or the fanout per cache line (ART [LKN13]); the layout techniques of clustering and

compression go back to Chilimbi et al. [CHL99]. The cache-oblivious counter-position is instantiated as well: Bender et al. ground the cache-oblivious B-tree on the van Emde Boas layout [BDFC05], building on the order-maintenance/packed-memory primitive [BCD⁺02], the cache-oblivious traversal of ordered sets with a locality-preserving $O(k/B)$ bound [BCDFC02], and the layout of a tree of fixed topology that minimizes the expected number of block transfers in a multi-level memory model nearly optimally even under unknown block size [BDFC02, BDFC00]; Saikkonen and Soisalon-Soininen maintain a cache-sensitive layout invariant even under updates [SSS08, SSS16], and Graefe/Larson [GL01] compare both approaches across the CPU cache levels, while all practical trie and B⁺ indexes are designed cache-aware. Table 2.1 orders the designs named here by which hardware parameters they explicitly bind into the design.

Table 2.1: Cache-near structure designs and the hardware parameters they explicitly bind into the design.

Design	Cache line	Page/TLB	SIMD width	Prefetch distance	parameter-free
CSS tree [RR99]	•	—	—	—	—
CSB ⁺ tree [RR00]	•	—	—	—	—
pB ⁺ tree [CGM01]	•	—	—	•	—
Fractal prefetching [CGM02]	•	•	—	•	—
FAST [KCS ⁺ 10]	•	•	•	—	—
ART [LKN13]	○	—	•	—	—
Cache-oblivious B-tree [BDFC05]	—	—	—	—	•
Node-size study Hankins/Patel [HP03]	•	—	—	—	—

Legend: • = explicit design parameter of the original work; ○ = considered indirectly (ART grades its node sizes cache-line-orientedly without parameterizing the line size); — = not addressed. Classification by this thesis based on the cited original works.

Two observations on the table lead further. First, every design binds only *few* parameters firmly — no known design treats the parameter set of the previous sections as an open input space. Second, both philosophies leave the same question open: how can *measured* locality knowledge of the concrete target platform — whose cache properties are predominantly hard-wired and only to a lesser extent configurable (Section 2.2.5) — be combined systematically with the cache-near settings that are actually selectable? This question is taken up again in Chapter 3 (Section 3.1).

2.3 Workloads and Benchmark Conventions

Whether a search structure makes good use of the hardware properties from Section 2.2 only shows under load; for this, fixed terms and benchmark conventions have become established. A *load* (operation) is a single call of the search interface — an exact lookup (also called a point query: the search for exactly one key), an insertion, a deletion, or a range scan. A *workload* (in this thesis synonymously: *load profile*) is a structured profile of such operations: an operation mix (e.g. 50 % read / 50 % update) over a key access distribution (uniform, Zipf-distributed, latest values) on a dataset of defined size and key characteristic. Realistic workloads are thereby sequences of many consecutive interface calls — not isolated, unrealistic single accesses.

The most widespread workload convention is the Yahoo! Cloud Serving Benchmark (YCSB) [CST⁺10]. It defines a family of key-value workloads: YCSB-A (50/50 read/update, Zipf-distributed), YCSB-B (95/5 read/update), YCSB-C (read only), YCSB-D (latest values), YCSB-E (short range scans), and

YCSB-F (read-modify-write). Alongside this, the literature relies on further established families: the index benchmark *SOSD* [KMvR⁺19], the benchmarks of the *TPC* family (transactional TPC-C, analytical TPC-DS) [Tra24], the integration into real systems (such as SuRF [ZLL⁺18] as a Bloom-filter replacement in RocksDB), real string corpora (URLs, DNA, dictionary terms), and — for the hardware-near works — *SPEC CPU* [Sta17] and *CloudSuite* [FAK⁺12]. These frameworks produce different loads that are not directly comparable with one another.

The *goal of workload variation* is to break a methodological bias: each publication typically chooses the load profile under which its own algorithm performs best. Only when all load profiles run against all comparison candidates can bias-reduced comparability be gained. For this thesis, the conventions named above provide the raw material of a common set of load profiles; how this set is constructed and driven over all candidates is developed in Chapter 3, and the corresponding measurement matrix belongs to the evaluation methodology (Chapter 5).

Observation under load uses an established set of metrics. At the cache level one counts the misses per level, expressed as a *miss rate* (misses per access) or — more robust when comparing programs of different access intensity — as *MPKI* (misses per thousand instructions); at the core level, *IPC* and *CPI* (instructions per cycle and its reciprocal) measure the achieved execution density, and stall-cycle breakdowns show which share of the clock cycles is spent on waiting memory accesses [HP19]; that this share can dominate under database loads was precisely the finding of Ailamaki et al. [ADHW99] (Section 2.2.6). For cache-aware layouts, the *cache-line utilization* is added — the share of actually used bytes per loaded line (Figure 2.2) — which directly measures what a layout gains at the transfer unit.

These quantities are collected via the processors’ *performance monitoring counters* (PMC): a few programmable count registers per core to which a hardware event (such as an L2 miss or a dTLB miss) is assigned via event selection; they are made accessible by tools such as `perf` [dM10] or VTune [Rei05]. Their limits are just as well documented: counter values are not strictly deterministic across repetitions and, depending on the event, systematically overcount [WM08], and if more events are to be observed than registers exist, the tool multiplexes the counters — the result is then an extrapolation, not a count. A reliable comparison therefore needs repetitions and an explicitly reported dispersion figure [HB15, WM08]. Which of these metrics this thesis collects and how they are ordered into measurement categories is developed in Chapter 3.

2.4 Software Means and Synthesis into the Abstraction

With which software-system means — compile-time metaprogramming, static use of the hardware properties from Section 2.2 — can the abstraction left open there be *realized*? Available are the established means of software-engineering design: the named design patterns [GHJV94], the static polymorphism of the *Curiously Recurring Template Pattern* (CRTP) [Cop95, VJG17]⁵ together with *policy-based design* [Ale01], and the composition from interchangeable building blocks as described by invasive software composition from gray-box components [Aßm03], generative programming [CE00], and the feature-oriented software product lines [ABKS13, BO92]; C++23 modules structure the translation units

⁵CRTP: a class derives from a class template to which it passes itself as the template argument (`class Derived: Base<Derived>`); the base can thereby access members of the derived class already at compile time and replaces virtual calls with statically resolved ones — without run-time indirection and without a table of virtual functions. The pattern was named and described by Coplien in 1995 in the *C++ Report* [Cop95].

of such building-block compositions but by themselves do not yet guarantee portable ABI stability [Xu24] — relevant as soon as pre-compiled building blocks are to be exchanged; how this thesis instead keeps the binary interface of its pre-compiled building blocks stable is shown in Chapter 4.

For interchangeable building blocks to become a concrete system, what is needed besides the composition techniques is a *form of description* for the configuration chosen in each case. The established answer is *declarative configuration models*: feature models have, since the FODA study⁶, described configurable variant spaces as feature hierarchies with selection constraints [KCH⁺90], generative programming produces the system variant in the solution space from such configuration knowledge in the problem space [CE00], and the product-line literature systematizes the associated binding times from compile time to run time [ABKS13]. Industrial tooling follows the same principle: Maven’s *Project Object Model* is the declarative — held as an XML file — manifestation of *what* makes up a modular Java system, while the build lifecycle takes over the *when* and *how* [Apa26]; CMake produces a native buildsysteem per platform from one declarative description via interchangeable *generators* [Kit26]; and Mokhov, Mitchell, and Peyton Jones show that build systems as a whole can be scientifically captured and recombined as a combination of a few scheduler and rebuilder building blocks over declarative dependency descriptions [MMP18]. That declarative models can accompany a system beyond build time into operation is developed by the models@run.time line [BBF09]; Götz shows in the TU Dresden MQuAT dissertation how declarative (configuration) models steer the self-optimization of component-based systems through contract negotiation [Göt13].

For this thesis, both strands together form the toolbox: the composition techniques with which the hardware properties measured per target platform can be turned into a concrete layout at compile time, and the declarative configuration models with which the chosen configuration can be described and reproduced; how both are combined for this purpose is developed in Chapter 3.

2.5 Heuristics as Decision Rules

Both the adaptive and learned indexes (Section 2.1) and the replacement strategies of the caches (Section 2.2) make their decisions *heuristically*. A *heuristic* is a decision rule without a guarantee of optimality: it maps observable features — such as the cache properties determined per target platform (Section 2.2) or the measured load profile (Section 2.3) — to a concrete configuration without proving that configuration best. It takes the place of an exact solution wherever the latter is unknown or too expensive to determine; the cache’s own heuristic replacement strategies (LRU, RRIP; Section 2.2) are the hardware-side archetype of this way of thinking [JTJE10].

Heuristics can be distinguished by *when* their decision is made. A *static heuristic* is fixed at compile time: the chosen configuration is compiled firmly into the translated program and no longer changes at run time. A *dynamic heuristic*, by contrast, selects the configuration at run time, depending on the actually observed behaviour, and may adjust it during operation. Both variants presuppose the same *heuristic function* but shift the moment of its evaluation.

Formally, this heuristic function is a *mapping* from a measurement result or load profile to a configuration: $h : \mathcal{M} \rightarrow \mathcal{K}$, where $\mathcal{M}$ denotes the space of collected features (measurements, load-profile metrics) and

⁶FODA (*Feature-Oriented Domain Analysis*): the domain-analysis method of the Software Engineering Institute, which captures the common and the variable characteristics (*features*) of a product family in a feature tree with mandatory, optional, and alternative nodes together with selection constraints [KCH⁺90].

$\mathcal{K}$ the space of admissible configurations. The present thesis first measures this relationship systematically (Chapter 3); the *automated* determination of h itself — the mechanism that derives the respective best configuration from the measurement results — is the objective and outlook of this thesis (Section 6.3). Finally, the terms used partly synonymously in the literature must be delimited. A heuristic is called *adaptive* if it adjusts its decision to changing conditions; *dynamic* refers more narrowly to the moment — evaluation at run time rather than at compile time; and *measurement-driven* (equivalently *measured-value-driven*) emphasizes the data source — the decision rests on collected measurements rather than on fixed model assumptions. An adaptive heuristic is thus always dynamic, a dynamic one not necessarily adaptive, and both may, but need not, be measurement-driven.

With this, the foundations are assembled: the landscape of search structures, the hardware properties that shape their performance, the workload conventions under which they are compared, the software means of abstraction, and the notion of heuristics that connects measured knowledge with configuration choice. How exactly these means allow a search algorithm to be decomposed into orthogonal, individually measurable design decisions is developed in Chapter 3; the concrete realization of the building blocks introduced there is shown in Chapter 4.

3 Concepts of a Cache-Aware Measurement System

Building on the foundations from Chapter 2, this chapter first develops the problem statement and the requirements for a measurement system, and as their *solution* the anatomy model and the measurement system built upon it — demonstrated on the candidate under test PRT-ART. In doing so, the chapter newly introduces three orderings that Chapters 4 to 6 presuppose: the triad of the axes into measurement, system, and organ axes, the four carrier stages of the apparatus — planner, CacheEngineBuilder, tier binary, and hybrid tier binary — with their identity stamp, and the store in which what is built and what is measured lies under stamp keys.

3.1 Problem Statement and Requirements for a Measurement System

The starting point is the finding of the foundations (Chapter 2): the performance of cache-aware search structures is decided not by asymptotic complexity but by cache-line behaviour under real load. This behaviour, however, is the result of many interlocking design decisions — how nodes branch, how pages are arranged in memory, how allocation, prefetching, and synchronization are done —, and in the published methods these decisions have grown together monolithically: whoever measures a method always measures the whole bundle, never the individual decision. Precisely this is the problem statement of this thesis, the *separability problem*: the contribution of each individual design decision to cache-line behaviour shall become visible in isolation — per decision as well as for the whole algorithm.

The state of the art sharpens this problem in two directions. In substance, each of the studied works optimizes only a few design decisions in isolation, and in the studied corpus *not a single method occupies all design dimensions together*: some dimensions are filled exclusively with baseline or textbook variants (the mapping between algorithm and cache traversal), others appear only as a *measurement target* (the dTLB), the spatial search class is missing entirely, and decisions such as I/O, migration, filtering, and buffering are each occupied individually but never together with the search decisions in one design. Methodologically a second gap is added: results are reported in incomparable settings, and the frequent choice of scenario-specific, often favourably tailored load profiles hampers a fair cross-comparison; what is missing is a bias-reduced measurement of *all* load profiles against *all* methods on a common basis.

From this double gap follow four requirements for a solution. It must, first, *decompose* cache-aware search algorithms into interchangeable design constituents, so that each decision becomes individually graspable; second, *reconstruct* the state-of-the-art designs as explicit configurations of these constituents, so that the known and the new become comparable in the same space; third, *measure* the cache-line-aware behaviour per constituent as well as for the whole algorithm on a common basis — all load profiles against all methods, bias-reduced; and fourth, derive the best overall algorithm from the measurements and *deliver* it system-optimized — realized as a ranking per interface function and metric, as a Pareto front of non-dominated candidates, and as a delivery artefact of binary, version sidecar, and manifest

(Section 3.6.1). The cache-aware focus is no side condition but the guiding quantity: precisely the cache-line behaviour that decides performance shall become measurable in isolation per design decision. The measurement requirement sharpens, at the juxtaposition of cache-aware and cache-oblivious designs (Section 2.2), into a third, intermediate path: the actual cache properties (line size, hierarchy depth, topology) are *determined* per target platform and enter a system-optimized binary as fixed quantities, while the *configurable* cache-near settings are permuted as sub-axis parameters against an already-loaded binary — in the measurement realm these are the registered dynamic dimensions prefetch distance, pool budget, batch size, inline threshold, thread count, and workload as well as, where architecture and operating system permit, the hardware prefetcher, graded by what hardware and operating system allow, from fully runtime-configurable (mutable Linux with MSR access) through boot- or compile-static (immutable operating systems) to fixed (read-only cache registers on ARM and RISC-V); which grade a platform offers is determined by the planner’s platform probe. Thus cache-oblivious behaviour is augmented with *measured* locality knowledge.

The four requirements condense into a target picture, the *design space*: each partial decision — say node type, memory layout, or allocator — spans an axis of its own, and every state-of-the-art algorithm is a point in the Cartesian product of these axes (Figure 3.1).

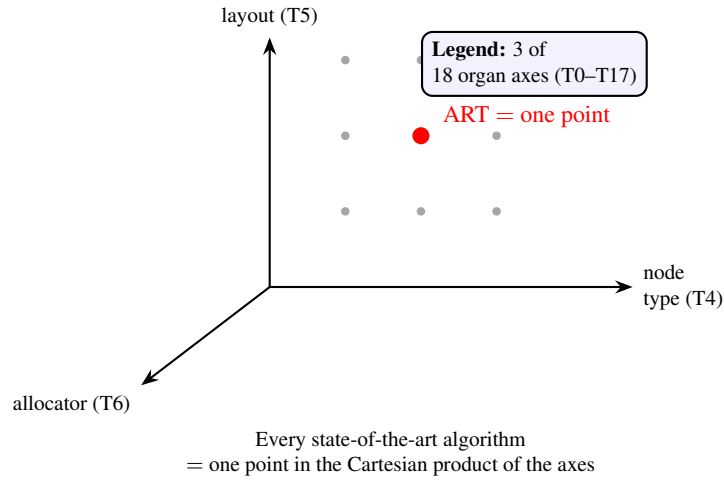


Figure 3.1: The design space: the axes span a multidimensional Cartesian product; a concrete algorithm is a point within it. For illustration, three of the eighteen organ axes are chosen — the real storage organ of node type (T4), memory layout (T5) and allocator (T6); the full design space has eighteen organ dimensions; to these is added an optional nineteenth organ axis for disk I/O, which does not describe an assignment but describes how other axes are described (Section 3.4.1).

Once the design space is spanned this way, the separability problem becomes solvable: varying a design decision means moving along exactly one axis while all others stay fixed — its contribution becomes measurable in isolation. What this thesis contributes is consequently the systematic, orthogonal axis decomposition of the known designs together with a *permuted measurement architecture* that reconstructs them as explicit configurations and measures them on a common basis; for cache-aware algorithms cannot be differentiated without a separation of the properties that decisively govern caching. The four requirements at the same time describe a processing chain — from the definition of an axis assignment through its generation and measurement to the evaluation —, and a system of this size remains manageable

only if these stations are developed as separate layers with fixed contracts; the layer model for this is developed in Section 3.3.4. Which of the catalogued instances the framework adopts and which it replaces as obsolete is developed dialectically by the following Section 3.2; the solution concept (Section 3.3) and the measurement system built upon it (Section 3.4) are the solution that closes the double gap.

3.2 State of the Art as Axis Instances

The methods known from Chapter 2 now appear systematically as axis compositions; the complete catalogues of this dissection are gathered in the present section. This adoption proceeds *dialectically* (thesis–antithesis–synthesis): per axis a foreign organ is *adopted* (thesis) or an obsolete assumption is *replaced* (antithesis), from which the own contribution emerges as synthesis. The guiding idea is the “third way” from Section 3.1 — measured locality knowledge instead of hard-wired or ignored cache awareness —, which makes each axis explicit as an orthogonal, permutable, and individually measurable organ. For many axes the antithesis is not an obsolete foreign instance but the implicit assumption of not regarding the decision as selectable at all; the synthesis then consists in making it an explicit axis in the first place (marked “—” in the replaced column). Two cluster assignments are deliberate approximations: the cache-traversal axis (T1) draws its inspiration from the layout theory as well as from prefetching, and the reclamation sub-axis of the allocator (T6) shares its sources with the synchronization (T8) — an inter-organ wiring, not a cluster violation. The dialectical comparison is carried by the living-being metaphor: every catalogued method is a living being whose organ — the assignment of an axis — is adopted or replaced, and the framework is the living being that arises from the adopted organs. The metaphor itself (organ, blood, habitat, laboratory) is defined in Section 3.3.1; here it is only touched upon. Table 3.1 summarizes this adoption per composition axis.

Table 3.1: Dialectical adoption of the state-of-the-art instances per composition axis (in canonical slot order T0–T17; telemetry and target ISA belong to the system sphere and to the measurement tooling, respectively, Section 3.4.1. Cluster codes: A trie/node, B hybrid/layout, C allocation, D prefetching, E synchronization, F hardware/measurement/engineering)

Axis		adopted (thesis)	replaced (antithesis)	synthesis core
T0	Search-Algorithm (A)	ART, HOT, START, SuRF, Wormhole; k-ary/interpolation search	monolith “structure is method”	search paradigm as a permutable axis
T1	Cache-Traversal (B)	cache-line stride (Chen), cache-oblivious (Bender), HBM (Schmidt)	implicit textbook linear search	algorithm and cache view separated
T2	Mapping (B)	linear 1:1 mapping; hash redirect, permutation index	—	direct vs. pool-relative mapping made explicit
T3	Path-Compression (A)	byte-wise (ART), Patricia (HOT), macro node (CoCo), multi-byte (START)	uncompressed raw path	compression granularity selectable
T4	Node-Type (A)	ART Node4/16/48/256	fixed node size	adaptive node class + storage delegation
T5	Memory-Layout (B)	AoS-aligned, SoA, LOUDS bit-packed, CSS/CSB ⁺ , AoSoA	—	four layout dimensions permutable

Axis	adopted	replaced	synthesis core
T6 Allocator (C)	A01–A23 (Hoard, mimalloc, jemalloc, TCMalloc, ...) + RCU/Hazard	algorithm-hardwired allocator	allocation as shared interface organ (seven sub-axes)
T7 Prefetch (D)	distance estimator (ART), hardware (Wormhole), stride (Chen), adaptive	prefetch not isolated as an axis	path-oriented bundle prefetch (own)
T8 Concurrency (E)	OLC (ART), RCU, hazard pointers; lock-/wait-free	coarse locking as the only assumption	pattern $\perp$ reclamation as two sub-axes
T9 Serialization (F)	raw binary, varint (ART), succinct (LOUDS/FST), LZ77	—	encoding/compression selectable
T10 Value-Handle (F)	inline, external pool, immutable shared (RCU), versioned	implicitly fixed embedded value	placement, ownership and versioning (+ chain ref)
T11 Index-Organization (B)	clustered, IOT, non-clustered, heap	—	storage order as its own axis
T12 I/O-Dispatch (F)	buffered, mmap, direct I/O	mmap as anti-pattern	persistence decoupled from the search structure
T13 Migration (F)	hot/cold, multi-tier NVM, LeCaR-ML	tiering never combined with search axes	migration logic as a measurable axis
T14 Filter (F)	Bloom, Cuckoo, SuRF-range, Xor	filter never integrated with search axes	filter as an orthogonal organ
T15 Queuing-q1 (F)	Disruptor ring, delta chain, CoW, epoch-RCU, SPSC/MPMC	operations buffer never an axis	buffer with access-pattern classes
T16 Queuing-q2 (F)	eager/lazy, watermark (LSM), timed	flush policy implicit	adaptive EWMA flush policy (own)
T17 Persistence-Target (F)	in-memory storage as the standard target	implicit in-memory assumption	persistence target as its own axis (pinned to in-memory in the measurement catalogue)
<i>Quantities of the system and measurement sphere (Section 3.4.1):</i>			
target_isa (sys-tem)	vendor-spec x86-64/AArch64/Power/RISC-V (+ SIMD)	—	target ISA as a system complex axis
Telemetry (sys-tem/meas.)	density tracker, insert counter, HdrHistogram, path counter	per-node counter (cache-line ping-pong)	coherence-friendly leaf-only; split in two: measurement sub-axis + build quantity

The systematic core is the axis-wise *dissection*: every overall algorithm is decomposed into its organs, and per axis the contributing organ algorithms are listed, with a brief reference to the originating living being. The framework carries *eighteen organ composition axes* in fixed slot order (T0–T17); alongside them stand the quantities of the system sphere — target ISA, SIMD and extension hardware as well as the two-way-split telemetry (the triad of axis categories is ordered by Section 3.4.1) — and the candidate build axis page type. Besides the eighteen canonical organ main axes — a *main axis* is an axis whose chosen assignment is compiled compile-statically into the binary — the organ catalogue carries an optional *organ meta-meta axis* for disk I/O (*meta-meta* means: the axis does not describe an assignment but describes how other axes are described; the short definition together with the ordering of levels is given in Section 3.4.1),

which takes effect only with a disk-writeback target and is built in compile-statically. Many axes are refined into *sub-axes* — axes beneath a main axis: as static sub-axes a compile-time choice among partial algorithms (say SA1–SA4 or AA1–AA7), as dynamic sub-axes runtime parameters that are varied without recompilation — and provide other axes with distributed interfaces for their detail operations — exactly these *axis interfaces* are captured by the micro-benchmarking (Section 3.4.6), which collects runtime and success parameters per interface call (Section 3.4). Table 3.2 gives the overview of which publications contribute an organ per axis; the thematic origin clusters of the prior works thereby appear *dissolved* into their organ contributions.

Table 3.2: Axis dissection of the search methods: eighteen organ axes in canonical slot order (T0–T17), the quantities of the system and measurement sphere, the candidate build axis, and the optional organ meta-meta axis; per axis the sources contributing an organ.

Axis	Sub-axes	Contributing sources
T0 search_algo	SA1–SA4	P01,P02,P05,P07,P10; k-ary (TUD 2009), interpol., Eytzinger, skip list, B-tree, hash
T1 cache_traversal	CT1–CT2	cache-line default; P17,P21,P32
T2 mapping	MP1–MP2	P03,P07; CE baseline
T3 path_compression	PC1–PC3	P01,P02,P04,P05,P09
T4 node_type	NT1–NT3	P01,P02,P03,P04,P05,P06,P11,P12,P13
T5 memory_layout	HM1–HM3 ^b	P06,P09,P11,P12,P32; default
T6 allocator ^a	AA1–AA7	A01–A23; P29,P30
T7 prefetch	PF1–PF3	P07,P21,P23,P25,P26,P27
T8 concurrency	CC1–CC2	P08,P29,P30; Dijkstra,Courtois,Herlihy,Michael/Scott
T9 serialization	SR1–SR3	P01,P09,P10; LZ77
T10 value_handle	VH1–VH3	P01,P03,P07,P29
T11 index_organization	io1–io3	Bayer/McCreight,Comer,Oracle
T12 io_dispatch	IO1–IO3	Stonebraker,Crotty; baselines
T13 migration_policy	MG1–MG3	anti-caching,NVM-tiering,LeCaR; none
T14 filter	FT1–FT3	Bloom,Cuckoo,P10,Xor
T15 queuing_q1	QS1–QS6	Disruptor, Bw-tree, Driscoll, P29, Pugh, LSM, Lamport, Michael/Scott
T16 queuing_q2	FS1–FS4	eager/lazy/watermark/Kafka/RocksDB
T17 persistence_target	—	CE baseline (in-memory); disk writeback as a disabled option
Sys. target_isa	IS1–IS3	x86-64/AArch64/Power/RISC-V (vendor-spec)
Sys. external_utils: SIMD	SE1–SE3	SSE2/AVX2/AVX-512/NEON/SVE2/RVV; GPU: CUDA
Sys. external_utils: ext. HW	HW1–HW4	HW strategy per publication (Tab. 3.4)
Sys./meas. telemetry	TM1–TM3	P01,P02,P16,P17,P26,P28; HdrHistogram (Gil Tene)
Build (candidate) page_type	PG1–PG3	P01,P02,P04,P05,P06,P10,P11,P12,P13,P20; PRT-ART (2026)
Organ meta-meta disk_io	staging (runtime)	CE baseline; optional, disabled (Section 3.4.1)

^a Corpus of 23 works (A01–A23); the code provides 26 vendor implementations.

^b HM4 (stride pattern) is a planned fourth sub-axis without a carrying layout building block; the code carries no tag for it (Appendix D, Section “T5 Memory Layout”).

The *search axis* (T0) comprises the most organs — as original bindings ART [LKN13], HOT [BZP⁺18], START [FJKN20], SuRF [ZLL⁺18], and Wormhole [WNJ19], plus generic search methods such as k-ary search [SGL09], interpolation search [PIA78], Eytzinger-layout search [KM17], skip list [Pug90], B-tree [BM72], and open-addressing hashing [Knu98]. The traversal axes separate the algorithm and cache views: in the code T1 contains the three traversal building blocks `LinearFanout`, `HashLookup`, and `BinarySearchFanout`; the related literature spans the axis from cache-line walking through stride

prefetch [CGM01] and HBM proximity [SKK⁺25] to cache-oblivious recursion [FLPR99] (Appendix D), while T2 (mapping) remains a sparsely occupied axis with the linear 1:1 mapping (`DirectPlacement`) and the pool-relative offset mapping (`PoolRelative`) — hash redirect [WNJ19] (MP04) and permutation index [MKM12] (MP03) are carried by the code as roadmap values without a registry entry. The *path compression* (T3) instantiates the byte-wise variant [LKN13], the Patricia split [Mor68, BZP⁺18], and the pass-through variant `None`; the macro node [BFTV24] and the multi-byte rewiring [FJKN20] are related literature of the axis without a building block of their own (the sub-axis PC1 carries the granularity `multi-byte` as a tag), and the *node-type axis* (T4) bundles in the code the four adaptive ART node formats Node4/16/48/256 [LKN13]; HOT compound [BZP⁺18], Internal/Border [MKM12], Decision/Span [SSL⁺21], LOUDS-Dense/Sparse [ZLL⁺18], and the CSS/CSB⁺/Hankins nodes are paper node formats (reconstruction targets of this thesis, not a CE baseline); the PRT-ART profile family is brought in by the candidate under test via its build axis page type.

The *memory-layout axis* (T5) ranges in the code from the cache-line-aligned AoS (`AoSStrictMemoryLayout`, `CacheLineAlignedMemoryLayout`) through SoA and AoSoA to the bit-packed LOUDS layout (`PackedBitmapMemoryLayout`) [Jac89]; pointer-free arrangement [RR99], sibling cluster [RR00], and embedded tree [SSL⁺21] are related literature of the axis (Appendix D). The *allocator axis* (T6) is the most densely occupied with the corpus A01–A23 and is refined into seven sub-axes — from free-list topology through thread locality/NUMA affinity [UHL⁺17] and the allocation strategy to reclamation (epoch, RCU [McK01], hazard pointers [Mic04a], QSBR); the code provides 26 allocator implementations for the 23-work corpus (individual works contribute several variants); the 23 allocator profiles with licence and workload affinity are listed by Table 3.3, ten of them attached with adapter and original code. The *value axis* (T10) distinguishes inline value [LKN13], external pool [WNJ19], versioned pointer [MKM12], immutable shared ref [McK01], and the chained external reference (`ChainRef`, a local re-implementation without a paper template) — five of five building blocks of the registry —, the *serialization axis* (T9) raw binary, succinct LOUDS/FST, and varint [LKN13].

On the hardware-near side, the *prefetch axis* (T7) contains four building blocks in the code — no prefetch (`NonePrefetch`, measurement baseline), the distance-estimated software prefetch after ART [LKN13] (`DistanceEstimatorPrefetch`), the explicit hardware prefetch instruction from Wormhole [WNJ19] (`HardwarePrefetch`), and the candidate’s path-oriented bundle prefetch (`PathOrientedPrefetch`, this thesis); the related literature spans the axis from software-fixed [CGM01] through adaptively distanced [Kha11] and fill-buffer-aware [MWR25] to path-oriented [ZSZ⁺24] and hierarchical bundle prefetch [ZGH⁺25] (Appendix D). The system axis *target ISA* and the SIMD extension of the extension-hardware hub (both system sphere, Section 3.4.1) carry the vendor specifications (x86-64, AArch64, Power, RISC-V with SSE2/AVX2/AVX-512, NEON/SVE2, RVV). Two cross-cutting strategy axes record what an algorithm actively *uses* and how it *distributes* work — the *hardware strategy* (SIMD family, cache-level targeting, NUMA, prefetch hardware, atomic family) and the *scheduling strategy* (worker-pool layout, SIMD worker limit — hardware-bound typically two of N cores —, heterogeneous P-/E-core dispatch, co-routines, and batch granularity) —, which Table 3.4 assigns per publication. The *telemetry* — in the triad of this thesis not an organ axis but split in two, into a measurement-tooling sub-axis and a build quantity of the system sphere (Section 3.4.1) — instantiates density tracker [LKN13], insert counter [BZP⁺18], HdrHistogram, path-read counter [ZSZ⁺24], and the probability hints (P16/P17 Bender [BDFC00, BDFC05]) — in the code in two families: the CRTP organ registry `AllTelemetries`

Table 3.3: Allocator profiles (23, all with `expected_workload` tag) with licence, workload affinity, and attachment state

Profile	Family	Licence	expected_workload	Code
hoard	A01	Apache-2.0	YCSB_A	yes
slab	A02	none	YCSB_A	—
michael_lockfree	A03	LGPL-2.1-or-later	YCSB_B	yes
mimalloc	A04	MIT	YCSB_A	yes
jemalloc	A05	BSD-2	YCSB_B	yes
tcmalloc	A06	Apache-2.0	YCSB_C	yes
snmalloc	A07	MIT	YCSB_A	yes
scalloc	A08	BSD-3	YCSB_A	yes
numalloc	A09	unknown	YCSB_B	—
rpmalloc	A10	Public Domain	YCSB_C	yes
lrmalloc	A11	MIT	YCSB_B	yes
cama	A12	none	YCSB_A	—
starmalloc	A13	Apache-2.0	YCSB_B	—
tcmalloc_warehouse	A14	Apache-2.0	YCSB_C	—
hmalloc	A15	none	YCSB_B	—
pim_malloc	A16	MIT	YCSB_E	—
crystalline	A17	unknown	YCSB_B	—
exgen_malloc	A18	unknown	YCSB_C	—
buddy	A19	none	YCSB_C	—
dlmalloc	A20	Public Domain (CC0)	YCSB_C	yes
ptmalloc2	A21	LGPL-2.1+	YCSB_B	—
pmr_resource	A22	Standard-Library	YCSB_B	—
vmem_magazines	A23	none	YCSB_A	—

`expected_workload` = YCSB workload class (A/B/C/E); licence verbatim from the profile (none = no OSS code, unknown = not recorded in the profile); Code = adapter and original code present in the repository (yes) or documentation dossier without adapter (—). Two reference quantities: 23 profiles A01–A23 (profile count of this table) versus 26 allocator wrappers of the code (wrapper count, Appendix D); there A22 splits into the three standard-library baselines A22a–A22c (`std_malloc`, `pmr_resource`, `pool_resource`), and the allocator front A24 reconstructed from the SOTA profile P33 joins without an allocator profile of its own.

with four strategies (leaf-only counter, density tracker, insert counter, latency histogram) and the vtable family `ITelemetryStrategy` with six kinds, among them path-read counter and probability hints (Appendix D) —; the coherence-friendly leaf-only variants go back to Kuehn (P28 [Kue23]; personal communication 2023, beyond the published DaMoN-2023 state on the cache- and distribution-aware B⁺ layout), while the counter in all (including inner) nodes counts as an *anti-pattern* (cache-line ping-pong).

The *concurrency axis* (T8) splits into the two sub-axes pattern and reclamation (pattern $\perp$ reclamation, Table 3.1): the patterns range from the serial zero point through coarse locking [Dij65], reader/writer locks [CHP71], and optimistic lock coupling [LABN16] to lock-free [MS96], wait-free [Her91], as well as RCU [McK01] and hazard-pointer [Mic04a] methods; a locking mode within a pattern (read-only, read-write, upgradeable, optimistic validation) is a consumer-less axis draft conserved in the code (`locking_mode.hpp`) and not a building block: implemented are the patterns as sub-axis CC1 and the reclamation as CC2 (Appendix D). The *I/O axis* (T12) instantiates the RAM-only zero point of the axis (`InMemoryOnly`, measurement baseline without I/O latency) as well as buffered [Sto81], direct, and memory-mapped I/O [CLP22] — four of four building blocks of the registry. The axis thereby builds on the block-transfer thinking of the external-memory tradition (Section 2.2): for I/O-efficient data-structure engineering, TPIE provides a reference library of the external-memory line (ESA 2002) [APV02]; the

Table 3.4: Target hardware and scheduling profiles assigned in this thesis per reconstructed SOTA method (selection; the profiles are reconstruction targets of this thesis, not necessarily original-paper properties; the hardware-strategy column follows the vocabulary of the consumer-less axis draft `hardware_strategy.hpp` conserved in the code, the target ISA of each reconstruction stands in the `<isa>` field of its SOTA profile (system sphere, in the code a build constant; P03 and P31: `X86_AVX2`, P27 and P32: `X86_AVX512`); the axis catalogue HW1–HW4 with the platform profiles follows in Appendix D, Section “Build-HW”; there is no paper-by-paper list).

Publication	Hardware strategy	Scheduling strategy
P01 ART	AVX2, L1-aware, none, CAS	thread-per-core
P02 HOT	AVX2 (bit vector), L1-aware, CAS	thread-per-core
P03 Masstree	scalar (profile ISA <code>X86_AVX2</code>), L1-aware, CAS	thread-per-core + work-stealing
P05 START	AVX2 (multi-byte span), L1-aware, CAS	thread-per-core
P20 B-Trees Are Back	AVX2, HBM-aware, NUMA-aware, CAS	work-stealing + hybrid
P27 hp-soft	PREFETCH (instr. bundle), all-cache, CAS	thread-per-core + co-routine
P28 Kuehn	PREFETCH (scalar counter), L1-aware, CAS	thread-per-core + sampling
P31 Ungethuem	AVX-512 (HBM, profile ISA <code>X86_AVX2</code>), HBM-aware, NUMA-aware, CAS	hybrid (P/E cores)
P32 Schmidt	AVX-512 (SIMD stride), HBM-aware, NUMA-aware, CAS	hybrid
PRT-ART	AVX2, L1-aware (TLB-aligned), local NUMA, PREFETCH, CAS (OLC)	thread-per-core + 2-SIMD-worker limit + hybrid + co-routine + micro-batch

textbook foundation is supplied by Mehlhorn and Sanders [MS08]. The *migration axis* (T13) instantiates hot/cold separation and multi-stage tier migration over the stages RAM→SSD→HDD (`kTiers = 3` in the code; the NVM stage of the paper template [vRLK⁺18] is not implemented in the code), and the *buffering axes* (T15/T16) range from the Disruptor ring buffer [TFB⁺11] through the Bw-tree delta chain [LLS13], the copy-on-write buffer [DSST89], and the tombstone buffer [OCGO96] to lock-free SPSC [Lam77, Lam83] and MPMC queues [MS96]. The *index-organization axis* (T11) distinguishes clustered arrangement [BM72], heap storage, non-clustered secondary index [Com79], and the index-organized table (IOT, data embedded in the leaf pages of the index) — four of four building blocks of the registry —, the *filter axis* (T14) the Bloom [Blo70], Cuckoo [FAKM14], SuRF [ZLL⁺18], and Xor filter [GL20]. The *persistence-target axis* (T17), finally, makes the storage target explicit — in-memory as the standard, a disk-writeback target as a disabled option in combination with the optional, disabled organ meta-meta axis `disk_io` (Section 3.4.1) —; in the golden reference catalogue it is pinned to its in-memory representative, so that this fixed regression detector counts $2^{17} = 131072$ identities, while the cardinality of the real permutation space is computed dynamically by the planner from the activated catalogues (Section 3.4.6). This recombination is not reached in reality, since not all axis organs are compatible with one another.

The *loads*, too, appear as instances. The workloads used in the literature stem from *several*, mutually

incomparable benchmark frameworks; Table 3.5 maps them to the publications that use them. Most widespread is the Yahoo! Cloud Serving Benchmark (YCSB) [CST⁺10]; besides it appear the index benchmark SOSD [KMvR⁺19], the TPC family [Tra24], the CPU suite SPEC CPU [Sta17], the scale-out suite CloudSuite [FAK⁺12], and — for the allocator corpus — mimalloc-bench [Lc21] and the classics Larson, threadtest, and shbench. Since these frameworks produce incomparable loads, the thesis abstracts them into a common set of load profiles; in the measurement realm the load framework is registered as a meta-meta axis and carries YCSB as the only enabled building block, while TPC, SOSD, SPEC CPU, CloudSuite, and the allocator suites are attached via a *dataset loader* (Section 3.4.1).

Table 3.5: Benchmark frameworks and the publications that use them

Framework	Type	Publications that use it
YCSB [CST ⁺ 10]	KV workload	P02, P03, P07, P20, P26
SOSD [KMvR ⁺ 19]	index benchmark	P05
TPC-C / TPC-DS [Tra24]	OLTP / analytical	P19
real string corpora	string datasets	P04, P06, P10
RocksDB integration	real system	P10 (SuRF)
SPEC CPU [Sta17]	CPU suite	P23, P24, P27, A18
CloudSuite [FAK ⁺ 12]	scale-out server	P24, P27
IBM DB2	production DBMS	P22
mimalloc-bench [Lc21]	allocator suite	A04, A07, A09, A10, A18
Larson / threadtest / shbench	allocator classics	A01, A03, P08, P30
LADDIS	kernel allocator	A02
AggSum / Memory-Stream	bandwidth	P32
custom microbenchmarks	per publication (Zipf/uniform)	P01, P04, P06, P11–P14, P21, P28

From this analysis the thesis distils *fourteen* distinct load profiles (LP01–LP14, Table 3.6): they give the heterogeneous loads a common, uint64-based operation model — the op-mix names the shares of insert, lookup, delete, scan, and read-modify-write over defined key distributions — and are each derived from concrete publications.

Table 3.6: Load-profile catalogue (14 profiles, derived from the 33-paper analysis)

LP	Key	Op-mix	Distribution	neg%	Derived from
LP01	bulk-insert-dense	100/0/0/0/0	uniform-seq	0	P01, P02, P03, P05, P12
LP02	bulk-insert-sparse	100/0/0/0/0	uniform-rand	0	P01, P02, P08
LP03	insert-value-length-sweep	100/0/0/0/0	uniform-seq	0	P01, P08
LP04	static-read-positive	0/100/0/0/0	uniform	0	P02, P05, P11, P12, P16, P25
LP05	static-read-zipfian	0/100/0/0/0	zipfian(.99)	0	P10, P20, P25, P28
LP06	static-read-negsweep	0/100/0/0/0	uniform	0–100	P04, P06, P22
LP07	static-read-string-corpus	0/100/0/0/0	real-corpus	50	P04, P06, P10
LP08	range-scan-heavy	0/0/0/100/0	uniform-seq	0	P01, P10, P12, P32
LP09	insert-lookup-5050	50/50/0/0/0	uniform	0	P03, P29
LP10	delete-heavy-post-build	50/0/50/0/0	uniform-rand	0	P01, P08
LP11	mixed-ycsb-oltp	25/50/0/0/25	zipfian(.99)	0	P10, P19, P20
LP12	concurrent-rmw-stress	25/9/36/9/9	uniform	0	P07, P29
LP13	concurrent-read-scaling	0/100/0/0/0	uniform	0	P07, P08, P25, P28
LP14	dynamic-realistic-trace	9/91/0/0/0	tpcc-nonuniform	0	P19, P20

Op-mix = shares of insert/lookup/delete/scan/RMW; neg% = share of negative lookups.

The state of the art dissected here rests on a fixed corpus: the analysis draws on 33 deeply analysed search-algorithm publications (P01–P33) and a parallel allocator corpus of 23 works (A01–A23), whose axis assignments are persisted as XML profiles in the `comdare-cache-engine` (three of them as abstract profiles, see below); the complete building-block and allocator matrices are given in the appendix. The axis-wise dissection of this corpus follows a provenance table (sources checked against the original publications) over roughly 110 organ algorithms; every state-of-the-art algorithm thereby appears as a point in the Cartesian product of the axes (Figure 3.1). That the axis framework also accommodates non-tree-like methods is shown by a pure hash table as a counter-check within the Map family — as two selectable T0 building blocks, of which exactly one is chosen per permutation and both of which are permuted: an open-addressing hash after Knuth (S14, implemented) and the SwissTable [Goo18] (S22, present as a scalar re-implementation of the Swiss-table principle, disabled by default). The counter-check satisfies the point-query subset of the `std::map` interface (`find/insert/erase`) as the chosen hull over the Cartesian-constructible overall search algorithms; the order-based operations drop out by principle, since a hash table is unordered. Across this subset it occupies a limited axis subset: allocation, prefetch, concurrency, and memory layout remain shared — and on the system side the SIMD group lookup of the template [Goo18] is envisioned at the SIMD extension, the replica checks the control-byte groups scalarly —, while the tree- and trie-specific axes do not drop out: the path compression (T3) carries for this the pass-through variant (`None`), the node type (T4) possesses no `None` variant in the code and is regularly assigned one of the four ART node formats, which the hash building block does not use for its search, and the ordered range iteration is not an organ axis but the additive scan capability of the module boundary (Appendix F), which the hash building block does not offer; the counter-check thus requires no family of its own.

The publications stem from six thematic *origin clusters* that here serve only the holistic placement of each living being before it is decomposed into its organs. The *trie family* comprises ART (P01) [LKN13], HOT (P02) [BZP⁺18], Masstree (P03) [MKM12], CoCo-Trie (P04) [BFTV24], START (P05) [FJKN20], and the succinct tries LOUDS (P09) [Jac89] and SuRF (P10) [ZLL⁺18]. The *hybrid and B⁺ family* unites the B²-tree (P06) [SSL⁺21], Wormhole (P07) [WNJ19], and the cache-sensitive CSS, CSB⁺, and Hankins trees (P11–P13) [RR99, RR00, HP03]. The *layout theory* ranges from Samuel (P14) [SPB05] and Graefe/Larson (P15) [GL01] through the cache-oblivious designs of Bender et al. (P16/P17) [BDFC00, BDFC05, BDFC02, BCDFC02, BCD⁺02] to the layout-invariant trees of Saikkonen/Soisalon-Soininen (P18/P19) [SSS08, SSS16]. The *prefetching* comprises *B-Trees Are Back* (P20) [MBL25], the prefetching B⁺ trees of Chen et al. (P21/P22) [CGM01, CGM02], Khan (P23) [Kha11], Naderan-Tahan (P24) [NTSA16], Mahling (P25) [MWR25], as well as Zhang (P26/P27) [ZSZ⁺24, ZGH⁺25] and Kuehn (P28) [Kue23]. The *synchronization* comprises the optimistic lock coupling of ART (P08) [LABN16], RCU (P29) [McK01], and hazard pointers (P30) [Mic04a]; the hardware-near *TU-Dresden line*, finally, Ungethüm (P31) [UHL⁺17], Schmidt (P32) [SKK⁺25], and VAMPIR (P33) [BH23]. The *parallel allocator corpus* (A01–A23) spans from the classics `dlmalloc` [Lea96], Hoard [BMBW00], Slab together with its `magazines/vmem` extension [Bon94, BA01], and Michael’s lock-free allocation [Mic04b] through the multi-core-scalable `mimalloc` [LZdM19], `jemalloc` [Eva06], `TCMalloc` [GM05], `snmalloc` [LDPB19], `scalloc` [AKLS15], `rpmalloc` [Jan17], and `lrmalloc` [LR18] to the cache-aware/predictable CAMA [HBHR11], `NUMAlloc` [YZZ⁺23], `StarMalloc` [RFP24], and `PIM-malloc` [VIA26]. Figure 3.2 condenses the most important reconstructed

search structures into a gallery — per method its origin class, the characteristic structural idea, and the axes at which it makes its main contribution; all thus appear as points in the common design space.

ART (P01) Trie adaptive nodes 4/16/48/256, byte-wise branching, path compression <i>axes T4, T3</i>	HOT (P02) Trie height-optimized dynamic span, compact SIMD nodes <i>axes T4, T0</i>	Masstree (P03) Trie + B ⁺ trie of B ⁺ -trees, 8-byte slices, B ⁺ -leaf nodes <i>axes T4, T10</i>	CoCo-Trie (P04) Succinct compact-collapsed trie, succinct-encoded paths <i>axes T3, T4</i>
START (P05) Trie self-tuning adaptive radix span <i>axes T0, T4</i>	B²-Tree (P06) Hybrid B ⁺ with binary inner nodes, cache-aware <i>axes T4, T5</i>	Wormhole (P07) Hybrid trie + hash + B ⁺ -leaf, $O(\log L)$ exact lookup <i>axes T0, T2</i>	LOUDS (P09) Succinct unary degree sequence, bitmap-encoded trie <i>axes T9, T5</i>
SuRF (P10) Succinct succinct range filter over LOUDS <i>axes T14, T9</i>	CSS-Tree (P11) cache-sensitive pointerless implicit tree, arithmetic navigation <i>axes T4, T5</i>	CSB⁺-Tree (P12) cache-sensitive B ⁺ with <i>one</i> child-group pointer <i>axes T4, T5</i>	SwissTable Hash (counter-check) SIMD control bytes; non-tree, in the same axis system <i>axis T0 + SIMD (system)</i>

Figure 3.2: Gallery of the most important reconstructed search structures: per method (with paper ID) its origin class, the characteristic structural idea, and the axes at which it makes its main contribution. Trie, hybrid, succinct, and cache-sensitive methods stand next to a non-tree hash counter-check (SwissTable) — all as points in the same axis design space. The complete axis dissection is given in Table 3.2.

For each SOTA algorithm (*state of the art* — the state of the art from Section 2.1) and each allocator a profile XML records the axis assignment: the organ assignment (`node`, `traversal`, `value_handle`, `concurrency`, `allocator`, `prefetch`, `layout`, `reclamation`), the candidate build axis page, and — separately from it, because not an organ assignment — the target ISA (`isa`, `system sphere`) and the telemetry (`telemetry`, `measurement tooling`); in addition a key-value signature and an optional `<expected_workload>` tag that overrides the measurement heuristic (Table 3.7).

For each of the 33 corpus works and each of the 23 allocators there exists a profile XML with `<expected_workload>` tag. The profiles are of two types — the type is marked in the profile XML and anchored compile-statically in the candidate registry of the cache engine: 30 *full profiles* describe standalone search designs completely; three *abstract profiles* enter only dissected — the optimistic lock coupling of ART [LABN16] as a concurrency organ, the succinct LOUDS representation [Jac89] as a memory-layout organ (e.g. in SuRF [ZLL⁺18]), and the profiling tool VAMPIR [BH23] as a measurement technology. An abstract profile supplies some organs itself and leaves the rest to the default toolkit; it is the same type that the candidate under test PRT-ART corresponds to (Section 3.5), and full profiles of foreign candidates under test can be integrated along the same path. The design envisions per axis a separate permutation over *profile ranges* — a value range per axis instead of a single value; the persisted profiles contain a single value per axis, the range permutation is not implemented.

Across all 33 profiles these affinities distribute into 13× YCSB_C (read-heavy), 10× YCSB_A (zipf-distributed read and update), 6× YCSB_E (range scans), and 4× YCSB_B (95/5 read/update); PRT-ART itself is tagged in its candidate profile with YCSB_F (read-modify-write, for the DensityTracker updates). The core finding of this load-profile analysis is a systematic *bias*: each publication chooses the load profile under which its own design performs best. Static, read-only-oriented tries and filters (HOT [BZP⁺18],

Table 3.7: SOTA profiles (33: 30 full profiles, 3 abstract) with workload affinity

Profile	Paper ref.	Type	expected_workload
art	P01 (Leis 2013)	full	YCSB_C
hot	P02 (Binna 2018)	full	YCSB_C
masstree	P03 (Mao 2012)	full	YCSB_A
coco_trie	P04 (Boffa 2024)	full	YCSB_C
start	P05 (Fent 2020)	full	YCSB_C
b2tree	P06 (Schmeisser 2022)	full	YCSB_E
wormhole	P07 (Wu 2019)	full	YCSB_A
olc	P08 (Leis 2016)	abstract	YCSB_B
louds	P09 (Jacobson 1989)	abstract	YCSB_A
surf	P10 (Zhang 2018)	full	YCSB_A
css_tree	P11 (Rao/Ross 1999)	full	YCSB_C
csb_tree	P12 (Rao/Ross 2000)	full	YCSB_E
hankins	P13 (Hankins 2003)	full	YCSB_C
samuel	P14 (Samuel 2005)	full	YCSB_C
graefe	P15 (Graefe 2001)	full	YCSB_C
bender_treelayout	P16 (Bender 2002)	full	YCSB_C
bender_cacheoblivious	P17 (Bender 2005)	full	YCSB_C
saikkonen_multilevel	P18 (Saikkonen 2008)	full	YCSB_E
saikkonen_layoutinvariant	P19 (Saikkonen 2016)	full	YCSB_E
btreesareback	P20 (Mueller 2025)	full	YCSB_E
chen_prefetch	P21 (Chen 2001)	full	YCSB_A
chen_fractal	P22 (Chen 2002)	full	YCSB_A
khan_adaptive	P23 (Khan 2010)	full	YCSB_A
naderan_tahan	P24 (Naderan-Tahan 2016)	full	YCSB_A
mahling	P25 (Mahling 2025)	full	YCSB_E
zhang_fgcs	P26 (Zhang 2024)	full	YCSB_A
zhang_asplos	P27 (Zhang 2025)	full	YCSB_A
kuehn	P28 (Kuehn 2023)	full	YCSB_C
rcu	P29 (McKenney 2001)	full	YCSB_B
hazard_pointers	P30 (Michael 2004)	full	YCSB_B
ungethuem	P31 (Ungethuem 2017)	full	YCSB_C
to_stride	P32 (Schmidt 2025)	full	YCSB_C
vampir	P33 (Berthold 2023)	abstract	YCSB_B

expected_workload = YCSB workload class (A/B/C/E/F); type: full = full profile of a standalone search design, abstract = abstract profile that enters only dissected.

CoCo [BFTV24], B²-tree [SSL⁺21], SuRF [ZLL⁺18], CSS [RR99]) frame “build-once, 100 % positive lookups” and avoid updates; compressed tries and filters (CoCo, B²-tree, SuRF) make the negative-query share the primary axis, because they abort early on misses; zipfian-centred works (B-Trees Are Back [MBL25], Mahling [MWR25], Kuehn [Kue23]) favour cache-aware reordering layouts; write- and concurrency-heavy works (ART [LKN13], Masstree [MKM12], Wormhole [WNJ19], OLC-ART [LABN16], RCU [McK01]) conversely compete only where static structures fail; and prefetch- and hardware-near works (CSB⁺ [RR00], Chen [CGM01], Schmidt [SKK⁺25]) measure, via node size and prefetch, cache and TLB rather than the algorithm logic. Fourteen of the publications contribute no distinct load profile to the catalogue — partly survey, theory, and hardware works without an index load profile of their own, partly works whose custom microbenchmarks are subsumed by profiles already catalogued. Only when *all* load profiles run against *all* living-being binaries (and the container hulls, Section 3.3.2) — the workload as a dynamic axis (Chapter 5) — can an advantage that exists only under the self-chosen load profile be made visible and bias-reduced comparability be gained.

Finally, the quality criteria of scientific measurement (cf. the definitional classes in Section 3.4), too, appear as concrete technologies. *Hardware performance counters* (PMC, via `perf`) [dM10] are nearly universal and deliver cache-miss rates, dTLB misses, branch misses, and IPC/CPI — the majority of the works P01–P28 report such counters —; *profilers* such as OTF2/VAMPIR [BH23] and VTune [Rei05] collect trace-based runtime profiles, *simulators* such as `gem5` [BBB⁺11] provide hardware-accurate estimates where real counters are absent (so P25 Mahling [MWR25], P27 Zhang [ZGH⁺25]), and the *HdrHistogram* [Ten12] collects the latency percentiles (p50/p95/p99). On the collection the evaluation builds: instead of arithmetic means, median and percentiles are reported, differences tested for significance with the distribution-free *Mann-Whitney U test* [MW47] (with Holm correction of the family-wise error rate [Hol79]) and quantified via the effect measure *Cliff's δ* [Cli93], following the recommendations for scientific benchmarking [HB15]. A methodological line of its own is the hardware-near measurement methodology of TU Dresden (Habich group): Ungethüm et al. [UHL⁺17] systematize the hardware optimizations for database machines, Schmidt et al. [SKK⁺25] measure stride versus sequential accesses on HBM with dTLB and L2/L3 counters, and VAMPIR [BH23] profiles non-functional properties of heterogeneous memory. From all these technologies follows the set of measurement categories — cache-line utilization, cache misses (L1/L2/L3), dTLB misses, memory footprint, branch misses, IPC/CPI, latency (mean and percentiles p50–p999), throughput, energy, and fill-buffer occupancy —, whose time- and observer-based subset the present apparatus already resolves (wall clock, HdrHistogram in the downstream evaluation, per-axis observers, the statistics triad of median/percentiles, significance test, and effect measure, the two-phase operation loop, and the conformance gate against `std::map`), while the counter-based PMC categories are mandatorily built into the measurement path and their assignment per microarchitecture is bound to an event catalogue (Section 3.4.1); the complete measurement architecture is developed by the measurement system (Section 3.4) and Chapter 5.

3.3 Solution Concept: the Anatomy Model

From the dialectical adoption of the state of the art emerges the solution concept of this thesis; this section develops it as the *anatomy model* and in doing so defines the terminology and conceptual special constructs on which the following sections build.

3.3.1 Axis Composition and the Living-Being Metaphor

The central idea is to treat an algorithm not as a monolith but as a *composition of orthogonal axes* — each axis a partial decision of a cache-aware or system-aware index (how pages are arranged, how nodes branch, how memory is allocated). A concrete algorithm is then a point in the Cartesian product of all axes, a state-of-the-art design an explicit configuration. The decisive property is *modular exchangeability*: a new axis variant can be studied in isolation — its effect measured per axis *and* for the whole algorithm — without reimplementing the others; this is precisely what solves the separability problem, since the contribution of each constituent becomes measurable in isolation while the other axes are held fixed. That this exchangeability is modular shows in the code: the reference compositions of ART and PRT-ART differ in exactly one axis, the path compression (Section 3.5). This decomposition rests on the observation of shared concerns of different search algorithms and publications, as well as on the comparability of the goals of different search algorithms, and can therefore be carried by a metaphor. First, search algorithms

share, in the large, the same *tree anatomy* in different expressions — from balanced search trees through tries and radix trees to custom constructs; even a sorted key–value list is, strictly speaking, a single leaf of a tree whose values point onward via references to data leaves. The main differences between the algorithm expressions thereby come down to the fan-out of the nodes, the tree depth, the tree filters and rule sets, and the implementation details of the serialization of memory pages, search pages, and tree nodes. The tree structure is thus universal across all search-algorithm architectures and forms the canvas on which the axes are placed.

Non-tree-like search methods — such as the hash counter-check of Section 3.2 — remain the same family with a restricted profile, for a separate family would arise only if the base axis set differed fundamentally: a hash table, too, is a tree that searches from the root of its interface over a fan-out of hashes — only the core algorithm of the node is a different one. Therefore every method that translates a key into a value, no matter how, belongs to the family Map and its genus *SearchAlgorithm* (Section 3.3.2). Second, a vivid metaphor treats a concrete algorithm as a *living being* whose *organs* are the axes. The metaphor is canonically cast: the *blood* is the liquid organ that connects all organ axes and is condensed in the planner into a compile-time configuration; the system axes are the *habitat* (the planet) of the animal with the possibilities and resources found there; the measurement axes are the equipment of the *laboratory* in which the animal is examined (Figure 3.3). An animal can live very well without measurement, but only the healthiest animal shall be put into productive use — and every animal shall be adapted as well as possible to its habitat: if it lacks a property, it cannot use this part of its habitat (the hardware) to its advantage; or the measurement finds out that its specialization should not be geared to this habitat at all, because it negatively affects the quality of the animal there — a monkey shall climb and a fish shall swim, not the other way round, because different objectives have led to different designs. This metaphor is owed to the author’s background in biomedical engineering and is meant to ease the analysis through vividness; from here on the thesis works consistently with this model.

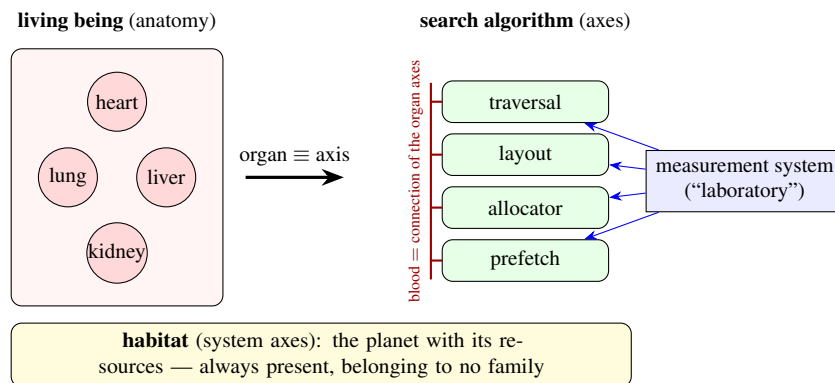


Figure 3.3: The anatomy bridge: just as a living being consists of interacting organs, a search algorithm is a composition of axis-organs; the blood connects the organ axes, the habitat (the system axes) carries the animal, and the measurement system takes the role of the laboratory that measures each organ in isolation.

The permutation of the axes is, in this reading, an organ exchange: exchanging organs between living beings produces new composite algorithms with hitherto unknown cache-line behaviour. The organs themselves are won from the state of the art — the design building blocks of the known methods (Section 2.1) are decomposed into orthogonal axes and collected in a shared axis matrix, so that each axis

harvests the contributions of the entire corpus (Figure 3.4).

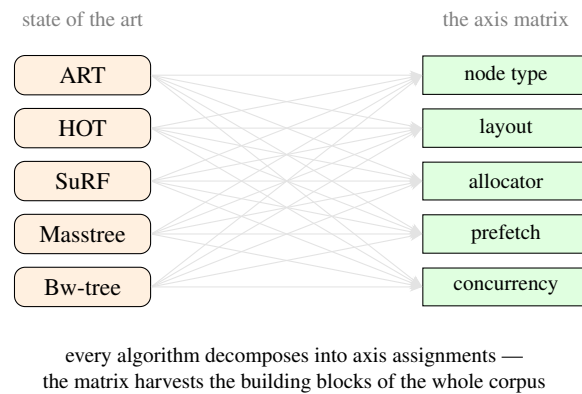


Figure 3.4: Synthesis state of the art → axes: the design building blocks of the known methods are decomposed into orthogonal axes; each axis collects the contributions of the entire corpus — the inverse of Figure 3.1 (there an algorithm is a point, here an axis is a collecting basin).

The *anatomy* thus won is more than the enumeration of the organs: it is their *wiring*. Each organ exports a generalized interface through which the others use its services without knowing its concrete assignment — thus the allocation organ provides a shared allocation interface that node layout, value placement, and serialization use for their memory needs (Figure 3.5); the hub of this wiring is in the code the storage organ `LayoutAwareChunkedStore`, which composes node type, layout, and allocator (“storage-organ interface” is the concept name, not a code type). The usage rules are broadly drawn: an axis may use foreign axis interfaces, call itself recursively, and use other families — the search family accesses axes in its interface functions and uses container genera of its own, and parts of the axis pool are reused for the containers (Section 3.4). In software-engineering terms this is the *feature interaction* of feature-oriented software product lines [ABKS13, BO92].

An axis corresponds to a feature, a living being to a feature composition. The *family* is the feature *core* composition: the shared interface core that all genera of a family share and that is used identically in the metaprogramming across all variations — just as different map categories (hash map, flat map) share a common core of interface functions; for the container family this core consists of identity, axis observation, and size. The *genus* (alias: tier subclass, Section 3.3.2) is the feature *special* composition: the real tier interface beyond the core — `SearchAlgorithm`, `Set`, `Sequence`, `Adapter`, `View`, and `FunctionInterfaceReroute` — and thus a static feature-set definition. The anatomy is the wiring between the features. Which axis uses which is shown by the inter-organ usage graph (Figure 3.6). Composing these pairwise *organ couplings* — an organ coupling is the mutual dependency of two axes in which the one uses the interface of the other (mandatorily or optionally) or restricts its admissible assignments; sub-question RQ1.b (Section 1.3) addresses it for node type, SIMD, and path compression — across all organs together yields a directed inter-organ usage graph.

3.3.2 The Three-Level Model and the Four Families

These organs are embedded in *one* architecture — a single, self-contained hierarchy without parallel trees. At its root stands the `ExecutionEngine` as the common, measurable abstraction over everything measurable — it persists because `IExecutionEngine` forms the common measurable root and

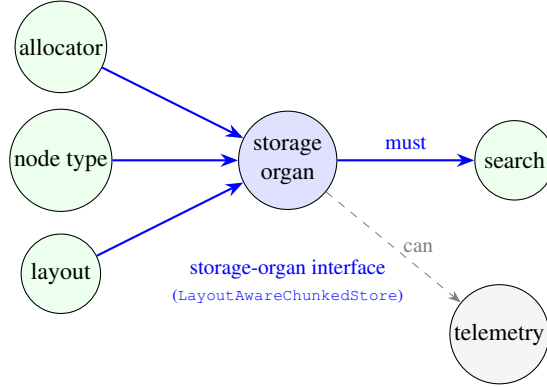


Figure 3.5: Axis = organ, anatomy = wiring: the storage organ (composition of node type, layout, and allocator) exports a generalized storage-organ interface (in the code `LayoutAwareChunkedStore`) over which the search runs (solid, *mandatory*); further organs such as the telemetry *can* use it via optional observer hooks (dashed, *optional*), but need not.

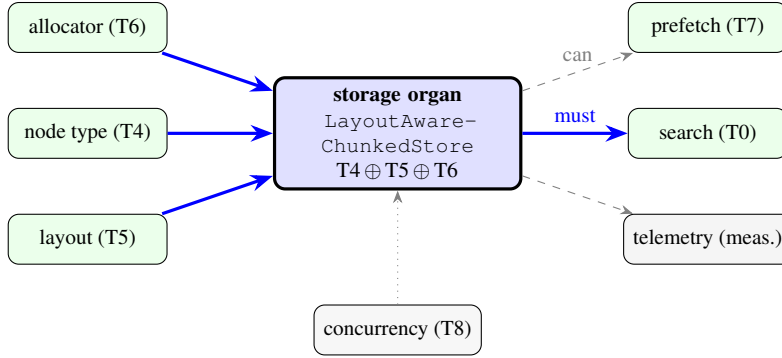


Figure 3.6: Inter-organ usage graph (“which axis uses which”): there is exactly *one* canonical hub — the storage organ `LayoutAwareChunkedStore`, which composes node type (T4), layout (T5), and allocator (T6). *Mandatorily* (solid) the store consumes these three axes, and the search (T0) runs over it. All remaining couplings are *optional* (dashed): prefetch (T7) and the telemetry (measurement instrumentation, Section 3.4.1) read the store only via `requires`-gated observer hooks — an axis *can* use foreign interfaces, but *need* not. The coupling of concurrency (T8) to the store is not connected (dotted). The graph is an example wiring of the storage organ, not a structure evaluated in the code.

`IAAnatomyBase` the anatomy-bearing view immediately beneath it; beneath that, three levels can be distinguished: the *family* as the outward-visible algorithm interface (the *probe dock*), beneath it the *genus* (alias: *tier subclass*) with a fixed axis set, beneath that the *organs* themselves (Figure 3.7). The separation of levels can be read off at the ABI boundary: every tier binary exports two letter-identical C symbols that answer family and genus without an instance, whereby the family is not maintained as a second datum but derived from the genus — the loader thus determines the family via a family-independent symbol, without knowing it in advance. “Living being” and `SearchAlgorithm` denote the same entity — metaphor and technical term —; the anatomy is not a second model but the *body* of this living being: the fixed composition of its eighteen axis-organs.

At the topmost level stand four families. The `Map` family — the key-value-oriented probe dock and the focus of this thesis — possesses as its sole genus `SearchAlgorithm`, the full eighteen-axis

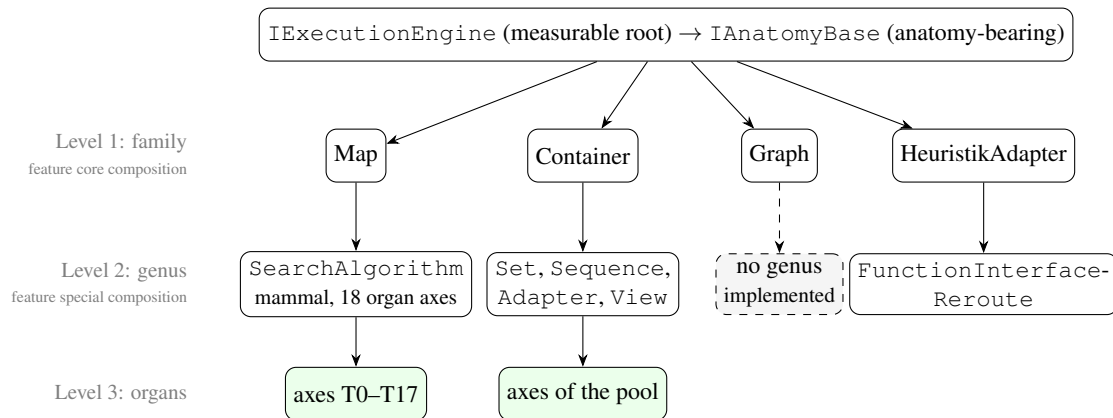


Figure 3.7: The three-level model: family (level 1, outer interface, feature core composition) → genus (level 2, fixed axis set, feature special composition) → organs/axes (level 3). Four families: Map possesses as its sole genus SearchAlgorithm, the “mammal” with the eighteen organ axes; Container the genera Set/Sequence/Adapter/View, which are formed from the same axis pool; Graph is laid out as an interface, a genus is not implemented; HeuristikAdapter possesses the reroute genus FunctionInterfaceReroute of the hybrid tier.

“mammal”; the Container family (keyless storage) fans out into the genera Set, Sequence, Adapter, and View, which are likewise formed from axes of the shared pool and whose slot count per genus is fixed compile-statically; the Graph family is laid out as an interface, a genus is not implemented; and the HeuristikAdapter family possesses the reroute genus FunctionInterfaceReroute — the family of the hybrid tier (Section 3.6.2), which by metaprogramming inherits the interfaces of a family together with a genus and passes requests through to the actual tier binaries; classification (of which kind the reroute is) and pass-through (the inherited target genus that the binary reports outwardly) lie on separate levels (Figure 3.8). Mnemonic: “Adapter” without prefix always means the container genus; the hybrid family is always called HeuristikAdapter. The kingdom “Animalia” ($\equiv$ IAnatomyBase) shown there as the root is the anatomy-bearing view immediately beneath the measurable root IExecutionEngine of Figure 3.7; the code separation of the two interfaces is deepened in Chapter 4. Non-tree-like methods thus require no family of their own but occupy a restricted axis subset of the same family.

The anatomy is carried by four *carrier stages*: the planner, the *CacheEngineBuilder* (CEB) — the measurement tool that the planner generates per target platform and measurement task —, the tier binary, and the hybrid tier binary optionally inserted between the CEB probe dock and the tier binary. The planner resolves the user XML against the three offering catalogues and chooses the measurement axes at its runtime; the CEB contains this measurement choice compiled in, knows the platform, and permutes system and organ axes into tier binaries; the tier binary contains the organ permutation; the hybrid tier binary additionally carries its attached tiers. Control sits in the planner, and the chain of releases is directed: the measurement axes determine the CEB type, the CEB defines the system axes and releases the build space, and this space permutes system and organ axes into the tier binary. No stage passes through layers — each contains its layer compiled in fixedly; only the configuration travels from the XML to the finished tier binary. The interfaces of the carriers follow a three-surface model: surface 1 is the genus interface of a tier binary (runtime, Abstract Factory), surface 2 the stamp (compile time, ABI-stable, Section 3.4.4), surface 3 the measurement cut-through (IMessVisitor), which crosses the

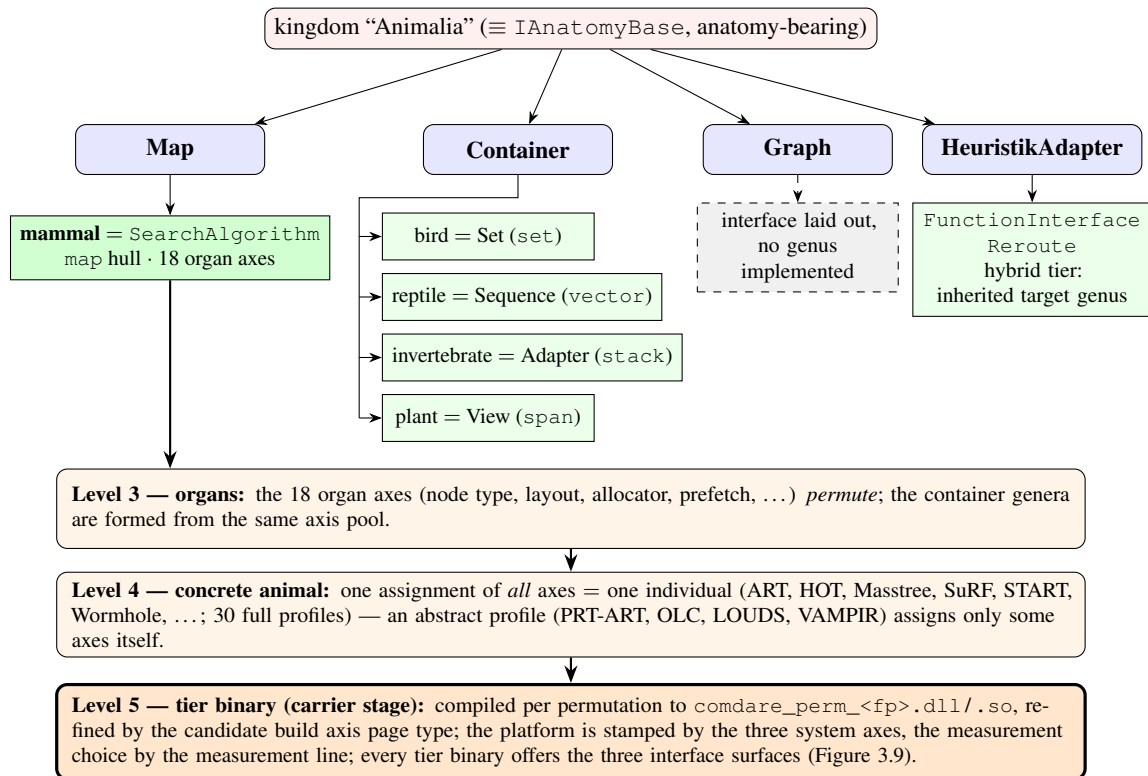


Figure 3.8: The family/tier hierarchy across five levels. Map, Container, Graph, and HeuristikAdapter are the four *families* (level 1, outer interface). Each family hosts its *own* genera (level 2, alias tier subclasses): Map exclusively the “mammal” (genus SearchAlgorithm, full eighteen-axis anatomy), Container every *non-mammal* genus (bird/reptile/invertebrate/plant = Set/Sequence/Adapter/View), Graph a laid-out interface without an implemented genus, HeuristikAdapter the reroute genus of the hybrid tier. The mammal branch is drawn out down to the individual: the eighteen axis organs (level 3) permute into a concrete axis composition = one concrete animal (level 4, e.g. ART, HOT, Masstree), which is compiled per permutation into a standalone tier binary (level 5, the carrier stage of the tier binary, additionally refined by the candidate build axis page type and platform-stamped by the three system axes, Section 3.4.1). *Every tier binary is thus an animal of its own with an actual binary genus interface and three interface surfaces*; the hierarchy reaches from the family down to the single compiled permutation. Cf. the condensed model in Figure 3.7.

measurement as a seam of its own, so that family and genus interfaces remain unchanged. On tier and hybrid tier binary all three surfaces are pronounced; planner and CEB possess the stamp surface, and the CEB additionally carries its own measurement-gate assignment, whose coverage against the tier gates (CEB off, tier on) is proven by a test. The contracts between planner, CEB, tier, and hybrid are distributed per stage locally over seven stamp interfaces — an overall self-version is carried only by the planner, measurement and system line are carried by CEB, tier, and hybrid, the organ line only by tier and hybrid (the absence at the CEB is the static barrier “the CEB carries no organ axes”), fingerprint and overall stamp are carried by all four, and the attached tiers are carried only by the hybrid (Figure 3.9).

Across the carrier stages runs a *run chain* of four work modes, which is registered in the measurement realm as a sub-axis and is traversed cumulatively in fixed order: *build* builds the binaries and does not measure; *measure* is the deterministic single-thread measurement run whose result fills the measurement

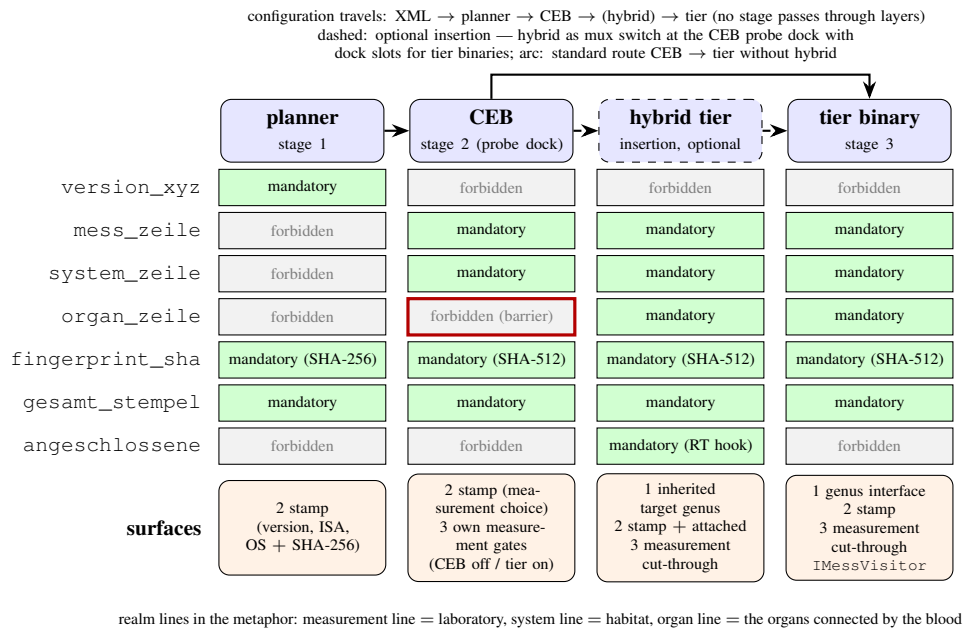


Figure 3.9: Carrier stages and interface surfaces: the three-valued admission matrix of the seven stamp interfaces over the four carrier stages planner, CacheEngineBuilder (CEB), hybrid tier binary (optional insertion), and tier binary (28 cells; green = mandatory, grey = forbidden; the cell organ line/CEB is the static barrier “the CEB carries no organ axes”) and beneath it the interface surfaces per stage — surface 1 genus interface (runtime), surface 2 stamp (compile time, ABI-stable), surface 3 measurement cut-through. The configuration travels from the XML via planner and CEB to the tier binary; the hybrid tier binary is a stage optionally inserted between the CEB probe dock and the tier binary (dashed: mux switch at the probe dock with dock slots for tier binaries; the column order of the matrix follows the processing topology with the hybrid at its insertion position, the arc is the standard route without hybrid); no stage passes through layers, each contains its layer compiled in.

store; *compare* reads the measurement store with options of its own and computes which tier binary or hybrid recombination must, according to the measurement, be the best; *release* generates from that the fully optimized binary without measurement instrumentation and re-measures its wall clock. On the hybrid level the same chain repeats synchronously (Figure 3.10). There is no debug state: `-debug` is a switch of the planner shell for the wiring check — a flag, not a work mode. Implementation status: the four modes are selectable and validatable and take effect on the build of the binaries; the mode-specific procedure of compare (replay comparison instead of measuring) and release (generation of the optimal binary from the measured values) is envisioned but not implemented — compare and release are selectable, validatable labels with release-equivalent build semantics.

3.3.3 Design Patterns and Compile-Time Patterns

The actual contribution thus lies less in a single structure than in a *tool*: instead of solving the comparison problem directly, this thesis first builds this tool — the permutation and measurement apparatus — with which arbitrary axis combinations can be generated and fairly measured. Scientifically this apparatus situates itself in the software engineering of *composition*: invasive software composition from gray-box components [Aßm03], generative programming [CE00], and feature-oriented software product

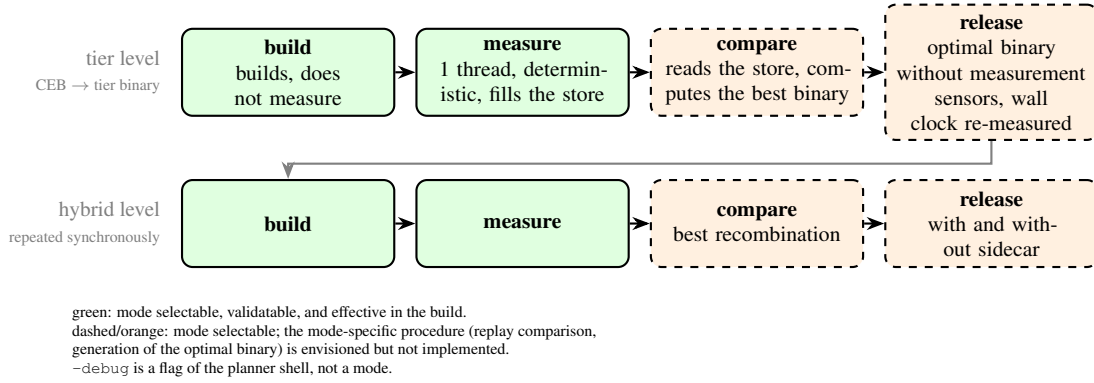


Figure 3.10: The run chain of the four work modes build → measure → compare → release as a registered sub-axis of the measurement realm, cumulative in this order and repeated synchronously on the hybrid level; compare reads the measurement store and computes the best binary or recombination, release generates the fully optimized binary without measurement sensors and re-measures its wall clock. A debug state does not exist. The assignment to the operating target pictures measurement, evaluation, working, and hybrid mode is given by Section 3.6.2.

lines [ABKS13, BO92]. The axis concept itself stems from prior work by the author under the Comdare brand, originally for the deduplication optimization of the UltiHash database; ComdareDB is its successor product as a concept without code, not a further entity. The contribution of this thesis is the transfer of the principle to cache-aware search structures as compiled, machine-optimized binaries — its design-pattern-driven, measurement-oriented realization. In software-engineering terms the apparatus is a composition of named design patterns and two new compile-time patterns (Figure 3.11).

The classical patterns are thereby not ornament but the wiring of the apparatus: *Strategy* encapsulates each of the eighteen organ axes as an interchangeable algorithm, *Director* and *Builder* emit the plan (one walk, one builder per output syntax), an *Abstract Factory* constructs the docks of the hybrid stage and the result workbooks at exactly one place, an *Iterator* traverses the lazily materialized experiment tree, *Adapter* binds foreign code behind the stable engine interface, *Observer* collects the measurement snapshots per axis, a *Visitor* is the measurement seam at the `dlopen` transition, *Chain of Responsibility* forms the hardware collection chain and the selection filter chain, and a *Memento* (copy-on-write) secures the state between warm-up and measurement [GHJV94]. The measurement hulls of the axes hold no component instance and no full interface and are expressly not a *Decorator*, and a *Composite* in the sense of the catalogue has no counterpart in the code. Not used in the hot path and in the organ axes are *Singleton*, *Flyweight*, and *Bridge*: host-side registries use Meyers singletons (the one instance as a function-local static object) [Mey05], the catalogue generation flyweight tables per axis, and “Bridge” occurs in the code only as a colloquial designation for connecting pieces — in comments on ABI adapters of the genera, platform probe, code generation, permutation commands, CMake lists, and comparison pipeline, for the CI bridge jobs of the planner, and the microarchitecture Sandy Bridge as well as in a version constant of the cache-line seam —, not as a pattern in the sense of the catalogue. The *Command* pattern fulfils several roles — the compile-time command base of the axes, the system complex axis as a command wrapper, and the typed control command channel —; the command channel of the hybrid mode, the operation in which a heuristic tier switches between preloaded tier binaries at the ABI boundary (Section 3.6.2), is one of these roles, and a *Facade* bundles the appendix generation and the reading plan emission (pattern register with code locations: Section 4.5.1, Appendix E, ADR-11).

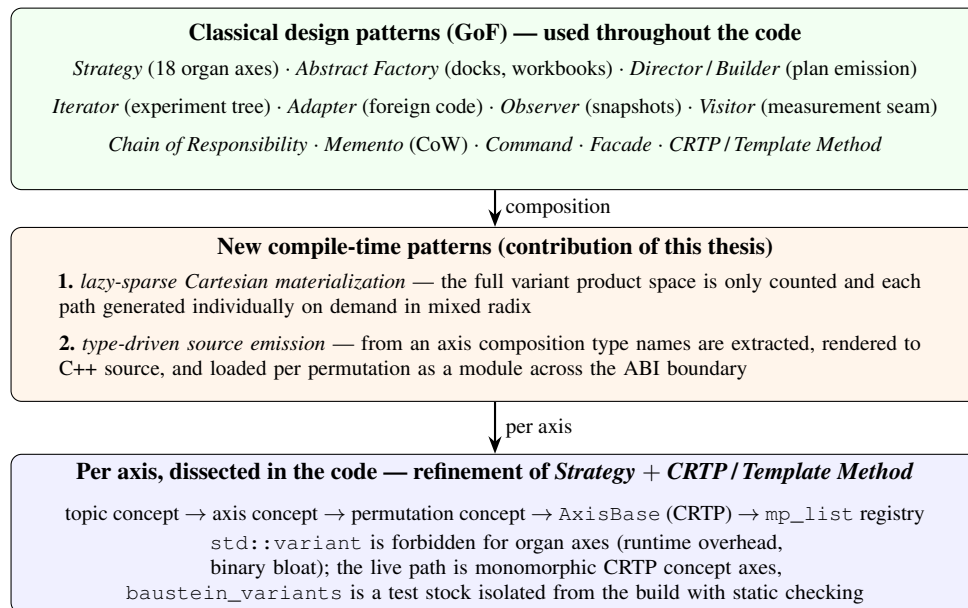


Figure 3.11: Design patterns as building blocks of the architecture: above the classical GoF patterns used throughout the code with their respective role, in the middle the two new compile-time patterns named in this thesis, below the code-dissected *three-stage concept chain* per axis (topic concept → axis concept → cache-engine permutation concept → CRTP base → `mp_list` registry) as a refinement of *Strategy/CRTP*; a literal `std::variant` occurs in no organ axis (forbidden) but only in a test stock isolated from the build. Their composition forms the design-pattern-driven, measurement-oriented architecture (Chapter 4).

Static polymorphism is provided by the *Curiously Recurring Template Pattern* (CRTP) together with *Template-Method* concept checks; the choice between candidate under test and cache-engine default per axis is made by the merge strategy of the merge plan (`Verbund1_CeOnly`, `Verbund2_Replace`, `Verbund2_Hybrid`, `Verbund3_Union`; Section 3.4.6), the axes themselves are monomorphic CRTP concept types, and *policy-based design* captures the composition as a tuple of axis policies [Ale01]. C++23 modules thereby structure the translation units but by themselves do not yet guarantee portable ABI stability [Xu24] — only the C-stable module boundary establishes that. Beyond the established patterns the thesis names two constructions for which there is no known scientific designation: the *lazy-sparse Cartesian materialization*, which counts the combinatorially exploding variant product space of all axis catalogues only arithmetically and generates each path individually on demand in mixed radix, rather than blowing up the compiler with the full type tree; and the *type-driven source emission*, which extracts fully qualified type names from a compile-time axis composition, renders them to C++ source, and loads each permutation as a standalone module across the ABI boundary — thus the candidate under test PRT-ART is compiled, by metaprogramming, in matrix layers against the cache engine.

How this *one* architecture becomes visible in the code through an ABI-stable hull is deepened in Chapter 4 (Section 4.3.1); before that, the following subsection orders the apparatus as a whole: in which layers it is organized from experiment definition to evaluation and how it is completed.

3.3.4 Five Interface Layers: E4 down to E0

The anatomy model orders the *measured*: what a living being is and which organs it consists of. Orthogonal to that, the apparatus that generates and measures such living beings needs a second ordering principle — a view of the *experiment pipeline* that transforms a declared axis assignment into measured results. This pipeline is organized into five layers E4 down to E0, complemented by the cross-cutting concern M (Figure 3.12). The numbering runs from the most abstract to the most concrete level. *E4* is the definition and evaluation layer — the level that a user of the framework typically works with: experiments are described declaratively as XML profiles, and at the end of the pipeline the same layer receives the measurement results as a return channel and prepares them into tables and diagrams. *E3* is the permutation B⁺-tree: it unfolds the declared assignment into the experiment tree, where the compile-static axes determine the identity of the binary to be built and the dynamic axes span the runtime dimension. *E2* are the tier binaries themselves: per permutation one standalone binary loaded across the ABI-stable interface (level 5 of the family hierarchy, Figure 3.8). *E1* is the runtime configuration — the most precise detail layer, in which the dynamic sub-axis parameters are varied on the already-loaded binary. *E0*, finally, is the infrastructure (build, tooling, and platform substrate); its completion forms the last step of the order. The E identifiers are a concept ordering of the pipeline, not a code constant; per layer its code location is named: for E4 the XML parse seam of the experiment entry and the evaluation generators of the umbrella project, for E3 the experiment plan director, for E2 the ABI declaration of the anatomy modules, for E1 the registered dynamic dimensions of the measurement realm, and for E0 the CMake configuration. The measurement system permeates as the cross-cutting concern *M* all layers without being one itself — this special role is deepened in Section 3.4.

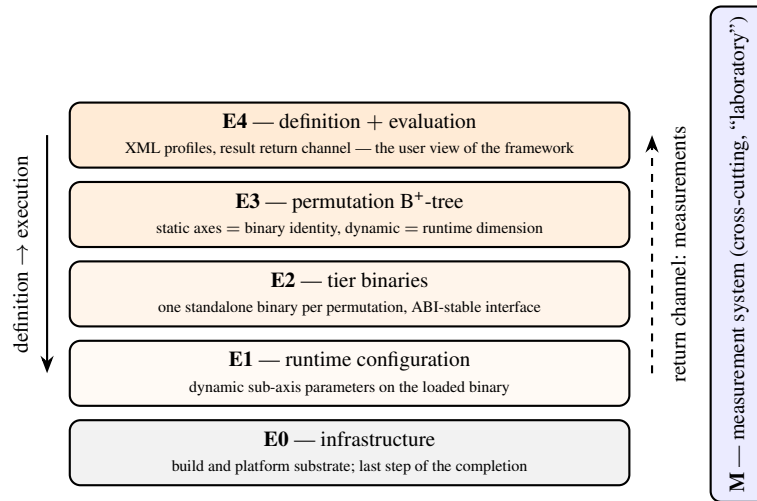


Figure 3.12: The five interface layers of the experiment pipeline: from the declarative definition and evaluation layer E4 (user view) through the permutation B⁺-tree (E3) and the tier binaries (E2) to the runtime configuration (E1); the infrastructure (E0) supports everything; its completion forms the last step. The return channel leads the measurements back to the evaluation in E4; the measurement system permeates all layers as the cross-cutting concern M (laboratory equipment, Section 3.4). Each layer is completed individually, with a fixed interface contract to its neighbouring level.

Fundamental is the completion principle of this layering: each layer is completed *individually*, with an explicitly *fixed interface contract* to its respective neighbouring level — the E4→E3 boundary, say, lays

down how an XML definition is translated into axis levels, the E3→E2 boundary how a tree path becomes a build order, and the E2→E1 boundary which runtime knobs a binary accepts across the ABI. The test question per boundary is always: does the layer pass its definition on to the next *losslessly*? A system whose product space rules out any complete materialization (Section 3.4.6) is too large for simultaneous changes across several levels; only the fixed contracts make each level individually acceptable before the next one begins. This principle corresponds to the matrix programming style of the apparatus (Section 3.3.3): just as the candidate under test is compiled in matrix layers against the cache engine, the apparatus itself is developed in interface layers against fixed contracts; how each layer is thereby tested individually — against a substitute of its neighbouring level — is shown in Section 4.4.

The E-layers are expressly to be distinguished from the three-level model (Section 3.3.2), however similar both may look at first glance: the three-level model is the *anatomy view* — it orders *one* living being from the family through the genus to the organs and thus describes the structure of the measured. The E-layers are the *pipeline view* — they order the path of an experiment from the definition through tree, binary, and runtime back to the evaluation and thus describe the process of measuring. Both views are orthogonal to each other: the same living being that anatomically decomposes into family, genus, and organs is *configured* through the layers — its axis assignment travels from the XML via the tree into the build order, while each carrier stage contains its layer compiled in fixedly (Section 3.3.2). Now that it is settled what is composed and in which layers the apparatus processes it, the following section turns to the question of how the living beings thus composed can be measured fairly.

3.4 The Measurement System

To compare structurally different search algorithms *fairly*, the thesis maps them all onto a single, well-understood interface: the C++ contract of the ordered map hull `std::map<Key, Value>` with exact lookup (`find`), insertion/update (`insert`, `emplace`), deletion (`erase`), and order-based navigation (`lower_bound/upper_bound`, range iteration) [ISO24]. The axes describe the *inner* behaviour behind this fixed interface, so that the comparison of two conformant `std::map` implementations becomes the object of the measurement; in parallel the sequence hull `std::vector` (indexed access, resizing, contiguous storage) serves as a container comparison case, to check whether an axis composite also satisfies the container and not only the search semantics. Behind the fixed outer interface the system distinguishes several *levels of dynamics* that determine *when* an axis decision is made: the topmost split of compile time versus run time; the genus, which occupies only a subset of the axes; the axis algorithms compiled into a binary at compile time (static recombination); the dynamic sub-axes, whose runtime parameters — the registered dynamic dimensions prefetch distance, pool budget, batch size, inline threshold, thread count, and workload — are varied without recompilation; and the static sub-axes as a compile-time choice among partial algorithms of differing complexity. Therein lies the structure of every axis that sub-question RQ3.b (Section 1.3) addresses: the *main axis* is the compile-time-fixed choice — for organ axes a segment of the `binary_id`, for system axes a stamp entry together with build version tag —, the *sub-axes* are its refinement, dynamically as runtime-variable parameters that the measurement registry carries as dynamic dimensions of the stage `runtime`, or statically as compile-time tuples of their main axis. The idea of regarding data structures not individually but as a design space shapes Idreos’ *Periodic Table of Data Structures* [IZA⁺18] and the *Data Calculator* [IZH⁺18]: structures

arise from the combination of few design primitives whose costs can be synthesized — which coincides directly with the axis dissection of this thesis (Section 3.2), in which the classical families are mere reading groupings of orthogonal axis assignments. Against the static design-space enumeration of the Data Calculator and classical *policy-based design* [Ale01], however, this taxonomy expressly carries compile- and runtime-variable axis levels together; an established name for this grading is not documented in the literature reviewed (cf. Section 3.3.3).

The container genera are formed from the same axis pool as the search algorithms and are no standard containers — out of necessity, not convenience: only thus do the axes permitted in the compiled artefact of a tier binary remain controllable, only thus does the micro measurement reproduce the same runtime on every run, and only thus do the properties of `SearchAlgorithm` and container that resemble each other remain seamlessly comparable over a uniform interface; with the standard containers of the C++ library this control would be lost. The shared core of the container family consists of identity, axis observation, and size (Section 3.3.1); the genera `Set`, `Sequence`, `Adapter`, and `View` are implemented. The hulls `std::map(SearchAlgorithm)` and `std::vector(container)` are to be carried out completely in the implementation; the measurement concentrates on the required interface functions, the implementation of all functions is envisioned. The order-based and iterative operations — `find` is a member function of the `std::map` contract [ISO24], range searches are not — are bundled by a library of its own over the iterators of the genus interfaces, which covers all Cartesian products of the axes equally at compile time; this genus iterator library is not implemented.

These building blocks are only as meaningful as their measurement. Scientific measurement is subject to established quality criteria [HB15]: *reproducibility* (fixed seeds, deterministic order, documented configuration, so that runs are methodically repeatable and statistically comparable), *validity* (internal: causally isolated effects; construct: the metric actually measures the intended concept; external: transferability to other platforms and loads), *fairness* (identical keys, workloads, and warm-up; missing capabilities as *N/A* rather than covertly emulated), and *dispersion* as part of the statement (percentiles instead of arithmetic means, with significance testing; the distributional view is provided by HDR histograms in the downstream evaluation). Reported percentiles follow the nearest-rank method: the rank $R(q, n) = \lceil q \cdot n \rceil$ is one-based, the corresponding array index $k = R - 1$ zero-based; the median is the special case $q = 0.5$ of the same method and yields, for even n , the lower middle value, not the mean of the two middle values; p95, p99, and p999 are the same ranks with $q = 0.95$, $q = 0.99$, and $q = 0.999$. On these criteria the measurement patterns used build: a *two-phase operation loop* separates a discarded first run from the actual measurement — the first run is the warm-up of a potentially cold cache, so that the measurement starts in a definitely warm cache, and the parameter difference between cold first run and warm measurement is itself a source of insight: it proves an actually previously cold cache and estimates the speed-up that the method achieves against a cold cache; only rows with a valid second phase enter the evaluation —; the collection is *hybrid* — isolated axis algorithms directly against one another, a composed living being centrally over an ABI-stable per-axis observer interface —; a *conformance gate* checks the `std::map` contract before every measurement, so that only comparable things are compared; and a plausibility criterion discards every run in which a measurement with measurement sensors is not slower than the same without. Where such a criterion needs a threshold, it is *platform-calibrated* — determined per machine signature from measurement runs on the respective platform, not set as a fixed model assumption —; implementation status: the platform-calibrated thresholds are not available; their

determination presupposes the measurement runs (Section 6.1). These concepts form the *definitional classes* of measurement; their technical *instances* — hardware performance counters, OTF2/VAMPIR profiling, the TU-Dresden hardware measurement methodology — have already been introduced as concrete technologies in Section 3.2.

The measurement-tooling main axis spans three *measurement levels*, in exactly this and no other order: *wallclock* (*w*) — the wall clock of the overall runs in the CacheEngineBuilder over the load frameworks; *macro* (*ma*) — the time of the genus interface functions of the tier binary; *micro* (*mi*) — the time and the success parameters per axis interface. Subsets of the three levels are permitted per stage, every other order is syntactically wrong; *w/ma/mi* is the registered alias. Every interface call is measured from beginning to end for its *success parameters* and its overall runtime; the measuring instrument is the measurement checkpoint (`checkpoint_measure`), which collects the axis-specific success parameters in the *mi* layers, and from the *ma* level on all parameters including the PMC counters are merged (full join of the *mi* success parameters). A measurement channel is always addressed via its path *axis* → *genus* → *category*, never via a free name. Three points are not implemented at this point: the catalogue of the success parameters per axis (subject of future work), the attachment of the checkpoint to the measurement loop, and the collector mechanics that gathers the parameters.

3.4.1 The Triad: Measurement, System, and Organ Axes

Within these definitional classes the measurement system answers the categorization question laid out as a question in the introduction (Section 1.3): the axes of the design space separate along the three concerns named there — the measuring itself, the build and target platform, the measured — into *three* axis categories, which are developed in the following as standalone *realms* with one class root each, one offering catalogue each, and one stamp line each (Section 3.4.4). Binding is the canonical overall order of all main axes, and in it the realms are presented here: first the measurement tooling, then the three system axes in their fixed order, then the organ slots T0–T17. Two terms are anticipated here: every tier binary possesses an identity *stamp* — one line per realm plus a fingerprint — and is stored under this fingerprint in the *store*, the stock of binaries and measured values from which the apparatus recognizes what exists instead of building or measuring it anew (Sections 3.4.4 and 3.4.5). This triad is the second shape of the separability problem: not only the design decisions among one another, but also the measured, the platform, and the measurement instrument must stay separated, so that a measured effect can be attributed unambiguously. The measurement axes settle the question of whether a measurement apparatus is to be built into the cache engine and the tier binaries at all.

The *measurement axes* are the concerns of measuring itself — in the metaphor the equipment of the laboratory. The measurement realm comprises (a) the load framework (`load_framework`) as a meta-meta axis, which brings the workload conventions of Section 2.3 into the apparatus — with YCSB as the only enabled building block, the remaining frameworks via the dataset loader (Section 3.2); (b) the measurement-tooling main axis with the three measurement levels wallclock, macro, and micro (Section 3.4), which alone occupies the measurement stamp line; (c) the measurement categories (Section 3.2) as a *sub-axis* of this tooling, which manifest as columns of the result tables, not as main axes of their own — the registry carries sixteen categories, yet with the collection per axis interface and its parameter set their number grows dynamically (sixteen categories times parameter sets) and is never a fixed count; and (d) the run methodology as sub-axis `work_mode` with the four modes build, measure, compare, and release

(Section 3.3.2), where `-debug` is a flag of the planner shell and not a mode. None of these sub-axes ever becomes part of a binary identity: measurement axes are strictly binary-id-neutral. Within the categories there is a *regime bipartition*: the time- and observer-based regime (runtime clocks and observer hulls; nine categories, among them latency, throughput, memory footprint, and fill-buffer occupancy) and the counter-based PMC regime (hardware performance counters; seven categories, among them cache, dTLB, and branch misses, IPC/CPI, and energy). The regime is a property of the measurement axis, not of the measured organ: the same organ axis can be illuminated by both regimes. The observer-based subset is resolved by the apparatus, the PMC categories are mandatorily built into the measurement path, and the separate measurement of P- and E-cores on Intel and AMD is laid out as a scheduling dimension of the target ISA. Not implemented at this point are the collection source per axis-interface parameter set and the attachment of the P-/E-core measurement to the run profile.

The *system axes* are the concerns of the build and target platform. Canonically there are *exactly three* main axes in fixed order: the target ISA (`target_isa`), the operating system (`operating_system`), and the extension hardware (`external_utils`); the latter is exclusively the collecting hub of the hardware meta axes (SIMD/vector extensions, GPU/FPGA/NPU, external accelerators), and the three are bundled by the command wrapper `build_target_complex` (Section 3.4.2). In the anatomy metaphor the system axes are not another organ but the *habitat* of the animal: always present as soon as measurement takes place, belonging to no family — and an animal is the better, the more precisely it is adapted to it. From this follow three invariants. First, system axes never touch the binary identity (in the registry: `binary_id = never`) — no measurement instrument and no platform quantity changes *what* is built in terms of organs; conversely, the effects of the organ axes must never be blended with the effects of the platform and the measurement instruments. Second, they are nevertheless permutable: the planner releases which system-axis assignments the CacheEngineBuilder builds — this is the release of a target platform for building, which sub-question RQ1.b (Section 1.3) presupposes —, and the build space of the CEB permutes system and organ axes together into tier binaries; the system sub-axes permute as well — the scheduling with its five dimensions on the wrapper of the target ISA (Section 3.2). Third, different build worlds of the same organ permutation are different artefacts and must differ in the fingerprint: build control quantities such as optimization level and SIMD extension therefore multiply only the *build* matrix — they appear in the build version tag of the binary and in the toolchain member of the fingerprint (Section 3.4.4) —, never the organ permutation space.

The *organ axes* are the *measured*: organs of the living being, part of the families, permutable — every assignment changes the identity of the tier binary. Canonically there are *eighteen* organ main axes in fixed slot order T0–T17 (Table 3.2); the T numbers denote the slots of the composition, not an implementation numbering. The persistence target is the eighteenth organ axis; telemetry and target ISA do not belong to the organs but to the measurement tooling and to the system sphere, respectively. To these is added an optional *organ meta-meta axis* — *meta-meta* in the sense of the indirection of a description: an axis that does not describe an assignment but describes how other axes are described. In the four-stage ordering of levels of this thesis (M0 the binary code, M1 conception and runtime implementation of the main and sub-axes, M2 the literal classification of the main and sub-axes carried under their term — in the code the registry line —, M3 the meta-meta axes; the assignment of M3 is set, the cut between M2 and M1 is a reading) meta-meta is the topmost stage M3; in the stamp the meta-meta level of an entry is its bracket depth — a main axis is bracketless, a meta-meta hangs as a bracket appendage on its realm line

(Section 3.4.4) —, and the bit encoding of this level in the entry is described by Section 4.5 —, the disk I/O `disk_io`: it stamps in the bracket appendage of the organ line, does not touch the `binary_id`, takes effect only with a disk-writeback target, and is disabled; in the golden reference catalogue its appendix remains empty.

Conceptually the triad has two realm roots: the system realm is rooted at the `CacheEngineBuilder`, which knows the platform and builds the tier binaries; the measurement realm is rooted at the experiment *planner*, which resolves the experiments from the XML description (Section 3.4.6). Each of the three categories thus possesses its own abstract class root and its own offering catalogue (three registry XML); the code shape of these roots is developed in Section 4.5.

The assignment of the further quantities is unambiguous: the compiler is a sub-axis group of the build toolchain in the outer complex bracket — `build_toolchain` with compiler, optimization level, and atomics capability (Section 3.4.2); the scheduling is a sub-axis on the wrapper of the target ISA with five dimensions; the load framework belongs to the measurement realm; the target ISA is the complex axis `target_isa`; and the telemetry is *split in two*, since counter instrumentation must be built in for measuring: as a sweep sub-axis of the measurement tooling in the planner it generates a compile-time main quantity in the builder.

This last pattern is generic and carries a name of its own: the *two-way-split main axis* — an axis that runs across the runtime of the preceding stage *and* the compile time of the following stage, along the chain planner → builder → tier binary; the sub-axis release of the preceding stage becomes the main-axis assumption of the following stage. The load framework is the first instance (the planner chooses it at runtime, the builder contains it compiled in), the telemetry the second, and the hardware detection (Section 3.4.3) the third.

System and organ axes are *two-phase*: the planner releases their assignment at its runtime, the following stage contains it compiled in. Measurement axes are *three-phase*: the planner chooses them at runtime; the `CacheEngineBuilder` contains the measurement choice compiled in as its provenance — *N* tooling configurations yield *N* CEB pipelines —; and the tier or hybrid binary carries the measurement-gate state of its own translation unit as a fingerprint member, so that two compiled artefacts of the same source with and without measurement sensors possess different identities. The measurement axes thus define the contracts across CEB and tier/hybrid and permeate them at compile time, because the planner releases their properties at runtime; control sits in the planner, the contract is local per carrier stage (Figure 3.9). That the gate combination CEB off, tier on is covered is proven by a test of its own.

All three axis realms are bound by a methodological principle, the *principle of declared non-observation*: a non-collected value is a missing value (NA semantics in the sense of the statistics of missing data [Rub76]), never a zero and never a phantom value. Where a value cannot truly be collected — an organ axis without an actual runtime parameter, a counter without hardware access, a measurement source that does not exist on the platform —, the system reports *explicitly invalid or empty* instead of a plausible phantom value. Better a measurement marked invalid than a synthetic effect: a phantom value would systematically poison the heuristic derivation (Section 3.6), whereas a declared gap stays visible and can be closed deliberately. The principle holds throughout — from the axis wiring (only axes with an actually measurable runtime knob report runtime effects; the rest count as compile-static) through the availability reporting of the counter sources to the evaluation, which generates no empty plot from an empty input.

What these measurement patterns concretely collect are three opposing components — *latency, throughput,*

and *memory consumption* —, against which every axis permutation can be spanned as a trade-off profile: the same permutation that is a point in the design space in Figure 3.1 appears here as a point inside the trade-off triangle (Figure 3.13); with several competing target quantities the “best” permutation is no single winner but an element of the Pareto front of the non-dominated candidates [Ehr05] (Section 3.6.1). The fourth measurement component, central for cache-aware search structures — the cache-aware behaviour at the node, data, and index level — is collected via hardware performance counters: PMC capture is mandatorily built into every official measurement run and is set by the emitted measurement jobs at all driver build sites; only the hardware-free test cell builds without it. The assignment is microarchitecture-bound: for AMD Zen 5 (prod1) the event catalogue for L2 demand misses and foreign-CCX fills is cross-checked, for Intel no catalogue entry exists, and the last-level counter fails to open on Zen 5; each of these gaps appears as a missing value, never as a zero. The separate measurement of the P- and E-cores on Intel and AMD is laid out as the scheduling dimension `hetero_core_dispatch` of the target ISA (values `None`, `HybridAware`, `PCoresOnly`, `ECoresOnly`) and via the core-pinning building blocks; the measurement model carries six core modes for this: four pinning modes per core kind (P- or E-cores, pinned to a single core or to all cores of the kind), the unpinned reference run as the fifth, and *HybridAware* as the sixth mode¹. The cache-line-awareness measurement presupposes the *HybridAware* mode, because cache-line effects otherwise cannot be measured deliberately at specific hybrid cores; the mode is therefore a mandatory constituent of the measurement model, not an option. Implementation status: the pinning actuator is implemented; the attachment of the core modes to the run profile — for prod2, the Intel Core i9-12900K (Alder Lake) with P- and E-cores — is missing.

3.4.2 Complex Axes: the Target ISA as a Fixed Recombination

To the outside, the three system main axes behave like *one*: a *complex axis* is a bracket over a fixed recombination of several main axes — all members remain main axes but appear together as one (bundled as a command bracket in the sense of the *Command* pattern). The *outer* complex bracket spans `target_isa × operating_system × external_utils` and contains as its sub-axis group the build toolchain `build_toolchain` (compiler, optimization level, atomics capability); as a bracket it is itself *no* member of the canonical order. The target ISA is *in itself* a second, inner complex axis: a fixed recombination of the identity members RAM frequency, CAS latency, and CPU fabrication, recursively wrapped by the outer bracket; on its wrapper hang the agreed sub-system axes (scheduling, NUMA node, memory page size). The members are *declared* per machine — in the same user XML that also displays the experiment (Section 3.4.6) — and determined per operating system and ISA; on an exact property match the same axis is reusable across formally different machine types, because the host name serves only the instance lookup — the class identity is the property tuple. In the identity stamp (Section 3.4.4) a complex axis occupies exactly *one* field of its type line, no bracket per member; the operating-system distribution is a stamp variable there, not a fourth expression. The CAS latency is a purely *declared* member (provenance level “declared, not measured”, Section 3.4.3); the thesis reports this instead of feigning a measured value.

¹HybridAware: the run knows the heterogeneous core topology of the machine, pins its threads at runtime via the affinity interface of the operating system (under Linux `sched_setaffinity`) to selected cores of both kinds, and measures there with the counter set belonging to the core kind; unlike `PCoresOnly` and `ECoresOnly` it does not fix the core kind for the whole run, unlike `None` it does not run unpinned.

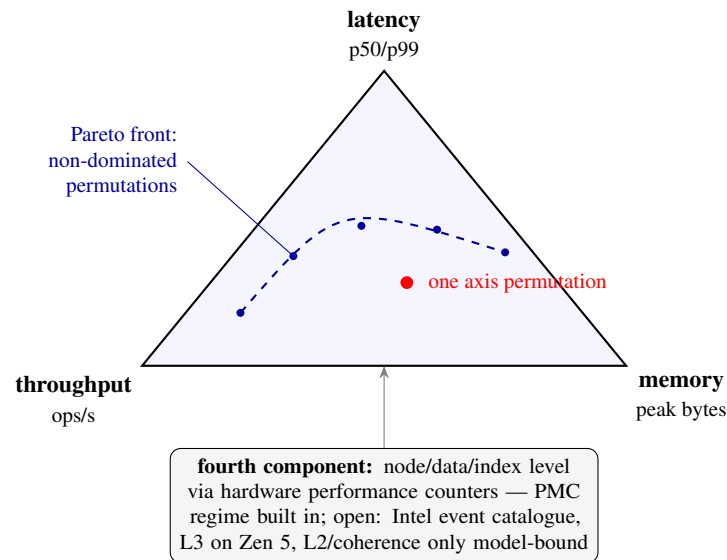


Figure 3.13: The measurement trade-off triangle: every axis permutation spans against the three opposing measurement components latency, throughput, and memory consumption and is a point inside it (cf. the design space in Figure 3.1, where the same permutation is a point); the Pareto front of the non-dominated permutations is the bridge from the three quantities to the “best binary”. The fourth component, central for cache-aware search structures — the behaviour at the node, data, and index level via hardware performance counters — is mandatorily built into the measurement path; open are the Intel event catalogue, the last-level counter on Zen 5, and the only model-bound L2/coherence assignment.

3.4.3 Hardware Detection as a Two-Way-Split Measurement Axis

So that the declared platform members do not become silently ageing constants, the concept introduces hardware detection as a further two-way-split main axis — and specifically as a *measurement* axis: it measures properties of the hardware but changes neither system nor organ axes. *Level 1* lies with the planner: at *its* runtime it coarsely detects the cell of ISA vendor family and operating system and selects by metaprogramming which fine-grained detection software including operating-system accesses is compiled into the `CacheEngineBuilder` — a compile-time cell selection, not a runtime switch across operating systems. The selection matrix is *total* over all declared ISA complexes and all documented operating systems: unimplemented cells exist as a declared declaration-only specialization, and a new catalogue entry without a cell breaks the compilation instead of leaving a silent gap. *Level 2* lies with the builder: with the compiled-in detection it collects at its runtime the actual properties of the machine and thereby sets the compile-time stamps of the tier binaries correctly. The stamp of this measurement axis itself contains the vendor detection family and the operating system — as the device segment of the measurement stamp line (Section 3.4.4) —, never a measured value. The core topology of hybrid CPUs (P- and E-cores) is carried as a sub-axis of the target ISA (scheduling dimension `hetero_core_dispatch`), so that cache-line effects can be measured deliberately per core type (six-mode model with `HybridAware` as the mandatory mode, Section 3.4.1).

Every collection result contains a *provenance* level together with the collection time, in descending order of trust: *configured-measured* (privileged system query), *SPD JEDEC base value* (read unprivileged from the module memory, deliberately only the JEDEC nominal rate; overclocking profiles appear as a

separate offer field, never as the actual value), and *declared-not-measured* (user XML). A lower level may displace a higher one only with a marker that travels along — otherwise a declared nominal rate would stand in the result table while the machine possibly runs faster: worse than a nominal rate declared as such. What is not collected receives an explicit n/a token, never a zero in the value column (principle of declared non-observation, Section 3.4.1); four error classes separate missing, unreadable, corrupt, and unknown-format sources, and a corrupt source never degrades silently to the next level. The gain of this axis is therefore not a better single value but the *automated resubmission*: a tool result collected once and then fixed — thus the RAM nominal rate of 4800 MT/s recorded for prod2 is marked as declared in the system registry, with `dmidecode` as the declaration source — is replaced by a collection that takes place anew on every run and carries its trust level along. Provenance vocabulary, SPD parser, error domain, collection chain, and platform selection matrix of this axis are implemented (Chapter 4).

3.4.4 Identity Stamps: One Line per Realm

The triad becomes readable on the tier binary itself. Its *permutation* identity is the `binary_id`: the path of eighteen `axis=value` segments in canonical slot order — one segment per organ axis, and *only* organ axes. In addition, every tier binary contains compiled-in *stamp lines*, exactly one per realm. The *organ line* records for each of the eighteen main axes the chosen algorithm together with its algorithm version. The *system line* holds the three system main axes; as their version they carry their *build* version — the ordered suffix tag of toolchain, optimization level, extension, and further build control quantities, in which an empty member simply produces no segment —, because a platform axis has no algorithm version. The *measurement line* carries the set of compiled-in measurement toolings as bracket-less segments and appends the load framework — the meta-meta main axis of the measurement realm — as a bracket appendage at its *end* (pattern `measurement_tooling=wallclock@1.0.0.c; [load_framework=yetsb@1.0.0.c]`); an *empty* measurement line is thereby an explicitly declared statement — no measurement tooling compiled in — and remains empty even without this appendix. A complex axis occupies exactly one field in its line (Section 3.4.2). The entry grammar is defined once for all lines: `axis=algo@X.Y.Z[.flag]*` with recursive composite brackets — `c{p.e}` is the base `c` with the sub-members `p` and `e` and is never read flat as `cpe`; the bracket depth of an entry is its meta-meta level, zero the main axis. The flag `e` denotes an efficiency-core algorithm. A fourth *realm* line for the mixing strategy of a candidate under test, by contrast, cannot exist: a candidate under test mixes against the organ axes and is thus already described in the organ line. The mixing becomes recognizable there in the axis name, which the candidate under test stamps under its own namespace prefix (pattern `pvt-art.memory.abc=pvt_patricia@2.3.4.c{p.e}`), so that different mixing constellations remain distinguishable by name — not by the version numbering. It is exactly on this name level that the hybrid and full-join states remain distinguishable at the compiled artefact; byte-different mixed builds therefore never possess organ segments of the same name.

The stamp is closed per carrier stage by a *fingerprint*; the stamp thereby takes on, besides the identity, the roles of cache, store, and recognition key as well as classification. The tier binary carries a SHA-512 over a preimage of eleven members in fixed order: the format identifier (`fingerprint_format=6`), the measurement, system, and organ line in this order, the sub-axis value set, the toolchain member, the enabled sets of the build axes, the overlay source hash, the actually effective measurement-gate state of the translation unit, the hybrid composite line, and the base of the build version; on this one comparable

value hangs the recognition of an already built permutation in the store (Section 3.4.5), and the format identifier as the first member leads, on every layout evolution, deterministically to a mismatch instead of a silent collision. The planner carries a SHA-256 over its version, the target ISA, and the operating system; the CacheEngineBuilder carries a SHA-512 as the provenance of its compiled-in measurement choice, which is expressly not a reuse criterion, because it deliberately does not contain toolchain and cell. Measured *values* — say, a collected RAM rate — never appear in stamp or `binary_id`: what is stamped is the compile-time platform *class*, never a measured value (Section 3.4.3).

3.4.5 The Store: Stocks under Stamp Keys

The *store* is the persistent stock of the apparatus, in which what is built and what is measured lies under stamp keys (Figure 3.14). The store tree is organized by family, genus, and realm — `gattung= → genus= → realm= (binaries or messdaten)` —, and the hybrid family is sorted into it like every other family. Four stocks are fixed compile-statically in the key schema: stock 1 are the tier binaries, keyed by their SHA-512 fingerprint, the measurement cell bracketed beside it; stock 2 are the measured values, keyed by the fingerprint of the measured binary and the hardware identity of the measuring machine (a class property of CPU fabrication and RAM pair, never the host name), the platform cell bracketed beside it — a measured value never enters its own key; stock 3 are the synthesis batches, the synthesized measurement curves as files, keyed by machine, full stamp, and the reference to measurement level and channel; stock 4 are the solutions, keyed by the canonicalized hash of the user XML, the machine, and the references to the participating stocks. Measurement is always switched on; in the store only what is missing there is re-measured, and a permutation is recognized via its fingerprint instead of being built anew. Because the measurement-gate state is a fingerprint member, per permutation the binary with and the binary without measurement sensors lie side by side as two entries of stock 1 — the precondition for a release without measurement sensors to be re-measured against the measured version (Section 3.6.2).

Two consequences are properties of the store, not of its implementation: an algorithm version change alters the organ line and thus the fingerprint, hence triggers the rebuild of the affected store stack together with re-measurement; and because the hardware identity is part of the measured-value key, every machine configuration has its own measured-value stock. Implementation status: stock 1 and 2 as well as the key schema of all four stocks are implemented; the mechanics of stock 3 and 4, the curve facet, the storage path of the hybrid tier, and the target selection via the XML form the full store build-out and are not implemented.

3.4.6 Subsystems and the Chain from the XML to the Tier Binary

The apparatus is organized as *one* main channel: *one* user XML is the only source of the entire process. It *displays* which experiments, axes, settings, and procedures are to be carried out, while per realm a *registry* catalogue holds the *offer* of what exists — the declarative configuration model of Section 2.4, modelled on the Java POM: a declarative manifest that is resolved against a repository; an unregistered reference is a hard validation error, not a silent omission. The experiment *planner* resolves the XML against the three offering catalogues and builds, per target platform and measurement task, *one* CacheEngineBuilder (CEB), into which the chosen measurement axes are compiled; this builder orchestrates with them the build of the tier binaries and stamps them with its platform (Section 3.4.4); the tier binary, finally, contains

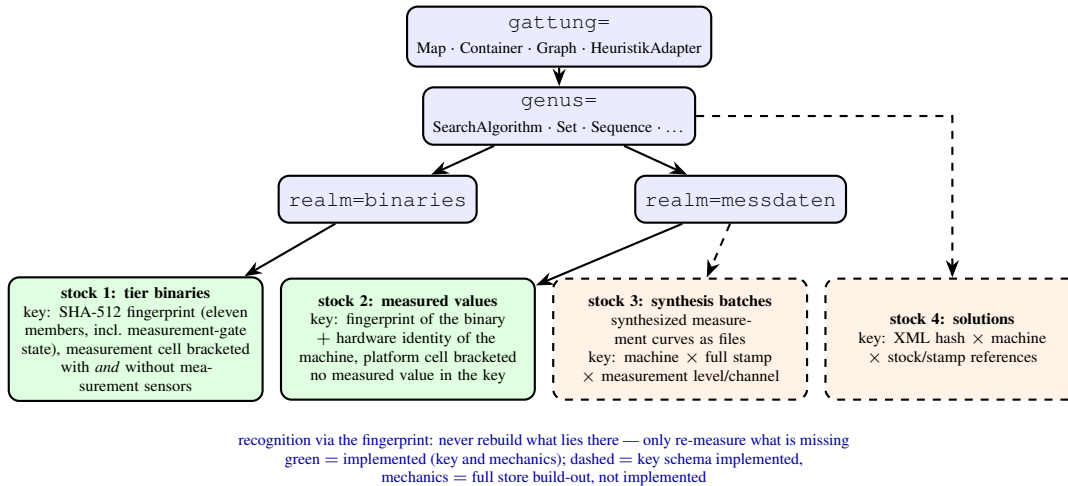


Figure 3.14: The store: the store tree is organized by family, genus, and realm; beneath lie four stocks under stamp keys — stock 1 the tier binaries (SHA-512 fingerprint; the binary with and the one without measurement sensors are two entries, because the measurement-gate state is a fingerprint member), stock 2 the measured values (fingerprint of the binary plus hardware identity of the machine), stock 3 the synthesis batches with the synthesized measurement curves as files, and stock 4 the solutions per user XML. The apparatus recognizes what exists via the fingerprint and re-measures only what is missing.

the organ permutation. Each stage of the chain thus feeds from its kind-specific offer — measurement catalogue → planner, system catalogue → builder, organ catalogue → tier binary —, and the tier binary stays readable in two strictly separated layers: the hardware and environment layer of the system axes and the application-logic layer of the organ axes; as little as one equates an operating system with the application front end, as little do these two layers merge in the binary. The installation is controlled in two ways: as the planner via shell directly or via API command line with a headless server that serves the planner as a cell on Kubernetes or bare metal; an optional, command-line-guided creation mode declares the available XML settings from the detected hardware and offers only what is selectable. Of these ways the planner shell is implemented; the headless service and the creation mode are not implemented.

Building on these measurement patterns and this chain, the measurement system separates four subsystems and tests in three stages that feed the three mandatory measurement series; it reduces the permutation space deliberately. Four formally separated subsystems with distinct responsibilities form the model: the `messung_driver` (the evaluation orchestrator that runs the pipeline per mandatory measurement series A/B/C; it resides in the umbrella project repository), the `CacheEngineBuilder` (an autonomous platform-measurement system that executes the seven-phase pipeline of Section 4.3.2 via its `ExperimentDriver`), the `CacheEngine` (the axis building-block library with the default implementations per axis), and the candidate under test `PRT-ART` (which supplies some axis-organs itself and consumes cache-engine services for the rest); cache engine, candidate under test, and this thesis reside in source repositories of their own — four repositories in total —, and the planner role of the chain is realized as a binary of its own (`comdare-experiment-planner`), the builder role as the measurement driver (`comdare-messung-driver`); the self-stamp of the measurement driver (`-version`) outputs the lines of both roles — the planner line and the line of the CEB contract version — from one driver binary (Chapter 4). The `ExecutionEngine` is thereby not a fifth subsystem but the shared

measurable root (the `IExecutionEngine` interface) through which these four are connected: beneath it, the candidate under test and every engine register as `IExecutionEngine`, and the builder enumerates the permutation space per registered engine. Defining is the *bidirectional* cache-engine $\leftrightarrow$ candidate relationship: the cache engine instantiates the candidate under test per permutation configuration, and the candidate under test calls cache-engine services (telemetry, prefetch, heuristics) at runtime during its own lookups. This symmetry founds the *multi-candidate* capability, with which measurement series A compares the candidate under test against many SOTA adapters in a single pipeline (Figure 3.15).

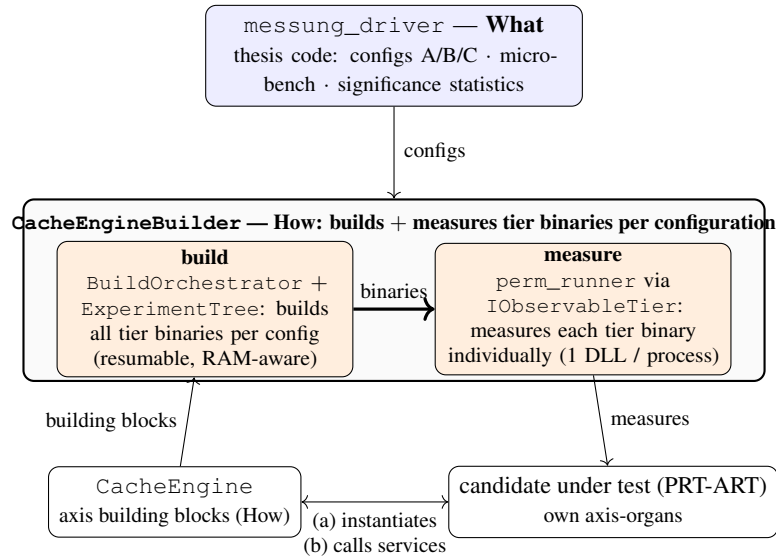


Figure 3.15: The M-model focused on the build and measurement phases. The outer `messung_driver` loop (*What*, thesis code) feeds the configurations of the three measurement series A/B/C; the `CacheEngineBuilder` (*How*, library) splits into the two genuinely interesting phases: **build** (`BuildOrchestrator/ExperimentTree` builds a standalone tier binary per configuration, resumable and RAM-aware) and **measure** (`perm_runner` drives and measures each tier binary individually through the observer interface `IObservableTier`). The `CacheEngine` supplies the default axis building blocks, the candidate under test PRT-ART its own organs; their bidirectional relationship ((a) instantiates, (b) calls services) founds the multi-candidate capability. The `ExecutionEngine` remains the shared measurable root.

The builder orchestrates a *three-stage evaluation*: **Stage 1** (cache-engine only) uses the default variant per axis, **Stage 2** (candidate only) replaces the overridden axes by the candidate variants and otherwise keeps the defaults, and **Stage 3** (full join, non-redundant) combines default and all candidate variants; in the code these are the merge strategies of the merge plan — Stage 1 is `Verbund1_CeOnly`, Stage 2 splits into replacing (`Verbund2_Replace`) and hybrid mixing (`Verbund2_Hybrid`), Stage 3 is `Verbund3_Union` —, which summarize the three-stage view. Each stage can produce up to thousands of non-redundant permutation binaries that are loaded at runtime across the C-stable module boundary as C++23 modules; what is materialized is always a sparse, capped subset, the full product space is only counted (Figure 3.16).

These stages do *not* map one-to-one onto the three mandatory measurement series but feed them: the default binaries of Stage 1 (including the reconstructed SOTA profiles) and the candidate binaries of Stage 2 *jointly* form the comparison series A; the non-redundant full join of Stage 3 is the basis of the

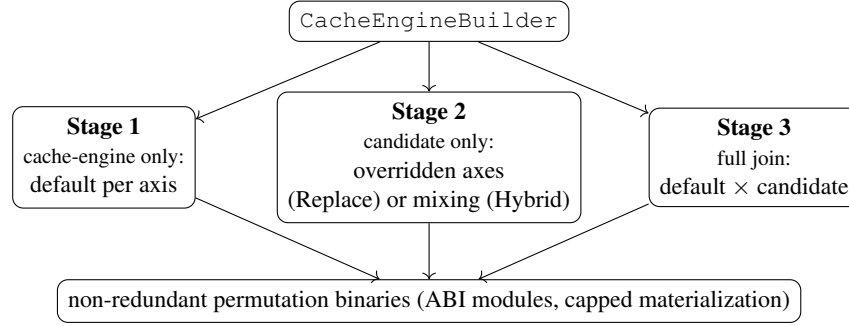


Figure 3.16: The builder’s three-stage evaluation: each stage produces non-redundant permutation binaries; their mapping onto the measurement series A/B/C is shown by Table 3.8.

variation series B; and the merge/regression series C (old vs. new) is bound to no single stage but runs build-spanning as an old-versus-new comparison — over successive states of the same configuration as well as over concrete merge points (Table 3.8).

Table 3.8: Mapping of the three-stage evaluation onto the mandatory measurement series

Evaluation stage	Content	feeds series
Stage 1 (cache-engine only)	default building blocks + SOTA profiles	A (baseline also for C)
Stage 2 (candidate only)	overridden axes = candidate variants	A
Stage 3 (full join)	default $\times$ candidate, non-redundant	B
build-spanning	old vs. new (regression and merge points)	C

The three evaluation stages map not one-to-one but in this assignment onto the three mandatory measurement series A/B/C; series C is build-/version-spanning (not bound to a single evaluation stage).

The three measurement series — all executed by the `messung_driver` via the `ExperimentDriver` library, configured either as three fixed series or via an external `messreihen.xml` — are thus: *series A* (PRT-ART vs. state of the art) with the modes `A_defined` (fast verification against the 8 Rank-1 full profiles) and `A_full` (all 33 profiles, 30 full profiles and 3 abstract; the standard, since only the full run delivers the bias-reduced measurement of all load profiles against all methods, Section 3.1); *series B* (systematic variation of the axis decisions under otherwise equal conditions, to isolate the contribution of each axis); and *series C* (merge/regression old vs. new: concrete merge points between PRT-ART building blocks and cache-engine permutations — say the PRT-ART OLC with `tcmalloc` and ART page, the 4+1+2 pool with the HOT page, the path prefetch with the B²-tree page, or the MultiLevel layout with the Wormhole page —, run build-spanning as an old-versus-new comparison). Each series is collected in three granularities that correspond to the three measurement levels (Section 3.4). *Micro-benchmarking* (level micro) measures each axis over its *axis interface* — the set of functions that *all* algorithms of this axis must supply (every allocator provides memory, whichever one it is): the axis-specific main properties, measured internally. The axis is thereby *not* an interface function of the search algorithm; its family interface functions *use* the axis interfaces — just as axis algorithms may use foreign axis interfaces, since containers, say, are needed everywhere. *Macro-benchmarking* (level macro) measures the individual `std::map` interface operations (insertion, point and range lookup, update, deletion) against example loads, and *overall benchmarking* (level wallclock, in the `CacheEngineBuilder` over the load frameworks) runs the YCSB load profiles over every recombination — w/ma/mi, in this order. As a

counter-check to the internal micro measurement, the apparatus additionally measures three *wallclock levels*: the wallclock of the axis interfaces under each algorithm, the wallclock of the family interface functions, and the wallclock of a test load from the load frameworks. The architectural obligation of this cut is that all axis algorithms use exclusively axis interfaces instead of generic operating-system calls — compiled in inline by metaprogramming, without `std::variant` —, so that the micro measurement of an axis actually measures the axis. Precisely this methodology is the answer to the separability problem: by measuring each constituent first in isolation over its axis interface before it appears in the operation and overall context, the contributions become separately measurable instead of remaining blended in the overall throughput.

The system reduces the space thus spanned deliberately. The golden reference catalogue — per organ axis two enabled representatives, the persistence axis pinned to its in-memory representative — counts $2^{17} = 131072$ binary identities per system permutation (per combination of the build control quantities, Section 3.4.1), each with its full eighteen-segment path (Section 3.4.4), and serves as the regression detector of the whole system with a fixed CRC-64 checksum over all identities; the cardinality of the real permutation space over all axis catalogues including their sub-axes, by contrast, is computed dynamically by the planner from its inputs (subcommand `simulate`) — it lies orders of magnitude above and is never a fixed count. Three mechanisms dampen the explosion: the `<expected_workload>` profile filter routes only compatible workloads; the `Defined` mode runs only the declared tuples, from which the measurement series of the manuscript are fed; and the `Full` or `Full-Sampled` mode runs the full space or a deterministic, seed-stable $1:n$ subset of it (default $n = 1000$). The default workload is the full permutation over all available workloads; the routing of this default is envisioned but not implemented (Section 5.1.3). Full and sampled runs run on the registered machine signatures — `prod1` (AMD Zen 5), `prod2` (Intel Core i9-12900K, Alder Lake), and an ODROID board with Intel Gracemont cores — observing the axis compatibility reported by the platform probe; membership of the ZIH cluster of TU Dresden is declared as a sub-axis of the target ISA, a run there is not measured (n/a).

3.5 PRT-ART as Demonstrator

PRT-ART is the *candidate under test* on which the load-bearing capacity of the system shows: an abstract living being that supplies some axis-organs itself and obtains the rest via the merge strategy of the merge plan from the default toolkit of the cache engine (Section 3.4.6) — as monomorphic CRTP concept axes, without runtime fallback; the hybrid `PrtArtSearchEngine<Ts...>` API remains unaffected by this, and the runtime binding occurs via the execution-engine adapter, not via a search-engine layer of its own. The candidate repo is loaded via the plugin controller of the cache engine (`COMDARE_CE_PRUEFLINGE`): its entry snippet must expressly give its *assurance* via `comdare_pruefling_deklarieren` — with the single registered capability `pruefling_slots_v1`, the candidate slots —, because presence may only refute, never permit: the deposited evidence headers can only break the assurance, never open a gate, and a requested but non-declaring candidate under test breaks the configuration with an error message instead of being skipped silently; a candidate-own adapter path is deactivated and not part of this assurance. What PRT-ART contributes lies on three levels. First, the *candidate slots* in its own repository: the build axis page type, the prefetch, the value handle (`ChainRef`), and the telemetry tooling with the hypothesis metrics H1/H2/H3 (in the triad a measurement-tooling

sub-axis, Section 3.4.1); the assessment of the hypothesis metrics is contained in Chapters 5 and 6. Second, the reference composition of the cache engine, `PrtArtComposition`, which differs from the ART composition in exactly one axis — the path compression as redirect organ, the *R* in PRT-ART; the optimistic lock coupling is inherited from ART, not a PRT-ART contribution. Third, the codegen-capable candidate composition, which takes sixteen of eighteen axes from the ART composition and replaces prefetch and value handle by the candidate slots. Its further own building blocks, the 4+1+2 pool allocator (free-list sub-axis) and the `MultiLevel` layout, reside in the candidate repo and run in the measurement path via the cache-engine default (attachment envisioned as a subsequent implementation step, but not implemented); for the remaining axes it reuses existing SOTA building blocks via the permutation configuration. Thus the candidate under test demonstrates that the measurement system holds: a novel living being arises from a few own organs plus the default toolkit (Figure 3.17) and is measurable over the same probe dock as every reconstructed SOTA method.

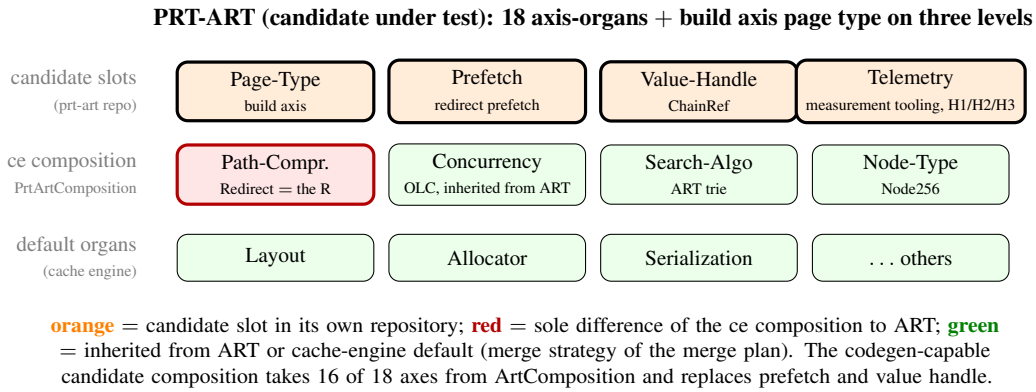


Figure 3.17: PRT-ART as a demonstrator in the M-model on three levels: the four candidate slots in its own repository (page type as build axis, prefetch, value handle, telemetry as measurement-tooling sub-axis, Section 3.4.1); the reference composition of the cache engine, which differs from ART in exactly one axis — the path compression as redirect organ —, while the optimistic lock coupling is inherited from ART; and the codegen-capable candidate composition, which takes sixteen of eighteen axes from the ART composition and replaces prefetch and value handle. The further own building blocks (4+1+2 pool allocator, `MultiLevel` layout) run in the measurement path via the cache-engine default (attachment: subsequent implementation step, not implemented). Thus a complete, measurable living being arises from a few own plus many default organs.

3.6 Sharpening through Heuristics

The measurement series deliver more than rankings: from them heuristics can be won that sharpen the measurement system step by step from a test bench into a self-optimizing delivery system. This section develops this sharpening as a concept in three stages — the heuristic loop over learned load profiles, the four operating modes of the builder up to the hybrid mode as the target picture of this thesis, and the measurement-curve type system together with the filter chain and measurement-error detection that feed these modes with data.

3.6.1 Heuristic Loop and Learned Load Profiles

From the measurement series one can derive the best axis composition per load and data-type profile and store it as a reusable profile.

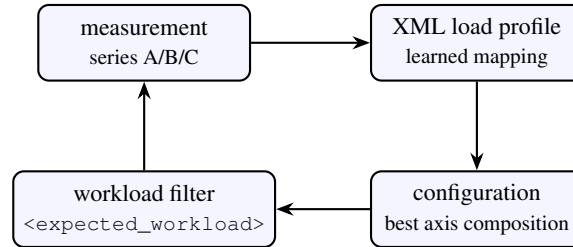


Figure 3.18: The heuristic loop: from the measurement series the best axis composition per load profile is learned, stored as an XML load profile, and fed back via the workload filter into the next measurement.

The full loop is an outlook mechanism; realized of it are the automatic selection and the binary delivery (Section 1.4). The selector reads the measured-value workbook — the `xlsx` workbook is the output and the standard of the apparatus; the CSV form is not a single child document but an ordered tree of CSV files generated from the workbook with the same columns (one overall folder per workbook, beneath it one folder per sheet, therein the CSV files of the individual sub-axis sweeps) —, ranks per interface function and metric over the valid two-phase rows (median, nearest rank), determines with competing target quantities the Pareto front of the non-dominated candidates instead of a single winner, resolves the `binary_id` to the binary in the store, and delivers binary, version sidecar, and manifest as a standalone artefact. The heuristic curves arise as natural cubic splines over the sampling points per axis and x -dimension; a decision lambda tree reads the break-even thresholds from the spline intersections and chooses at a query coordinate backwards the leading binary — synthetic sampling points it reports as scaffolding, without valid curves it makes no decision. Beyond mere ranking, a *heuristic extraction* is envisioned as a follow-up phase after **Persist** — the last phase of the seven-phase pipeline (Section 4.3.2) — that automatically derives from the collected measurements which axis configuration is best per workload and dataset. The intended mechanism is an ML classification in three steps: first, features are extracted from the measured-value workbook per permutation — the load-profile and dataset features (workload type, key distribution, write/read share, alphabet and prefix characteristics, as well as key and value data type) and the axis assignment —, with the best performance measured per load pattern (say throughput or tail latency) as the target quantity; over these an *ML classifier* learns the mapping from load-profile/axis features to the best configuration; and it delivers per workload/dataset class the recommended axis composition without re-running the entire permutation space. The result is persisted as a reusable *XML load profile* — symmetric to the measurement-series XML configurations — and can be fed back into the `<expected_workload>` filter without a full re-measurement; this closes the loop between measurement, learned heuristic, and configuration-guiding selection (Figure 3.18). This *measure* $\rightarrow$ *profile* $\rightarrow$ *config* $\rightarrow$ *filter* loop is the classical, established form of *empirical, measurement-driven autotuning*: it stands in the line of autotuning frameworks such as OpenTuner [AKV⁺14] and the measure-in-the-loop autotuners FFTW [FJ98] and ATLAS [WPD01]; the load-profile-driven configuration choice with workload filter finds its database counterpart in OtterTune [APGZ17] and in the Database Tuning Advisor with workload compression [ACK⁺04], the online runtime-near variant in Active Harmony [TCH02];

the term “software-in-the-loop” itself originates from the X-in-the-loop validation taxonomy [TMS⁺16]. The ML classification is not implemented and remains reserved for future work (Chapter 6); the loop itself, however, is only the first stage of a larger operating target picture, which the following subsection develops.

3.6.2 The Four Operating Modes of the Builder

The heuristic loop is the core of an operating regime of *four* operating modes of the builder interface that build on one another. From the run chain build → measure → compare → release (Section 3.3.2, Figure 3.10) these operating target pictures are to be distinguished, in which the apparatus is operated; the assignment is unambiguous: the *measurement mode* comprises build and measure, the *evaluation mode* is compare, the *working mode* is the release of a single binary, and the *hybrid mode* is the release of a hybrid recombination, whose chain repeats on the hybrid level. The first mode is the operation of the apparatus; modes two through four are the declared goal of this thesis and are recorded here as a concept. The hybrid mode is thereby an extended, optional construct: the core of the thesis is the measuring through and generating of the tier binaries together with break-even analysis before every application of the hybrid, and the thesis does not go in the chain beyond the comparison stage of the hybrid level (*hybrid-compare*) after the delivery stage of the single binary (*single-release*). To be distinguished from these four operating modes is the *paper isolation mode* asked for in sub-question RQ2.c (Section 1.3): an operating mode that offers the design constituents of exactly one publication as an *axis set* — the closed axis set of this one reference design, with measurement and system axes still free-standing — in isolation, in order to measure the publication faithfully; it is not implemented and remains outlook (Section 6.3).

In the *measurement mode* — the operation that Section 3.4 describes — the builder runs the measurement series over the admitted permutations. After completion of the measurement the framework knows the properties of all admitted axis algorithms against the recombination of all admitted load-profile frameworks and workloads. The *evaluation mode* condenses these measurements: from them emerge minimal strands of the permutations of all properties against one another for certain *workload clusters* — for groups of similar loads, the small set of axis compositions that dominates them remains. For this, workloads are clustered according to sequences, as similar as possible, of equal access patterns on the family and genus interfaces of the tier binaries: every load framework runs through its planned strategy once dry, and sequence, frequency, and requirements of its interface calls are noted — what a run of the framework statically gives of itself. From this arises per profile a directed transition graph of the interface accesses with absolute and relative successor probabilities (the mapping of an interface onto itself is admitted as an edge); its edge matrix is evaluated with the EM algorithm [DLR77], and the distances of the centres of the probability point clouds from one another yield per profile a *point-cloud word* of variable length (shorter when interfaces are not called — null-filtered). The lexicographic sorting of these words is the measurement order: adjacent words are similar loads and are — no matter from which framework they stem — run one after another. This procedure is not implemented in the code; it is at the same time the fourth evaluation component of the hybrid tier (Section 3.6.3).

The *working mode* is the production goal: the builder interface keeps all tier binaries relevant for the currently stochastically frequent workload load and interface function preloaded in main memory and internally switches the tier binary with its specifically optimal axis permutations at the ABI-stable interface — the same binary-switching mechanism that the measurement mode uses, only load-driven instead of

measurement-driven. In this, the compile-time principle remains preserved at the correct level: there is no runtime switch *inside* the tier — the binaries themselves remain compile-time-permuted; switching happens *between* preloaded binaries at the deliberate ABI boundary. The probe dock of the family (Section 3.3.2) is called in this mode the *working dock*: the same interface through which the measurement mode tests now delivers the productive answers. The dock layer of this switch is implemented — the dock contracts, the probe dock, the dock factory as the sole construction site, the dock field with attachment and detachment, the parser of the hybrid section of the XML, and the binary proxy, which makes a hybrid binary outwardly an ordinary tier binary —; router, eviction, and the `.so` export of the switch are not implemented. The hybrid always compiles with the help of the CacheEngineBuilder, never by itself.

On switching from one tier binary to another, every participating tier binary must contain a further *state surface* compiled in, which reads and re-sets the state in the hybrid mode — sparse: every binary keeps its state and pulls in only the versioned changed blocks, a checkpoint procedure after the *Memento* pattern [GHJV94], which bypasses the interfaces and tracks the memory block-wise only for changed regions. Main memory multiplies thereby, because all participating binaries hold the total state, of which always one is in front and current. The state surface is a compiled-in, metaprogrammed sidecar library that occurs only in the hybrid pattern — in single operation a tier binary does not contain it; therefore two release artefacts are stored, a release for single operation without sidecar and a release for the hybrid operation with sidecar (Section 3.4.5). The hybrid binary organizes the states in the protocol of the switch and enforces the checkpointing for all metaprogrammatically attached tier binaries: the computed switch per hybrid request generates a memory reconciliation between the tier binaries and then passes the request through to the chosen tier binary. State surface and sidecar are not implemented.

The *hybrid mode*, finally — the goal of this thesis — combines working mode and measurement. Its centrepiece is that the heuristic adapter is a *family* of its own with its own genus — the fourth family `HeuristikAdapter` with the reroute genus `FunctionInterfaceReroute` (Section 3.3.2): it adopts by metaprogramming the family and genus interfaces of its tier binaries, is itself compiled into a tier binary, and docks at the probe dock of the builder like every single tier; it passes commands via the *Command* pattern [GHJV94] on to the single tier binaries statically assigned to it. To the outside it is indistinguishable from a single binary, because its binary exports the same mandatory symbols as every other and reports as genus the inherited target genus — classification (the kind of the reroute) and pass-through lie on separate levels. The recombination of the heuristic tier binaries to be used is thereby itself permutable against the existing single tier binaries; the heuristic adapter can therefore use the measurement function curves for estimates and compile an optimal multidimensional recombination that is unambiguously usable as a search-algorithm hull through its ABI-stable family interface — to the outside a *virtual whole tier binary*. At the same time the hybrid mode measures along: over the optimized recombination the combined wall-clock time as well as macro- and micro-benchmarks are logged per measurement command across all tier binaries and operations.

Thereby the hybrid mode delivers the *heuristic counter-proof*: after entry into the working mode the heuristic tier binary is measured anew — its wall-clock behaviour in comparison against the best single binaries and against the reconstructed paper algorithms decides whether the heuristic recombination reaches the performance predicted by the curves. The curves that the counter-proof checks are the spline curves per axis and x -dimension with their break-even intersections (Section 3.6.1); in the stage compare it is computed from the measurement store which tier binary or hybrid recombination must, according to

the measurement, be the best, and in the stage release the maximally optimized compiled binary without compiled-in measurement sensors is measured through once more — whether its wall clock without measurement insight even undercuts the previously measured total time. The comparison against the reconstructed paper algorithms presupposes per publication a wall-clock reference, which is missing as a data source and is to be determined per method from the respective publication; the paper comparison takes precedence over the Pareto selection. The goal of the thesis is thus formulated twice over: to determine the best recombination or the best single binary against all axis properties — especially cache-line awareness — and in the end to provide both in production-ready form, the best single binary *and* the heuristic tier binary consisting of the best mixed cache-aware binaries for the measured loads. All four modes shall thereby be documented automatically via the evaluation chain of the definition layer (Section 3.3.4); the measured-value tables and diagrams of this thesis itself are the product of the compare stage of this chain, and the assessment of the counter-proof is defined by Chapter 5.

3.6.3 Measurement-Curve Type System, Filter Chain, and Measurement-Error Detection

For the evaluation and the hybrid mode to hold, the evaluation needs more than flat result tables. As a target picture this thesis formulates a *measurement-curve type system* for this; implemented of it is a first approach — a curve fit over one axis and one x -dimension as a natural cubic spline (Section 3.6.1) —, while the type-system key framework $\times$ workload $\times$ operation $\times$ size is not implemented. Every tier-binary total permutation receives per measurement axis a wall-clock measurement curve over the working set — say the lookup latency as a function of the fill size —, separated by load-profile framework, workload type, working-set size, operation type, and observer type and per measurement level wallclock, macro, and micro (Section 3.4); every axis is swept separately in the experiment tree compile-statically as well as runtime-dynamically, so that every axis receives a complete overview of its properties and individual measurements. Taken together over all permutations there thus arises per tier binary a multidimensional measurement database, from which the heuristic curves of the loop (Section 3.6.1) are read; the synthesized curves are kept as files in the store (Section 3.4.5). The cardinality of every dimension of this type system is to be organized strategically into classes and hierarchies: design-fixed constants (say the eighteen organ axes), registered, dynamically growing quantities (the measurement categories, Section 3.4.1), profile-bound products (what an experiment profile admits as assignments), and runtime-free iteration spaces (the dynamic sampling points per static binary) — thus the system axes remain able to support the evaluation across all levels.

The compare stage is composed of five evaluation components that are gradations of a chain, not alternatives: first, the ranking per metric with Pareto front and delivery (implemented, Section 3.6.1); second, the spline curves with the break-even decision (implemented as a first approach); third, the filter chain over the permutation families (mechanism implemented, evaluation links not implemented, next paragraph); fourth, the live ranking of the preloaded tier binaries in the hybrid tier from the interface accesses recorded at runtime and their transition probabilities — the same clustering procedure as for the measurement order (Section 3.6.2), which holds the known interface suitabilities of a tier binary against the measured requirements and passes the request through to the best binary (not implemented); and fifth, the measurement-error interpolation over the measurement constellations (not implemented, paragraph after next).

From the evaluations there emerges a *filter chain*: every chain link checks, based on the curves so far, whether a permutation family is pursued further, and otherwise passes the decision on to the next link — the design pattern *Chain of Responsibility* [GHJV94]. The chain is located in the CacheEngineBuilder, because it controls the *generation* of the next build round and closes the feedback edge from the selection to the provisioning: evaluation, filter, then the decision which permutations are built next and which are discarded. Since in compare build and measure have already been run through, it at the same time filters from the measured values for the release which binary is statistically the fastest for a purpose according to the properties demanded in the XML — the default is the execution time, further filters are not the subject of this thesis. Implemented are the mechanism (with the verdicts Pass, Reject, and pass-on: a candidate survives exactly when no link rejects it) and a core link for the resumption of interrupted runs; the evaluation links — the paper comparison, which lacks the reference source, and the Pareto dominance, which lacks the type-system key — follow as further links, with the precedence of the paper comparison over the Pareto selection.

Finally, the hybrid mode needs a control of the measurement error that the measurement instances themselves introduce — observers cost time. The three measurement facilities — the wall clock in the CacheEngineBuilder, the genus interface time of the tier binary (macro), and the axis interface time (micro) — can be built as a three-phase contract construction between CEB and tier binary in all arrangements: with three facilities $3! = 6$ probe-dock constellations, which only need to be measured at the tier binary, with an interposed hybrid tier binary $4! = 24$ constellations of the chain CEB–hybrid–tier; over several hybrid stages connected in series (hybrid meta-meta stages, compile-statically extensible via the XML) the number is dynamically $n \times m$ and is never presented statically. From the differences of all measured values with built-in measurement sensors against the respectively most accurate remaining total time, related to the number of the finest interface accesses, a mathematical procedure interpolates the measurement error over the constellation matrix and computes it out of the published results — the fifth evaluation component. The three previously named build variants of the same experiment — first, observers in all tier binaries *and* in the heuristic hybrid tier, second, observers only in the heuristic tier binary, third, observers in none, only the wall clock — are three of these constellations. Complementarily follows a *fourth step* of the final heuristic assembly: the tier binaries subordinate to the heuristic tier (not the finally optimized main binary) are rebuilt without micro- and macro-benchmarks and without observers and compared purely on the wall clock against all known tier-binary permutations and against the known paper algorithms — with the paper comparison taking precedence. The validity criterion of every constellation is that the measurement with measurement sensors is slower than the same without. What is laid is the foundation: the measurement gates per compiled artefact as a fingerprint member, whose coverage CEB off, tier on is proven by a test, and the declared non-observation per measured value (Section 3.4.1); the interpolation and the constellation matrix are not implemented.

This keeps clearly apart what is implemented and what is a goal. Implemented are the measurement apparatus with its merge stages, the evaluation chain up to tables and diagrams, the curve fit as a first approach together with break-even tree, the Pareto selection with delivery, the filter-chain mechanism with resumption link, the dock and classification layer of the hybrid tier together with XML parser and binary proxy, and the measurement gates per compiled artefact. The measurement runs of the complete measurement series are not available. Not implemented are the evaluation links of the filter chain, router, eviction, and `.so` export of the hybrid switch, the state synchronization with sidecar, the workload

clustering, the measurement-error interpolation, the type-system key of the measurement curves, the collection per axis-interface parameter set, and the attachment of the P-/E-core measurement to the run profile. The assessment of the heuristic counter-proof is taken up by Chapter 5; the placement of the outlook beyond this thesis is given by Chapter 6. First, Chapter 4 deepens the concrete implementation of the building blocks introduced here.

4 Implementation and Concrete Algorithm Components

This chapter descends into detail: first the repository architecture with the carrier-stage chain (Section 3.3.2) and the four work modes of a permutation, then each key point of PRT-ART individually and visually, next the axis implementation together with the *one* architecture in code, the code shape of the build chain, the levels E4–E0 in code and the contract surfaces at the carrier binaries, after that the quality assurance via contract tests per layer including the two-gate protocol and the code shape of the realm roots — system and measurement axes including the hardware detection — and finally the register of design patterns, the hybrid stage and the store. At its core, the *one* architecture is a self-optimizing compiler-compiler structure: the planner generates the build orders from the user XML, the builder generates the compiled artefacts from them, and the evaluation of the measured values feeds the next planning (Section 3.4.6). The apparatus thus stands in the line of the empirical autotuning systems that generate candidate implementations, measure them and make the selection from the measured values [WPD01, FJ98, AKV⁺14, BDG⁺18]; the difference is that here it is not the parameters of a fixed implementation that are permuted but the organ assignment of a living being, and a separate binary is generated per permutation.

4.1 Four-Repository Architecture

The implementation is distributed across four Git repositories with clearly separated responsibilities:

- `comdare-cache-engine` — **how** measurement is done: the building-block library and the builder (XML config parser, `generate_module_from_profile` code generation, permutation loop, module loader, the seven-phase `ExperimentDriver` with the phases `phase1_enumerate` to `phase7_export` and the experiment planner `ExperimentPlanDirector`, Section 4.3.2); the ABI headers under `abi/` — the living ABI core `module_abi_v1` as well as the test cluster, isolated from the build, of `baustein_variants` and `resolve_baustein` (Section 4.3) —; the 33 SOTA profiles under `algorithm_profiles/sota` as executable XML — 30 full profiles and three abstract profiles (Section 3.2) — and the test suites. The library `search_engine` is a declared, content-empty skeleton (seventeen files, eight of them placeholders and nine build descriptions without source code), and `execution_engine` contains three content files (result aggregator and experiment demonstration); the living root of the execution engine lies under `libs/cache_engine/execution_engine` (Section 4.3.1). The source tree follows a Pitchfork layout (`apps/` for executables, `libs/` for libraries, `libs/common` for cross-cutting code); added to these are `libs/traeger` with the four carrier stages (below), `libs/cache_engine/heuristik` with six headers (axis spline, break-even, measurement-curve loader, offline clustering of the workloads, feature vector, optimization catalogue) and `libs/test_infra` (benchmark suite, workload generator, test data).

- `comdare-prt-art` — the **candidate under test**: the hybrid `PrtArtSearchEngine` with the $1/2/N$ parameter rule (one parameter demands the function interfaces of a vector, two those of a map, more than two those of a map onto a tuple), its ABI adapters and its building blocks (4+1+2 allocator pools — seven pools: four page pools with the roles trie hull, dense pages, MultiLevel and decision span, one rest pool and two value pools for static and dynamic values —, OLC with reserved blocks, MultiLevel layout, path-oriented prefetch, DensityTracker, hypothesis metrics H1/H2/H3) as well as the candidate-side comparison re-implementations of the rank-2/3 works (`legacy_reimpl`, fourteen directories P11–P27 without P15, P20 and P25), which are compared against the actual adapters of the cache engine in the SOTA admission test — rank 1 (centrality rank, Section 1.3) comprises the eight independent search designs — all with public source code and original adapter — as well as the works P08 and P09 that enter as organs; of the rank-2/3 works, P20, P25, P29 and P30 likewise possess original adapters via `ext/`, the fourteen works in `legacy_reimpl` are re-implemented from the publication without public source code (Section 3.2). Pool, prefetch and layout components are descriptor skeletons without a memory backend of their own; which candidate building blocks reach the measurement apparatus is decided by the slot assignment (Section 4.2).
- `probst-Diplomarbeit-cache-engine` — **what** is tested by the user: the experiment configurations (XML together with the XSD schema), the measurement driver `messung_driver` (binary `comdare-messung-driver`, the CEB role) and the evaluation toolchain of nine numbered programs from `01_sample_data_generator` to `09_tex_formatter` (Figure 4.8).
- the **manuscript repository** — the versioned experiment execution directly in the document: the generator output of the toolchain (table and diagram files per measurement series) is committed under `anhang/<language>/tabellen/` and included by the manuscript, the repository has its own CI with the jobs `lint:secrets`, `lint:latex` and `thesis:pdf`, and the project repository triggers the manuscript build after a measurement run (`trigger:thesis`). A measured value in the document is thus a versioned artefact state with provenance, not a copied number — that is the reason why the manuscript is a Git repository of its own with its own pipeline.

The fourfold order of the code is not the repository boundary but the *carrier-stage chain* planner → CEB → tier with the hybrid optionally inserted between the CEB probe dock and the tier (Section 3.3.2): the stamp contract knows exactly four carriers, and the source tree maps them under `libs/traeger` as four subprojects in stage form — `comdare_planner`, `comdare_ceb`, `comdare_tier_emission` and `comdare_hybrid` —, where in the build order stage $N + 1$ depends only on stage N (no hybrid without a tier), never the reverse, and each stage closes onto its predecessor stage with a strict configuration check. Implementation status of this order: the subprojects form a scaffold (per stage a build description and a description file, planner and CEB with an include directory of their own), and the actual implementation lies as a monolith under `libs/cache_engine`; its partition along the stages is planned but not implemented. The candidate under test is not a stage but the transversal supplier of organ building blocks; builder and CacheEngine share the engine repository — the builder as an executable (`apps/`) on top of its builder library (`libs/cache_engine/builder`), the CacheEngine as a library (`libs/cache_engine`). The stage chain is at the same time the layer at whose binaries the contracts are partitioned into surfaces (Section 4.3.2); Figure 4.1 shows it with its binaries, legend forms

and surfaces. Across this, the terminology separates two engine terms that must not be short-circuited. The

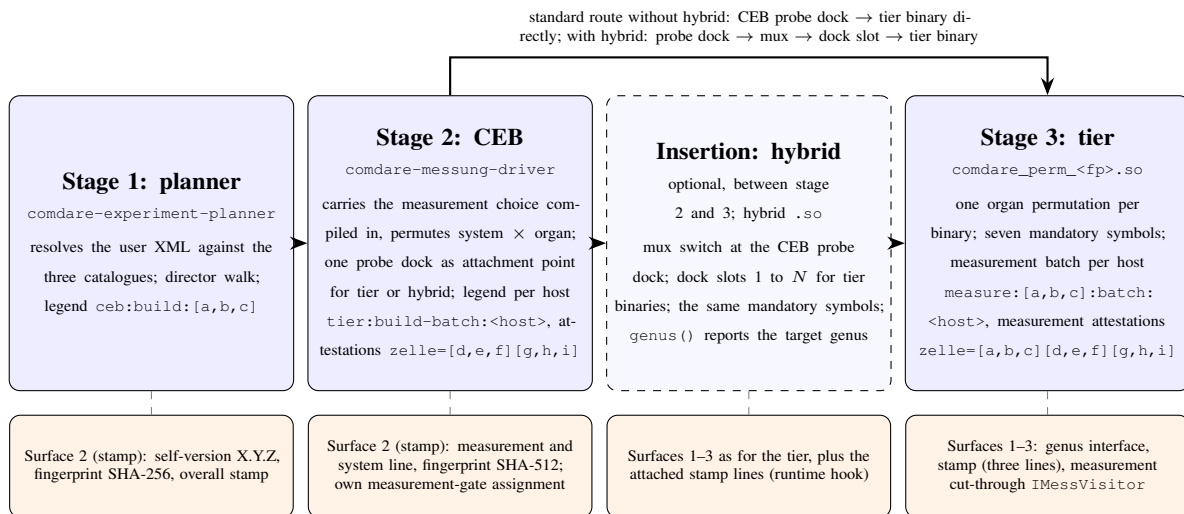


Figure 4.1: The carrier-stage chain in code: four carriers, of which the hybrid is a stage optionally inserted between the CEB probe dock and the tier binary (stamp chain in fixed order planner → CEB → [hybrid] → tier), per stage the binary, its legend form in the emitted pipeline and the contract surfaces expressed at it. In the stamp chain, stage $N + 1$ depends only on stage N ; the standard route leads from the CEB probe dock directly to the tier binary, the hybrid is a mux switch at the probe dock with dock slots for tier binaries (build dependency: no hybrid without a tier); only the measurement attestations contain all three axis brackets. The surfaces partition the contracts (Section 4.3.2); the subprojects under `libs/traeger` are a scaffold, the implementation lies as a monolith under `libs/cache_engine`.

`CacheEngine` (`libs/cache_engine`) is the *library of axis building blocks*: per axis it provides the interchangeable design constituents as concept/C RTP components¹ and contains *no* measurement logic. The `ExecutionEngine`, by contrast, is the *execution and measurement infrastructure* lying across all axes: its abstract measurable root is `IExecutionEngine` (Section 3.4) with the three engine kinds Anatomy, Virus and Hybrid; the concrete runtime adapter `SearchAlgorithmAbiAdapter<A>` provides, via the `SearchAlgorithmAnatomy`, the OS primitives (cache-line-aligned allocation, NUMA-conformant read pinning, coherence-aware writing); the measurement and observer couplings of the adapter are switched via `COMDARE_MEASUREMENT_ON`, and the measurement aggregator is kept host-side by the `ExperimentDriver`; the CMake option `COMDARE_EXPERIMENT_MODE` marks experiment compiles and encodes at compile time the invariant that an experiment build is always a measurement build — it does not itself switch any hooks, and an experiment build without measurement mode breaks the configuration. In short: the `CacheEngine` supplies the *building blocks*, the `ExecutionEngine` the *tools*; this separation keeps the production path measurement-free.

What happens with a permutation is a registry, not a label: the four *work modes* build, measure, compare and release are carried as a sub-axis of the measurement realm (Table 4.1; enum index, execution order and registry line are the same order, Section 3.3.2). build builds the binaries without measuring and may run in parallel; measure is the deterministic single-thread measurement run and the only mode with measurement switched on; compare reads the measurement store and compares

¹C RTP: the *Curiously Recurring Template Pattern* [Cop95], the static polymorphism in which a class derives from a class template to which it passes itself as the template argument (Section 2.4); virtual calls are thereby replaced by statically resolved ones.

views already measured — the stage *before* the release; `release` produces the reference binary without instrumentation. All four build as CMake Release; a measuring debug regime no longer exists, because numbers collected in parallel are not comparable with single-thread measurements, and the wiring check it performed is realized as the switch `-debug` of the planner shell — a flag, not a state. Implementation status: `compare` is a selectable label with build semantics identical to `release`; its mode-specific execution (replay comparison instead of measuring) is planned but not implemented (Section 3.3.2).

Mode	CMake type	measures	measurement threads	purpose / status
<code>build</code>	Release	no	—	build without measurement, in parallel; the step before every measurement
<code>measure</code>	Release	yes	1	deterministic measurement run, run-to-run stable; fills the measurement store
<code>compare</code>	Release	no	—	reads the measurement store, stage before <code>release</code> ; execution planned, not implemented (label)
<code>release</code>	Release	no	—	reference binary without measurement instrumentation

Table 4.1: The four work modes of a permutation as a registered sub-axis of the measurement realm (`WorkMode`, four entries; a fifth breaks the compilation). No mode measures in parallel; only `measure` switches the measurement on.

4.2 PRT-ART in Detail

Building on the state of the art, this section describes each key point of PRT-ART individually and visually; for each key point, the related technique from which it emerges is named first. Pool allocation with role-pure pools stands in the line of the size-class and thread-heap allocators Hoard, jemalloc and mimalloc [BMBW00, Eva06, LZdM19]; optimistic lock coupling is the synchronization technique of the ART [LABN16]; software prefetch along the tree paths follows the path-bundle heuristic of the prefetching B⁺-trees of Chen, Gibbons and Mowry [CGM01] (the related literature of the building block in the code) and hierarchical prefetching [ZGH⁺25]; cache-line-aware node layouts go back to the ART and to FAST [LKN13, KCS⁺10]; coherence-aware telemetry answers the false-sharing problem of multiprocessor caches [TLH94] in the research line of Kuehn [Kue23]. The DensityTracker and the hypothesis metrics H1/H2/H3 are own contributions without an external template.

The candidate under test contributes six key points of its own, two of them on the telemetry axis, which in the tripartition is split in two — a sweep sub-axis of the measurement tooling in the planner and a compile-time quantity in the builder (Section 3.4.1). On the *allocator axis*, a *4+1+2 pool allocator* — four page pools with the roles trie hull, dense pages, MultiLevel and decision span, one rest pool and two value pools for static and dynamic values — replaces the allocation hard-wired into the algorithm with a cache-line-aligned, low-fragmentation free list. On the *concurrency axis* — itself divided into pattern (8.1) and locking mode (8.2) — PRT-ART contributes *optimistic lock coupling (OLC) with reserved value blocks* (`olc_with_reserved_blocks`), coordinated by a concurrency manager that carries the disciplines OLC, hardware and software transactional memory (HTM, STM), lock-free, read-copy-update (RCU) and hazard pointers [Mic04a]; memory reclamation is here a sub-axis of the allocator (epoch, RCU, hazard-pointer and QSBR strategies selectable per permutation). On the *prefetch axis*, a *path-oriented*

bundle prefetch delivers the hot search paths — the frequently traversed node sequences — ahead of time, on the *layout axis* a *MultiLevel layout*. Particularly cache-critical is the *telemetry axis*: per-node counters cause cache-line ping-pong on multi-core systems and falsify precisely those measurements they collect. Following the Kuehn research line [Kue23], PRT-ART therefore offers cache-coherence-aware strategies — *leaf-only counters* (main variant), *sampling* (every N th leaf access) and *offline recompute* (bottom-up aggregation before the reordering) —, while the classical per-node counter is retained only as a labelled anti-pattern. The candidate under test adds the `DensityTracker` (`density_tracker`) and the hypothesis metrics H1/H2/H3 as telemetry organs of its own. In the engine source tree, the building-block implementations of these three telemetry strategies (TM1–TM3) and the ISA sub-axes (IS1–IS3) lie under the directories of the organ axes, while telemetry regime and target ISA are carried in the registry of the system axes and the permutation-flag structure has an ISA and telemetry bank of its own (Section 4.3.2); this double location is the existing state of the code, not an assignment question of the model.

These key points are *building blocks of the candidate repo*; which of them reach the measurement apparatus is decided by the slot assignment, not by the contribution list (Section 3.5). The candidate under test provides four slot hulls: the page type of the build axis (`PrtArtBPlusPageType`), the prefetch (`PrtArtRedirectPrefetch`, T7), the `ChainRef` value handle (`PrtArtChainRefHandle`, T10) and a telemetry wrapper with the per-node counter as anti-pattern comparison baseline. Its codegen-capable slot composition `PrtArtCompositionDemo` takes over sixteen of the eighteen organs from the ART reference composition of the cache engine and replaces exactly two — prefetch and value handle —; the engine-side reference composition `PrtArtComposition` differs from the ART composition in exactly one axis, the path-compression redirect (T3, the R in PRT-ART), and inherits the optimistic lock coupling from ART. Overridden in the measurement apparatus are thus the path-compression redirect, the prefetch and the value handle (together with the build axis page type); the 4+1+2 pool allocator (T6) and the `MultiLevel layout` (T5) run in the measurement path via the cache-engine default, the reserved OLC value blocks via the OLC discipline inherited from ART, and the telemetry strategies appear as a sub-axis of the measurement tooling, not as a composition slot. The binding of pool allocator and layout to the measurement path is planned but not implemented (update of the candidate integration). Condensed into a gallery, Figure 4.2 shows the six own key points of PRT-ART, each located at its axis. The remaining two of the eight building-block layers from the contribution list (Section 1.4) — the inline/external/`ChainRef` value handles and the signalling-bit serialization — do not appear as gallery points of their own: the former provide the value-handle slot overridden in the apparatus (T10), the latter is catalogued as the `VarLenSerialization` building block of the serialization axis (T9) with signalling bits in Appendix D.

4.3 Axis Implementation and Measurement-System Levels

Each axis is implemented as a generalized interface; alongside lie the levels of the measurement system from code generation to the seven-phase pipeline. The section closes with the level picture E4–E0 in code (Figure 4.9) and the contract surfaces at the carrier binaries; the register of the design patterns used follows at the end of the chapter (Figure 4.12). Each axis is a topic directory that contains a two-layer *concept* [SS12] (a broad topic concept and a narrow axis concept) as well as a *CRTP base class* secured by a `requires` clause. The live path of building-block selection is monomorphic: per axis, a complete type list of all known building blocks (`All*`) is carried, from which a compile-time filter over the

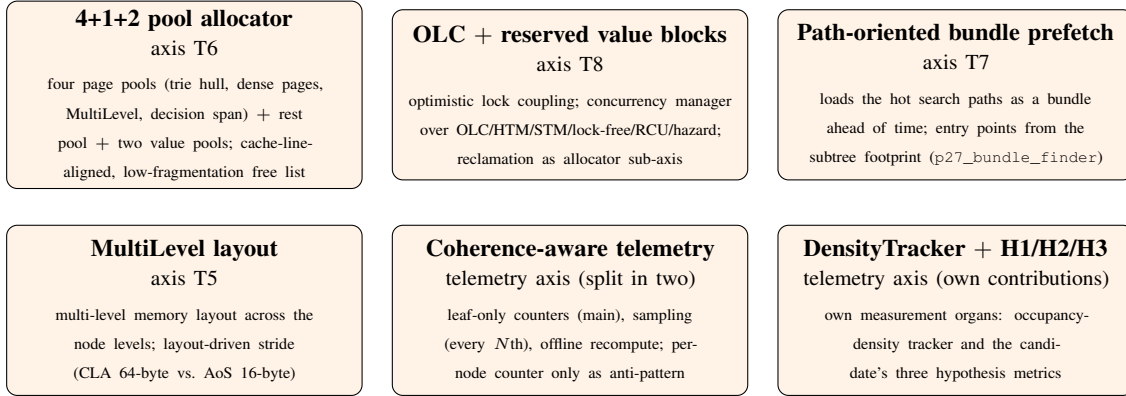


Figure 4.2: Gallery of the six own key points of PRT-ART, each at its axis: a 4+1+2 pool allocator (T6), optimistic lock coupling with reserved value blocks (T8), path-oriented bundle prefetch (T7), a MultiLevel layout (T5) and cache-coherence-aware telemetry together with the DensityTracker and the hypothesis metrics H1/H2/H3 — the telemetry axis is split in two, a sweep sub-axis of the measurement tooling and a compile-time quantity of the builder (Section 3.4.1). The tiles show the *building blocks of the candidate repo*; overridden in the apparatus are the path-compression redirect (T3, engine-side reference composition), the prefetch (T7) and the ChainRef value handle (T10, slot composition of the candidate under test) as well as the build axis page type; the building blocks for T5/T6 run in the measurement path via the cache-engine default, OLC is inherited from ART, and the remaining organ axes the slot composition draws from the ART reference of the cache engine (sixteen of eighteen).

enable flag of each building block (`mp_filter<is_enabled>`, Boost.Mp11 [Dim19]) forms the `Enabled*` list, and a composition assigns exactly one type of this list per slot. The hot path contains neither `virtual` calls nor runtime switches — the zero-cost abstraction from Section 3.3, in the sense of the C++ principle that an unused abstraction costs nothing [Str12]. Every SOTA design and every allocator is integrated via a thin *adapter* that provides the original implementation behind the engine ABI: allocator vendors are attached via per-vendor include shims (`#if COMDARE_AXIS_06_USE_<X>` with forward stubs), where `USE` is the product of the enable flag of the configuration and the detection flag `COMDARE_HAVE_<X>` of the build system — a missing vendor switches the building block off instead of breaking the build —; SOTA building blocks are switched on or off at compile time via `constexpr` enable flags together with `mp_filter` (the `COMDARE_AXIS_*_USE_/ENABLE_` flag family), each with a safe fallback chosen per family: allocator adapters replace the missing vendor in `if constexpr` branches with the portable aligned allocation (`portable_aligned_alloc`), the vendor shim supplies only forward stubs for the never-executed branch, and the axis value `StdMalloc` is a building block of its own, not a makeshift; data-structure adapters of foreign search techniques possess, behind their enable flag, a self-contained re-implementation as body, while the binding to the original code runs via the library build with the original compiler — a stub that throws an exception at runtime does not occur in the organ axes (the types thrown there are contract violations such as `invalid_argument`, `length_error` or `logic_error` and the allocation errors `bad_alloc` and `bad_array_new_length`, no `runtime_error`) —, so that the build remains independently runnable. The organ axes not occupied by the candidate under test draw their building block from the cache engine by the same route (Section 4.2). The integration of hierarchical prefetching [ZGH⁺25] additionally provides a compile-time call-graph analyser (`p27_bundle_finder`, a C++23 port of the

original tool `hp_soft` that identifies functions with a subtree footprint above a threshold as bundle entry points).

A design decision delimits this live path from a rejected alternative: a variant-based building-block resolution — `resolve_baustein` over `baustein_variants`, a concept-checked `if constexpr` over tag specializations — is expressly forbidden for static organ axes because it produces runtime overhead and code bloat; it exists only as a test cluster isolated from the build, with one test as consumer of its resolution path (a second test reads from it only a building-block identifier) and a strictness check that breaks any wiring into the codegen path. The reason is the parsimony of the compiled artefact: a tier binary maps exactly the one permutation being run; `std::variant`, `virtual` and runtime switches are forbidden in tier and hybrid binaries because they map at runtime onto several compile-time targets — whoever needs another permutation needs another compiled artefact. The one permitted variant site of the house is the dock array of the hybrid stage (`HybridDockVariant`, Section 4.5.1), whose zone a test checks by reading the source files and permitting exactly one occurrence; besides this, the code tolerates `std::variant` in two axis-foreign building blocks (the error domain `axis_error` and the pressure state `pressure_state`), which belong to no organ, and in the three expected/result seams of the hardware probes (NUMA pages, NUMA core binding, operating system) that consume the error sums of `axis_error`; the CEB itself contains no variant site: its substance under `builder/` lies within the counting range of the same check, and the one registry header of its carrier scaffold under `libs/traeger/ceb` outside this range is likewise variant-free. The ban is thus stricter than the mere avoidance of virtual calls [Ale01]: it forbids every form of runtime branching between building blocks.

Within each axis category there are main axes, sub-axes and meta-meta axes. In code, the category is the trichotomy `AxisCategory` with the fixed order `mess < system < organ` — an order that a `static_assert` holds against silent re-sorting —, and the family of an axis is the discriminator `AxisKind` with six values: measurement-system axis and measurement meta-meta, system-configuration axis and system meta-meta, organ axis and organ meta-meta. A sub-axis is itself a full axis (the recursion of the layer model remains open), and a sub-axis can at the same time be a permutation axis — the two roles do not exclude each other. Table 4.2 counts the inventory per category: the organ realm comprises eighteen main axes, twenty sub-axis headers and the organ meta-meta `disk_io`; the system realm three main axes, the complex axis `build_target_complex` as command wrapper of their recombination with the sub-axis group `build_toolchain` (compiler, optimization level, atomics), the scheduling with five dimensions and the system meta-metas under the hub of the extension hardware; the measurement realm the tooling main axis (wallclock/macro/micro), four registered sub-axes (sixteen categories, work mode, measurement framework, write-back method) and the measurement meta-meta `load_framework`. The designation meta-meta follows the four-level meta-level architecture of the Object Management Group (MOF) [Obj16], in which each level is the description language of the one beneath it; the reading M0–M3 for this apparatus is developed in Section 4.5.

The eighteen organ axes (T0–T17, Section 3.4.1) are here not abstract but each a concrete *strategy concept* with several interchangeable building blocks, from which a concrete composition chooses exactly one per slot; their number is, as `kOrganAxisCount = 18`, the single source of the organ line, and their names and slot order are carried by the array `kCompositionAxisNames`. Figure 4.3 gathers all eighteen as a gallery — per axis the short name, the sub-axis family and a representative selection of the building blocks; the complete building-block matrix with all origin sources is given in Appendix D, the SOTA

Category	Main axes	Sub-axes	Meta-meta axes	Single source
Measurement	tooling (3: wall-clock, macro, micro)	16 categories; work mode (4); measurement framework (1); write-back method (4)	load_framework (YCSB)	four registry headers, measurement registry XML
System	3: target_isa, operating_system, external_utils	complex axis build_target_complex with group build_toolchain (3); target_isa_complex (3 members); scheduling (5 dimensions)	system meta-metas under the hub external_utils	kSystemAxisOrder, system registry XML
Organ	18 (T0–T17)	20 sub-axis headers (e.g. SA1–SA4, AA1–AA7)	disk_io (binary_id = never)	kOrganAxisCount, kComposition–AxisNames, organ registry XML

Table 4.2: Axis typology per category (`AxisCategory` with the order `mess < system < organ`; family discriminator `AxisKind` with six values). The numbers are structural numbers of the source tree (registry headers and generated registry XML), not measured values.

contributions per axis in Table 3.2. Next to the eighteen canonical main axes stands the organ meta-meta `disk_io`, which never touches the `binary_id` (Section 3.4.1); a nineteenth, optional organ axis is planned but not implemented; only its implementation introduces it as slot T18. That no composition omits an organ is enforced by the `IsComposition` concept: it demands per algorithm the assignment of exactly these eighteen organs — the fifteen search axes, the two queuing axes and the persistence target —, so that no composition compiles that leaves an axis open; the complete organ set is thus not a convention but a compile-time-checked condition.

4.3.1 The One Architecture in Code

The *one* architecture from Section 3.3 is, in code, a single hierarchy. Its root is `IExecutionEngine` with three engine kinds — Anatomy, Virus, Hybrid — and a contract of seven methods: `warm_up`, `run`, `reset` and `shutdown` as measurement lifecycle as well as `engine_name`, `engine_kind` and `lifecycle_state` as reports. Beneath it lie, on level 1, the four families Map, Container, Graph and HeuristikAdapter (Section 3.3.2) — Graph as a declared stub without a genus — and, on level 2, the six genera SearchAlgorithm, Set, Sequence, Adapter, View and FunctionInterfaceReroute; in code the family is the derivation of the genus (`gattung_of`), not a second label. The living being SearchAlgorithm and its ABI runtime view — the “SearchEngine” — are two views of the same entity across the module boundary, not a parallel construct: the body is `SearchAlgorithmAnatomy<C>`, the composition and wiring of the eighteen organs, and the view is `SearchAlgorithmAbiAdapter<A>`, which exports it across the module boundary. The layering `CacheEngine` $\rightarrow$ `ExecutionEngine` $\rightarrow$ `SearchEngine` (cf. Section 4.1) thus separates roles, not class trees: the `CacheEngine` as the axis building-block library, the `ExecutionEngine` as the living root of the execution and measurement tools (OS primitives; in experiment mode the measurement aggregator over exactly one observer interface) and the `SearchEngine` as the search-API view measured over precisely this interface — the library skeleton of the same name, `libs/search_engine`, contains no code (Section 4.1). Axis-less *viruses* stand as siblings beneath the same root (in the existing code a graph breadth-first search) but possess no anatomy (Figure 4.4). In

T0 Search algorithm SA1-SA4 Array256, OriginalART/HOT, KArY, Interpol., Hash, SwissTable ... (22: 18 base + 4 per-K) <i>Def. Array256</i>	T1 Cache traversal CT1-CT2 Linear-fanout, Hash- lookup, Binary-fanout <i>Def. Linear</i>	T2 Slot mapping MP1-MP2 Direct-placement, Pool-relative <i>Def. Direct</i>	T3 Path compression PC1-PC3 None, Byte-wise (ART), Patricia <i>Def. None</i>
T4 Node type NT1-NT3 Node4 / 16 / 48 / 256 <i>Def. Node256</i>	T5 Memory layout HM1-HM3 AoS, Cache-line-aligned, SoA, Bitmap (LOUDS), AoSoA <i>Def. Cache-line</i>	T6 Allocator AA1-AA7 Hoard, Slab, Mimalloc, Jemalloc, TCMalloc, Buddy ... (26) <i>Def. Std-malloc</i>	T7 Prefetch PF1-PF3 None, Distance-estimator (ART), Hardware, Path-orient. <i>Def. None</i>
T8 Concurrency CC1-CC2 None, Blocking, R/W, OLC, Lock-free, RCU, Hazard <i>Def. None</i>	T9 Serialization SR1-SR3 Raw-binary, VarLen (LEB128), Succinct, LZ4/Delta <i>Def. Raw-binary</i>	T10 Value handle VH1-VH3 Inline, External-pool, RCU- shared, MVCC, Chain-ref <i>Def. Inline</i>	T11 Index org. io1-io3 (index_organization) Clustered, Heap, IOT (em- bedded), Non-clustered <i>Def. Heap</i>
T12 I/O dispatch IO1-IO3 (io_dispatch) In-memory, Buffered, Di- rect (O_DIRECT), Mmap <i>Def. In-memory</i>	T13 Migration MG1-MG3 None, Hot/Cold, Tier- based, Adaptive (ML) <i>Def. None</i>	T14 Filter FT1-FT3 Bloom, Cuckoo, Range-SuRF (LOUDS), Xor; None (de- clared, disabled via option) <i>Def. Bloom</i>	T15 Op. buffer (Q1) QS1-QS6 None, Append, FIFO/LIFO, Ring, Priority ... (15) <i>Def. No-buffer</i>
T16 Flush policy (Q2) FS1-FS4 Eager, Lazy, Watermark, Timed, Adaptive-LSM <i>Def. Lazy</i>	T17 Persistence target 18th slot (ORG-18) In-memory; disk write-back (declared, disabled via option) <i>Def. In-memory</i>	Organ meta- meta disk_io binary_id = never, stage ct; runtime sub-axis staging; stamps in the bracket appendage of the organ line (Section 3.4.1)	Reading Each axis = a Concept with interchangeable building blocks (<i>Strategy</i>); a composition chooses exactly one per slot.

Figure 4.3: Gallery of the eighteen organ axes (T0–T17, canonical slot order): per axis the implementation concept — short name, sub-axis family and a representative selection of the interchangeable building blocks. Each axis is a `Concept` interface with several strategy building blocks (*Strategy* pattern); a concrete composition occupies exactly one per slot. The *default* (“Def.”) is a derived comparison baseline, not a code flag. The numbers in parentheses (T0, T6, T15) are the *catalogues* of the complete type lists (`All*`); the *enabled inventory* of a build configuration is the subset formed by the enable filter, which the generated registry XML reports per axis (for instance four search algorithms and three allocators in the in-house default). The T numbers are slots of the composition, not directory numbers of the implementation; the abbreviations io1–io3 and IO1–IO3 belong to different axes (index organization and I/O dispatch). Next to the eighteen slots stands the organ meta-meta `disk_io`; the persistence target is pinned to its in-memory representative in the canonical reference catalogue (Section 3.4.1). The complete matrix with origin sources is given in Appendix D.

this hierarchy the compiler-compiler character of the apparatus is anchored: every composition is a type from which the builder generates a separate compiled artefact per permutation.

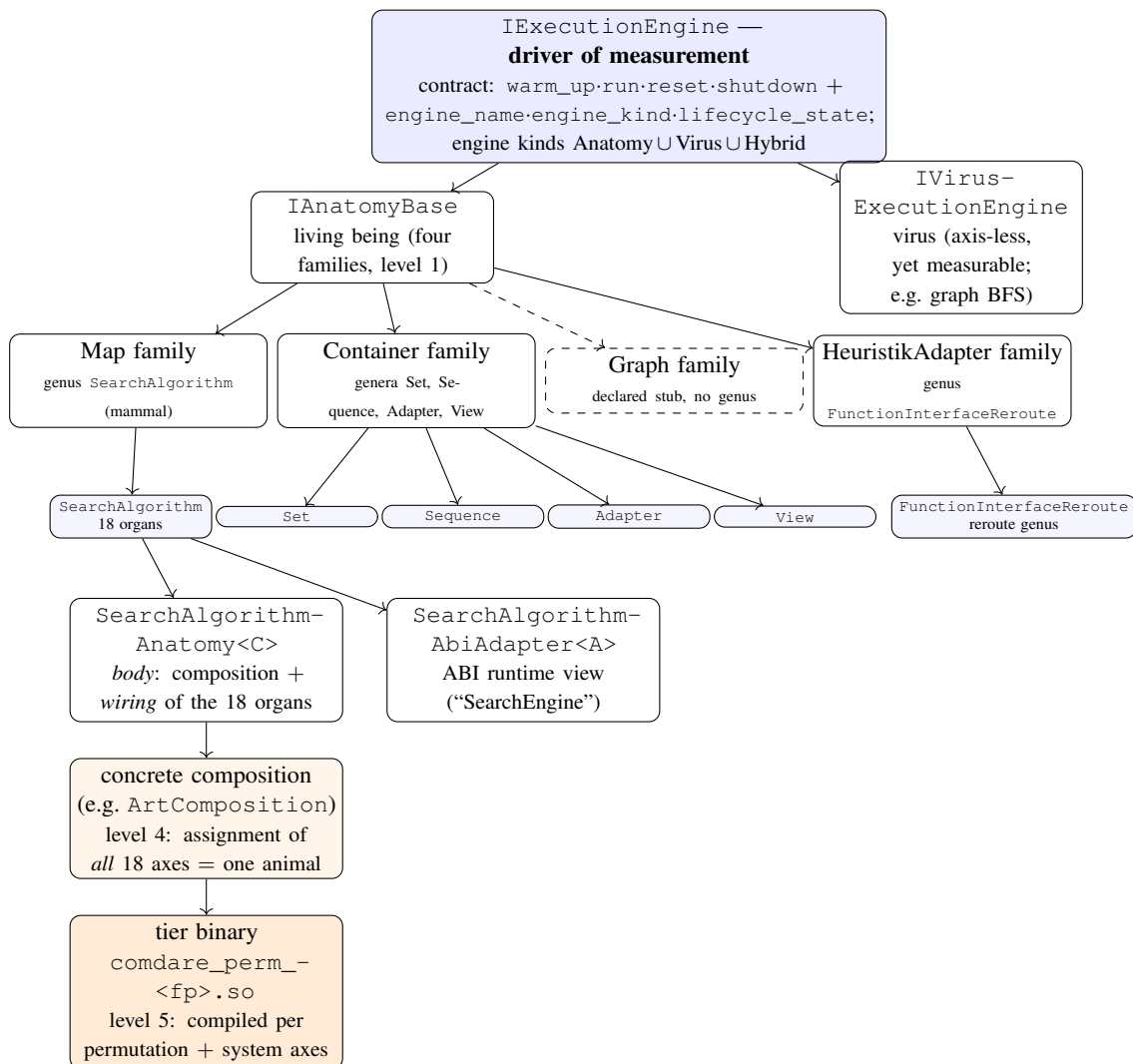


Figure 4.4: The *one* architecture as a single hierarchy without parallel trees. At the root stands `IExecutionEngine` as the *driver of measurement* — the universal measurable contract of measurement lifecycle (`warm_up/run/reset/shutdown`) and reports (`engine_name/engine_kind/lifecycle_state`) across all three engine kinds `Anatomy`, `Virus` and `Hybrid`; beneath it branch the anatomy-bearing *families* (level 1: `Map`, `Container`, `Graph`, `HeuristikAdapter`) with their *genera* (level 2) and the axis-less *viruses*. `SearchAlgorithm` is the focal living being (mammal); its anatomy is its *body* — the fixed composition *and wiring* of the 18 organs —, and “`SearchEngine`” only its ABI runtime view. The mammal branch is drawn out down to the individual: a concrete axis composition (level 4) is compiled per permutation as a standalone tier binary (level 5). `Container` possesses an organ set of its own with the genera `Set`, `Sequence`, `Adapter` and `View`, whose arities are carried by the genus build admission; `Graph` is declared and possesses no genus; the family `HeuristikAdapter` carries the reroute genus of the hybrid stage.

Concretely, the module boundary is a C-stable interface — the ABI (application binary interface) of the apparatus — that exports exactly one living being per permutation (Figure 4.5): every tier binary contains seven mandatory symbols with C linkage — the four primal symbols ABI version, magic number, factory

comdare_create_anatomy and destroy, plus the two reports comdare_anatomy_gattung and comdare_anatomy_genus, spelled identically across all families, as well as the version lines of the stamp. The identical spelling is a condition, not a convenience: the family-independent loader loads every binary via the same loading route (POSIX `dlopen` [The18]) and can determine the family *before* it constructs an instance — family-independent names with family-specific values; the family is here derived from the genus. The ABI carries major version 9, whose seam is the genus interface. The variadic 1/2/N hybrid API belongs to the candidate under test: its `PrtArtSearchEngine<Ts...>` specializes by parameter count — one parameter demands all function interfaces of a `std::vector`, two those of a `std::map`, more than two those of a `std::map` onto a tuple —, and the slot arities of the genera of the cache engine (eighteen for the genus `SearchAlgorithm`, each their own for Set, Sequence, Adapter and View from the genus build admission) are to be read separately from it. Complex keys are reduced by a fingerprint component of the cache engine (`fixed_length_fingerprint`, FNV-1a base) to 16-byte binary strings of fixed length, which the candidate under test uses as the key type of its memory model (`BinaryKey`); this key fingerprint is to be distinguished from the three identity fingerprints of the apparatus — the 64-bit field `permutation_fingerprint` in the ABI POD of the module, the SHA-512 tier fingerprint (format 6, eleven members) and the SHA-256 planner fingerprint (Section 3.4.4). How these and the remaining code interfaces relate in detail is summarized by the UML map in Figure 4.6.

```
// C-stable module boundary: seven mandatory symbols per tier binary
extern "C" {
    std::uint64_t comdare_anatomy_abi_version();    // ABI major 9
    std::uint64_t comdare_anatomy_abi_magic();     // compatibility probe
    IAnatomyBase* comdare_create_anatomy();        // one being per permutation
    void          comdare_destroy_anatomy(IAnatomyBase*);
    std::uint8_t  comdare_anatomy_gattung();       // same name for every family
    std::uint8_t  comdare_anatomy_genus();         // hybrid: the target genus
    AnatomyVersionLines const* comdare_anatomy_version_lines();
}
// The hybrid .so exports the same set: exactly one loading route.

// Candidate view: variadic 1/2/N API (key K, values V)
// 1 parameter -> std::vector<V>
// 2 parameters -> std::map<K, V>
// N > 2         -> std::map<K, std::tuple<V1, ..., VN>>
template<class K, class First, class... Rest> class PrtArtSearchEngine;
```

Figure 4.5: The ABI-stable C module boundary (seven mandatory symbols; the hybrid `.so` exports the same set and reports its target genus via `genus`) and the variadic 1/2/N API as the view of the candidate under test. Complex keys are reduced by a fingerprint component to 16-byte binary strings of fixed length; the identity of a binary is carried separately from this by the `permutation_fingerprint` field of the ABI POD, the SHA-512 tier fingerprint and the SHA-256 planner fingerprint.

4.3.2 From Profile to Tier Binary: Planner, Builder and Measurement Pipeline

The chain from the user XML to the tier binary (Section 3.4.6) has, in code, a named shape of two binaries. Stage 1, the planner role, is `comdare-experiment-planner` with the subcommands `validate`, `plan dump|ci|cmake`, `cache-key`, `fingerprint`, `status`, `check-size`, `simulate`, `version` and `help`. `validate` checks, purely read-only, the profile resolved by the

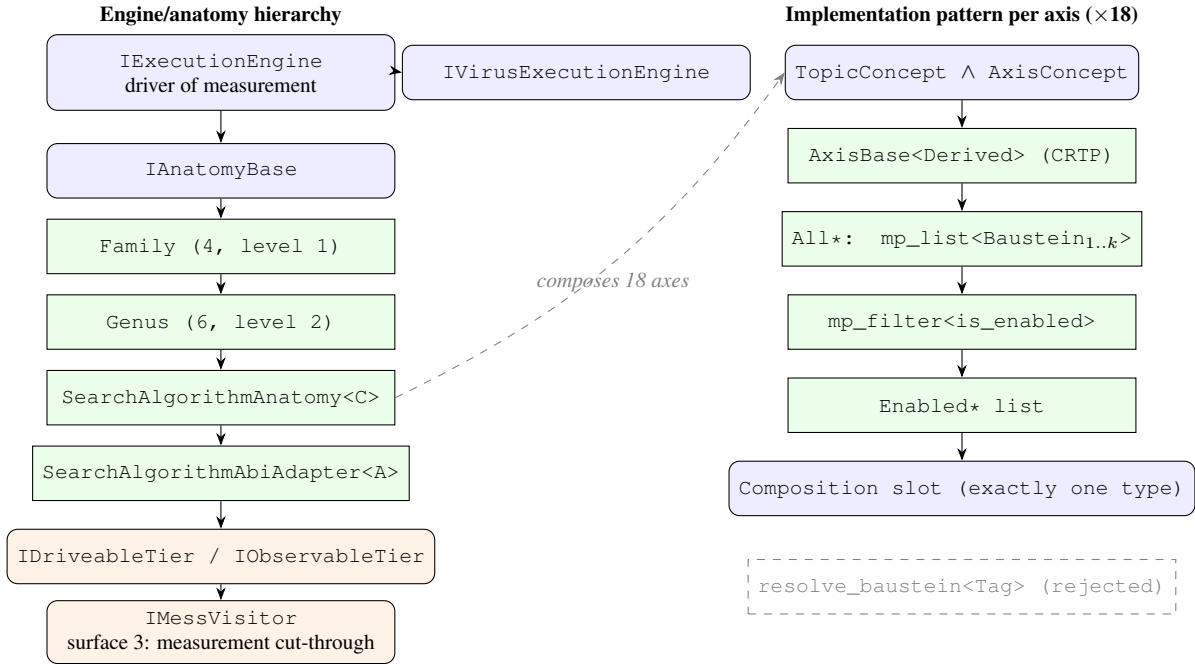


Figure 4.6: UML map of the code interfaces: on the left the engine/anatomy hierarchy — the measurable root **IExecutionEngine** as the *driver of measurement* ($\text{Anatomy} \cup \text{Virus} \cup \text{Hybrid}$) over **IAatomyBase**, the four families and six genera and the axis-less **IVirusExecutionEngine**, beneath it the *body* **SearchAlgorithmAnatomy<C>** (composition and wiring of the eighteen axes) and its ABI runtime view **SearchAlgorithmAbiAdapter<A>** (“SearchEngine”), which is measured via the drive/observer contracts **IDriveableTier/IObservableTier** and whose measurement crosses as surface 3 through the visitor **IMessVisitor**; on the right the live path repeated per axis — a two-layer Concept, a requires-secured CRTP base, the complete **mp_list** building-block family, the compile-time filter over the enable flag and the composition slot, which contains exactly one type. The variant-based resolution **resolve_baustein** is a rejected alternative isolated from the build (dashed).

XML parser against the compiled-in registry catalogues (an unregistered reference is a hard validation error, Section 3.4.6), builds no binary in doing so and does not measure. **plan** drives the deterministic walk of the **ExperimentPlanDirector** — separated along the design patterns *Director* and *Builder* [GHJV94]: the director owns the *one* enumeration walk over measurement combinations, system permutations and organ steps, and per output syntax (text plan, CI pipeline description, CMake fragment) a concrete builder emits via hook methods —; two runs are byte-identical, and the plan header annotates the registry trio of the three offering catalogues organ/system/measurement. **check-size** and **simulate** compute the measurement set before the run from the inputs of the walk (Section 4.4). Stage 2, the CEB role, is the measurement driver **comdare-messung-driver** in the project repository with the subcommands **tier ci|cmake, run, version** and **help**: it emits the view of the second chain stage — system permutations and tier build orders of its released build space — and drives the measurement run; a planner topic addressed to it is answered by its dispatcher with the reference to the planner binary instead of silently executing it. The planner steers the CEB orders, the CEB steers the tier orders: the separation of roles is thus not merely surface but two modules. The emitted build and check attestations keep the layer separation of the chain visible — their legends

have three forms: `ceb:build:[a,b,c]` on stage 1; on stage 2 per host a build and check batch `tier:build-batch:<host>`, whose attestations per step contain the separate system and organ brackets `zelle=[d,e,f][g,h,i]` and which bundles the organ build volumes internally into slices of 4096 binaries each (the milestone batches in Section 5.1.7); and per host a measurement batch `measure:[a,b,c]:batch:<host>`, whose measurement attestations alone contain all three brackets `zelle=[a,b,c][d,e,f][g,h,i]` (Figure 4.1); the brackets name only main axes, sub-axes are pure runtime permutation and appear in no build legend.

The installation has two control paths: the planner directly via shell, and an API command line with a headless service that serves the planner as a cell on Kubernetes or on bare metal, so that a complete measurement series can be triggered and observed without an interactive session. Added to this is a command-line-guided creation mode for the user XML with hardware filter: options not available on the target machine are not selectable in it, and the mode is optional — a hand-written XML remains equivalent. Present in the code are the planner’s hardware probe, which queries the host at runtime, and the three machine signatures of the system registry; implementation status: the shell path is implemented; service and creation mode are planned but not implemented.

Two properties secure the *one* main channel. First, the text plan (format v1.1) reports per build step the actually generated file stem (`built_stem`) — produced by the *same* orchestrator function that the build orchestrator also calls during the build: the plan claims no name that the build does not produce, and a stem that cannot be determined is written as “-” instead of synthesized. Second, there is *one* XML source and *one* channel that builds it: variant formation lies in the C++23 metaprogramming, CMake is confined to the role of building, and a second emission channel for permutation source texts does not exist in the source tree.

Inside the CEB, the builder transforms each profile into a precompiled C++23 module. Each permutation is identified by a bit-packed *permutation flag* structure that comprises ten banks of 64 bits each (page type, node, traversal, value handle, layout, allocator, prefetch, concurrency, ISA and telemetry) and is serialized hexadecimally; the allocator sub-axes AA1–AA7 carry a flag bank of their own. A constraint filter of the flag structure (`is_valid_combination`, four rules, for instance: leaf-only counters only together with retroactive aggregation) excludes incoherent combinations. The measurement path identifies the eighteen organ axes via the string-serialized eighteen-segment path (the `binary_id`, Section 3.4.4) together with the three fingerprints; the permutation count over the axes follows a mixed-radix encoding in which each axis is a digit with its own base — the shape of a product space as described by the variability models of software product lines [PBvdL05]. Over these building blocks runs the *seven-phase measurement pipeline* (Figure 4.7) of the `ExperimentDriver` library per series with the phases of the code: (1) `ENUMERATE` (parse XML configs, build the permutation list), (2) `GENERATE` (one `comdare_perm_<fp>.cpp` per permutation plus aggregator `CMakeLists`, in addition `generate_module_from_profile` for the SOTA profiles), (3) `COMPILE` (CMake configuration and build of all permutation modules), (4) `LINK+LOAD` (`dlopen` of all modules), (5) `EXECUTE` (`create_instance` + profile-aware `run_workload`), (6) `MEASURE` (aggregate binary measurement records) and (7) `PERSIST` (CSV and JSON, optionally binary); two opt-in phases compile missing modules in the running experiment (`phase3_hot_compile_missing`) and run a functional test per module — creating and destroying an instance, a `run_workload` with an empty workload descriptor and the check of the ABI version (`phase4b_functional_tests`). The measurement-series modes `Defined`, `Full`

and `FullSampled` (deterministic 1: n subset of the full space, Section 3.4.6) choose which permutations a series comprises. The downstream evaluation toolchain (Figure 4.8) is a sequence of nine numbered programs from `sample_data_generator` to `tex_formatter`; its output is the `xlsx` workbook, the standard; the CSV form is not a single child document but an ordered tree of CSV files — one overall folder per workbook, beneath it one folder per sheet, therein the CSV files of the individual sub-axis runs —, which is always generated from the workbook (`xlsx` to `csv`, as a strategy of the same factory) and, per XML, lies at the same target instead of the workbook or in addition to it (four registered write-back methods, Section 4.5). Implementation status: the seam writes the CSV projection as one flat file (`projektion_oeffnen` in `ergebnis_mappe_naht`); the tree form is planned but not implemented. The measurement couplings are separated from the production path by the `COMDARE_MEASUREMENT_ON` gating (Section 4.1), and continuous integration [Fow06] runs across all four repositories (GitLab CI; the repositories are additionally mirrored to GitHub).

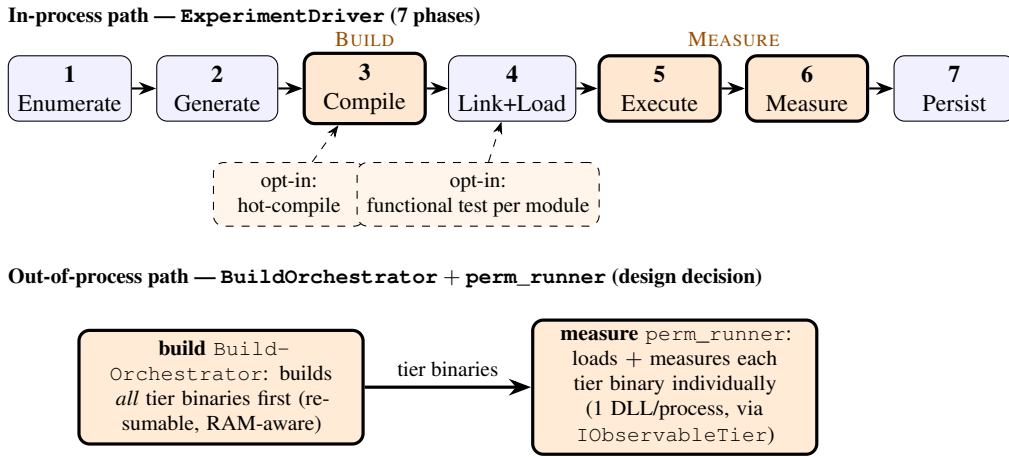


Figure 4.7: The measurement pipeline in two forms. The **in-process path** (top) is the seven-phase pipeline of the `ExperimentDriver` per measurement series (Enumerate → Generate → Compile → Link+Load → Execute → Measure → Persist; two opt-in phases compile missing modules in the running experiment and run a functional test per module with ABI version check). Highlighted are the substantial phases: `COMPILE` (the *build* of the tier binaries) and `EXECUTE/MEASURE` (the *measurement*); Generate/Link+Load/Persist are wiring. The **out-of-process path** (bottom) is a design decision for process isolation: the `BuildOrchestrator` *first* builds *all* tier binaries (resumable, RAM-aware), after which `perm_runner` loads and measures each binary individually via the observer interface `IObservableTier` — one DLL per process, because the DLL contains no `main` and the runner is the executable that loads it; no state is shared between permutations, and a cluster task runs exactly one runner with exactly one DLL.

The levels E4–E0 from Section 3.3.4 each have a place in code, and the boundaries between them each have a contract test (Section 4.4); Figure 4.9 shows this code shape without explaining the levels anew. E4, the definition and evaluation layer, is the XML profile with the planner (`profile_facade/planner`) and the evaluation programs 04 and 08 of the toolchain; E3, the permutation B^+ -tree, is the experiment tree (`experiment_tree`) with the `binary_id` as path; E2, the tier binaries, is the ABI adapter with the module ABI declaration; E1, the runtime configuration, is the interface `IResourceControllableTier` together with the dynamic dimensions of the measurement registry XML; E0, the infrastructure, is CI, runners and store; the cross-cutting concern M is the system-axis root

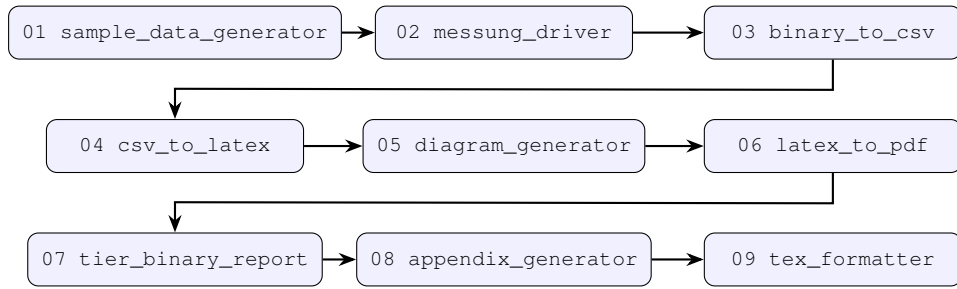


Figure 4.8: The downstream evaluation toolchain as a nine-stage chain: `sample_data_generator` produces deterministic test data without real hardware, `messung_driver` drives the measurement series (the CEB role), `binary_to_csv` deserializes the binary measurement records, `csv_to_latex` renders the booktabs tables with a building-block fact sheet per permutation ID and produces no file on empty input, `diagram_generator` the A4-aware TikZ diagrams (three basic plot types bar, scatter and heatmap as well as the families of the measurement appendix built on them: ECDF staircase, range bars, stacked per-axis attribution, sweep curve, 3D raw-data view, ratio matrix, normalized bars, Pareto scatter and the observer detail table), `latex_to_pdf` typesets the manuscript, `tier_binary_report` rasters per tier binary the interface contract against the planned set of the text plan, `appendix_generator` generates, as an in-process facade per language, the core and presentation files of the measurement appendix from *one* WIDE matrix, and `tex_formatter` normalizes the generated files. The output of the chain is the `xlsx` workbook; the CSV form is not a single child document but the ordered tree of CSV files generated from the workbook (one folder per sheet), written in code as one flat file (the tree form is planned but not implemented; Section 4.5).

SystemAxis (Section 4.5).

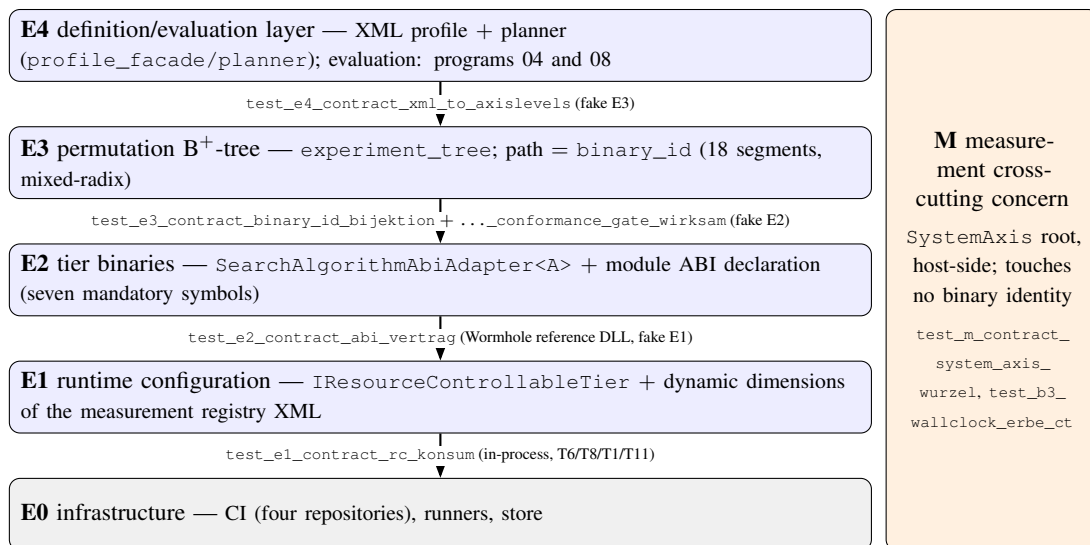


Figure 4.9: The levels E4–E0 of the layer model (Section 3.3.4) at their place in code, with the contract test per boundary (fake of the neighbouring level) and the measurement cross-cutting concern M, which crosses all levels host-side without touching the binary identity.

The contracts between the carrier stages are partitioned into surfaces at the binaries. The three-surface model (Section 3.3.2) is present in code by name: surface 1 is the genus interface of the tier binary (runtime, abstract factory), surface 2 the stamp (compile time, ABI-stable) and surface 3 the measurement cut-through `IMessVisitor` — a visitor at the `dlopen` transition with double dispatch, which exists

so that family and genus interfaces remain unchanged: the measurement crosses as a seam of its own instead of forcing measurement fields into the domain interfaces. The stamp contract is executed as a matrix of seven interfaces and four carriers — 28 cells with three-valued admission (mandatory, permitted, forbidden), of which the existing code occupies only mandatory and forbidden (Table 4.3): the static self-version is carried only by the planner; measurement and system line are carried by CEB, tier and hybrid; the organ line is carried only by tier and hybrid — the absence at the CEB is the static barrier “the CEB carries no organ axes” —; fingerprint and overall stamp are carried by all four, and the attached stamp lines are carried only by the hybrid as runtime hook. The contract is checked not in the base but beneath every heir: the base is empty, and only a `static_assert(StempelVertrag<Erbin, Traeger>)` directly beneath the definition turns an unparseable line, a missing mandatory or a forbidden interface into a compilation error — a form of consumer-driven contract [Rob06], in which the expectation of the consuming carrier drives the contract.

Interface	Content	Planner	CEB	Hybrid	Tier
VersionXyz	static self-version X.Y.Z[.flag]	M	F	F	F
MessZeile	stamp line of the measurement realm	F	M	M	M
SystemZeile	stamp line of the system realm	F	M	M	M
OrganZeile	stamp line of the organ realm	F	F	M	M
FingerprintSha	fingerprint hex (planner SHA-256, otherwise SHA-512)	M	M	M	M
GesamtStempel	the one composite of the heir	M	M	M	M
Angeschlossen	attached stamp lines (runtime hook)	F	F	M	F

Table 4.3: The stamp contract as a matrix of seven interfaces and four carriers (28 cells, three-valued admission M = mandatory, P = permitted, F = forbidden; P is unoccupied in the existing code). The forbidden cell of the organ line at the CEB is the static barrier “the CEB carries no organ axes”; the cells are checked per heir via `static_assert`. The column order follows, like Figure 3.9, the processing topology with the hybrid tier binary as the optional insertion between CEB and tier binary.

4.4 Quality Assurance: Contract Tests per Layer

The layer model E4–E0 with its fixed interface contracts was developed as a concept in Section 3.3.4; this section describes its implementation as test infrastructure. The reason lies in the layering itself: changes that alter several layers at once produce errors from interactions between the levels that neither local tests nor line-by-line review make visible (ADR-5, Appendix E). For a system whose product space rules out any complete materialization (Section 3.4.6), changes across several levels at once are thus practically not acceptable; the consequence is to complete each layer individually — with an explicitly fixed interface contract to the neighbouring level, its own test harness and its own definition of acceptance.

The carrying test principle reads: *each layer is tested against a compile-time substitute of its neighbouring level*. The contract tests are ordinary test targets of the build pipeline; the neighbouring level is replaced in each case by a *fake* — a template- and concept-based stand-in in the test scope, no runtime switch in production code. A layer can thus be finished and accepted before the deeper level even consumes its values, and the test question of every boundary — does the layer pass its definition on to the next one *without loss*? (Section 3.3.4) — turns from a design principle into an executable acceptance test.

Per layer this takes concrete shape as follows (Figure 4.9): for the definition and evaluation layer E4

the harness (`test_e4_contract_xml_to_axislevels`) checks the translation of the declarative XML profiles into the axis-level structure — golden fixtures compare the axis levels emitted from a profile structurally against an expected reference, with a fake E3 replacing the permutation tree; neither tree construction nor build orders are needed to accept the boundary $E4 \rightarrow E3$. For the permutation B^+ -tree E3 the harness (`test_e3_contract_binary_id_bijektion`) checks the bijection of the binary identity: from axis-level fixtures the permutation enumeration is run and, per path, the mixed-radix-encoded binary identifier is translated back and forth (counting plus signature round trip) — against a fake E2 that builds not a single binary; added to this is a conformance gate in the full-run path (`test_e3_contract_conformance_gate_wirksam`), which checks every imported binary against the fixed contract before the measurement. For the tier binaries E2 the contract is the ABI itself (`test_e2_contract_abi_vertrag`): the Wormhole reference DLL is loaded and must satisfy the fixed interface sequence — `IAnatomyBase`, `IObservableTier` and `IResourceControllableTier` (present at every adapter) together with a fixed capability report. The response to a control-quantity request (`apply`) is here not an error code but the *number of axes that actually accepted the value*: a request above the reported capabilities becomes visible because this number stays below the number of requested axes — in the Wormhole pilot, of five controllable axes exactly two (batch size and inline threshold) actually accept the value, a null pointer yields zero —, and that is the completeness rule of the control quantities: nothing is accepted silently, every response counts only the real consumption. The accompanying fake E1 creates the runtime control quantities and applies them, but demands no consumption effect — that belongs to the acceptance of E1. For the runtime configuration E1, finally, edge tests (`test_e1_contract_rc_konsum`, in-process over the axes T6, T8, T1 and T11) check the corner cases of the dynamic regime: that a reused tier is completely reset between two runs, that a pool budget below the live data set is treated as a named state instead of silently violated and that every applied control quantity is visible as a named column in the persisted result file, plus the preservation of the two-phase operation loop (discarded warm-up, measurement run) as an invariant. Persistence is checked at the result file, not at the in-process state; the lead here is the standard `xlsx` — the write-back registry carries it as the fourth method and default evaluation format, the `xlsx` writer lies in the storage layer of the store —, and the CSV form is not a single child document but an ordered tree of CSV files (one overall folder per workbook, beneath it one folder per sheet, therein the CSV files of the individual sub-axis runs), which is always generated from the workbook and, per XML, lies at the same target instead of the workbook or in addition to it. Implementation status: the E1 check reads the CSV output of the persist phase, which the seam writes as one flat file (`projektion_oeffnen`); the check at the `xlsx` generated per binary and the tree form of the CSV storage are planned but not implemented.

Above the layer harnesses, a *two-gate protocol* secures every implementation package. Its purpose is the separation of two questions: gate 1 secures the engine repository against itself — the unit and contract tests —, gate 2 secures the integration into the project repository, in which the chain $\text{XML} \rightarrow \text{binaries} \rightarrow \text{measurement} \rightarrow \text{appendix}$ actually runs; only both together prove that a package does not break the apparatus (continuous integration and delivery as safeguarding stages [Fow06, HF10]). *Gate 1* is the strict gate of the engine repository: the full test suite² runs in four cells, gcc and clang each as release and debug build, without change skip, and only a run without failed tests is accepted, without an exception list. The double run has two reasons: the compiler matrix forces every static check through two translators, and the

²Extent of the test suite: 551 registered tests (`cctest -N`) in the gcc release build tree of 17 September 2026.

repetition of the measurement run — the two-phase operation loop of discarded warm-up and measurement run in the single-thread-deterministic mode `measure` — makes the run-to-run stability of the number itself the object of testing. Part of gate 1 is the golden anchoring of the permutation space, and here two notions are to be separated (Section 3.4.6): The *reference catalogue* of the golden test holds $2^{17} = 131072$ binary identities — seventeen axes with two representatives each, the persistence target pinned to one — and is confirmed against a fixed CRC-64 checksum (`0x56F1B721C72DC10E`, CRC-64/ECMA-182 over the identities in canonical order; the reference is literal tool output, and its change is permissible only within a coordinated interface change), while a small, materializable catalogue of $4 \cdot 4 \cdot 5 \cdot 4 = 320$ identities serves as a byte-exact check of the serialization form — a characterization test in the sense of Feathers [Fea04], which uses a fixed cardinality as a regression detector. The *cardinality* of the space planned in a complete measurement series is different from this and never static: the planner counts it per axis from the possibilities declared in the XML in the director walk (`check-size`, `simulate`) and multiplies the levels over all axes; it differs per platform through missing releases, and every number of the computation comes in as input, never as a constant. *Gate 2* is the integration gate of the project repository: its pipeline builds without error and without an exception set — the rule of this work knows no tolerated failing tests; the two declared failure tolerances of the measurement machines lie in the engine repository and are a platform boundary (an arm64 smoke build without an available runner) and a manual relock maintenance job (it must not remain in the executed state), not a test set.

This protocol is methodology, not a system component: it describes *how* changes to the apparatus are accepted, not what the binaries contain. The directory and suite structure on which it runs is documented in Appendix B.

Completion follows the layering top-down — E4, E3, E2, then the measurement cross-cutting concern M immediately before E1 —, and the infrastructure E0 forms the last step of the consolidation (Section 3.3.4). The degree of maturity can be read from the test inventory: all four boundaries possess their harness (above), and the cross-cutting concern M possesses one of its own that gets by without any tier binary (`test_m_contract_system_axis_wurzel` for the system-axis root and the measurement visibility, `test_b3_wallclock_erbe_ct` for the wallclock inheritance): a synthetic measurement source is driven through the system-axis root and its values traced into the result columns. The measurement axis here drives the release of the measurement source via the system-axis root in the shape of the CEB: the planner delegates to the CEB interface which binary recombination is to be measured against which loads, and the CEB releases the source. Implementation status: the hardware acquisition chain runs at the release seam of the builder (Section 4.5); the CEB-driven release path of the measurement source is planned but not implemented. The code shape of the cross-cutting concern is described in the following section.

4.5 Realm Roots in Code: System and Measurement Axes

Four abstraction levels order the notions of this section (Figure 4.10), following the four-level meta-level architecture of the Object Management Group (MOF) [Obj16], in which M0 denotes the runtime instances, M1 the model, M2 the metamodel and M3 the meta-metamodel: M3 are the *meta-meta axes* in the sense of the indirection of a description — an axis that describes how other axes are described (the measurement meta-meta `load_framework`, the organ meta-meta `disk_io`, the system meta-metas under the hub

of the extension hardware); M2 are, as a reading, the *meta axes*, the literal classification of the main and sub-axes carried under their notion — in code the registry line with category, family and building-block list; M1 are conception and runtime implementation of the main and sub-axes — the CRTP axis with its concept; M0 is the binary code — the tier binary in which exactly one assignment is compiled in. In the stamp, the meta-meta level of an entry is mechanically its bracket depth: bits 3–5 of an entry encode the level, a main axis is bracket-less, a meta-meta hangs as a bracket appendage on its line, and a depth above seven aborts with an error message; the renderer of the meta-meta entries is at the same time the bottleneck at which two version checks check every stamped member, so that no new meta-meta can bypass this check. The short form `c{p.e}` of a version flag here denotes `c` with the sub-members `p` and `e`, never a flat abbreviation. Per realm there is a dedicated meta-meta discriminator (`measurement_meta_meta`, `system_meta_meta`, `organ_meta_meta`); PMC acquisition — performance monitoring counters, the hardware counters of the CPU — is a meta-meta measurement axis because it describes with which counter instrument other measurements are collected.

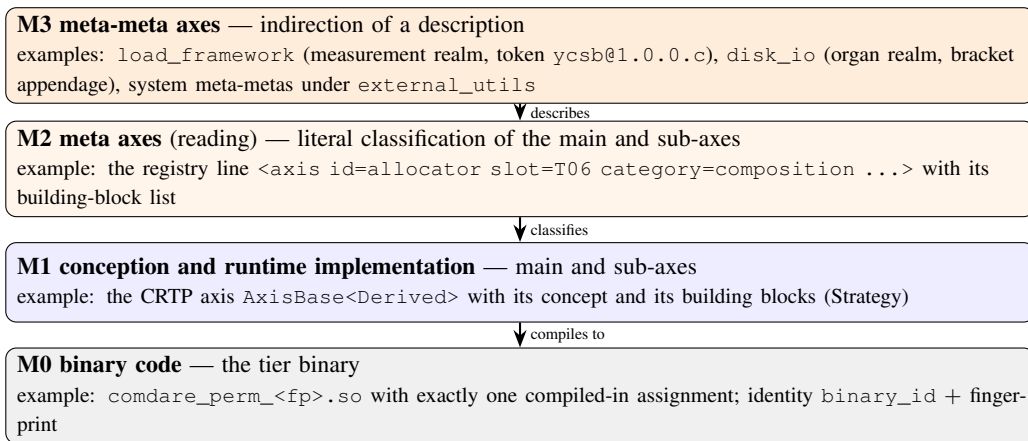


Figure 4.10: The four abstraction levels M0–M3 as a reading of the axis notions, following the meta-level architecture of the MOF: meta-meta axes (M3) describe how other axes are described; the registry line (M2) classifies main and sub-axes; the CRTP axis (M1) is their conception and runtime implementation; the tier binary (M0) is the binary code of exactly one assignment.

The tripartition of the axis realms (Section 3.4.1) is executed in code as a type separation. All axes share a common family roof — `topics::Axis<Derived>`, an empty, non-polymorphic CRTP base whose concept check (`is_empty` and `not is_polymorphic`) rejects any smuggled-in vtable already at compile time — and declare their category via the trichotomy `AxisCategory` and their family via the discriminator `AxisKind` with six values (Section 4.3). Per family there is a dedicated CRTP root: `OrganAxis` and `OrganMetaMetaAxis` for the organ realm, `SystemAxis` for the measurement-system axes of the host-side cross-cutting concern, `CebSystemAxis` for the build and steering axes of the CEB configuration, `SystemMetaMetaAxis` for the system meta axes under the extension hub and `MeasurementMetaMetaAxis` for the measurement-realm axes of the planner. An axis is its family through its base class, not through a re-attachable label — `CebSystemAxis` returns its kind `axis_kind()` as the hard return value `system_config`, which is why the measurement realm possesses a root of its own instead of a reused one: the system realm roots at the CEB, the measurement realm at the planner (Section 3.4.1).

On the system side, the canonical order itself is code: the system main axes `target_isa`,

`operating_system` and `external_utils` are held in their binding sequence by the constant three-element array `kSystemAxisOrder`, and every consumer that needs an order — the stamp line, the generator block sequence, the XML header, the suffix segments — must read it there instead of re-deriving it (order single source with drift checks). Three compile-time *re-entry guards* fix the cut of the main axes: `static_assert` checks reject compiler, scheduling and load framework as system main axes — the compiler is a sub-axis group `build_toolchain` of the complex wrapper `build_target_complex` with a global optimization identity in the toolchain member of the fingerprint, the scheduling a sub-axis at `target_isa` with five dimensions, the load framework a meta-meta main axis of the measurement realm —, and whoever re-enters one of the three breaks the build and must revoke the assignment explicitly instead of silently revising it. The build version identifier of the system line (Section 3.4.4) likewise draws from a single declarative source: the ordered eight-segment sequence `kSuffixSegmentOrder` of the build control quantities — compiler identifier, optimization, SIMD extension, CEB, target ISA, telemetry, build type and gate contribution —, in which an empty member produces no segment and the gate segment stands at the end, so that every suffix comparison sees an unchanged beginning and an appendage (prefix stability); the four-part cell path of the storage follows the same order member by member, with a check that binds both orders to each other.

On the measurement side, the meta-meta root `MeasurementMetaMetaAxis` is the planner's own; its first carrier is the load framework `load_framework` (stamp token `ycsb@1.0.0.c`), and like all measurement-realm axes it is strictly binary-id-neutral (registry XML: `binary_id = never`). The measurement-tooling *main* axis holds exactly three versioned entries — wallclock, macro, micro — in the only permitted order wallclock/macro/micro (alias `w/ma/mi`, Section 3.4); its check turns a fourth, unagreed tooling into a compilation error. It fans out four sub-axis registries, one file each (Table 4.4): the work mode (`WorkMode`: build, measure, compare, release, Section 4.1), the measurement framework (`MeasurementFramework`: exactly one entry, YCSB), the write-back method (`WritebackMethod`: CSV, LaTeX table, comparison metrics and `xlsx` — four values, `xlsx` as default evaluation format, expressly no PDF, because the PDF compilation is not a write-back channel of the measurement machine) and the sixteen measurement categories (below); each registry is a `constexpr` catalogue with the static check index = enum value, and a further entry breaks the compilation. The counter instrumentation resides in the micro tooling; a PMC vendor registry supplements it with the hardware form of the counters (below). On the compile side, two separate gates mirror this tooling ladder: the *statistics gate* (`COMDARE_CE_ENABLE_STATISTICS`, an `#ifdef`) switches the observer statistics of the macro regime, the value-based *segment-timing gate* (`COMDARE_CE_ENABLE_SEGMENT_TIMING`, an `#if`) the eighteen per-axis segment timers of the fine grain. An unset segment-timing gate *inherits* the state of the statistics gate — existing builds do not change a single byte through the separation —, and a set segment-timing gate without the statistics gate aborts the compilation with an error message, because the segment timers write into structures of the observer block. The actually effective gate state of every translation unit travels as a dedicated measurement-gate member with nine `;`-separated fields into the fingerprint preimage of the stamp (Section 3.4.4) — in the off state `mg=m0;s0;st0;x0;tw0;tm0;tmi0;hm0;hmi0`: measurement, statistics, segment timing, experiment mode, then wallclock, macro and micro of the tier stage and macro and micro of the hybrid stage on positions of their own —; the stamp thus never claims an outfit that the build does not carry. The target picture next to this registry state is the axis-interface model from Section 3.4.6: micro-benchmarking measures an axis via its axis interface, and the three

measurement levels provide the counter-measurement; the registry is aligned to this model.

Registry (file)	Values	Count	Check
run_ methodology_ registry	WorkMode: build, measure, compare, release	4	index = enum, execution order; only measure measures
measurement_ framework_ registry	MeasurementFramework: Ycsb	1	a second framework breaks at compile time
writeback_ method_registry	WritebackMethod: Csv, LaTeXTable, ComparisonMetrics, Xlsx (default)	4	a fifth method breaks at compile time; no PDF
measurement_ axis_registry	sixteen measurement categories with regime (9 time/observer, 7 PMC)	16	index = category, no empty name, regime = root assignment

Table 4.4: The four sub-axis registries of the measurement tooling: one `constexpr` file each with index-equals-enum check; the main axis itself holds the three toolings wallclock, macro and micro.

Switching off the measurement is a staged model over the three measurement levels (Figure 4.11). The measuring instrument `checkpoint_measure` always collects the input parameters at a *point in time*; the first layer, the wallclock, is the minimum of a time-measuring, otherwise parameter-empty checkpoint and thus the minimal measuring instrument of the CEB. If macro or micro benchmarking is active, the wallclock checkpoint also performs the parameter bracket — a compile-time special case, because in macro and micro, parameters and time measurement always coincide. The first shut-off level switches off the parameter collection in macro and micro, so that the wallclock remains in place for `measure` and `compare`; the second, permutative shut-off level lets the wallclock fall away as well for `release`, and with it the surfaces and interfaces of the measurement fall away: for the release only the core code of the binary remains standing. Implementation status: the code contains the gates G2 and G3 and the release mode without instrumentation (`measurement_on off`); the two-level shut-off as a permutation quantity is planned but not implemented.

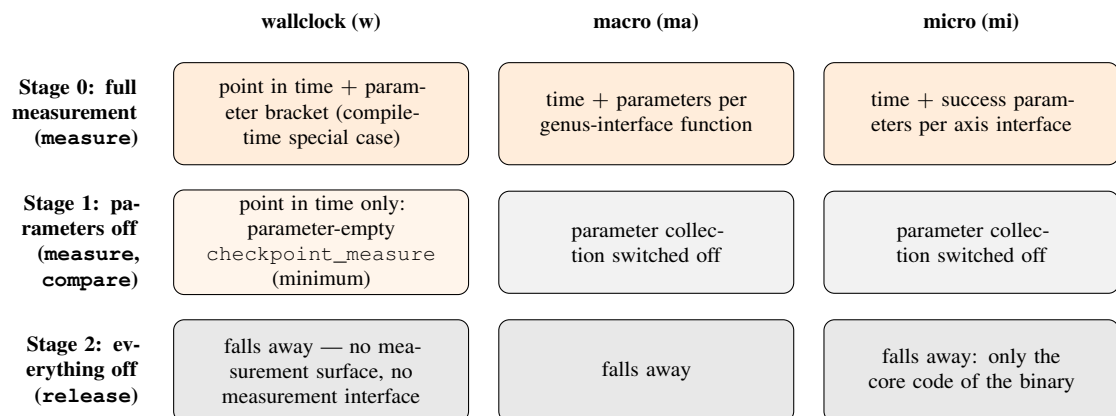


Figure 4.11: The two shut-off levels of the measurement over the three measurement levels: `checkpoint_measure` collects at a point in time; the wallclock is the parameter-empty minimum but, with macro or micro active, also performs the parameter bracket; stage 1 switches off the parameter collection, stage 2 lets the wallclock together with the measurement surfaces fall away as well for the release.

The PMC tooling is multi-layered because the hardware counters differ per core type and vendor (access via the `perf_event_open` interface of the Linux kernel [Lin26b]). A generic event list (`pmc_event_set`) decides whether PMC is built in at all: the planner's host probe checks exactly the events that the real source opens later, so that the decision and the measurement read the same source. A model-bound RAW catalogue (`pmc_raw_event_katalog`) fills the channels for L2 misses and coherence per microarchitecture: for AMD Zen 5 (family 0x1A, measured on the Ryzen 9 9950X3D of prod1) the assignment is cross-checked — event 0x64/umask 0x9 for the demand misses in L2 and event 0x43/umask 0x14 for demand fills from the cache of another CCX, against the named `perf` events with identical counter readings (17 314 023 against 17 314 023); for Intel no entry exists, because a RAW assignment without cross-checking by measurement would be a guess [Int24b] — a declared gap that appears as a missing value. The semantics are disclosed: `cache_misses_l2` captures on Zen 5 the demand misses without L2 prefetch, `coherence_invalidations` the cache-to-cache fills from a foreign CCX — the observable coherence measure of the core PMU, not a literal invalidation counter. A vendor registry (`pmc_vendor_registry`) carries the PMC as *one* feature with two values — AMD and Intel are two hardware components, not one PMC — with a version identifier each, because a CEB is built on exactly one host and the built-in counter form co-determines its fingerprint; the planner's hardware probe detects per run on each machine for itself whether PMC exists, and the CI runs two counter paths, `pmc:amd` on prod1 and `pmc:intel` on prod2, the Intel Core i9-12900K (Alder Lake) with P- and E-cores. The separate measurement of the P- and E-cores is laid out as a scheduling dimension and via the core-pinning building blocks (six-mode model with HybridAware as mandatory mode, Section 3.4.1); implementation status: the runtime pinning per core type is planned but not implemented.

The host-side measurement cross-cutting concern of the system realm — the cross-cutting concern M, which permeates all layers without ever touching the binary identity (Section 3.4.1) — roots in the CRTP base `SystemAxis` (file `system_axis.hpp`), which docks only reading onto the host-side result structures. It is implemented as a zero-cost abstraction — an empty, non-polymorphic base without virtual calls in the hot path, whose conformance a dedicated concept enforces — and translates the concept invariants into compile-time completeness gates: a `consteval` gate checks that each of the sixteen measurement categories is assigned to exactly one of the two regimes `TimeObserver` and `PmcCounter` (nine time- and observer-based, seven counter-based PMC categories, Section 3.4.1), and a second one rejects every axis that mixes categories of both regimes — the regime-mixing ban thus breaks the compilation rather than only the evaluation.

The sixteen measurement categories are carried by a single `constexpr` registry (`measurement_axis_registry.hpp`): per category the name and the regime derived from the one source. A static gate enforces that the table index matches the category value, that no name is empty and that no regime deviates from the root assignment — a seventeenth category or a silent index drift thereby becomes a compilation error; the growth of the category set runs via the channel addresses of the axis-interface model (below), not past this check. Conceptually the categories are a *sub-axis* of the measurement tooling and manifest as columns of the result tables (Section 3.4.1); the registry names are at the same time the reporting vocabulary of the evaluation layer (the column names of the E4 reports), and a compile-time iteration over all entries makes the registry available to metaprogramming.

The acquisition abstracts a vendor-neutral measurement-source interface (`i_measurement_source.hpp`). `IMeasurementSource` describes a source by its *structural* capabilities — which

of the ten event semantics (cycles, instructions, L1/L2/L3 cache, dTLB and branch misses, memory stalls, coherence invalidations, energy) it can express at all — and delivers per measurement interval a `MeasuredDelta` with *per-event validity*: the value zero with the validity flag set is an actual zero measurement; an unset flag means *not measured*. The capability report never gives a measurement guarantee — the validity per measurement is carried by the validity flags alone. As a guaranteed, expressly *degraded* fallback there is the wall-clock source (`WallClockSource`): it delivers only a nanosecond proxy for the cycles and leaves all hardware events invalid; the existing PMC family is attached by the adapter `PmcSourceAdapter`, which advertises only the seven structurally expressible channels — L1d, L2, L3, dTLB and branch misses, coherence invalidations and energy —, while cycles, instructions and memory stalls are structurally not expressible in the counter POD and remain invalid.

In the axis-interface model, the measurement-source interface for micro-benchmarking is collected per axis interface specifically for its parameter set and gathered in stages over the macro benchmarking and the wallclock in the CEB at parameters; the category set thereby grows via the channel addresses axis → genus → category (hierarchy depth three) to hundreds of measurement categories, one per axis interface and parameter set. Implementation status: sixteen categories, statically fixed by the registry (above). Added to this is the *unthrottled re-measurement*: in an experiment, the actions of every axis are rolled back in order via the checkpoint of the first-called axis-interface function behind the genus interface through a control surface of its own, so that afterwards all axes can be measured anew from the entry point without measurement — for this the CEB, in single mode, takes over the inner state of a tier binary with measurement sensors into the analogous tier binary without measurement sensors (only the wallclock checkpoint) and measures anew from the same point, staggered by the load-profile input of the tier binary per macro-benchmarking interface, recursively. A simple re-traversal of different permutations does not suffice for this, and a complete restart allocates a new memory block and worsens the comparability if it is not possible to roll back exactly to the last checkpoint. Implementation status: planned but not implemented. The test criterion of the re-measurement is the plausibility invariant that the binary with measurement sensors is slower than the one without (Chapter 5).

The principle of explicitly coded non-observation (Section 3.4.1) — a missing value is a state of its own, encoded as a validity flag or as NaN according to IEEE 754 [IEE19], never a zero value; in the terminology of missing data it is *missing*, not *null* [Rub76] — is programmed into these building blocks rather than documented (named in code as the vocabulary `honest-0`, `honest-100` and `honest-empty` in comments and probe-dock messages; the measurement building blocks encode it as sample status `NotApplicable` and `SourceUnavailable` (`SampleStatus`) and as `*_source_available` flags in the counter POD). The wall-clock system axis refuses to label its mean as a percentile: it answers requests for p50 through p999 as invalid, for the total time divided by the operation count is a mean, and a mean reported as a percentile would be a phantom value — actual percentiles arise only downstream in the evaluation: nearest-rank over the collected raw latencies (the only conversion, the type-1 quantile according to Hyndman and Fan [HF96]), supplemented by the HDR histogram [Ten12] (high dynamic range), which at three significant digits resolves with a relative bucket tolerance of $1/1024 \approx 0.0977\%$ — derived from its bucket geometry: $2 \cdot 10^3$ values with unit resolution, $\lceil \log_2 2000 \rceil = 11$, hence 2^{10} sub-buckets per half —; in the measurement path itself deliberately no histogram bucket stands, because it would irreversibly quantize the raw value already at recording. The cache-line utilization (CLU) is collected as an actual utilization ratio — used payload bytes against occupied cache lines times line width,

the fields `field_bytes`, `cache_lines` and `line_bytes` of the axis schema —, not as a raw line counter, which would be precisely the *inverse* metric. Memory footprint and fill-buffer utilization remain invalid until an actual peak column or a hardware source exists (the software operation buffer of the buffering axis is no proxy for the hardware line-fill buffer), and IPC/CPI remains invalid as long as the counter POD possesses no instruction and cycle columns.

The hardware detection is the acquisition chain of the system main axis `target_isa` — its sub-axis `target_isa_complex` carries per named recombination the members CPU fabrication, RAM frequency and CAS latency — and at the same time a *measurement* axis (Section 3.4.3), whose planner view is the host probe; it is present as code in both stages. The first stage comprises the provenance vocabulary (the three trust levels `ConfiguredMeasured`, `SpdJedecBase` and `DeclaredNotMeasured` as a dedicated result type; “not collected” is deliberately not a fourth level but a state of its own), a DDR5 SPD parser (serial presence detect, the module data record according to JEDEC [JED25]) as pure, `constexpr`-capable byte functions over a passed-in memory range — 141 lines, without I/O and without vendoring of third-party code, with `std::expected` as result type; overclocking profiles (XMP, extreme memory profile) it recognizes as a separate offer, never as an actual value — as well as a dedicated error domain with four classes (`QuelleFehlt`, `QuelleUnlesbar`, `QuelleKorrupt`, `FormatUnbekannt`), so that a corrupt source never silently degrades to the next stage. The second stage wires the acquisition: the runtime chain boot cache (the filtered SMBIOS/DMI table of the system [Dis24]) → SPD → declaration is a chain of responsibility [GHJV94] without virtual calls — its members are heterogeneous types, their wiring is a type list [Ale01], so that the order (the essence of such a chain) resides in the type instead of in a runtime list, and a static check holds that every provenance level occurs exactly once; the chain stops at the highest reachable trust level, the delivering stage determines the provenance of the result (no member can claim a foreign origin), it receives its source paths by injection (every stage is testable without the target platform) and runs exclusively, buffered once, at the release seam of the builder. Which chain applies is decided by a compile-time factory (`hardware_probe_factory`) over the cell of ISA complex and operating system: its primary template is declared but not defined — a cell without a conscious decision is an incomplete type and breaks the build —, a totality check covers the full matrix of all declared ISA complexes and operating systems, and unimplemented cells (Windows, macOS) are declared declaration-only cells with expressly occupied upper levels, so that the degradation path reports “asked, does not exist here” instead of “skipped”.

The comparison between declaration and acquisition is decided by a named runtime verdict with exactly seven outcomes — match, deviation, declared only, not collected, no declaration, incomparable levels and label contradiction —, where only provenance-compatible quantities are compared at all (a fixed DMI result only against a configured measurement, a fixed SPD result only against the SPD base) and a deviation is mapped onto the error class “hardware extension missing”; no verdict and no measured value ever reaches the stamp or identity APIs (Section 3.4.4). With this, the automated resubmission demanded in Section 3.4.3 is implemented in code: the `prod2` value of 4800 MT/s declared in the system registry is a fixed `dmidecode` result and remains carried as a note of its origin (`DeclarationOrigin`), the code fixes not the number but only the presence of a value, and the acquisition takes place anew on every run and carries its trust level along — in the measurement, stage 1 delivers on `prod1` the configured clock of 5600 MT/s with XMP active and on `prod2` 4800 MT/s.

At the evaluation end of the cross-cutting concern, the first approach to the curve fitting from Section 3.6.3

is present as a skeleton (`curve_fit.hpp` in the builder): a measurement-axis curve over the working-set size is fitted by least squares to the model $y = a \log_2 x + b$ — the form expected for cache-hierarchy curves, whose steps map the capacity boundaries of the hierarchy [HS89]. Here too: no fit without valid data. The fit status distinguishes *no data*, *underdetermined* (fewer than two points or no spread of the x values) and *invalid data* (undefined logarithm, non-finite values) expressly from the success case — never an *Ok* with NaN coefficients —, and the result reader accepts only strictly numeric cells and counts skipped rows as a diagnostic instead of silently turning them into unoccupied points. On the same skeleton stands, per measurement curve, a cubic spline over the abscissa $\log_2 x$, optionally natural (twice continuously differentiable, but capable of overshoot) or as a monotonicity-preserving cubic Hermite spline according to Fritsch and Carlson [FC80], which on monotone data produces no spurious extrema and thus no false intersections; without at least two points with x spread no spline arises. The heuristics family next to it (`heuristik/`) contains the axis spline as a strategy seam with compile-time method choice, the break-even finder over the intersections of two curves of the same axis, the measurement-curve loader, the offline clustering of the workloads with its feature vector and the optimization catalogue of the axes; break-even curves are formed only over the w/ma/mi-measured organ-axis parameters. The curve formation itself is developed in Section 3.6.3; here stands only its code shape.

Implemented with this are the realm roots including order and re-entry guards, the registries, the source abstraction with degraded fallback, the two-stage hardware detection and the first fit approach with spline; the binding of these cross-cutting building blocks to runner and probe dock (the collector of the success parameters) and the mapping of the registry names onto the existing columns of the result files are not implemented. The three concept invariants of the system axes read: they *permute* the build and measurement space — the complex axis `build_target_complex` is the command wrapper of the recombination of the three main axes with the sub-axis group `build_toolchain` and the scheduling with five dimensions, and this system multiplier multiplies the organ space per platform —; they do not stand in the organ permutation tree of the `binary_id`; and they never touch the binary identity (registry XML: `binary_id = never`; only the eighteen organ main axes enter the eighteen-segment path). The construction itself preserves this: the root docks only reading onto the host-side result structures and remains invisible to the ABI boundary.

4.5.1 Pattern Register, Hybrid Stage and Store

The design patterns of the apparatus are not a random inventory but a register (Figure 4.12); the naming follows the catalogue of Gamma et al. [GHJV94] (“Gang of Four”, GoF), and every pattern is executed here at compile time instead of runtime-polymorphically, because the hot path may contain no runtime branching (Section 4.3). The pattern of every axis is *Strategy*: interchangeable building blocks behind a concept, bound via CRTP [Cop95] instead of via a virtual base. Plan emission is handled by *Director* and *Builder*: one walk, one builder per output syntax, plus the legend forms as formatting single source. An *Abstract Factory* constructs the docks of the hybrid stage at exactly one place and builds the result workbooks of the store (the `xlsx` workbook as standard; the CSV tree — one folder per sheet, always generated from the workbook — as a selectable strategy of the same factory, instead of the workbook or in addition to it; implementation status: the CSV projection is implemented as one flat file, the tree form is planned but not implemented). The measurement seam `IMessVisitor` at the `dlopen` transition is a *Visitor*: the host passes the sink in, the tier calls the matching `visit` method (double dispatch). The

hardware acquisition chain is a *Chain of Responsibility* with a type list instead of handler pointers, and likewise the selection filter chain of the CEB, which passes candidates through with the verdicts Pass, Reject and PassToNext. A *Memento* saves and restores the state of stateful axes per tier (`memento_all` after the warm-up; as a plan the state takeover of the hybrid stage, below). The pattern *Command* is the compile-time command base of the axes with measurement-visitor slot and limitation report — and the complex axis `build_target_complex` as command wrapper of the system recombination. A *Facade*, finally, bundles the appendix generation (program 08: a facade over the libraries 04 and 05, in-process) and the purely reading plan emission of the planner.

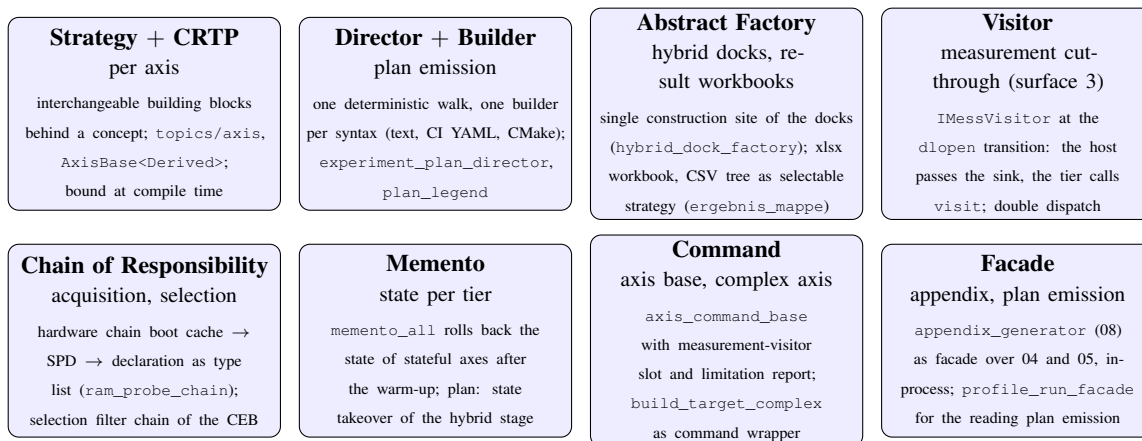


Figure 4.12: The register of the design patterns of the apparatus (naming after Gamma et al.): per pattern the role and the code location. All eight are executed at compile time — binding via concepts, CRTP and type lists instead of via virtual calls —, because tier binaries may contain no runtime branching between building blocks.

The hybrid stage is the fourth carrier, inserted as an optional mux stage between the CEB probe dock and the tier binary (Section 3.6.2); its code state lies under `hybrid/` (Figure 4.13). Implemented are the dock contracts — four contract forms `standard`, `rollback`, `scan` and `resource_control` as a `constexpr` registry with sixteen status constants (`ok` and fifteen named rejection reasons) —, the minimal probe dock, the dock factory as the single construction site, the dock array with `HybridDockVariant` (the one permitted variant site, Section 4.3) and the runtime cap `max_docks`, which rejects the $(N + 1)$ -th docking with “array full”, the parser of the `<hybrid_tier>` section (two-way switch enabled, exclusive per chain, `max_docks` mandatory with the stage switched on) and the binary proxy, which makes a hybrid `.so` an ordinary tier binary to the outside: it satisfies `IAnatomyBase`, holds the dock array and passes the drive through to the target that sits at a dock. Not implemented are the `.so` export of the hybrid stage (it depends on an open layering requirement, because a hybrid `.so` would have to link builder code), the eviction and the router (it needs actual measurement curves). Measured against the file list that the README of the directory `hybrid/` envisages for the dock, proxy and router layer, six of the nine files are implemented (dock contracts, probe dock, dock factory, dock array, XML parser, binary proxy) and three are not (the tier module of the `.so` export — the two module sources under `tests/unit` demonstrate only the buildability of the module ABI —, the eviction and the router); besides these, the directory carries the four headers of the classification layer (classification, concept gate, strategy per genus, synthesis matrix) and three further headers for docking, the module ABI and the stamp chain, thirteen headers in total. The Hybrid Module ABI v1 exports the same mandatory symbols as the

plain tiers including family and genus, and `comdare_anatomy_genus` reports the *target* genus (what the module is for the caller), never the reroute genus of the classification — the same family-independent loader loads the hybrid `.so` via the one loading route; the stamp chain of the hybrid stage and the hybrid positions of the measurement-gate member (`hm/hmi`) carry its identity, and the genus build admission carries six entries, the sixth for the reroute genus with the dock cap (default 32) as arity. The hybrid live ranking is planned as the fourth evaluation component of the compare stage, and the state reconciliation of the hybrid stage is planned via memento with two release artefacts, single and hybrid, in the store (Section 3.6.2); implementation status: neither is implemented, nor is the fix of the schema drift that the XSD does not carry the `<hybrid_tier>` section. The hybrid mode compiles only with the help of the CEB.

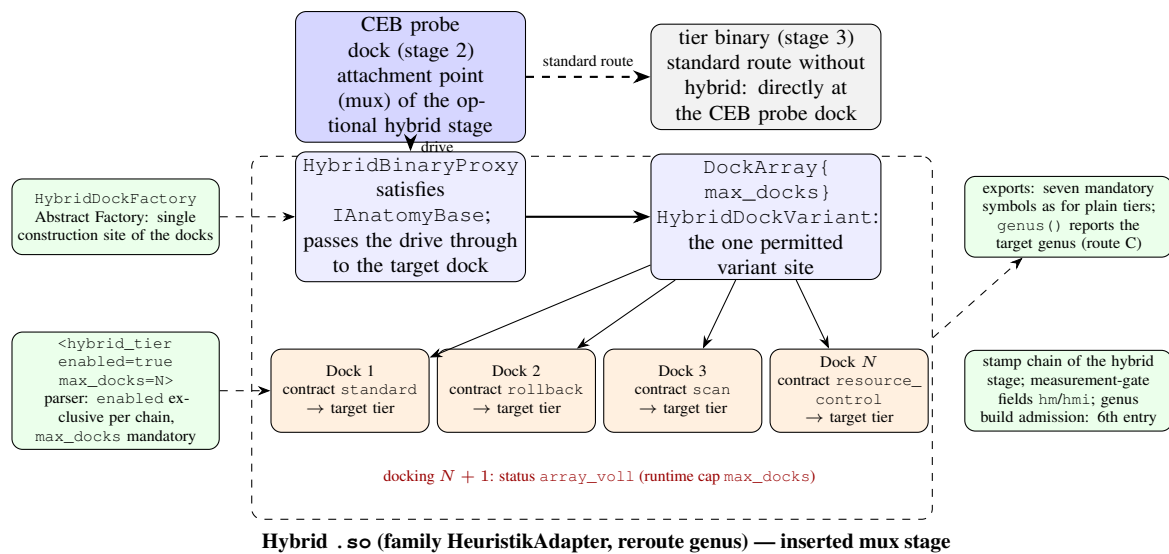


Figure 4.13: The hybrid stage in code: the mux stage optionally inserted between the CEB probe dock and the tier binary, attached at the CEB probe dock and carrying dock slots for tier binaries (standard route without hybrid: probe dock directly to the tier binary). The binary proxy makes the hybrid `.so` an ordinary tier binary to the outside and holds the dock array with the runtime cap `max_docks`; per dock one of four contracts applies, the dock factory is the single construction site, the `<hybrid_tier>` section of the XML switches the stage and sets the cap. The exports are spelled identically to the plain tiers, `genus()` reports the target genus. Not implemented: `.so` export, eviction, router.

The store (Section 3.4.5) has in code a compile-time-static key schema (`bestand_schluessel_schema`) that declares the four stocks without itself implementing store mechanics: stock 1, the binaries, is keyed via a key policy whose single SHA-512 key is identical to the tier fingerprint, the cell bracketed beside it; stock 2, the measured values, via the 128-hex fingerprint of the measured binary and the hardware identity of the measuring machine, the cell bracketed beside it and never fused into the digest; stock 3, the synthesis batches, and stock 4, the solutions, are declared as a work step of the full expansion of the store (Table 4.5). Drift between schema and mechanics breaks when the mechanics are built, not only in the store. The version lock (`axis_version.lock`, format v3, 719 records of three lines each) binds every source file of the axes — the heuristics headers and the overlay cut over which the overlay member of the tier fingerprint hashes — to its content digest and its stamped algorithm version: a deviation between lock and discovery is a failure, a downgrade of a version-bearing header to digest-only

is a failure without a write path, an upgrade is legitimate and demands the deliberate regeneration of the lock; the CI gate `contract:axis-version-lock` enforces this. From the schema follow the store properties of the concept: the measurement is always switched on, in the store only what is missing there is re-measured; a version change of an axis changes stamp and fingerprint and rebuilds the affected store stack; and because the hardware identity stands in the measured-value key, every machine type has its own stock of measured values. Implementation status: schema, lock and the stocks 1 and 2 are implemented; the full expansion — planner leaf function, CEB full placement, hybrid storage, commit skip, rebuild flag, XML target choice — and the binaries with and without measurement sensors together with curve files as store entries are planned but not implemented.

Stock	Key components	Source in code	Implementation status
1 binaries	the one SHA-512 key (= tier fingerprint); cell bracket beside it	key policy of the stock log	implemented
2 measured values	128-hex fingerprint of the measured binary; hardware identity of the measuring machine; cell bracketed beside it, never fused	measured-value key source of the stock log	implemented
3 synthesis batches	machine; full stamp; reference to measurement level and channel	schema declared	not implemented (full expansion)
4 solutions	canonicalized hash of the user XML; machine; references to the participating stocks	schema declared	not implemented (full expansion)

Table 4.5: The four store stocks and their key components according to the compile-time-static key schema; the version lock binds the axis sources to digest and stamped version.

With this, the apparatus is fully described — from the four repositories through the candidate under test, the eighteen organ axes and the carrier-stage chain to the chain from plan to tier binary, its contract surfaces, the layer-wise quality assurance, the realm roots in code, the pattern register, the hybrid stage and the store. Chapter 5 lays out the methodology by which this apparatus is assessed.

5 Evaluation

The evaluation first discloses the methodology and, building on it, analyses the measurement series. Its guiding question is cache-line awareness: Which design decision changes the cache-near measured quantities — L1, L2, and last-level misses, dTLB misses, branch misses, and the latency per genus-interface call — on which platform, and to what degree? Three evaluation objects structure the chapter: the *design space* (the hypotheses H1–H3 and the axis sensitivity analysis), the *apparatus* itself (the heuristic counter-proof and the measurement error that the measurement instances introduce), and the *automation* of the evaluation from the experiment XML, for at its core the thesis automates precisely this evaluation. The methodology part (Section 5.1) unfolds, in eight subsections, hypotheses and measured quantities, the XML-driven chain, workloads and datasets, platforms and regimes, measurement levels and counters, build scope, fairness, and the counter-proof; the results part (Section 5.2) presents, in seven subsections, the identity of the results, the data flow, the measurement series, the available measurements, the sensitivity analysis, the validity check, and the summary.

The evaluation follows the chain customary in empirical software and systems research of research question, hypothesis, measured quantity, experimental setup, execution, result, validity check, and discussion [WRH⁺12, Sha03]: every hypothesis names its measured quantity and its acceptance criterion before the measurement, every experimental setup its platform, its regime, and its repetitions, and every result its uncertainty [GBE07, KJ13]. Because the object is an artefact — the measurement apparatus with its axes —, the assessment of the artefact according to the rules of design-science research¹ is added: utility, rigour, and traceability [HMPR04, PTRC07, Wie14]. Performance measurements are reported according to the reporting rules of Hoeffler and Belli [HB15] and Blackburn et al. [BDH⁺16]: reference quantity, repetitions, dispersion, platform, and values not collected (n/a) stand at every result. The three evaluation objects structure the subsections; cache-line awareness is their common guiding axis.

The guiding axis of the evaluation is cache-line awareness: Compared are the cache-near measured quantities — L1, L2, and last-level misses, dTLB misses, branch misses, energy, and the latency percentiles per genus-interface function; they are sixteen registered measurement categories with one regime ordinal per category — per axis assignment, per family- and genus-interface call (the channel address always reads axis → genus → category), and per platform, that is, per registered machine signature: prod1 (AMD Zen 5), prod2 (Intel Core i9-12900K, Alder Lake), and the ODROID board with Intel Gracemont cores. The question is not which algorithm is faster, but which axis moves the cache-line-related costs on which microarchitecture. The PMC categories (performance monitoring counters, the hardware counters of the processors) are the primary measured quantities for this, the time categories the cross-check; because hardware counters are trustworthy only after a cross-check by measurement [WM08], Section 5.1.5 discloses the semantics of every counter per model.

¹Design science: the research direction of information systems that makes a constructed artefact (here the measurement apparatus) itself the object of assessment and demands for it utility, rigour of procedure, and traceability [HMPR04].

5.1 Methodology

5.1.1 Hypotheses and Measured Quantities

Three *hypotheses* guide the analysis; each names its measured quantity and its acceptance criterion before the measurement:

H1 (page-type cost) The mean access cost per page-type variant differs systematically by workload: dense page types win, as expected, under Zipf distribution (YCSB-A), range-optimized pages under range scans (YCSB-E). The measured quantities are the latency percentiles and the cache-miss categories per workload; H1 counts as supported if the ranking of the page-type variants flips between YCSB-A and YCSB-E on the same platform and the margin exceeds the uncertainty band of the build variants (Section 5.1.8).

H2 (code quality per source) The code-quality score inherited from the original source repository correlates measurably with the achieved throughput. Two instruments supply the score: first, the documented manual audit per source repository over seven assessment axes (style, tests, documentation, maintainability, maintenance status, licence, build system; scale 1–5) for 22 assessed sources — a record among the audits of the cache-engine repository —, second, the tool-computed instrument that actually enters the measurement row: the weighted `cppcheck` finding density per thousand physical lines (weights error 3, warning 2, performance 1, portability 1, style 0.5) over the vendored original sources; the record lists 33 entries, 22 of them as n/a because the reconstruction possesses no vendored original source, and the value travels as column `h2_code_quality_score` of the WIDE schema (kept in the code as field `h2_score`, record `sota_h2_scores.xml`) in every measurement row of a SOTA series. Both are complemented by the documented per-function `is_original` marking (Appendix D). Acceptance criterion: H2 counts as supported if the tool-computed score over the substantiated SOTA profiles shows, with the throughput in the same cell (platform, workload, regime), a rank-correlation coefficient beyond a threshold named before the measurement; n/a profiles do not enter the correlation, and the manual audit is the qualitative classification.

H3 (inline vs. external vs. chain distribution) The observed distribution of the three `ValueHandle` variants correlates with page density: dense pages prefer inline, sparse pages external, and PRT-ART heuristics `ChainRef`. The registry of the axis `value_handle` lists five building blocks (Inline, External-Pool, Immutable-Shared-Ref, Versioned-Pointer, Chain-Ref; only Inline is bound to the golden reference catalogue); H3 regards the three variants decisive for PRT-ART as a reasoned excerpt. The measured quantity is the share per variant over page density; H3 counts as supported if the share moves monotonically with density.

These *evaluation hypotheses* are not to be confused with the identically named PRT-ART telemetry metrics H1–H3 (Section 3.5): the former are testable statements about the design space, the latter measurement organs of the candidate under test that, among other things, supply the data for this test.

5.1.2 Fully Automatic Experiment Evaluation from the XML

The evaluation is itself an object of the automation (Figure 5.1): A user XML with the root `<comdare_experiment>` declares *what* is measured, *where*, and *when* — never the *how*. Decisive for the evaluation are the sections `workloads`, `datasets`, `measurement_tooling`, `measurement_categories`, `run_methodology`, `writeback_methods`, and `output`; the profile validator checks them before every run against the registries (axis names and values, merge bindings, fairness modes, records). From the XML the plan arises deterministically — the director walk of the planner, retrievable as a pure text emission even without a build —, from it the build jobs of the CacheEngineBuilder and the tier binaries, whose measurement is deposited in the workbook (xlsx); the CSV form is the ordered tree of CSV files generated from the workbook (one folder per sheet, therein the files of the individual sub-axis sweeps; not a single child document), and the generators of the tool repository (binary-to-CSV, CSV-to-LaTeX, diagram generator, appendix generator) render fact sheets, tables, and TikZ diagrams per language into a `tabellen` directory that the manuscript includes. No step of this chain is executed by hand; every number in the appendix carries the run that produced it (Appendix A). Implementation status: The chain is demonstrated in a first end-to-end run — the smoke measurement series over 43 permutations in the appendix is its product —; the XML section `<publish>` for the store write-back, the command-line-guided creation mode of the XML (Section 4.3.2), and the third output factory LaTeX (Section 5.2.2) are planned but not implemented.

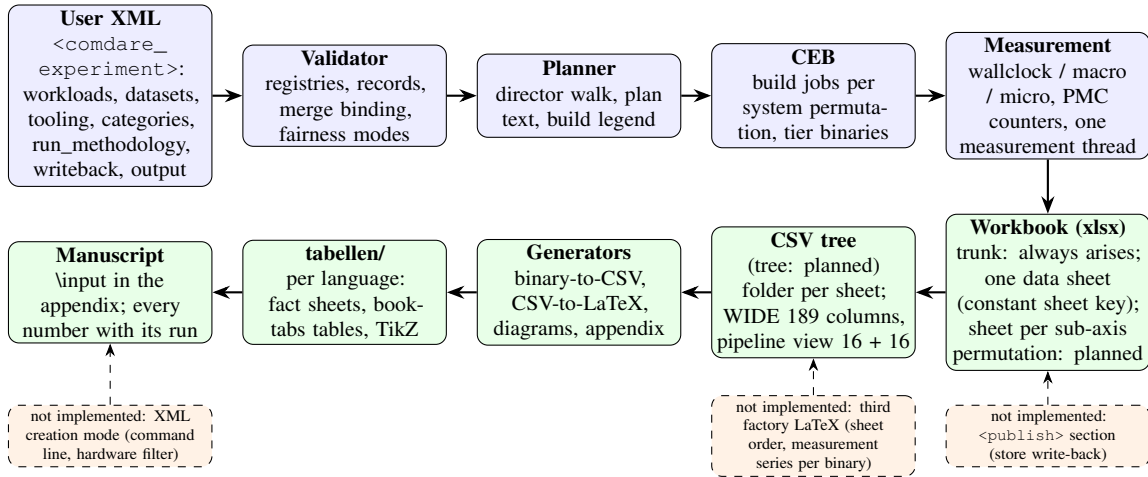


Figure 5.1: The automated evaluation chain from the user XML to the manuscript. Upper row: The XML declares what, where, and when; the validator checks it against the registries; the planner emits the deterministic plan; the CEB builds the tier binaries per system permutation; the measurement collects the three measurement levels and the counters with one measurement thread. Lower row: The workbook is the trunk (one data sheet under a constant sheet key; the fan-out into one sheet per sub-axis permutation is planned but not implemented), the CSV form its ordered tree of CSV files generated from it (one folder per sheet; not a single child document; implemented is a flat file), the generators render per language into the `tabellen` directory, the manuscript includes the files — every number carries its run. Dashed: parts not implemented (XML creation mode, `<publish>` section, third factory).

5.1.3 Workloads and Datasets

Phase 5 of the measurement pipeline (Execute, Section 4.3.2) derives the YCSB workload per permutation as a fallback heuristic from the profile’s traversal tag (Table 5.1), unless an `<expected_workload>` tag of the profile overrides the heuristic (Section 3.2); the routing matches by substring (`RANGE`, `HOT_PATH`) and not over a closed tag list, and the value of the `<expected_workload>` tag is resolved against the six tokens `YCSB_A` to `YCSB_F`. The global default workload is the permutation over all available workloads; the runtime dimension `workload` is registered for this as a dynamic dimension of the measurement registry, fed from the load framework YCSB [CST⁺10]. Implementation status: For profile-less modules the driver falls back to exactly one workload, the run’s workload option; the full permutation is planned but not implemented. The data basis is real string datasets of differing characteristics (Table 5.2), each with a reproducibility record of source, checksum, line count, preprocessing, and seed rule for the synthetic miss, update, and delete keys; four of the six datasets (`url`, `protein`, `tpcds-id`, `trec-terms`) possess a record actually computed against the preprocessed files, `dna` the record of the Pizza&Chili source, and `xml` possesses a specification without checksum because the file is not available locally — there is no synthetic checksum (n/a). The records lie as XML files per dataset in the tool repository, and a canon test checks the computed records against the six-element canon of the table.

Table 5.1: Profile-aware workload routing (fallback heuristic)

Traversal tag	YCSB workload	Characteristic
<code>RANGE_SCAN / RANGE_HOT_PATH</code>	YCSB-E	range queries
<code>HOT_PATH / PARTIAL_HOT_PATH</code>	YCSB-A	50/50 read/update, Zipf
<code>STANDARD (default)</code>	YCSB-C	read-only

Table 5.2: Real datasets, their characteristics, and their reproducibility record

Dataset	Characteristic	Record
<code>url</code>	very long keys, long common prefixes, medium alphabet	real
<code>dna</code>	small alphabet, high repetition, deep prefix entropy	Pizza&Chili record
<code>protein</code>	biological sequences, small to medium alphabet	real
<code>xml</code>	structured strings with recurring prefixes	n/a (file not local)
<code>tpcds-id</code>	database-like IDs, sorted and unsorted	real
<code>trec-terms</code>	dictionary/search-index terms with defined miss keys	real

5.1.4 Platforms, Vendor Lanes, and Regimes

The measurements target the registered measurement machines of the apparatus: `prod1`, the AMD machine with a Ryzen 9 9950X3D (Zen 5; machine signature `prod1_zen5` with 22 features including AVX-512), at the same time the development workstation of the verification; `prod2`, the Intel machine with a Core i9-12900K (Alder Lake; signature `prod2_alder_lake` with nine features, without AVX-512, with P and E cores); and an ODR0ID board with Intel Gracemont cores (signature `odroid_gracemont`, nine features). The ZIH cluster of TU Dresden is declared as sub-axis IS3 of the target ISA (Barnard, Capella, GraceHopper, N/A); the Barnard node possesses Intel Sapphire Rapids CPUs, two Xeon Platinum 8470 per node [Zen26a], and Capella is the GPU cluster with AMD Genoa CPUs and NVIDIA H100. In no

machine signature is a ZIH node registered, and a run there is not measured (n/a) — in accordance with Section 3.4. An HBM platform is registered in no signature and in no axis as a platform; the platform probe carries an HBM feature and the allocator axis knows PIM special cases, nothing more — it is planned but not registered. An `IPlatformProbe` (Phase 1) reports per platform the runnable subset of the axes (for instance AVX-512 only where available), whereby the exclusion of an unavailable extension is always decided *machine-side* — with a warning and a named error class, never as a silent profile filter, for the experiment profile stays identical for all machines; per platform a scalar and an AVX2 build are prepared, an AVX-512 build only where the signature carries the feature. The available axis options are checkable per platform via the planner probe; a command-line-guided XML creation mode with platform filter is planned but not implemented (Section 4.3.2). On hybrid CPUs, P and E cores are measured separately rather than aggregated together, on Intel as on AMD; the measurement mechanics for this are the core PMU domains of the kernel (on Intel the sysfs names `cpu_core` and `cpu_atom`), while the naming of the sub-axis stays vendor-neutral — the sysfs names stand explicitly on the prohibition list of the ISA sub-axis labels. The core modes follow the six-mode model from Section 3.4.1: four pinning modes per core type (P or E cores, a single core or all cores of the type), the unpinned reference run, and *HybridAware* as the sixth mode (scheduling dimension `hetero_core_dispatch`), which is mandatory for the cache-line-awareness measurement; the attachment of the core pinning and the core modes to the run profile is planned but not implemented (Section 4.5).

The productive measurement operation is a *two-machine operation* — the AMD machine `prod1` alongside the Intel machine `prod2` —, and the counter-based collection carries for this two vendor lanes, each individually activatable, `pmc:amd` on `prod1` and `pmc:intel` on `prod2`, each with its own PMU resource group; the lanes deliberately run in parallel, because different hosts possess different PMUs and a shared group would serialize them without reason. Completeness is an evaluation rule, not a CI gate: A measurement series counts as complete only once both lanes have proven and delivered their counter prerequisites; a lane run on one side only is reported as a partial measurement, never silently carried as a whole (Section 5.2.6). The measurement plan moreover provides, on the production target machines, *two operating-system regimes* for every measurement, to separate production fidelity from measurement depth: An *immutable* operating system (Talos) mirrors production use and enforces boot-/compile-static cache settings (Section 2.2); a *root Linux with full hardware-counter access* (`perf` and the model-specific registers, MSR), by contrast, is what opens access in the first place to the counter-based measurement categories (Section 3.2) and to the runtime-configurable cache-near axis parameters (prefetch distance, hardware prefetcher). The second regime is *designed, not carried out*: The reported measurements all originate from the root-Linux regime of the two production machines; the Talos regime is planned but not measured.

5.1.5 Measurement Levels, Collectors, and Counters

To be separated from this is which counters that access actually populates (Table 5.3): The measurement chain opens via `perf_event_open` [Lin26b, dM10] four generic counters — L1D misses, last-level misses, dTLB misses, and branch misses (`PERF_COUNT_HW_BRANCH_MISSES`, portable on Intel and AMD). For L2 misses and coherence invalidations there is no portable generic counter; both are

collected model-bound via a RAW event catalogue whose encodings are cross-checked: For Zen 5² `l2_cache_req_stat.ic_dc_miss_in_l2` is the event 0x64 with mask 0x09 (demand misses in L2 without L2 prefetch) and `ls_dmnd_fills_from_sys.remote_cache` the event 0x43 with mask 0x14 (demand fills from the cache of another CCX, a core complex with its own L3); a literal invalidation counter does not exist in the core event set, the column carries exactly this assignment. The encodings stem from the event table of the Linux kernel for Zen 5 [Lin25], from which `perf` binds the named events. Models without a catalogue entry carry n/a, never a guessed encoding; for Intel no entry exists, because the cross-check on an Intel host is not available. The last-level event fails on Zen 5 at opening (ENOENT). IPC/CPI is declared as a category but not wired up — the counter bundle lacks the sources for instructions and cycles —, and the energy draw is read best-effort from the RAPL sysfs zone³ and stays empty without read permission on the zone. The principle of explicitly coded non-observation is realized as a field flag: Per counter the counter bundle carries a `*_source_available` flag, seven flags for seven counters; a number means that the source has delivered — a measured zero included —, n/a means that the source is missing, and this distinction travels end-to-end into workbook, tables, and diagrams. Hardware counters thereby count as trustworthy only after cross-checking [WM08]; the cross-check for Zen 5 is substantiated by measurement, the one for Intel is not available.

Category (column)	Source	Intel status	Validity flag
<code>cache_misses_l1</code>	generic perf event (L1D read miss)	generic: yes	own flag
<code>cache_misses_l2</code>	RAW catalogue Zen 5: event 0x64, mask 0x09 (demand misses in L2)	no catalogue entry, cross-check not performed (n/a)	own flag
<code>cache_misses_l3</code>	generic perf event (last level); fails on Zen 5 at opening (n/a)	generic: yes	own flag
<code>dtlb_misses</code>	generic perf event (dTLB read miss)	generic: yes	own flag
<code>branch_misses</code>	generic perf event (PERF_COUNT_HW_BRANCH_MISSES)	generic: yes	own flag
<code>coherence_invalidations</code>	RAW catalogue Zen 5: event 0x43, mask 0x14 (foreign-CCX fills)	no catalogue entry, cross-check not performed (n/a)	own flag
<code>energy_micro_joules</code>	RAPL sysfs zone, best-effort (read permission)	as AMD (read permission)	own flag
IPC/CPI	declared, not wired up (source missing)	n/a	—

Table 5.3: The counters of the measurement chain and their sources: four generic perf events (portable), two model-bound RAW events from the catalogue for Zen 5 (cross-checked by measurement), the RAPL energy as best-effort reading, and IPC/CPI as a declared, not wired-up category. Per counter the counter bundle carries a `*_source_available` flag; a number means “source has delivered”, n/a means “source is missing”. The Intel status names what is collected on prod2 without a catalogue entry.

Time and observer measurement are united in the measurement checkpoint (`checkpoint_measure`); their responsibilities are separated by the measurement-tooling main axis in exactly this order: *wallclock* (w) — the wall clock of the overall runs in the CacheEngineBuilder over the load frameworks, parameter-free; *macro* (ma) — the time of the genus-interface functions of the tier binary; *micro* (mi) — the time and

²Zen 5: the AMD microarchitecture of the Ryzen 9 9950X3D (Granite Ridge, CPUID family 26); prod1 possesses two L3 complexes of unequal size.

³RAPL (Running Average Power Limit): the energy counters of the processors, which the operating system reads out via the powercap interface in microjoules [DGH⁺10, KHN⁺18]; without read permission on the zone the value stays empty.

the success parameters per axis interface (Section 3.4, Figure 3.9); subsets of the three levels are allowed, every other order is syntactically wrong, and w/ma/mi is the registered alias. Three collectors take over the collection as building blocks of the measurement axis `collector` — wall clock, observer snapshot, and PMC —, and the sixteen measurement categories are the sub-axis of the tooling axis, not its main fan-out. The analysis keeps the three levels apart: axis interface, family and genus interface, and the wall clock of the CacheEngineBuilder are never mixed (Section 5.2.5); PMC measurements as a figure type of their own in the appendix generator are planned but not implemented. Implementation status: Registry and level canon are implemented; the order validation at the three serialization points and the collector as a self-contained component are not.

The time- and observer-based categories run identically under both regimes and secure comparability; the PMC categories are collected only under the privileged regime. Per category the registry carries the regime as an ordinal (0 for time and observer, 1 for counters): nine time/observer categories and seven counter categories. Operating system, kernel, and counter domain are to be logged per platform: The operating-system probe collects kernel and identifier of the system as a system axis, and the counter source records per counter on which core PMU domain it opened (generic or, on hybrid machines, the sysfs name of the domain); a column of their own in the measurement output these three items do not possess in the WIDE schema — the columns `platform` and `build_version` carry platform tag and build version per row, the log column per domain is not implemented.

5.1.6 Build Scope and Optimization Levels

The scope of the full build is fixed in the profile. The profile declares the build control quantities: two optimization levels (O2, O3) times two SIMD levels (no extension, AVX2), that is, four build variants per permutation (Section 3.4.1). Both levels are declared in the system registry as IEEE-754-deterministic; the registry carries five levels (O0 to O3 and Ofast), and excluded from the profile is Ofast alone, because it would break the floating-point determinism according to IEEE 754 and the run-to-run determinism of the apparatus as well as the CRC-64 checksum of the golden reference catalogue — the registry marks exactly this level as non-deterministic. Maximum optimization thus remains selectable: The default of the build is globally O2, O3 is selected via the XML and built alongside, and the selected level is identity-effective via the toolchain member of the fingerprint, so that O2 and O3 stocks are never tacitly confused as cache hits; O0 to O3 as a user choice in the XML and the pass-through of further compiler flags are planned but not implemented. Per build variant the golden reference catalogue is built: $2^{17} = 131072$ binary identities, because the golden profile activates for seventeen organ main axes two representatives each from the offering of the organ registry and the eighteenth (`persistence_target`) is fixed (Section 3.4.6); the system axes multiply the build matrix, never the `binary_id`. The cardinality of the real permutation space the planner computes dynamically from its inputs (subcommand `simulate`); it is not a fixed number. Results of the runs of the full build are not available (Section 5.2.4).

5.1.7 Fairness and Reproducibility

Fair comparability is secured by fixed rules. Each comparison separates a common minimal mode (common denominator: external value handles, no PRT-ART special paths) from the PRT-ART native mode (inline switching and adaptive page-type selection per branch point); the user XML declares the

mode per SOTA series as `fairness_mode` with the two permitted values `common_denominator` and `native`, and the validator rejects every other value. Implementation status: The mode is a series metadatum without influence on the measurement — it travels as a tag column of the measurement row and in the resumption stamp, never in the `binary_id` —, the composition definition of the minimal mode is data-bound, and two series of the same candidate under test with different modes therefore produce the same `binary_id`, which the validator reports as a warning. All methods receive identical, normalized byte keys as well as identical miss, update, and delete sets; the warm-up is the two-phase driver of the load-profile orchestrator (save, warm up, roll back, measure), otherwise measurement is cold, and each configuration is measured in several separate runs (repetition dimension `repetition`). Thread pinning: The actuator — a range-bound pinning of the measurement thread — is implemented; the core choice within the core complex is actuator policy without an emission contract, and the measurement emission of the planner reports the pinning as planned and the actual state as *unpinned*, because the actuator is not attached to the run profile — on `prod1` with its two L3 complexes of unequal size (96 and 32 MiB) an unpinned run is not reproducible, which the emission says literally. Missing capabilities of a baseline — for instance range scans of a hash table — are reported as *n/a* rather than covertly emulated; the mechanics for this are the sample statuses *NotApplicable*, *SourceUnavailable*, and *Failed* of the measurement axes, so that missing capabilities and missing sources carry *n/a*, never a zero. Provenance: toolchain (compiler identifier and version), flags, ISA summary, and the commit states of the four repositories are captured at configuration time as build provenance and written into the provenance manifest of the host exports; the allocator is, as an organ axis, part of every `binary_id`; a provenance per individual foreign library — its own compiler, its flags, its commit hash — is not logged (not implemented). The latency percentiles are determined by the evaluation tree of the cache engine nearest-rank from the raw values — Hyndman and Fan, type 1: $\text{rank } k = \lceil q \cdot n \rceil - 1$, without interpolation [HF96] —, so that every percentile is an actually measured value⁴; the presentation tools of the tool repository possess their own copies of the conversion (a declared gap), and the HDR histogram [Ten12] is a method of its own of the distributional view (Section 4.5), not a complement of the nearest rank. Layout and environment effects that produce wrong conclusions without a recognizable error [MDHS09] the apparatus counters with identical keys, identical build matrix per variant, and the repetition dimension; a randomized layout stabilization in the sense of Curtsinger and Berger [CB13] is not part of the apparatus and is carried in Section 5.2.6 as a limit.

For foreign libraries the original-compiler rule applies: Dissected axis functions are bound in at compile time and compiled into a once-again original binary block, so that every dissected sub-algorithm keeps its performance properties; where feasible or comparable, the foreign source is built with its original compiler. The proof of originality is implemented: the `is_original` marking per function (a macro of the cache engine together with the mixin base of the axis building blocks), the validator as a program of its own, and the manual relock gate of the CI, which re-anchors the lock files of the markings; the evidence per building block stands in Appendix D. The H2 dossier marks reconstructions without a vendored original source as *n/a* (22 of 33, Section 5.1.1). Implementation status: The automatic intake of the vendored sources into the build with their original compiler is planned but not implemented.

The measurement *operation*, too, is part of these fairness rules, and it runs through the store. The

⁴Nearest rank: The q -quantile is the smallest sample value below which at least the share q of the sample lies; interpolating methods (type 7), by contrast, produce values that were never measured.

store is the stock of all built tier binaries and their measured values, addressed via compile-time keys (Section 3.4.5; Figure 3.14 shows its structure): stock 1 keeps the binary under its SHA-512 fingerprint, stock 2 the measured values under the pair of the fingerprint of the measured binary and the hardware identity of the measuring machine; the cell of ISA, optimization, and extension stands bracketed beside it, never in the digest. Because the measurement-gate state is a fingerprint member, the binary with and the binary without measurement sensors lie side by side as two entries of stock 1. Measurement is always switched on; in the store only what is missing there is re-measured: A binary found is skipped if its fingerprint is known and its passed test attestation is inventoried at the binary, otherwise it is built or measured (Figure 5.2). The CEB controls the parallelism by producing parallel requests to the tier binaries — the execution threads arise in the CEB, in a worker pool with deterministic result order —, while the threading of the family stays in the tier binary. Implementation status: Stock 1 and 2 as well as the key schema of all four stocks are implemented; stock 3 and 4 (synthesized curves as files, solutions per user XML), the planner sheet function, the full placement of the CEB, the hybrid storage path, the commit skip, the rebuild flag, and the XML section `<publish>` form the full store build-out and are not implemented.

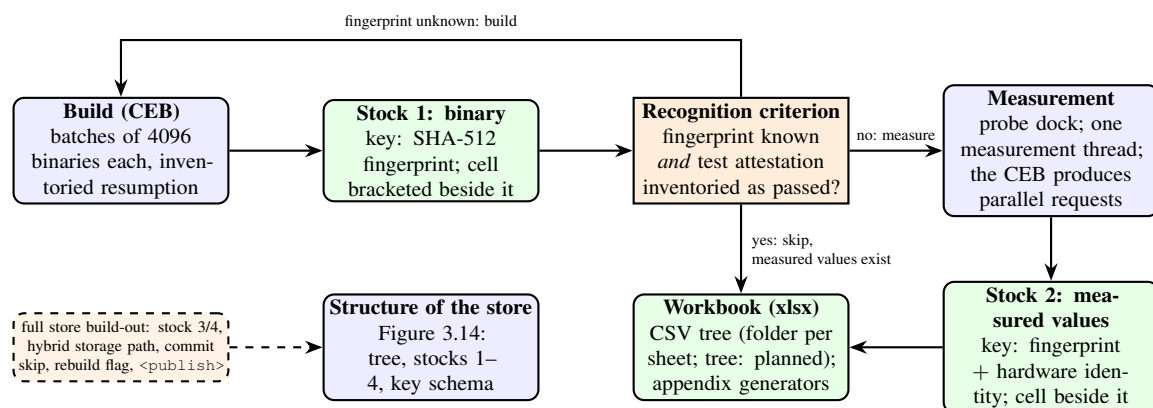


Figure 5.2: The store cycle in measurement operation: The CEB builds in batches of 4096 binaries each; every binary lies under its SHA-512 fingerprint in stock 1; the recognition criterion skips what is inventoried with a passed test attestation and otherwise lets build or measure; the measurement runs at the probe dock with one measurement thread, while the CEB produces the parallel requests; the measured values lie under fingerprint and hardware identity in stock 2 and flow into the workbook (the CSV tree form with one folder per sheet is planned; implemented is a flat file). The structure of the store (tree, four stocks, key schema) is shown by Figure 3.14; dashed the full store build-out.

Measurement remains strictly separated from building: measurement runs never start automatically with an error-free build but are triggered explicitly — an error-free build proves buildability, not a measurement result. The run methodology knows four work modes — `build`, `measure`, `compare`, and `release` (Section 4.3.2) —; the measure mode measures with exactly *one* measurement thread, a parallel measurement execution is armed via no registry row and reachable only via the state injection of the run entry, and `-debug` is a switch of the planner shell for the wiring check, which stays locked for users, not a work mode; the threading of the family itself is thereby left to the tier binary rather than overridden by the harness. A binary found in the store is skipped only if its passed test attestation is inventoried at the binary (Figure 5.2), and every measurement is preceded by the conformity check

at the probe dock (Section 4.4): the conformity gate of the genus contract before the post-compile attestations of the measurement interfaces, beside it the consistency gate of the measurement; build as well as measurement runs are processed per machine in milestone batches of 4096 binaries each at a stretch — the grain is a compile-time constant whose deviation is a compile error —, with an evenly distributed allocation of the jointly buildable binaries across the machines and inventoried resumption after interruption.

5.1.8 The Heuristic Counter-Proof as Fourth Dimension

Beyond the three hypotheses, the analysis receives a fourth, self-contained *evaluation dimension*: the heuristic counter-proof. Its concept is developed in Chapter 3 (Sections 3.6.2 and 3.6.3): The CacheEngineBuilder stage compiles the heuristically estimated recombination of the tier binaries into a hybrid tier binary — the hybrid module emitter lies in the builder tree of the CEB —, and the hybrid tier binary is a carrier stage of its own that is optionally inserted between the CEB probe dock and the tier binary: it docks at the probe dock, carries dock slots for tier binaries, and itself produces no code. The measurement-curve type system supplies the estimation basis (Figure 5.3): per axis one curve over its parameters and per measurement level wallclock, macro, and micro, from these the break-even points and the estimated recombination; the measurement-error detection is formulated by Section 3.6.3. The assessment question of this dimension reads: “Is the heuristically estimated recombination of the tier binaries faster than the existing paper algorithms?” Unlike H1–H3, which test statements about the design space, the counter-proof tests the apparatus of the thesis itself: whether the recombination estimated from the measurement curves reaches, in actual wall-clock behaviour, the performance predicted by the curves; it therefore stands *beside* the ranking of the sensitivity analysis (Section 5.2.5), not within it. Implementation status: Classification layer, dock layer, XML parser, and binary proxy of the hybrid stage are implemented (six of the nine files of the dock, proxy, and router layer envisaged in the README of the directory `hybrid/`, Section 4.5.1); router and eviction presuppose measurement curves and are not implemented.

The test plan for this is fixed in two stages; the corresponding measurement runs are not available (Section 5.2.4). The first stage quantifies the measurement error that the measurement instances themselves introduce, for the heuristic’s estimation basis is *observed* curves, while its claim concerns unobserved production behaviour. To this end, the same experiment is set up in three build variants:

1. observers in *all* tier binaries *and* in the heuristic hybrid tier,
2. observers *only* in the heuristic tier binary,
3. observers in *none* of the compiled artefacts — only the wall clock is measured.

The variants are kept apart via the measurement-gate member of the fingerprint (Section 3.4.4): It carries fields of its own for the tier level (`tm`, `tmi`) and the hybrid level (`hm`, `hmi`); the off state of all fields reads `mg=m0;s0;st0;x0;tw0;tm0;tmi0;hm0;hmi0`, and because the member enters the digest, the three variants lie side by side in the store as three binaries with three fingerprints. The differences between the three variants quantify the observer overhead separated by its origin — measurement organs of the subordinate animals (difference from variant 1 to 2) versus measurement commands of the hybrid tier (difference from variant 2 to 3) — and yield, per workload and platform, an *uncertainty band* below which

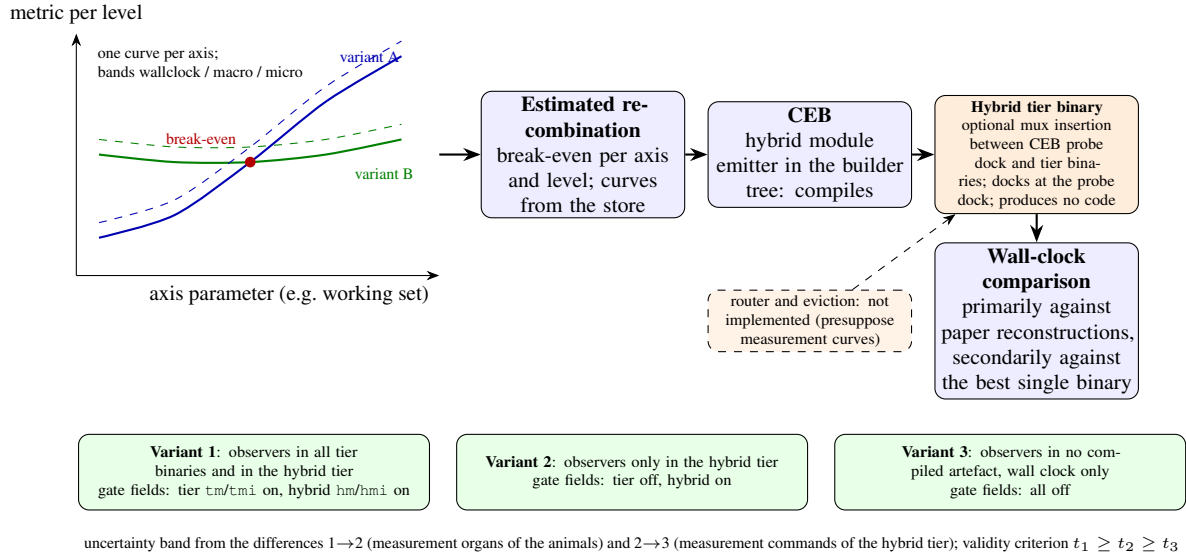


Figure 5.3: The measurement-curve type system in the heuristic counter-proof. Left: per axis one curve of the metric over the axis parameter, separated by measurement level (wallclock, macro, micro); the intersection of two variants is the break-even point from which the recombination is estimated. Middle and right: The CEB compiles the estimated recombination via the hybrid module emitter; the hybrid tier binary is the mux stage optionally inserted between the CEB probe dock and the tier binary, docks at the probe dock, carries dock slots for tier binaries, and itself produces no code; the comparison is made on the wall clock, primarily against the paper reconstructions, secondarily against the best single binary. Bottom: the three build variants of the measurement-error test plan, kept apart via the gate fields of the fingerprint. Dashed: router and eviction (not implemented).

two configurations count as indistinguishable; the band is formed from repetitions with a confidence interval, not from a single run [GBE07]. Validity criterion of the stage: With measurement sensors it must be slower than without — for the wall-clock times of the three variants $t_1 \geq t_2 \geq t_3$ holds; a violation is a measurement defect of the affected cell, not a result, and leads to the exclusion of the cell. Implementation status: This plausibility check is not implemented in the code; it connects to the measurement-error interpolation as the fifth evaluation component of the compare stage (Section 3.6.3), whose constellations the planner computes dynamically from the gate assignment, never as a fixed number.

The second stage is the complementary *fourth step* of the final heuristic assembly: The tier binaries subordinate to the heuristic tier — not the finally optimized main binary — are rebuilt without micro- and macro-benchmarks and without observers, so that no measurement artefact remains in the compiled artefact. The comparison is then made purely on the wall clock, in two comparison sets with a fixed precedence: primarily against the reconstructed paper algorithms — the 33 corpus papers with one profile XML each, 30 full profiles and 3 abstract ones (Section 3.2); the paper comparison takes precedence, for it answers the assessment question literally —, secondarily against all known tier-binary permutations, in particular the best single binary; this second comparison classifies whether a potential lead is attributable to the heuristic as a whole or merely to a single superior permutation.

From this follows the decision rule: The counter-proof counts as *delivered* per workload class if the observer-free rebuilt recombination beats the fastest paper reconstruction on the wall clock and the margin exceeds the uncertainty band of the build variants; it counts as *missed* if a paper algorithm or the best

single binary comes out ahead — this outcome, too, would be a result, for it would quantify for the first time how much estimation error lies between curve prognosis and real behaviour. The uncertainty band is to be distinguished from a measurement defect: Cells that violate the validity criterion of the first stage do not enter the decision. The measurement data of this dimension are not available; they are reported with series A–C in Section 5.2.4.

5.2 Results and Analysis

5.2.1 Identity of the Results: Stamp, Store, Workbook

Every result row is identifiable via the stamp system (Section 3.4.4): the `binary_id` — the path of eighteen `axis=value` segments in slot order —, the SHA-512 fingerprint over eleven members with the format identifier `fingerprint_format=6`, and the three compiled-in stamp lines per realm (organ, system, measurement). This identity travels unchanged through the chain: as store key (stock 1 and 2), as columns `permutation_id` and `fingerprint` of the measurement row, as sheet key of the workbook, and as identifier in every diagram and every table of the appendix — thus every presented number remains traceable to its binary, its axis versions, and its machine. The mixing strategy of a candidate under test is expressed not via a version suffix but via the merge tokens `replace`, `merge`, and `union` of the user XML and the hierarchically extended algorithm names of the organ line under the namespace prefix of the candidate under test; the flag `e` of the entry grammar denotes an efficiency-core algorithm (the E-core path of a hybrid CPU), not an experiment and not a mixture. Values of the measurement never appear in the stamp; what is stamped is the compile-time identity. Implementation status: The appendix tables carry `permutation_id` and `fingerprint` per row; the full stamp identity per diagram is planned but not implemented.

5.2.2 Data Flow from the Measurement to the Manuscript

Per series run the measurement stage writes the one official `measurements.csv` as a projection of the workbook (implemented is a flat file; planned is the CSV tree with one folder per sheet, below); the `<output>` paths of the user XML (pattern `_runs/<date>/<series>/`) the run entry resolves as run provenance. Three column lists are to be distinguished (Table 5.4): the WIDE schema with 189 columns (separator semicolon; the schema authority of the measurement output, whose header arises at runtime from four individual sources and is therefore anchored as a fixed schema), the pipeline view with 16 columns plus 16 additional columns, that is 32, for the LaTeX stages (separator comma), and the thirteen-column legacy form of the result aggregator over `ExecutionResult`. The pipeline view carries per permutation `permutation_id`, `fingerprint`, `succeeded`, `workload_used`, `op_count`, `total_cycles`, `cache_misses_l1`, `cache_misses_l2`, `cache_misses_l3`, `dtlb_misses`, `coherence_invalidations`, `energy_micro_joules`, `bytes_allocated`, `bytes_in_use_peak`, `external_frag`, and `internal_frag`; the full view adds six search counters (`search_insert` to `search_peak_occupancy`), `pmc_available`, `branch_misses`, `throughput_ops_per_sec`, and the seven `*_source_available` flags. `total_cycles` carries — under its legacy name — the representative p50 latency in nanoseconds, not cycles; the renaming or an actual cycle channel is a documented design decision.

Values not collected stand as n/a, never as zero.

Schema	Separator	Columns	Authority / consumer	Source in the code
WIDE (E4)	semicolon	189	measurement output; header at runtime from four individual sources, anchored as a fixed schema	iterator of the CacheEngineBuilder (<code>lazy_csv_header</code>); fixed schema
PIPELINE	comma	16 (+16 = 32)	LaTeX stages 04/05/06; full view as prefix superset	pipeline CSV schema: one list, four writers
F15-LEGACY	comma	13 (12 + 1)	result aggregator over ExecutionResult	result aggregator of the builder commands

Table 5.4: The three measurement column lists of the cache engine and their authority. `total_cycles` of the pipeline view carries the representative p50 latency in nanoseconds (legacy name), not cycles. F15-LEGACY comprises twelve columns and, appended at the end, `degeneriert` (12 + 1).

The workbook (xlsx) always arises — in memory, unconditionally, even if a profile persists CSV only; it is the output and the standard, and the CSV form is not a single child document but an ordered tree of CSV files — one overall folder per workbook, beneath it one folder per sheet, therein the CSV files of the individual sub-axis sweeps —, which is always generated from the workbook (xlsx to csv, as a strategy of the same factory) and is, as a presentation, its analogously mirrored counterpart: A row reaches the CSV tree only if the workbook has accepted it before, and if the workbook generation breaks, the CSV tree is invalid too; what goes to disk is decided by `<writeback_methods>` alone — xlsx, csv, or both (the workbook, the CSV tree, or both at the same target), as a strategy over the output formats with xlsx as default (Section 4.5); the xlsx writer lies in the store deposit and writes via `libxlsxwriter`. Implementation status: The seam writes the CSV projection as one flat file (`projektion_oeffnen` in `ergebnis_mappe_naht`); the tree form is planned but not implemented. The analysis enriches the projection via binary-to-CSV with profile ID, paper reference, and axis assignment, renders CSV-to-LaTeX fact sheets per permutation ID (an empty input produces no file, never an empty plot) and diagram-generator diagrams as TikZ bars, scatter plots, and heatmaps together with the families of the measurement appendix built on them (ECDF, range bars, sweep curves, 3D raw-data view, ratio matrices, normalized bars, Pareto scatter, per-axis attribution, observer detail table) with A4 cap and size check (a 1×1 , $1 \times n$, or $n \times 1$ matrix produces no image); the appendix generator writes per language into a `tabellen` directory, from which the manuscript includes the files — each language version includes 38 generated files of the appendix.

The workbook of a run is one file with one sheet per selected sub-axis permutation and an info sheet (system information of the measuring machine, main axes used of all three realms, constant meta axes); the sheets are numbered in the binding order measurement $\rightarrow$ system $\rightarrow$ organ (S001 to Snnn), and the columns stem exclusively from the WIDE header; columns whose value never changes within a run move into the metadata of the info sheet instead of being repeated in every row. The contract tests of the persistence check at the CSV output; a check at the standard xlsx is planned but not implemented (Section 4.4). Besides xlsx and csv, a third factory LaTeX is planned, which serializes the sheet order of the workbook when rolling out the appendix and carries per binary one table row as a temporal measurement series. Implementation status: Writer and projection are implemented; the seam writes one data sheet under a constant sheet key, and the projection is a flat file. Not implemented are: the fan-out into one sheet per sub-axis permutation plus info sheet (Section 6.2), the CSV tree (one folder per sheet), the check

at the `xlsx` produced per binary (the persistence check reads the CSV output), the export element with target filter and per-binary `xlsx`, the chart functions, the LaTeX factory, the warnings section of the info sheet, and the catch-up of the fixed WIDE schema.

5.2.3 Measurement Series and Merge Strategies

Three measurement series are evaluated (Table 3.8): series A — PRT-ART against the state of the art with the evaluation hypotheses H1–H3 (Section 5.1.1); series B — the systematic variation of the cache-engine permutations, fed from the union of stage 3; series C — merge/regression of old against new, across builds. Orthogonal to this assignment, the user XML declares per axis the `merge` mode of the candidate under test (Table 5.5): `replace` (the default, also with an empty attribute; the candidate axis replaces the cache-engine axis with fallback, merge strategy `Verbund2_Replace`), `merge` (cache engine and candidate under test hybrid per candidate under test, `Verbund2_Hybrid`), and `union` (the explicit union of all axis variants without discarding, bound to stage 3, `Verbund3_Union`); stage 1 is `Verbund1_CeOnly`, the absence of any candidate directive, and “stage 1 to 3” remains the alias of the merge-strategy names, the full join the alias of `union`. Series A works with `replace` and `merge`, series B with `union`. The states remain distinguishable at the binary via the hierarchically extended algorithm names of the organ line (Section 5.2.1); a stamp line of their own they do not possess, for the candidate under test mixes against the organ axes and thus already stands in their line, and a version suffix does not denote the mixture. For pure replacement builds as for pure cache-engine builds, the name extension consequently stays absent. Implementation status: The token `merge` is registered in the merge plan and declared in the golden core profile; materialized are the strategies `Verbund1_CeOnly`, `Verbund2_Replace`, and `Verbund3_Union`, while `Verbund2_Hybrid` remains without a production caller, because its materialization in the strategy enum is not implemented.

merge token	Merge strategy	Alias	Effect	Series
— (no directive)	<code>Verbund1_CeOnly</code>	stage 1	cache engine only: standard building blocks and SOTA profiles	A (baseline, also C)
empty / <code>replace</code>	<code>Verbund2_Replace</code>	stage 2 (replace)	the candidate axis replaces the cache-engine axis, with fallback	A
<code>merge</code>	<code>Verbund2_Hybrid</code>	stage 2 (hybrid)	cache engine and candidate under test hybrid per candidate under test; materialization open, no production caller	A
<code>union</code>	<code>Verbund3_Union</code>	stage 3 (full join)	union of all axis variants without discarding, bound to stage 3	B

Table 5.5: The `merge` tokens of the user XML and their merge strategies in the merge plan; the stage names are aliases, the full join is the alias of `union`. The series column follows the assignment from Table 3.8.

5.2.4 Available Measurements

Methodology and evaluation pipeline are in place and verified on real data: A smoke measurement series over 43 permutations — DLL measurement; 27 Cartesian combinations of SIMD, memory layout, and allocator plus 16 more over four two-valued quantities — is documented by Appendix A together with its limitations (Section A.10); it varies seven quantities (SIMD is a system axis, memory layout and

allocator are organ axes, the remaining four names are not registry building blocks and are not equated), the full measurement series varies all eighteen composition axes. The complete runs of the mandatory measurement series A–C on the target platforms are not available (n/a), because the full build is not carried out (Section 6.2); the same holds for the measurement data of the counter-proof (Section 5.1.8). No number of this chapter stems from any run other than the one reported in the appendix.

5.2.5 Axis Sensitivity Analysis

The core of the evaluation is the *axis sensitivity analysis*: It answers the separability problem of the task (Sections 1.3 and 3.1) in that the axis framework isolates each constituent as an interchangeable axis over an otherwise constant structure and the contribution of each design decision becomes attributable under all-else-equal conditions. The ranking runs over the three measurement levels in canonical order: *wallclock* — the overall benchmarking of the YCSB load profiles in the CacheEngineBuilder —, *macro* — the genus-interface operations of the tier binary — and *micro* — the axis interface, the set of functions that all algorithms of an axis must provide (Section 3.4.6); from the rows per permutation the analysis quantifies at the micro level which axis contributes most to the performance variance, the wallclock level checks the internal micro measurement from outside, and the measurement categories are thereby the sub-axis of the tooling axis (Section 5.1.5). The guiding axis is cache-line awareness: which axis moves the cache-near categories on which platform. From the ranking follows the recommendation of standard configurations per workload class; where ratios are aggregated over workloads or platforms, this is done by geometric, never arithmetic, mean [FW86]. The heuristic counter-proof stands beside this ranking (Section 5.1.8).

5.2.6 Validity and Limits

The validity of the evaluation is checked according to the four classes of empirical software research⁵ (Table 5.6; Section 3.4 carries the same classes as quality criteria of the measurement system). Construct validity: Do the categories measure cache-line awareness? The PMC semantics are disclosed per model (Section 5.1.5), and hardware counters count as trustworthy only when cross-checked [WM08]. Internal validity: The observer overhead is bounded via the uncertainty band of the three build variants (Section 5.1.8); layout and environment bias [MDHS09, CB13] are countered by identical keys, identical build matrix, and repetitions, a layout randomization is missing; warm-up and repetitions follow Section 5.1.7 [KJ13]. External validity: The registered measurement machines comprise two x86-64 microarchitectures and one Gracemont node; ZIH and HBM are declared, not measured (Section 5.1.4), the Intel RAW assignment is missing, and a lane run on one side only counts as a partial measurement. Conclusion validity: percentiles nearest-rank without interpolation, geometric aggregation, n/a instead of zero. The limitations of the measurement appendix (Table A.10, Section A.10) and of the conclusion (Section 6.2) are the substantiated list of these limits; no number is named without a run, and the reporting rules for performance measurements [HB15, BDH⁺16] apply to every number that a run delivers.

⁵Threats to validity: the classification, customary in empirical software research, of the threats to a conclusion into construct validity (does the measured quantity measure what is meant?), internal validity (is the effect causally isolated?), external validity (does the finding transfer to other platforms and loads?), and conclusion validity (do statistics and aggregation support the conclusion?) [WRH⁺12].

Class	Threat	Countermeasure in the apparatus	Implementation status
Construct	counters do not measure what is meant (model semantics, prefetch shares)	RAW catalogue per model with cross-check; semantics disclosure per column; n/a without catalogue entry	implemented (Zen 5); Intel entry: not implemented
Construct	time category misread as cycles	<code>total_cycles</code> declared as p50 latency in nanoseconds	implemented; renaming: design decision
Internal	observer overhead distorts the curves	three build variants, uncertainty band, validity criterion $t_1 \geq t_2 \geq t_3$	gate fields implemented; check: not implemented
Internal	layout/environment bias, change of the L3 complex	identical keys and build matrix; repetition dimension; pinning reported as planned	actuator implemented, attachment: not implemented; no layout randomization
Internal	warm-up and cold-start effects	two-phase driver (save, warm up, roll back, measure)	implemented
External	measurement machines do not cover the target platforms	registered signatures prod1, prod2, ODROID; ZIH declared as IS3	ZIH and HBM: declared, not measured (n/a)
External	one-sided vendor lane	completeness rule of the evaluation; partial measurement is reported	rule set; Intel lane: cross-check not performed
Conclusion	percentiles synthetic instead of measured, means distorted	nearest rank without interpolation; geometric aggregation	implemented (evaluation tree); copies in the tools: declared gap
Conclusion	zero instead of non-observation	field flags per counter, n/a end-to-end	implemented

Table 5.6: Validity of the evaluation according to the four classes of threats (construct, internal, external, conclusion): per threat the countermeasure in the apparatus and its implementation status — implemented, declared and not measured (n/a), or not implemented. No row names a number without a run.

5.2.7 Summary of the Evaluation

The evaluation records per evaluation object. Design space: The hypotheses H1–H3 carry measured quantity and criterion, and the sensitivity analysis is fixed as a procedure over the three measurement levels; evidence is available for the smoke series (43 permutations, seven quantities, Appendix A), for the mandatory measurement series not (n/a, full build not carried out). Apparatus: The counter-proof is fixed as a fourth dimension with test plan, validity criterion, and decision rule, its measurement data are not available; the measurement-error foundation — gate fields in the fingerprint, field flags per counter, n/a rule — is implemented. Automation: The chain from the experiment XML to the included appendix is demonstrated in a first end-to-end run and delivers every number with its run. The results are discussed against the leitmotif of the thesis — the transition from heuristic to informed cache management, that is, the question whether telemetry-driven, automatically adapted configurations outperform statically chosen ones; the cache-line effect from the synthesis curves is the first evaluation step after the full run, data-dependent and therefore not anticipated here. By the standards for research contributions [Sha03] and for empirical assessments [BDH⁺16], the contribution of this chapter is a validated experimental setup together with its automation, whose result figures will be available with the full measurement series. Chapter 6 answers the research questions on this basis.

6 Conclusion and Outlook

This chapter takes stock: for each research question RQ0–RQ4 (Section 1.3) it puts the sub-questions first, names the state of their answer in three grades — answered, structurally answered, open —, traces per question the chain of evidence through the chapters and its consequences, assigns the hypotheses to the target path, names the limits of the system from the user’s point of view, and sketches the next steps.

6.1 Answering the Research Questions

The answers follow the scheme claim, evidence, validation [Sha03]: a question counts as *answered* when a test or a measurement substantiates it; as *structurally answered* when the apparatus that delivers the answer is implemented and tested but the measurement is not available; as *open* when neither structure nor measurement is available. Three kinds of evidence occur: the structure (a code object that supports the answer), the test (a static check or a contract test that secures it), and the measurement (a number together with its run); for the question about the state of the art of the workloads (RQ2.d) the literature evidence is added. A “structurally answered” does not replace the number: without measured values the main question remains unanswered even if its structure is sound. Table 6.1 lists status, evidence, and remainder per main and sub-question; the form of reporting follows the rules of empirical software and systems research [WRH⁺12, RH09] and the demand to state the reference quantity, the platform, and the values not collected at every result [BDH⁺16, HB15].

Orthogonal to all five questions, the introduction posed a categorization question as a question: whether the axes of the design space separate cleanly into organ, system, and measurement axes — and how such a separation is to be accomplished where a design does not provide for it. Both are answered (Section 3.4.1): the axes separate along the three concerns into three *realms* — organ, system, measurement — with one class root each, one offering catalogue each (three generated registries: the organ-axis registry of the cache engine, the system-axis registry, and the measurement-axis registry), and one stamp line each at the binary (Section 3.4.4); the three line carriers of the fingerprint — organ, system, and measurement line — are, as types, pairwise unconstructible from one another, so that a swap does not compile. Only the organ main axes enter the `binary_id`; system and measurement axes are declared binary-id-neutral in their registries. The separation is held by compile-time *re-entry guards* (Section 4.5): `static_assert` checks reject compiler, scheduling, and load framework as system main axes — the scheduling is a sub-axis of the target ISA `target_isa` with five dimensions, the compiler a sub-axis group of the complex bracket `build_target_complex`, the load framework the meta-meta axis of the measurement realm —, and the flag grammar of the version stamps admits only catalogued flag chains. This triad is the second shape of the separability problem and supports the answers to the following sub-questions.

RQ0 (Main question). The question is by how much active, measurement-driven cache-aware decisions — page type, memory layout, value placement, prefetching — outperform their passive, statically

dimensioned counterparts on different CPU platforms; the extension RQ0.E asks whether a measured gain can be attributed unambiguously to the organ, the system, or the measurement concern. The *status*: RQ0 is open. Without the end-to-end runs per platform there is no number; the structure that delivers the number is implemented. The same axis-constant structure is built once actively — with a runtime-dynamic cache-line policy (the policy selector derives the cache-line adaptation from an evaluated profile aggregate of operation mix and working set) and measurement-driven selection of the best binary — and once passively — with a statically fixed organ choice of memory layout, prefetch, value placement, and node type as segments of the `binary_id` — and the two are compared under an identical compiler and flag basis (default `-O2`; `-O3` only as an opt-in under warning and with its own toolchain identity). The measurement machines surveyed are the three registered machine signatures: `prod1` (AMD Zen 5, Ryzen 9 9950X3D), `prod2` (Intel Core i9-12900K, Alder Lake — a hybrid CPU with P- and E-cores, without AVX-512), and an ODR0ID board with Intel Gracemont cores; the server platform Sapphire Rapids named in the question is provided for in the platform detection (ZIH node Barnard, Xeon 8470) but not measured (n/a; Section 5.1.4). RQ0.E is structurally answered, and the separation it requires is the triad: an organ effect permutes the `binary_id`, a platform concern stands at the binary as system stamp line and build version identifier without permuting it, and a measurement concern remains binary-id-neutral — no measurement instrument changes *what* is built. The *evidence* is Section 3.4.1 (triad; structure), Section 4.5 (re-entry guards; test), and Section 5.1 (measurement plan; structure). The *consequence*: a quantification presupposes the full build and the run of series A (Section 6.3); the effect of cache-line awareness from the synthesized measurement curves is the first evaluation step after the first measurement run (Section 3.6.3).

RQ1 (Composition). The question is how a search-algorithm design can be canonically decomposed into orthogonal axes and systematized in a permutation matrix, such that state-of-the-art methods become reconstructible bias-free as profiles, ideally from their original code, and arbitrarily permutable; RQ1.a asks about the category cut of the decomposition, RQ1.b about the organ couplings and the behaviour under a solely varied ISA/SIMD axis. The *status*: RQ1 is structurally answered, RQ1.a answered, RQ1.b open in its empirical part. The canonical decomposition carries: *eighteen* organ main axes in fixed slot order T0–T17 — as the constant name array of the axis-path serialization they are code, not convention — span the permutation matrix (Section 3.4.1); to these is added an optional *organ meta-meta axis* for disk input/output `disk_io`, carried in the organ registry as the nineteenth entry (a compile-time stage with the runtime option `staging`, binary-id-neutral), but not selectable via experiment profile, because the profile parser does not read its registry block; the profile connection is planned but not implemented. It is the thesis’s example of the meta-meta level M3, the indirection of a description: it stamps in the bracket appendage of the organ line, and the bracket depth of an entry is its meta-meta level (Section 3.4.4). Two reference quantities are to be kept apart: the golden reference catalogue varies seventeen axes over two values each and pins the persistence target to one value — $2^{17} = 131072$ binary identities per system permutation as a regression detector with a fixed CRC-64 checksum¹ over all identities (Section 4.4) —, whereas the cardinality of a planned space is not a constant but the planner’s computation per profile and platform (subcommand `simulate`); all eighteen axes are planned for variation (not measured). The 33 corpus works P01–P33 each possess a profile XML, among them 30 full profiles of standalone

¹CRC-64: a cyclic redundancy check with a 64-bit checksum (here the variant ECMA-182), computed over all identities in canonical order; a changed identity changes the checksum.

search designs (eight Rank-1 and 22 Rank-2/3) and three abstract profiles (OLC, LOUDS, VAMPIR) that occupy only individual axes (Section 3.2); the registry of the paper-derived candidates under test holds this translation at compile time against duplicates and gaps. The variadic mapping — one template parameter for the `std::vector` API, two for `std::map`, $N > 2$ for `std::map<K, std::tuple<...>` (Appendix F) — makes algorithm- as well as container-oriented patterns expressible on one engine, and the family triad Map, Container, Graph orders the genera beneath it (Section 3.3.2). RQ1.a is answered in the affirmative: the decomposition remains orthogonal only because platform and measurement concerns are kept out of the permutation matrix, and the re-entry guards hold this cut (Section 4.5). On RQ1.b: the coupling node type $\times$ SIMD $\times$ path compression named by the question falls apart — node type $\times$ path compression remains a not-fully-orthogonal organ coupling that is marked as such and restricted during permutation, whereas SIMD is not an organ axis but a build control quantity of the system axis `external_utils` (a runtime sub-axis with the options `no_extension`, `avx2`, and `avx512` and a feature catalogue of 23 flags, binary-id-neutral): one and the same `binary_id` is built under several system permutations and compared with an otherwise unchanged structure; the empirical answer on cache-update and cache-line fill behaviour is a measurement and is not available. The *evidence* is Section 3.4.1 (decomposition and realms; structure), Section 3.4.6 (catalogue and reference quantity; structure), and Section 4.4 (catalogue checks and golden checksum; test). The *consequence*: RQ1 structurally answered; RQ1.b (fill behaviour) presupposes the full build.

RQ2 (Measurement methodology). The question is how a reproducible measurement pipeline with hardware-counter collection must be designed so that every axis permutation is measured fairly — under an identical compiler and flag basis — in three measurement series over three granularities, and under which platform-calibrated criteria a platform counts as verified; RQ2.a asks whether the measurement instruments themselves must be axes of their own, RQ2.b about the re-measurement of contradictory literature findings, RQ2.c about extensibility, RQ2.d about the workloads that belong to the state of the art specifically for search algorithms. The *status*: RQ2 is structurally answered, RQ2.a and RQ2.d answered. Measurement is split into a production mode and an experiment mode, switched via the master switch of the measurement compilation (`COMDARE_MEASUREMENT_ON`): the experiment mode always measures — a release build without measurement sensors is a separate, deliverable artefact that forces the switch off, not a measurement run that has been switched off —, and in the store only what is missing there is re-measured; the compile-time-static key schema of the four stocks is the design of this recognition, the full build-out of the store is not implemented (Section 3.4.5). The `ExperimentDriver` controls the sequence host-side: the runner loads the modules and runs the phases load, execute, measure, and teardown, the driver runs execution and measurement per module and collects the results in the `ResultAggregator`. Every series is collected on three *measurement levels*, in exactly this order: the wall-clock level (`wallclock`; the overall runs in the `CacheEngineBuilder` over the load frameworks), the macro level (`macro`; the family and genus interface operations of the tier binary), and the micro level (`micro`; axis interface and success parameters); `w/ma/mi` is the registered alias, and the three levels are the measurement-tooling main axis, which per choice is compiled into one `CacheEngineBuilder` route (Section 3.4.6). RQ2.a is answered in the affirmative: the measurement instruments are a realm of their own in the planner — the load framework as meta-meta axis, the measurement tooling as main axis, the measurement categories as sub-axis (sixteen building blocks in the registry; planned is the collection

per axis-interface parameter set, whose catalogue per axis is not implemented) —, always present as soon as measurement takes place, and binary-id-neutral. The separation is accomplished by the double root cut: the system realm is rooted at the CacheEngineBuilder, the measurement realm at the planner, and the measurement axis drives, via the system-axis root, the release of the measurement source in the form of the CacheEngineBuilder type — the legend chain measurement axis $\rightarrow$ CacheEngineBuilder type $\rightarrow$ system axes $\rightarrow$ tier binary is code as a naming scheme (Section 3.4.1). For the workload, the driver passes a default workload through per module, and a profile brings its own; planned is the full permutation over all available workloads, so that no design wins only under its self-chosen load profile — the apparatus counters the *load-profile selection bias*² with the full matrix of all load profiles against all living-being binaries, because even inconspicuous setup decisions systematically distort measurement results [MDHS09]; implementation status: the full permutation is not implemented.

On RQ2.b (organ axes): the contradictory literature findings on the optimal node/cache-line size — one cache line per node for CSS and CSB⁺ trees [RR99, RR00] versus sixteen and more cache lines for Hankins and Patel [HP03] (nodes of 512 bytes and larger at 32-byte cache lines) as well as several per node for the prefetching B⁺-trees of Chen, Gibbons, and Mowry [CGM01] — are not adopted from their original setups but re-measured axis-isolated under otherwise equal conditions: node type, memory layout, and prefetch are organ axes that are varied individually while system and measurement axes stay constant; the number is a measurement and is not available. On RQ2.c (extensibility): a new candidate under test joins as a living being of its family into the genus — family is the first level (Map, Container, Graph), genus the second, and the genus build admission carries the slot sets per family as a compile-time verdict —; it docks at the probe dock, passes the three-stage evaluation (Section 3.4.6), and its merge with the cache engine bears one of the registered names: `Verbund1_CeOnly` (no candidate directive), `Verbund2_Replace` (the candidate axis replaces the cache-engine axis, with fallback), `Verbund2_Hybrid` (cache engine and candidate under test hybrid per candidate under test; declared, not materialized — the validator blocks the token `merge` as long as the strategy value is not implemented), or `Verbund3_Union` (union of all variants; prepared interface, Section 5.2.3). At the binary the merge remains distinguishable via the hierarchically extended algorithm names and flags of the organ line — a candidate organ carries its namespace prefix in the entry —; a stamp line of its own is not needed, and a version suffix does not denote the mixture: the flag `e` denotes an efficiency-core algorithm (Section 3.4.4). Whether the candidate under test matches an existing family, founds a new one, or constitutes a hybrid form is decided by the family cut (Section 3.3.2); a family of its own would arise only with a fundamentally different base axis set. The operating mode asked for in RQ2.c, which offers the design constituents of exactly one publication as a closed axis set in isolation, is not implemented and remains a subject of the outlook (Section 6.3). On RQ2.d (workload state of the art): the state of the art of the workloads for search algorithms is formed by the key-value convention YCSB with its six load profiles A–F (each an operation mix over a key access distribution, from 50/50 read/update Zipf-distributed to short range scans and read-modify-write), the index benchmark SOSD, the TPC family (TPC-C, TPC-DS), the integration into real systems (SuRF in RocksDB), real string corpora (URLs, DNA, dictionary terms), as well as SPEC CPU and CloudSuite in the hardware-near works (Section 2.3); these frameworks generate loads that are not directly comparable with one another, and each publication typically chooses the load profile under which

²Load-profile selection bias (workload selection bias): the distortion of a comparison that arises because the load profiles are chosen such that one design performs preferentially well under them; the benchmarking literature therefore demands a broad, justified set of load profiles [BGH⁺06, BDH⁺16].

its own design performs best. From these the thesis adopts YCSB as the registered load framework (the meta-meta axis `load_framework` of the measurement realm) with the runtime dimension `workload` over the six tokens YCSB_A to YCSB_F, routed per permutation from the traversal tag of the profile or its `<expected_workload>` tag, and runs them over real string datasets with a reproducibility record (`url`, `protein`, `tpcds-id`, `trec-terms`, `dna`, `xml`; Section 5.1.3), where `xml` possesses only a specification without checksum because the file is not available locally (n/a); the remaining conventions are not registered as load frameworks (n/a). Implementation status: for profile-less modules the driver falls back to exactly one workload, the workload option of the run; the full permutation over all load profiles is planned but not implemented (Section 5.1.3). On platform verification: a platform counts as verified when the conformance gate confirms, per loaded tier binary, the seventeen `std::map` operations marked in Appendix F — the operation set of its twelve core check blocks, among them `at`, `contains`, `count`, `insert_or_assign`, `erase`, `clear`, `lower_bound`, and `upper_bound` — against `std::map` as oracle and the platform probe covers the reported axis subset; the five drive operations of the module boundary (`insert`, `lookup`, `erase`, `clear`, `size`) are the ABI primitives from which the host composes the remaining operations, not the scope of the check, and the full build-out to the complete `std::map` interface is not implemented; the platform-calibrated thresholds are not available, their determination presupposes the measurement runs. The *evidence* is Section 3.4.6 (three-stage evaluation and merge; structure), Section 3.4 (measurement levels and realm; structure), Section 3.3.2 (family cut; structure), Appendix F (contract subset; test), and, for RQ2.d, Section 2.3 (workload conventions; literature) and Section 5.1.3 (workload routing and records; structure). The *consequence*: RQ2 structurally answered, RQ2.c conceptually answered, RQ2.d answered; the isolation mode, the number for RQ2.b, and the full permutation of the workloads are open.

RQ3 (Candidate comparison). The question is whether the candidate under test PRT-ART wins against the eight Rank-1 designs — ART [LKN13], HOT [BZP⁺18], Masstree, CoCo-Trie, START, B²-tree, Wormhole, SuRF — and the Rank-2/3 SOTA *not* through asymptotic novelty but through better microarchitectural properties — higher cache-line utilization, fewer LLC and dTLB misses, lower branch cost, smaller hot search paths — for exact lookup, prefix lookup, and prefix enumeration, measured in the mandatory measurement series A, B, and C; RQ3.a asks about the cheapest local page representation per branch point, RQ3.b about the compile-time force. The *status*: RQ3 is open, RQ3.b answered, RQ3.a structurally answered (prescription without measurement evidence). The comparison is not available and presupposes the first measurement runs; the apparatus for it is implemented: the PRT-ART profile, the three mandatory measurement series — the series is derived mechanically from the merge stage (`Verbund1` and `Verbund2` yield series A, `Verbund3` series B, series C is cross-build; Table 3.8) —, the `ExperimentDriver` with auto-pickup of the 33 profile XMLs from the profile directory, and the ABI-conformant adapter with the `std::map` core contract as a host-side composition layer over the exact-lookup primitive of the module boundary, flanked by insertion and deletion, with ordered iteration as an additive scan capability (Appendix F). The conformance gate checks the seventeen operations marked in Appendix F in twelve core check blocks against `std::map` as oracle; the full build-out to the complete interface is planned but not implemented. The notification hook of the measurement layer is a single-slot hook per axis, not compiled in by default (opt-in), and not part of the contract. On RQ3.a: array-indexed page — the direct-index pages `Array256` and `Array65535` — versus vector page — the vector

pages `Vector<u8,u8>` and `Vector<u16,u16>` — per branch point and the switching rules by occupancy density and access type are part of the PRT-ART profile; the candidate under test switches at fixed density thresholds of 25, 50, and 75 percent (Section 3.5), and the platform-calibrated rules are prescriptions without measurement evidence as long as the runs do not substantiate them. RQ3.b is answered in the affirmative and has become the construction principle: every axis divides into a compile-time-fixed *main axis* and runtime-variable *sub-axes* — the main axes are compiled in statically, for organ axes as a `binary_id` segment, for system axes as stamp together with the build version identifier, while sub-axes with the stage `runtime` vary their parameters without recompilation; the organ registry carries the stage per sub-axis, the system registry the sub-axis group `build_toolchain` of the complex bracket (Section 3.4). The *evidence* is Section 3.5 (candidate under test and page representation; structure), Section 3.2 (33 profiles; structure), Section 3.4 (main and sub-axis; structure), and Appendix F (core contract; test). The *consequence*: the number presupposes series A; the hypotheses H1–H3 supply the evaluation criteria.

Hypotheses and target path. The evaluation hypotheses H1–H3 (Section 5.1.1) were formulated before the apparatus; the target path developed across the chapters — triad of the realms, stamp identity, measurement levels — has not laid them as a foundation but ranks them as touchstones of the evaluation: H1 (page-type cost) and H3 (distribution of the value placement) are checked in series A per load profile, H2 (code quality per source) via the tool-computed quality record. Its data basis is the weighted `cppcheck` finding density per thousand physical lines over the vendored original sources; the record carries 33 entries, 22 of them as n/a because a reconstruction possesses no vendored original source, and PRT-ART possesses no original repository (n/a). The hypothesis check follows the requirement to name criterion and threshold before the measurement [WRH⁺ 12], and the number of repetitions per cell follows the required confidence [KJ13]. H2 counts as accepted if, over the full profiles with a vendored original source, the Spearman rank correlation between finding density and measured throughput per load profile and cell (platform, workload, regime) is negative with $|\rho| \geq 0.5$ at $\alpha = 0.05$; as rejected at $|\rho| < 0.3$; in between as undecided; below eight profiles with a score H2 is not testable. The manual audit over seven assessment axes remains the qualitative classification. The PRT-ART telemetry metrics H1–H3 are measurement organs of the candidate under test, not these hypotheses (Section 5.1.1); and the variant `ChainRef` of the value placement is the subject of H3, it is no abort criterion for H2.

RQ4 (Compile time vs. run time). The question is which axis properties must be fixed at compile time to produce platform-optimized, provenance-verifiable binaries per permutation, and which can or must be selected at run time (bit-encoded binary identity per permutation, runtime-adaptive layout selection) without violating comparability (identical compiler and flag basis) — and whether the measurement makes it possible to generate ISA+OS-optimized binaries for a search-algorithm recombination whose heuristic optimization is perfectly tailored to its runtime environment; RQ4.a asks whether the entire sequence — from the platform-specific build of all binary permutations via the measurement run and the selection of the best measured binary or binary recombination up to the evaluation in CSV/xlsx files, LaTeX, and diagrams — can be generated end to end from one XML description of the experiment; the extension RQ4.E asks whether the boundary runs along the axis categories. The *status*: RQ4 is structurally answered, RQ4.a structurally answered, RQ4.E answered. The boundary runs along the categories: the

three system main axes `target_isa`, `operating_system`, and `external_utils` together with their complex bracket `build_target_complex` (Section 3.4.2; no fourth main axis) belong to the compile-time identity of the binary; they appear as system stamp line and build version identifier, never as a `binary_id` segment. Provenance is secured by the tier fingerprint: a SHA-512 over eleven members (fingerprint format 6), among them the overlay member as a SHA-512 over the concatenated source files of the overlay cut — per axis the file set of its own implementation plus the shared anatomy substance —, so that a pure source-text change yields a new identity, and the toolchain member with the build-compiler identifier and the optimization identity; the planner fingerprint is a SHA-256 over its own preimage. The query `get_compiler()` of an axis, by contrast, returns the original compiler of the publication from the manifest of the vendored source — a statement of origin, not a build identifier. Measurement choices remain runtime-variable; layout and page selection remain selectable via the mixed-radix order of the `binary_id` (the static binary view) and the runtime-dynamic cache-line policy, without violating the common compiler and flag basis. Where an axis needs both sides of the boundary, it is two-way split: load framework, telemetry, and hardware detection run across the runtime of the planner *and* the compile time of the builder — the sub-axis release of the earlier stage becomes the main-axis assumption of the later stage (Section 3.4.1) —, and the hardware detection sets the compile-time stamps of the tier binaries from the actual properties acquired at builder runtime (Section 3.4.3). What is stamped is always the platform *class*, never a measured value; the *fit stamp*, which checks the stamped class on every measurement run against the runtime verdict of the hardware detection, is designed but not implemented (Section 6.3). The forward-looking sub-question of whether the measurement itself can tailor ISA- and operating-system-optimized binaries for a recombination is taken up by the outlook as heuristics automation; the operating-system dimension is provided for with three declared values of the system axis (`linux`, `windows`, `macos`, each with its own probe) and the two measurement regimes in the measurement plan — the second regime, an immutable operating system, is designed, not implemented (Section 5.1.4). On RQ4.a (declarative chain): the chain is laid out declaratively end to end and built as two binaries. The planner `comdare-experiment-planner` checks the user XML read-only against the compiled-in registry catalogues and emits the deterministic plan from the one director walk — two runs are byte-identical, the plan header annotates the registry trio, and every build step reports the actually generated file stem from the same orchestrator function that the build also calls —; the measurement driver `comdare-messung-driver` emits the system permutations and tier build orders of its released build space and runs the seven-phase measurement pipeline from `ENUMERATE` to `PERSIST` (CSV and JSON); the downstream nine-stage toolchain renders from that the `xlsx` workbook, the booktabs tables, the TikZ diagrams, and the appendix files that the manuscript includes — one XML source, one build channel, no step by hand (Section 4.3.2, Figure 5.1). The selection of the best binary or recombination is registered and validatable as the work mode `compare` of the run chain; its mode-specific sequence and the `release` are planned but not implemented — both are selectable, validatable labels with `release`-equivalent build semantics (Section 3.3.2). Implementation status: the chain is demonstrated in a first end-to-end run — the smoke measurement series over 43 permutations in the appendix is its product —; the headless service, the XML creation mode, the `<publish>` section, and the CSV tree form are planned but not implemented. The *evidence* is Section 3.4.2 (complex bracket; structure), Section 3.4.3 (two-way split detection; structure), Section 3.4.4 (stamp lines; structure), Section 4.5 (re-entry guards and order checks; test), and, for RQ4.a, Section 4.3.2 (planner, driver, and measurement pipeline; structure) and

Section 3.4.6 (chain from the XML to the tier binary; structure). The *consequence*: RQ4 and RQ4.a structurally answered; heuristics automation, fit stamp, and the sequence of `compare` and `release` open.

Question	Sub-question	Status	Evidence (sections; kind)	Remainder
RQ0	main question: active, measurement-driven versus passive, static	open	3.4.1 (structure), 4.5 (test), 5.1 (measurement plan)	number from series A after the full build
RQ0.E	attributability of a gain to organ, system, measurement	structurally answered	3.4.1, 3.4.4 (structure)	measurement that exercises the attribution
RQ1	canonical decomposition, permutation matrix, profiles	structurally answered	3.4.1, 3.4.6 (structure), 4.4 (test)	run over all eighteen axes; profile connection of the meta-meta axis
RQ1.a	category cut of the decomposition	answered	3.4.1 (structure), 4.5 (test)	—
RQ1.b	organ couplings, ISA/SIMD variation	open (measurement)	3.4.1 (structure)	cache-line fill behaviour per system permutation
RQ2	measurement pipeline, three series, three levels, verification	structurally answered	3.4.6, 3.4 (structure)	full permutation of the workloads; calibrated thresholds
RQ2.a	measurement axes as a realm of their own	answered	3.4.1 (structure), 4.5 (test)	catalogue of the success parameters per axis
RQ2.b	node/cache-line size	structurally answered	3.4 (structure)	measurement over node type, layout, prefetch
RQ2.c	axis-isolated extensibility, integration of new candidates under test	answered (conceptually)	3.3.2, 3.4.6 (structure), F (test)	paper-isolation mode; materialization of <code>Verbund2_Hybrid</code>
RQ2.d	workload state of the art	answered	2.3 (literature), 5.1.3 (structure)	full permutation of the workloads; only YCSB registered as load framework
RQ3	PRT-ART against SOTA in series A–C	open	3.5, 3.2 (structure)	series A; full build-out of the contract
RQ3.a	page representation, switching rules	structurally answered (prescription without measurement evidence)	3.5 (structure)	platform-calibrated rules from the runs
RQ3.b	compile-time force	answered	3.4 (structure), 4.4 (test)	—
RQ4	compile time versus run time, provenance	structurally answered	3.4.4, 3.4.3 (structure), 4.5 (test)	fit stamp; heuristics automation
RQ4.a	declarative chain from the XML to the evaluation	structurally answered	4.3.2, 3.4.6 (structure)	sequence of <code>compare</code> and <code>release</code> ; service, creation mode, CSV tree
RQ4.E	boundary along the axis categories	answered	3.4.2, 3.4.4 (structure)	—

Table 6.1: The chain of evidence per main and sub-question: status in three grades (answered, structurally answered, open), the supporting sections with their kind of evidence (structure, test, measurement; for RQ2.d the literature evidence), and the open remainder without a number.

Implementation status. The declared build scope — per system permutation the 131072 organ permutations of the golden reference catalogue; the catalogue comment estimates roughly 224 GB and roughly 24 hours per complete build, an estimate, not a measured value — has not been carried out; the

end-to-end runs of the measurement driver across the target platforms (x86-64 with AVX2 and AVX-512, aarch64 with NEON and SVE³) are not available, nor is the delivery measurement over the 320-identity catalogue. Substantiated by a run are two partial executions: the first end-to-end run of the chain from the experiment XML via build and measurement to the included appendix (Section 5.1.2) — a measurement job of the smoke stage on the AMD Zen 5 machine⁴ with 10000 operations and 671.5 ns per operation (p50 of the `lookup` operation 750 ns) as the real measured value of the path — and the calibration run on the AMD machine, which built 250 of 250 tier binaries and calibrated the build cadence to 0.521 s per binary and the space requirement to roughly 457 kB per binary; from this follows a storage requirement of roughly 240 GB for the complete build of 524288 tier binaries over the four system permutations of the golden profile (two optimization levels times two SIMD options). The smoke series — 43 permutations in two sub-blocks: 27 Cartesian SIMD×layout×allocator combinations plus 16 over node width, path compression, internal search, and telemetry — is available in Appendix A; its provenance is the data state under which it was collected: seven varied axes of that count, of which SIMD and telemetry are no longer organ axes in the canonical order. It approximates axis effects partly via wall-clock proxies, and individual effects are instrumentation artefacts⁵ rather than organ effects; the caveats per series are listed in Table A.10 (Section A.10). The apparatus-side acceptance per package runs through the two-gate protocol (Section 4.4). The *consequence*: RQ0 and RQ3 remain open as long as series A has not run; all eighteen axes are planned for variation (not measured).

6.2 Limitations

The following limits describe what a user cannot achieve with the system — apart from the natural and self-chosen limits of the machine on which they work; caveats of individual measurement series are listed in the measurement appendix (Table A.10), implementation states in the implementation chapter (Chapter 4), and the implementation status of the full build and the smoke series is stated in Section 6.1. Two limits follow from the store principle, the rest are derived from the architecture: machine-specific empirical optimization is the essential trait of cache-aware methods — ATLAS [WPD01] and FFTW [FJ98] survey the machine before they choose the code —, and cache-oblivious methods [FLPR99] are the opposite pole without machine binding; an apparatus that makes machine binding its principle inherits its costs. The module bodies are not a limit: the code generator of the measurement path emits per permutation the eighteen fully qualified axis types together with the umbrella include, the legacy path with the candidate stub is separated from the measurement path by a strict barrier, and the zero statements of the fixed-assigned pool families are explicitly coded non-observation with reason (Section 4.3.2; Table A.10).

- **Version change of an axis algorithm:** If the user changes the version of an axis algorithm, every tier binary whose overlay cut contains the file becomes a new identity — the overlay member of the fingerprint hashes all sources of the cut, and the version lock demands the deliberate regeneration commit for every change to a recorded file (Section 4.4); the store’s stock of these

³NEON and SVE: the SIMD extensions of the ARM architecture — NEON with a fixed 128-bit width, SVE (Scalable Vector Extension) with an implementation-dependent vector width.

⁴Provenance of the smoke run: pipeline 15800, measurement job 377503, host prod1 (Appendix A); a preceding first end-to-end run of the same chain (pipeline 15764, job 376333) yielded 1199.047 ns per operation at 10000 operations, its measuring machine is not documented and the difference not assessed.

⁵Instrumentation artefact (probe effect): the change of the observed behaviour by the measuring instrument itself [Gai86].

binaries together with measurement and evaluation has to be regenerated, because the fingerprint of the CacheEngineBuilder is provenance, not a reuse criterion. What the user cannot do: exchange a single algorithm and keep the measurements of the remaining binaries.

- **Store per machine type and hardware configuration:** Measured values are keyed to the hardware identity of the measuring machine — stock 2 of the store is addressed via the fingerprint of the measured binary and the hardware identity, the platform cell bracketed beside it (Section 3.4.5) —; the entire store has to be measured separately per machine type and hardware configuration. What the user cannot do: transfer measured values from one machine to another. That is the price of machine-specific optimization (ATLAS [WPD01], FFTW [FJ98]) and the reason for the three machine signatures of the system registry.
- **Coverage of the hardware counters per microarchitecture:** In the measurement path the counter collection is active — the emitted measurement jobs set the PMC switch, only the test build builds without hardware access. Collected are four generic counters (L1, last-level, dTLB, and branch misses via `perf_event_open`⁶) and two model-bound raw counters (L2 demand misses and fills from the cache of a foreign CCX as coherence observables), the latter only where the encoding is cross-checked by measurement — AMD Zen 5; without a catalogue entry the fields carry an explicitly coded non-observation⁷ (n/a), never a zero, and for Intel there is no raw assignment until an Intel host provides the cross-check. The last-level event fails to open on Zen 5 (no generic mapping), RAPL⁸ is best-effort and empty without zone read permission, and IPC is declared as a category, not collected. In the smoke series the hardware columns are not collected (n/a); the tabulated zeros of the appendix tables are presentation, not measurement (Table A.10, row 1). What the user cannot do: obtain L2 and coherence counters on models other than Zen 5 (Figure 3.13).
- **Hardware fit without automatic check:** The hardware detection is available in three functions: the acquisition (the ISA complex axis with the SPD/XMP declaration of the memory clocking and the planner probe, which asks the host for its counters per run), the compile-time factory (the tier stamps from the actual properties), and the runtime verdict; the machine declarations of the system registry are declarations with a preserved source (`dmidecode`). Not present are the wiring into runner and probe dock, the additive result write-back, the provenance round trip of the declarations, and the fit stamp. What the user cannot do: have the fit between machine and binary checked automatically (Sections 3.4.3 and 4.5).
- **Output without sheet fan-out:** The result workbook arises unconditionally in memory; `xlsx` is the default, the CSV form its selectable counterpart, always generated from it — not a single child document but an ordered tree of CSV files with one folder per sheet —, and both together are selectable (Section 5.2.1). The workbook possesses a constant sheet key — all rows in one sheet —; the fan-out into one sheet per sub-axis permutation plus an info sheet is planned as a filter chain of the evaluation but not implemented; the CSV projection is a flat file, the tree form not implemented.

⁶`perf_event_open(2)`: the Linux system call that opens a hardware or software counter as a file descriptor; generic events are mapped by the kernel per microarchitecture, model-specific ones require a raw encoding.

⁷Explicitly coded non-observation: the missing value is a state of its own (n/a with reason), never a zero; statistics speaks of planned missing data whose missingness mechanism is known and which therefore introduces no bias [GTOC06, Gra09].

⁸RAPL (Running Average Power Limit): the energy counters of the processor packages, readable under Linux via the `powercap` sysfs zone; the zone is often readable only with root privileges.

What the user cannot do: obtain results per sub-axis permutation as separate sheets or obtain the CSV storage as a tree.

- **Permutation space and budget:** A complete build of the golden reference catalogue costs, per system permutation, roughly 224 GB and roughly 24 hours according to the catalogue’s estimate, roughly 457 kB and 0.521 s per tier binary according to the calibration (Section 6.1); the planner computes the cardinality per profile and offers budget inputs in the subcommand `simulate` (bytes per binary, store budget, measurement subset, candidates). What the user cannot do: materialize the full product space of all axis catalogues — they choose catalogues or subsets, and the number of repetitions per measurement remains a trade-off between effort and confidence [KJ13].
- **No runtime switch between tier binaries:** The hybrid stage is implemented as classification, dock layer, XML parser, and binary proxy (Section 4.5.1); the `.so` export of the hybrid stage, the eviction, and the router are not implemented — reroute and router presuppose measurement curves. What the user cannot do: operate a hybrid tier that switches between preloaded binaries at run time; the state handover on switching — the memento reconciliation with the two release artefacts single and hybrid in the store — is likewise designed, not implemented.
- **Operating-system regime:** The system axis `operating_system` is declared with `linux`, `windows`, and `macos` and possesses one probe each; measurement takes place under root Linux with full counter access, the second regime (immutable operating system) is designed, not implemented (Section 5.1.4), and a Windows counter driver is not available (Table A.10, row 1). What the user cannot do: collect measurement series under Windows or macOS or without perf permissions (`perf_event_paranoid`, RAPL zones).
- **No isolated re-measurement of a single publication:** An operating mode that offers the design constituents of exactly one publication as a closed axis set is not implemented; per publication there exist the profile XML (33) and the replacement merge, not the isolation against all remaining axes. What the user cannot do: measure a publication in isolation, faithfully to the original (Section 6.3).
- **No automatic selection of the best configuration:** The heuristics-driven full automation (classifier and binary packager) is not implemented; present are the measurement-driven increment of the binary selection (ranking per metric from the evaluation output) and the runtime-dynamic cache-line policy. What the user cannot do: have an axis configuration recommended automatically from load-profile and hardware features; the recommendation remains a manual derivation from the evaluation (Section 6.3).

6.3 Outlook

In the short term the following steps are due; implemented parts appear with their open remainder, designed parts with their implementation status:

- (1) the complete build according to the declared scope (Section 5.1.6) and the delivery measurement over the 320-identity catalogue;

- (2) the end-to-end run of the three mandatory measurement series on the registered measurement machines together with the generation of the measurement diagrams;
- (3) the full build-out of the store: the binaries *with* and *without* measurement sensors as two stored release artefacts and the synthesized function curves as files — measurement is always switched on, in the store only what is missing is re-measured (Section 3.4.5); the build-out is not implemented;
- (4) the hardware detection: wiring into runner and probe dock, result write-back, provenance round trip, and fit stamp;
- (5) the sheet fan-out of the *xlsx* workbook per sub-axis permutation plus info sheet — the workbook writer is implemented, the fan-out is not implemented;
- (6) the cross-check of the Intel raw counters on prod2, so that L2 and coherence counters are collected there as well;
- (7) the second operating-system regime (immutable operating system) and the Windows counter driver;
- (8) the paper-isolation mode from RQ2.c: an operating mode that offers the design constituents of exactly one publication as a closed axis set in isolation, in order to measure them — with measurement and system axes remaining free-standing — faithfully to the original;
- (9) the profile connection of the organ meta-meta axis `disk_io` and the permutation build over the scheduling sub-axis;
- (10) the full permutation of the workloads against all living-being binaries (RQ2);
- (11) the evaluation of the H2 criterion after the run of series A.

Heuristic automation and binary delivery. The mechanism demanded by the thesis assignment (sub-task 7), which automatically derives the best configuration from the measurement results, is not implemented in its heuristics-driven *full* automation and is a subject of future work; a first increment of the automatic selection and binary shipping is available as a standalone tool (`best_binary_selector`) that ranks and delivers the best permutation per metric from the evaluation output — the *xlsx* workbook is the output and the default, the CSV form its ordered tree of CSV files generated from it (one folder per sheet, no single child document), and the tool reads the CSV projection, which the seam writes as one flat file (the tree form is planned but not implemented). Besides the selection (commitment 1), the same tool strand comprises the runtime-dynamic cache-line adaptation (commitment 2), which is pertinent to the cache-line awareness of the thesis: from an evaluated profile aggregate the policy selector derives the cache-line policy. The full build-out presupposes three components; the second is available, and toward the first as well as the third the tool provides a first, purely measurement-driven increment: (i) a *heuristic/ML classifier* that maps from the load-profile and axis features (operation mix, key characteristics, hardware identifier) to the recommended axis configuration; (ii) the implemented *XML parser* (import and export including the profile loader) for the heuristic policies stored as an XML load-profile result and reusable per architecture focus; and (iii) a *binary packager* that delivers the overall algorithm selected in this way as a standalone binary behind the unified search-algorithm interface. A prerequisite for all three is that the end-to-end measurement series are fully available — only the robust data basis across all permutations

and load profiles makes a measurement-driven, automatically controlled selection meaningful; without this automation the configuration recommendations of the evaluation scheme from Section 5.2 are derived manually once the data are available.

In the medium to long term: the measurement of the PIM allocator family on PIM hardware⁹ — the allocator A16 is integrated as an axis building block and falls back to the portable aligned allocation without PIM hardware [VIA26] —; the formal verification of selected permutation classes in the style of StarMalloc [RFP24]¹⁰ — the allocator family A13 is integrated, the verification itself is open —; the transfer of the anatomy principle to accelerator memory hierarchies (GPU), without a commitment of cluster availability; and the hybrid components live ranking, state handover, and measurement-error interpolation (Section 3.6.2). The four-repository architecture (Section 4.1) — cache engine, candidate under test, super-repository, manuscript — allows further candidates under test to be brought in analogously to PRT-ART without changing the building-block library: the candidate contract of the build configuration checks the code path and the registry path against each other, so that both halves of the integration point to the same place. Beyond the tree-like search structures treated here, the anatomy principle is extensible via higher-level abstractions to the further structure classes from Section 2.1 — hash-based, spatial, and flat structures —, so that the same measurement and permutation apparatus can be applied to new classes without a rebuild; herein lies an essential lever for future experimental progress.

⁹PIM (Processing-in-Memory): memory devices with computing units of their own, such as UPMEM DPUs or Samsung’s HBM-PIM; a PIM allocator requires such hardware.

¹⁰StarMalloc is verified in F* with the framework Steel, a separation logic for concurrent programs; the verification of an axis building block in this style is outlook.

Bibliography

- [ABKS13] Sven Apel, Don Batory, Christian Kästner, and Gunter Saake. *Feature-Oriented Software Product Lines: Concepts and Implementation*. Springer, 1 edition, 2013.
- [ACK⁺04] Sanjay Agrawal, Surajit Chaudhuri, Lubor Kollár, Arunprasad P. Marathe, Vivek R. Narasayya, and Manoj Syamala. Database tuning advisor for Microsoft SQL server 2005. In *30th International Conference on Very Large Data Bases (VLDB)*, pages 1110–1121, 2004.
- [ADHW99] Anastassia Ailamaki, David J. DeWitt, Mark D. Hill, and David A. Wood. DBMSs on a modern processor: Where does time go? In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*, pages 266–277, 1999.
- [Adv23] Advanced Micro Devices. Software optimization guide for the AMD Zen microarchitecture (family 19h). Advanced Micro Devices, Santa Clara, CA, USA, 2023.
- [AKLS15] Christoph Aigner, Christoph Kirsch, Michael Lippautz, and Ana Sokolova. Fast, multicore-scalable, low-fragmentation memory allocation. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 513–531. ACM, 2015.
- [AKV⁺14] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O’Reilly, and Saman Amarasinghe. Opentuner: An extensible framework for program autotuning. In *23rd International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pages 303–316, 2014.
- [Ale01] Andrei Alexandrescu. *Modern C++ Design: Generic Programming and Design Patterns Applied*. Addison-Wesley, 1 edition, 2001.
- [AMH08] Daniel J. Abadi, Samuel R. Madden, and Nabil Hachem. Column-stores vs. row-stores: How different are they really? In *ACM SIGMOD International Conference on Management of Data*, pages 967–980, 2008.
- [Apa26] Apache Software Foundation. Maven — POM reference. <https://maven.apache.org/pom.html>, accessed 2026-08-01, 2026.
- [APGZ17] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. Automatic database management system tuning through large-scale machine learning. In *ACM SIGMOD International Conference on Management of Data*, pages 1009–1024, 2017.
- [APK⁺22] Michael Andersch, Greg Palmer, Ronny Krashinsky, Nick Stam, Vishal Mehta, Gonzalo Brito, and Sridhar Ramaswamy. NVIDIA Hopper architecture in-depth.

- NVIDIA Technical Blog, 2022-03-22, <https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>, accessed 2026-09-19, 2022.
- [App24] Apple Inc. Apple silicon CPU optimization guide. Apple Inc., Cupertino, CA, USA, Version 4, 2024.
- [APV02] Lars Arge, Octavian Procopiuc, and Jeffrey Scott Vitter. Implementing I/O-efficient data structures using TPIE. In *Algorithms — ESA 2002, 10th Annual European Symposium*, volume 2461 of *Lecture Notes in Computer Science*, pages 88–100. Springer, 2002.
- [Arm24] Arm Limited. Arm architecture reference manual for A-profile architecture (DDI 0487). Arm Limited, Cambridge, UK, <https://developer.arm.com/documentation/ddi0487/latest>, accessed 2026-09-18, 2024.
- [Aßm03] Uwe Aßmann. *Invasive Software Composition*. Springer, 1 edition, 2003.
- [AV88] Alok Aggarwal and Jeffrey Scott Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.
- [AVL62] Georgy Adelson-Velsky and Evgenii Landis. An algorithm for the organization of information. *Doklady Akademii Nauk SSSR*, 146:263–266, 1962.
- [BA01] Jeff Bonwick and Jonathan Adams. Magazines and vmem: Extending the slab allocator to many cpus and arbitrary resources. In *Proceedings of the USENIX Annual Technical Conference (ATC)*. USENIX, 2001.
- [Bay72] Rudolf Bayer. Symmetric binary B-trees: Data structure and maintenance algorithms. *Acta Informatica*, 1(4):290–306, 1972.
- [BBB⁺11] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2):1–7, 2011.
- [BBF09] Gordon Blair, Nelly Bencomo, and Robert B. France. Models@run.time. *Computer*, 42(10):22–27, 2009.
- [BCD⁺02] Michael A. Bender, Richard Cole, Erik D. Demaine, Martin Farach-Colton, and Jack Zito. Two simplified algorithms for maintaining order in a list. In *Algorithms — ESA 2002, 10th Annual European Symposium*, volume 2461 of *Lecture Notes in Computer Science*, pages 152–164. Springer, 2002.
- [BCDFC02] Michael A. Bender, Richard Cole, Erik D. Demaine, and Martin Farach-Colton. Scanning and traversing: Maintaining data for traversals in a memory hierarchy. In *Algorithms — ESA 2002, 10th Annual European Symposium*, volume 2461 of *Lecture Notes in Computer Science*, pages 139–151. Springer, 2002.

- [BDFC00] Michael A. Bender, Erik D. Demaine, and Martin Farach-Colton. Cache-oblivious b-trees. In *Proceedings of the 41st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 399–409. IEEE, 2000.
- [BDFC02] Michael A. Bender, Erik D. Demaine, and Martin Farach-Colton. Efficient tree layout in a multilevel memory hierarchy. In *Algorithms — ESA 2002, 10th Annual European Symposium*, volume 2461 of *Lecture Notes in Computer Science*, pages 165–173. Springer, 2002.
- [BDFC05] Michael A. Bender, Erik D. Demaine, and Martin Farach-Colton. Cache-oblivious b-trees. *SIAM Journal on Computing*, 35(2):341–358, 2005.
- [BDG⁺18] Prasanna Balaprakash, Jack Dongarra, Todd Gamblin, Mary Hall, Jeffrey K. Hollingsworth, Boyana Norris, and Richard Vuduc. Autotuning in high-performance computing applications. *Proceedings of the IEEE*, 106(11):2068–2083, 2018.
- [BDH⁺16] Stephen M. Blackburn, Amer Diwan, Matthias Hauswirth, Peter F. Sweeney, José Nelson Amaral, Tim Brecht, Lubomír Bulej, Cliff Click, Lieven Eeckhout, Sebastian Fischmeister, Daniel Frampton, Laurie J. Hendren, Michael Hind, Antony L. Hosking, Richard E. Jones, Tomas Kalibera, Nathan Keynes, Nathaniel Nystrom, and Andreas Zeller. The truth, the whole truth, and nothing but the truth: A pragmatic guide to assessing empirical evaluations. *ACM Transactions on Programming Languages and Systems*, 38(4):1–20, 2016.
- [Ben75] Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.
- [BFTV24] Antonio Boffa, Paolo Ferragina, Francesco Tosoni, and Giorgio Vinciguerra. CoCo-trie: Data-aware compression and indexing of strings. *Information Systems*, 120:102316, 2024.
- [BGH⁺06] Stephen M. Blackburn, Robin Garner, Chris Hoffmann, Asjad M. Khang, Kathryn S. McKinley, Rotem Bentzur, Amer Diwan, Daniel Feinberg, Daniel Frampton, Samuel Z. Guyer, Martin Hirzel, Antony Hosking, Maria Jump, Han Lee, J. Eliot B. Moss, Aashish Phansalkar, Darko Stefanović, Thomas VanDrunen, Daniel von Dincklage, and Ben Wiederemann. The DaCapo benchmarks: Java benchmarking development and analysis. In *Proceedings of the 21st Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, pages 169–190, 2006.
- [BH23] Johannes Berthold and Dirk Habich. VAMPIR/OTF2 profiling and performance analysis for database workloads. In *Proceedings of the 19th International Workshop on Data Management on New Hardware (DaMoN)*, 2023. Poster presentation.
- [Blo70] Burton H. Bloom. Space/Time Trade-offs in Hash Coding with Allowable Errors. *Communications of the ACM*, 13(7):422–426, 1970.
- [BM72] Rudolf Bayer and Edward M. McCreight. Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3):173–189, 1972.

- [BMBW00] Emery D. Berger, Kathryn S. McKinley, Robert D. Blumofe, and Paul R. Wilson. Hoard: A scalable memory allocator for multithreaded applications. In *Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS IX)*, pages 117–128. ACM, 2000.
- [BO92] Don Batory and Sean O’Malley. The design and implementation of hierarchical software systems with reusable components. *ACM Transactions on Software Engineering and Methodology*, 1(4):355–398, 1992.
- [Bon94] Jeff Bonwick. The slab allocator: An object-caching kernel memory allocator. In *Proceedings of the USENIX Summer 1994 Technical Conference*, pages 87–98. USENIX, 1994.
- [BSB⁺23] A. Berthold, L. Schmidt, A. Bartuschka, W. Lehner, D. Habich, and H. Schirmeier. VAMPIR: Virtualized non-functional memory properties for data-pipeline scheduling. Poster, 29th ACM Symposium on Operating Systems Principles (SOSP 2023), 2023-10-24; poster list <https://sosp2023.mpi-sws.org/cfpo.html>, author list according to the publication list of the Faculty of Computer Science of TU Dresden, both accessed 2026-09-19, 2023.
- [BVA21] Konstantin Boyarinov, Sergey Vinogradov, and Ruslan Arutyunyan. Heterogeneous erasure overloads for associative containers. ISO/IEC JTC1/SC22/WG21 document P2077R3, 2021-10-15, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2077r3.html>, accessed 2026-09-19, 2021.
- [BVA23] Konstantin Boyarinov, Sergey Vinogradov, and Ruslan Arutyunyan. Extending associative containers with the remaining heterogeneous overloads. ISO/IEC JTC1/SC22/WG21 document P2363R5, 2023-02-10, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2363r5.html>, accessed 2026-09-19, 2023.
- [BZP⁺18] Robert Binna, Eva Zangerle, Martin Pichl, Günther Specht, and Viktor Leis. HOT: A height optimized trie index for main-memory database systems. In *ACM SIGMOD International Conference on Management of Data*, pages 521–534, 2018.
- [CB13] Charlie Curtsinger and Emery D. Berger. STABILIZER: Statistically sound performance evaluation. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 219–228, 2013.
- [CE00] Krzysztof Czarnecki and Ulrich W. Eisenecker. *Generative Programming: Methods, Tools, and Applications*. Addison-Wesley, 1 edition, 2000.
- [CGM01] Shimin Chen, Phillip B. Gibbons, and Todd C. Mowry. Improving index performance through prefetching. In *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*, pages 235–246. ACM, 2001.
- [CGM02] Shimin Chen, Phillip B. Gibbons, and Todd C. Mowry. Fractal prefetching b+-trees. In *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data*, pages 157–168. ACM, 2002.

- [CHL99] Trishul M. Chilimbi, Mark D. Hill, and James R. Larus. Cache-conscious structure layout. In *Proceedings of the ACM SIGPLAN 1999 Conference on Programming Language Design and Implementation (PLDI)*, pages 1–12, 1999.
- [CHP71] Pierre-Jacques Courtois, Frans Heymans, and David Lorge Parnas. Concurrent Control with “Readers” and “Writers”. *Communications of the ACM*, 14(10):667–668, 1971.
- [Cli93] Norman Cliff. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin*, 114(3):494–509, 1993.
- [CLP22] Andrew Crotty, Viktor Leis, and Andrew Pavlo. Are You Sure You Want to Use MMAP in Your Database Management System? In *12th Conference on Innovative Data Systems Research (CIDR 2022)*, Chaminade, CA, USA, January 9-12, 2022. www.cidrdb.org, 2022.
- [CLP23] Supawit Chockchowwat, Wenjie Liu, and Yongjoo Park. AirIndex: Versatile index tuning through data and storage. *Proceedings of the ACM on Management of Data*, 1(3):204:1–204:26, 2023. Article no. 204; presented at the ACM SIGMOD Conference 2024.
- [Com79] Douglas Comer. The ubiquitous B-tree. *ACM Computing Surveys*, 11(2):121–137, 1979.
- [Cop95] James O. Coplien. Curiously recurring template patterns. *C++ Report*, pages 24–27, February 1995. <https://dl.acm.org/doi/10.5555/229227.229229>.
- [cpp26a] cppreference.com. `std::map` — Online reference to the C++ standard library. <https://en.cppreference.com/w/cpp/container/map>, accessed 2026-09-19, 2026.
- [cpp26b] cppreference.com. `std::vector` — Online reference to the C++ standard library. <https://en.cppreference.com/w/cpp/container/vector>, accessed 2026-09-19, 2026.
- [Cre13] Creative Commons. Creative commons attribution 4.0 international public license. Legal Code, Version 4.0 of 2013-11-25, <https://creativecommons.org/licenses/by/4.0/legalcode>, accessed 2026-09-19, 2013.
- [CST⁺10] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with YCSB. In *1st ACM Symposium on Cloud Computing (SoCC)*, pages 143–154, 2010.
- [Des14] Cameron Desrochers. `moodycamel::ConcurrentQueue`: A fast multi-producer, multi-consumer lock-free queue for C++. <https://github.com/cameron314/concurrentqueue>, 2014.
- [DGH⁺10] Howard David, Eugene Gorbatoov, Ulf R. Hanebutte, Rahul Khanna, and Christian Le. RAPL: Memory power estimation and capping. In *Proceedings of the 16th ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED)*, pages 189–194, 2010.
- [Dij65] Edsger W. Dijkstra. Solution of a Problem in Concurrent Programming Control. *Communications of the ACM*, 8(9):569, 1965.

- [Dim19] Peter Dimov. Boost.Mp11: A C++11 metaprogramming library. Documentation of the Boost library Mp11 (Copyright 2017–2019), <https://www.boost.org/doc/libs/release/libs/mp11/doc/html/mp11.html>, accessed 2026-09-19, 2019.
- [Dio19] Louis Dionne. Making `std::vector constexpr`. ISO/IEC JTC1/SC22/WG21 document P1004R2, 2019-07-19, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1004r2.pdf>, accessed 2026-09-19, 2019.
- [Dis24] Distributed Management Task Force. System management BIOS (SMBIOS) reference specification, version 3.8.0. Technical Report DSP0134, DMTF, 2024. https://www.dmtf.org/sites/default/files/standards/documents/DSP0134_3.8.0.pdf, accessed 2026-09-19.
- [dlB59] René de la Briandais. File searching using variable length keys. In *Western Joint Computer Conference*, pages 295–298, 1959.
- [DLR77] Arthur P. Dempster, Nan M. Laird, and Donald B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1):1–22, 1977.
- [dM10] Arnaldo Carvalho de Melo. The new linux perf tools. In *Linux Kongress*, pages 1–42, 2010. <http://oldvger.kernel.org/~acme/perf/lk2010-perf-paper.pdf>.
- [DPT⁺13] Justin DeBrabant, Andrew Pavlo, Stephen Tu, Michael Stonebraker, and Stan Zdonik. Anti-caching: A new approach to database management system architecture. *Proceedings of the VLDB Endowment*, 6(14):1942–1953, 2013.
- [Dre07] Ulrich Drepper. What every programmer should know about memory. Technical report, Red Hat, Inc., 2007. White paper, <https://www.akkadia.org/drepper/cpumemory.pdf>.
- [DSST89] James R. Driscoll, Neil Sarnak, Daniel Dominic Sleator, and Robert Endre Tarjan. Making Data Structures Persistent. *Journal of Computer and System Sciences*, 38(1):86–124, 1989.
- [DWKN20] Robert Dietrich, Frank Winkler, Andreas Knüpfer, and Wolfgang E. Nagel. PIKA: Center-wide and job-aware cluster monitoring. In *2020 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 424–432. IEEE, 2020.
- [ED18] Michael T. M. Emmerich and André H. Deutz. A tutorial on multiobjective optimization: Fundamentals and evolutionary methods. *Natural Computing*, 17(3):585–609, 2018.
- [Ehr05] Matthias Ehrgott. *Multicriteria Optimization*. Springer, Berlin, Heidelberg, 2nd edition, 2005.
- [Eva06] Jason Evans. A scalable concurrent malloc(3) implementation for freebsd. In *Proceedings of BSDCan 2006*. BSDCan, 2006. <https://people.freebsd.org/~jasone/jemalloc/bsdcan2006/jemalloc.pdf>.

- [FAK⁺12] Michael Ferdman, Almutaz Adileh, Onur Kocberber, Stavros Volos, Mohammad Alisafae, Djordje Jevdjic, Cansu Kaynak, Adrian Daniel Popescu, Anastasia Ailamaki, and Babak Falsafi. Clearing the clouds: A study of emerging scale-out workloads on modern hardware. In *17th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 37–48, 2012.
- [FAKM14] Bin Fan, David G. Andersen, Michael Kaminsky, and Michael Mitzenmacher. Cuckoo Filter: Practically Better Than Bloom. In *Proceedings of the 10th ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT '14)*, pages 75–88. ACM, 2014.
- [FB74] Raphael A. Finkel and Jon Louis Bentley. Quad trees: A data structure for retrieval on composite keys. *Acta Informatica*, 4(1):1–9, 1974.
- [FC80] F. N. Fritsch and R. E. Carlson. Monotone piecewise cubic interpolation. *SIAM Journal on Numerical Analysis*, 17(2):238–246, 1980.
- [Fea04] Michael C. Feathers. *Working Effectively with Legacy Code*. Prentice Hall PTR, Upper Saddle River, NJ, 1 edition, 2004.
- [FJ98] Matteo Frigo and Steven G. Johnson. FFTW: An adaptive software architecture for the FFT. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, volume 3, pages 1381–1384, 1998.
- [FJKN20] Philipp Fent, Michael Jungmair, Andreas Kipf, and Thomas Neumann. START: Self-tuning adaptive radix tree. In *36th IEEE International Conference on Data Engineering Workshops (ICDEW)*, pages 147–153, 2020.
- [FLPR99] Matteo Frigo, Charles E. Leiserson, Harald Prokop, and Sridhar Ramachandran. Cache-oblivious algorithms. In *40th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 285–297, 1999.
- [Fow06] Martin Fowler. Continuous integration. <https://martinfowler.com/articles/continuousIntegration.html>, version of 2006-05-01, revised 2024-01-18, accessed 2026-09-19, 2006.
- [Fre60] Edward Fredkin. Trie memory. *Communications of the ACM*, 3(9):490–499, 1960.
- [Fre24] Free Software Foundation Europe e.V. REUSE specification — version 3.3. as of 2024-11-14, <https://reuse.software/spec-3.3/>, accessed 2026-09-19, 2024.
- [Fre26] Free Software Foundation. GNU coding standards, Section 7.2.6 standard targets for users. as of 2026-04-27, https://www.gnu.org/prep/standards/html_node/Standard-Targets.html, accessed 2026-09-19, 2026.
- [FW86] Philip J. Fleming and John J. Wallace. How not to lie with statistics: The correct way to summarize benchmark results. *Communications of the ACM*, 29(3):218–221, 1986.

- [FW93] Michael L. Fredman and Dan E. Willard. Surpassing the information theoretic bound with fusion trees. *Journal of Computer and System Sciences*, 47(3):424–436, 1993.
- [Gai86] Jason Gait. A probe effect in concurrent programs. *Software: Practice and Experience*, 16(3):225–233, 1986.
- [GBE07] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous Java performance evaluation. In *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, pages 57–76, 2007.
- [GHJV94] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1 edition, 1994.
- [Git26] Git Project. Git — gitmodules documentation. Git Reference Documentation, as of Git 2.54.0 (2026-04-20), <https://git-scm.com/docs/gitmodules>, accessed 2026-09-19, 2026.
- [GL01] Goetz Graefe and Per-Åke Larson. B-tree indexes and cpu caches. In *Proceedings of the 17th IEEE International Conference on Data Engineering (ICDE)*, pages 349–358. IEEE, 2001.
- [GL20] Thomas Mueller Graf and Daniel Lemire. Xor Filters: Faster and Smaller Than Bloom and Cuckoo Filters. *ACM Journal of Experimental Algorithmics*, 25(1):1.5:1–1.5:16, 2020.
- [GM05] Sanjay Ghemawat and Paul Menage. TCMalloc: Thread-caching malloc. Technical report, Google, 2005. Design documentation.
- [GMUW09] Hector Garcia-Molina, Jeffrey D. Ullman, and Jennifer Widom. *Database Systems: The Complete Book*. Pearson Prentice Hall, Upper Saddle River, NJ, USA, 2 edition, 2009.
- [Goo18] Google. Abseil swiss tables (flat_hash_map). <https://abseil.io/about/design/swisstables>, 2018.
- [Göt13] Sebastian Götz. *Multi-Quality Auto-Tuning by Contract Negotiation*. PhD thesis, Technische Universität Dresden, 2013. URN: urn:nbn:de:bsz:14-qucosa-119938.
- [Gra09] John W. Graham. Missing data analysis: Making it work in the real world. *Annual Review of Psychology*, 60(1):549–576, 2009.
- [GS78] Leo J. Guibas and Robert Sedgewick. A dichromatic framework for balanced trees. In *19th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 8–21, 1978.
- [GTOC06] John W. Graham, Bonnie J. Taylor, Allison E. Olchowski, and Patricio E. Cumsille. Planned missing data designs in psychological research. *Psychological Methods*, 11(4):323–343, 2006.
- [Gut84] Antonin Guttman. R-trees: A dynamic index structure for spatial searching. In *ACM SIGMOD International Conference on Management of Data*, pages 47–57, 1984.

- [HB15] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- [HBHR11] Jörg Herter, Peter Backes, Florian Hauptenthal, and Jan Reineke. CAMA: A predictable cache-aware memory allocator. In *23rd Euromicro Conference on Real-Time Systems (ECRTS)*, pages 23–32, 2011.
- [Her91] Maurice Herlihy. Wait-Free Synchronization. *ACM Transactions on Programming Languages and Systems*, 13(1):124–149, 1991.
- [HF96] Rob J. Hyndman and Yanan Fan. Sample quantiles in statistical packages. *The American Statistician*, 50(4):361–365, 1996.
- [HF10] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 1 edition, 2010.
- [HIKY12] Felix Halim, Stratos Idreos, Panagiotis Karras, and Roland H. C. Yap. Stochastic database cracking: Towards robust adaptive indexing in main-memory column-stores. *Proceedings of the VLDB Endowment*, 5(6):502–513, 2012.
- [HKT⁺21] A. H. Hunter, Chris Kennelly, Paul Turner, Darryl Gove, Tipp Moseley, and Parthasarathy Ranganathan. Beyond malloc efficiency to fleet efficiency: A hugepage-aware memory allocator. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 257–273, 2021. <https://www.usenix.org/conference/osdi21/presentation/hunter>.
- [HMPR04] Alan R. Hevner, Salvatore T. March, Jinsoo Park, and Sudha Ram. Design science in information systems research. *MIS Quarterly*, 28(1):75–106, 2004.
- [Hol79] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [HP03] Richard A. Hankins and Jignesh M. Patel. Effect of node size on the performance of cache-conscious b+-trees. In *Proceedings of the 2003 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, pages 120–131. ACM, 2003.
- [HP19] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6th edition, 2019.
- [HS89] Mark D. Hill and Alan Jay Smith. Evaluating associativity in CPU caches. *IEEE Transactions on Computers*, 38(12):1612–1630, 1989.
- [IEE19] IEEE. IEEE standard for floating-point arithmetic (IEEE std 754-2019). IEEE, New York, NY, USA, 2019.

- [IKM07] Stratos Idreos, Martin L. Kersten, and Stefan Manegold. Database cracking. In *3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, pages 68–78, 2007.
- [Int24a] Intel Corporation. Intel 64 and IA-32 architectures optimization reference manual. Intel Corporation, Santa Clara, CA, USA, 2024.
- [Int24b] Intel Corporation. Intel 64 and IA-32 architectures software developer’s manual, volume 3a: System programming guide. Intel Corporation, Santa Clara, CA, USA, <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>, accessed 2026-09-18, 2024.
- [Int26] Intel Corporation. Intel architecture instruction set extensions and future features programming reference. Document Number 319433-061, Intel Resource and Documentation Center, document ID 915637, <https://cdrdv2.intel.com/v1/dl/getContent/915637>, accessed 2026-09-19, 2026.
- [ISO24] ISO/IEC. ISO/IEC 14882:2024 — programming languages — c++. International Organization for Standardization, Geneva, Switzerland, <https://www.iso.org/standard/83626.html>, accessed 2026-09-18, 2024.
- [IZA⁺18] Stratos Idreos, Kostas Zoumpatianos, Manos Athanassoulis, Niv Dayan, Brian Hentschel, Michael S. Kester, Demi Guo, Lukas M. Maas, Wilson Qin, Abdul Wasay, and Yiyu Sun. The periodic table of data structures. *IEEE Data Engineering Bulletin*, 41(3):64–75, 2018.
- [IZH⁺18] Stratos Idreos, Kostas Zoumpatianos, Brian Hentschel, Michael S. Kester, and Demi Guo. The data calculator: Data structure design and cost synthesis from first principles, and learned cost models. In *ACM SIGMOD International Conference on Management of Data*, pages 535–550, 2018.
- [Jac89] Guy J. Jacobson. Space-efficient static trees and graphs. In *Proceedings of the 30th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 549–554. IEEE, 1989.
- [Jan17] Mattias Jansson. rpmalloc: A public domain license per-thread memory allocator. GitHub Repository, 2017.
- [JED20] JEDEC Solid State Technology Association. DDR5 SDRAM standard (JESD79-5). Technical report, JEDEC Solid State Technology Association, Arlington, VA, USA, July 2020. Norm.
- [JED25] JEDEC Solid State Technology Association. DDR5 serial presence detect (SPD) contents. Technical Report JESD400-5D, JEDEC, 2025. <https://www.jedec.org/standards-documents/docs/jesd400-5d01>, accessed 2026-09-19.
- [JNC22] Corentin Jabot, Eric Niebler, and Casey Carter. Conversions from ranges to containers. ISO/IEC JTC1/SC22/WG21 document P1206R7, 2022-01-21, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p1206r7.pdf>, accessed 2026-09-19, 2022.

- [JTJE10] Aamer Jaleel, Kevin B. Theobald, Simon C. Steely Jr., and Joel Emer. High performance cache replacement using re-reference interval prediction (RRIP). In *37th Annual International Symposium on Computer Architecture (ISCA)*, pages 60–71, 2010.
- [KAS21] Neal Koyle, Sujeet Ayyapureddi, and Mike Seibert. Micron DDR5: Client Module Features. White Paper CCM004676576390-11566, Rev. A, Micron Technology, Inc., October 2021. Online-Ressource.
- [KAZ18] Oliver Kopp, Anita Armbruster, and Olaf Zimmermann. Markdown architectural decision records: Format and tool support. In *Proceedings of the 10th Central European Workshop on Services and their Composition (ZEUS 2018), Dresden, Germany, February 8–9, 2018*, volume 2072 of *CEUR Workshop Proceedings*, pages 55–62. CEUR-WS.org, 2018. <https://ceur-ws.org/Vol-2072/paper9.pdf>, accessed 2026-09-19.
- [KBC⁺18] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. The case for learned index structures. In *ACM SIGMOD International Conference on Management of Data*, pages 489–504, 2018.
- [KBD⁺08] Andreas Knüpfer, Holger Brunst, Jens Doleschal, Matthias Jurenz, Matthias Lieber, Holger Mickler, Matthias S. Müller, and Wolfgang E. Nagel. The Vampir performance analysis tool-set. In *Tools for High Performance Computing*, pages 139–155. Springer, Berlin, Heidelberg, 2008.
- [KCH⁺90] Kyo C. Kang, Sholom G. Cohen, James A. Hess, William E. Novak, and A. Spencer Peterson. Feature-oriented domain analysis (FODA) feasibility study. Technical Report CMU/SEI-90-TR-021, Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA, USA, 1990. Technical Report.
- [KCS⁺10] Changkyu Kim, Jatin Chhugani, Nadathur Satish, Eric Sedlar, Anthony D. Nguyen, Tim Kaldewey, Victor W. Lee, Scott A. Brandt, and Pradeep Dubey. FAST: Fast architecture sensitive tree search on modern CPUs and GPUs. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 339–350, 2010.
- [Kha11] Minhaj Ahmad Khan. Data cache prefetching with dynamic adaptation. *The Computer Journal*, 54(5):815–823, 2011.
- [KHN⁺18] Kashif Nizam Khan, Mikael Hirki, Tapio Niemi, Jukka K. Nurminen, and Zhonghong Ou. RAPL in action: Experiences in using RAPL for power measurements. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, 3(2):1–26, 2018.
- [Kit26] Kitware, Inc. CMake documentation: cmake(1). <https://cmake.org/cmake/help/latest/manual/cmake.1.html>, accessed 2026-08-01, 2026.
- [KJ13] Tomas Kalibera and Richard Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, pages 63–74, 2013.

- [KM17] Paul-Virak Khuong and Pat Morin. Array Layouts for Comparison-Based Searching. *ACM Journal of Experimental Algorithmics*, 22:1.3:1–1.3:39, 2017.
- [KMvR⁺19] Andreas Kipf, Ryan Marcus, Alexander van Renen, Mihail Stoian, Alfons Kemper, Tim Kraska, and Thomas Neumann. SOSD: A benchmark for learned indexes. In *NeurIPS Workshop on Machine Learning for Systems*, 2019.
- [Kno65] Kenneth C. Knowlton. A fast storage allocator. *Communications of the ACM*, 8(10):623–625, 1965.
- [KNR11] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A Distributed Messaging System for Log Processing. In *6th International Workshop on Networking Meets Databases (NetDB)*, pages 1–7, 2011.
- [Knu98] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd edition, 1998.
- [Kro81] David Kroft. Lockup-free instruction fetch/prefetch cache organization. In *Proceedings of the 8th Annual Symposium on Computer Architecture (ISCA)*, pages 81–87, 1981.
- [Kue23] Lena Kuehn. Towards data-based cache optimization of b+-trees. In *Proceedings of the 19th International Workshop on Data Management on New Hardware (DaMoN)*, pages 8:1–8:8. ACM, 2023.
- [LABN16] Viktor Leis, Michael Andersen, Erik Birkedal, and Thomas Neumann. The art of practical synchronization. In *Proceedings of the International Workshop on Data Management on New Hardware (DaMoN)*, pages 8:1–8:8. ACM, 2016.
- [Lam77] Leslie Lamport. Concurrent Reading and Writing. *Communications of the ACM*, 20(11):806–811, 1977.
- [Lam83] Leslie Lamport. Specifying Concurrent Program Modules. *ACM Transactions on Programming Languages and Systems*, 5(2):190–222, 1983.
- [Lam13] Christoph Lameter. NUMA (non-uniform memory access): An overview. *ACM Queue*, 11(7):40–51, 2013.
- [LC86] Tobin J. Lehman and Michael J. Carey. A study of index structures for main memory database management systems. In *12th International Conference on Very Large Data Bases (VLDB)*, pages 294–303, 1986.
- [LC01] Steff Lewis and Mike Clarke. Forest plots: Trying to see the wood and the trees. *BMJ*, 322(7300):1479–1480, 2001.
- [Lc21] Daan Leijen and contributors. mimalloc-bench: Suite for benchmarking malloc implementations. <https://github.com/daanx/mimalloc-bench>, 2021.

- [LCL⁺25] Meng Li, Huayi Chai, Siqiang Luo, Haipeng Dai, Rong Gu, Jiaqi Zheng, and Guihai Chen. VEGA: An active-tuning learned index with group-wise learning granularity. *Proceedings of the ACM on Management of Data*, 3(1):86:1–86:26, 2025. SIGMOD 2025; article no. 86.
- [LDPB19] Paul Lietar, Derek Dreyer, Adam Parkinson, and Richard Bornat. smalloc: A message passing allocator. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on Memory Management (ISMM)*, pages 122–135. ACM, 2019.
- [Lea96] Doug Lea. A memory allocator. Design article and C implementation, 1996.
- [Lin25] Linux Kernel Community. Linux kernel source tree, tag v6.17: perf PMU event tables for AMD Zen 5 (tools/perf/pmu-events/arch/x86/amdzen5, files l2-cache.json and load-store.json). <https://github.com/torvalds/linux/tree/v6.17/tools/perf/pmu-events/arch/x86/amdzen5>, accessed 2026-09-19, 2025.
- [Lin26a] Linux Kernel Organization. RISC-V hardware probing interface. Linux Kernel Documentation, <https://docs.kernel.org/arch/riscv/hwprobe.html>, accessed 2026-09-18, 2026.
- [Lin26b] Linux man-pages project. perf_event_open(2) — linux manual page. https://man7.org/linux/man-pages/man2/perf_event_open.2.html, accessed 2026-09-19, 2026.
- [LJY25] Ruihao Li, Lizy K. John, and Neeraja J. Yadwadkar. Old is gold: Optimizing single-threaded applications with Exgen-Malloc. *IEEE Computer Architecture Letters*, 24(2), 2025.
- [LKN13] Viktor Leis, Alfons Kemper, and Thomas Neumann. The adaptive radix tree: ARTful indexing for main-memory databases. In *29th IEEE International Conference on Data Engineering (ICDE)*, pages 38–49, 2013.
- [LLS13] Justin J. Levandoski, David B. Lomet, and Sudipta Sengupta. The Bw-Tree: A B-tree for New Hardware Platforms. In *29th IEEE International Conference on Data Engineering (ICDE 2013), Brisbane, Australia, April 8-12, 2013*, pages 302–313. IEEE Computer Society, 2013.
- [LNOM08] Erik Lindholm, John Nickolls, Stuart Oberman, and John Montrym. NVIDIA Tesla: A unified graphics and computing architecture. *IEEE Micro*, 28(2):39–55, 2008.
- [LR18] Ricardo Leite and Rui Rocha. Lrmalloc: A modern and competitive lock-free dynamic memory allocator. In *Proceedings of the 13th International Conference on Parallel Processing and Applied Mathematics (VECPAR)*, pages 188–200. Springer, 2018.
- [LYTL19] Tianlin Li, Yiping Yao, Wenjie Tang, and Zhongwei Lin. HMalloc: A Hybrid, Scalable, and Lock-Free Memory Allocator for Multi-Threaded Applications. In *2019 IEEE 25th International Conference on Parallel and Distributed Systems (ICPADS)*, pages 406–409, 2019.

- [LZdM19] Daan Leijen, Benjamin Zorn, and Leonardo de Moura. Mimalloc: Free list sharding in action. In *Proceedings of the 17th Asian Symposium on Programming Languages and Systems (APLAS)*, pages 172–192. Springer, 2019.
- [LZS⁺23] Xuchuan Luo, Pengfei Zuo, Jiacheng Shen, Jiazhen Gu, Xin Wang, Michael R. Lyu, and Yangfan Zhou. SMART: A high-performance adaptive radix tree for disaggregated memory. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023.
- [MBL25] Marcus Müller, Lawrence Benson, and Viktor Leis. B-trees are back: Engineering fast and pageable node layouts. *Proceedings of the ACM on Management of Data*, 3(1), 2025.
- [McK01] Paul E. McKenney. Read-copy update. In *Proceedings of the 2001 Ottawa Linux Symposium (OLS)*, pages 338–367, 2001.
- [MDHS09] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 265–276, 2009.
- [Mey05] Scott Meyers. *Effective C++: 55 Specific Ways to Improve Your Programs and Designs*. Addison-Wesley Professional, 3 edition, 2005.
- [MF02] Giovanni Manzini and Paolo Ferragina. Engineering a lightweight suffix array construction algorithm. In *Algorithms — ESA 2002, 10th Annual European Symposium*, volume 2461 of *Lecture Notes in Computer Science*, pages 698–710. Springer, 2002.
- [Mic04a] Maged M. Michael. Hazard pointers: Safe memory reclamation for lock-free objects. *IEEE Transactions on Parallel and Distributed Systems*, 15(6):491–504, 2004.
- [Mic04b] Maged M. Michael. Scalable lock-free dynamic memory allocation. In *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation (PLDI)*, pages 35–46. ACM, 2004.
- [Mit16] Sparsh Mittal. A survey of recent prefetching techniques for processor caches. *ACM Computing Surveys*, 49(2):35:1–35:35, 2016.
- [MKM12] Yandong Mao, Eddie Kohler, and Robert Tappan Morris. Cache craftiness for fast multicore key-value storage. In *7th ACM European Conference on Computer Systems (EuroSys)*, pages 183–196, 2012.
- [MMP18] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP):79:1–79:29, 2018.
- [Mor68] Donald R. Morrison. PATRICIA — practical algorithm to retrieve information coded in alphanumeric. *Journal of the ACM*, 15(4):514–534, 1968.

- [MPU⁺23] Francesco Minervini, Oscar Palomar, Osman Unsal, Enrico Reggiani, Josue Quiroga, Joan Marimon, et al. Vitruvius+: An area-efficient RISC-V decoupled vector coprocessor for high performance computing applications. *ACM Transactions on Architecture and Code Optimization*, 20(2), 2023.
- [MS96] Maged M. Michael and Michael L. Scott. Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms. In *Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, pages 267–275. ACM, 1996.
- [MS08] Kurt Mehlhorn and Peter Sanders. *Algorithms and Data Structures: The Basic Toolbox*. Springer, 1 edition, 2008.
- [MTL78] Robert McGill, John W. Tukey, and Wayne A. Larsen. Variations of box plots. *The American Statistician*, 32(1):12–16, 1978.
- [MW47] Henry B. Mann and Donald R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1):50–60, 1947.
- [MWR25] Fabian Mahling, Marcel Weisgut, and Tilmann Rabl. Fetch me if you can: Evaluating CPU cache prefetching and its reliability on high latency memory. In *Proceedings of the 21st International Workshop on Data Management on New Hardware (DaMoN)*. ACM, 2025.
- [NR21] Ruslan Nikolaev and Binoy Ravindran. Brief announcement: Crystalline: Fast and memory efficient wait-free reclamation. In *35th International Symposium on Distributed Computing (DISC)*, volume 209 of *LIPIcs*, pages 60:1–60:4, 2021.
- [NR24] Ruslan Nikolaev and Binoy Ravindran. A family of fast and memory efficient lock- and wait-free reclamation. *Proceedings of the ACM on Programming Languages*, 8(PLDI), 2024.
- [NTSA16] Mahmood Naderan-Tahan and Hamid Sarbazi-Azad. Why does data prefetching not work for modern workloads? *The Computer Journal*, 59(2):244–259, 2016.
- [Nyg11] Michael Nygard. Documenting architecture decisions. Blog post, Cognitect, 2011-11-15, <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>, accessed 2026-09-19, 2011.
- [Obj16] Object Management Group. Meta object facility (MOF) core specification, version 2.5.1. OMG Document formal/19-10-01, <https://www.omg.org/spec/MOF/2.5.1>, accessed 2026-09-19, 2016.
- [OCGO96] Patrick E. O’Neil, Edward Cheng, Dieter Gawlick, and Elizabeth J. O’Neil. The Log-Structured Merge-Tree (LSM-Tree). *Acta Informatica*, 33(4):351–385, 1996.
- [PBvdL05] Klaus Pohl, Günter Böckle, and Frank van der Linden. *Software Product Line Engineering*. Springer, Berlin, Heidelberg, 1 edition, 2005.

- [PIA78] Yehoshua Perl, Alon Itai, and Haim Avni. Interpolation Search - A Log Log N Search. *Communications of the ACM*, 21(7):550–553, 1978.
- [Pik18] Colby Pike. cxx-pflR1: The pitchfork layout (PFL). Specification `data/spec.bs` in the repository `vector-of-bool/pitchfork`, <https://github.com/vector-of-bool/pitchfork>, accessed 2026-09-19, 2018.
- [PM12] Matt Pharr and William R. Mark. ispc: A SPMD compiler for high-performance CPU programming. In *2012 Innovative Parallel Computing (InPar)*, pages 1–13. IEEE, 2012.
- [PR04] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. *Journal of Algorithms*, 51(2):122–144, 2004.
- [PTRC07] Ken Peffers, Tuure Tuunanen, Marcus A. Rothenberger, and Samir Chatterjee. A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3):45–77, 2007.
- [Pug90] William W. Pugh. Skip Lists: A Probabilistic Alternative to Balanced Trees. *Communications of the ACM*, 33(6):668–676, 1990.
- [QJP⁺07] Moinuddin K. Qureshi, Aamer Jaleel, Yale N. Patt, Simon C. Steely Jr., and Joel Emer. Adaptive insertion policies for high performance caching. In *34th Annual International Symposium on Computer Architecture (ISCA)*, pages 381–391, 2007.
- [Rei05] James Reinders. *VTune Performance Analyzer Essentials: Measurement and Tuning Techniques for Software Developers*. Intel Press, Hillsboro, OR, USA, 1 edition, 2005.
- [RFP24] Antonin Reitz, Aymeric Fromherz, and Jonathan Protzenko. StarMalloc: Verifying a modern, hardened memory allocator. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA2), 2024.
- [RH09] Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009.
- [RIS21] RISC-V International. RISC-V V vector extension, version 1.0. https://docs.riscv.org/reference/isa/extensions/vector/_attachments/riscv-v-spec.pdf, 2021.
- [Rob06] Ian Robinson. Consumer-driven contracts: A service evolution pattern. <https://martinfowler.com/articles/consumerDrivenContracts.html>, 2006-06-12, accessed 2026-09-19, 2006.
- [RR99] Jun Rao and Kenneth A. Ross. Cache conscious indexing for decision-support in main memory. In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*, pages 78–89, 1999.
- [RR00] Jun Rao and Kenneth A. Ross. Making b+-trees cache conscious in main memory. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pages 475–486. ACM, 2000.

- [Rub76] Donald B. Rubin. Inference and missing data. *Biometrika*, 63(3):581–592, 1976.
- [SBB⁺17] Nigel Stephens, Stuart Biles, Matthias Boettcher, Mbou Eyole, Giacomo Gabrielli, Matt Horsnell, Grigorios Magklis, Alejandro Martinez, Nathanael Premillieu, Alastair Reid, Alejandro Rico, and Paul Walker. The ARM scalable vector extension. *IEEE Micro*, 37(2):26–39, 2017.
- [Sch06] Douglas C. Schmidt. Guest editor’s introduction: Model-driven engineering. *Computer*, 39(2):25–31, 2006.
- [SDF⁺00] Jagannathan Srinivasan, Souripriya Das, Chuck Freiwald, Eugene Inseok Chong, Mahesh Jagannath, Aravind Yalamanchi, Ramkumar Krishnan, Anh-Tuan Tran, Samuel DeFazio, and Jayanta Banerjee. Oracle8i index-organized table and its application to new domains. In *26th International Conference on Very Large Data Bases (VLDB)*, pages 285–296, 2000.
- [SGL09] Benjamin Schlegel, Rainer Gemulla, and Wolfgang Lehner. k-Ary Search on Modern Processors. In *Proceedings of the Fifth International Workshop on Data Management on New Hardware (DaMoN 2009)*, pages 52–60. ACM, 2009.
- [Sha03] Mary Shaw. Writing good software engineering research papers. In *Proceedings of the 25th International Conference on Software Engineering (ICSE)*, pages 726–736, 2003.
- [Sid26] Sidero Labs. Talos linux — linux OS for Kubernetes. Product page, <https://www.siderolabs.com/talos-linux/> (talos.dev redirects there), accessed 2026-09-19, 2026.
- [SKK⁺25] Lennart Schmidt, Roland Kühn, Matti Krause, Jens Teubner, Wolfgang Lehner, and Dirk Habich. To stride or not to stride the memory access? In *Proceedings of the 3rd Workshop on Disruptive Memory Systems (DIMES)*, pages 19–26. ACM, 2025.
- [SPB05] Curt Samuel, Torben Bach Pedersen, and Philippe Bonnet. Processor-conscious indexing. In *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*, pages 235–246. ACM, 2005.
- [SS12] Andrew Sutton and Bjarne Stroustrup. Design of concept libraries for C++. In *Software Language Engineering (SLE 2011), Lecture Notes in Computer Science 6940*, pages 97–118, Berlin, Heidelberg, 2012. Springer.
- [SSL⁺21] Josef Schmeißer, Maximilian E. Schüle, Viktor Leis, Thomas Neumann, and Alfons Kemper. The B²-tree: Cache-friendly string indexing within B-trees. In *Datenbanksysteme für Business, Technologie und Web (BTW)*, pages 39–58, 2021.
- [SSS08] Riku Saikkonen and Eljas Soisalon-Soininen. Cache-sensitive memory layout for binary trees. In *Fifth IFIP International Conference on Theoretical Computer Science (TCS 2008)*, volume 273 of *IFIP Advances in Information and Communication Technology*. Springer, 2008.

- [SSS16] Riku Saikkonen and Eljas Soisalon-Soininen. Cache-sensitive memory layout for dynamic binary trees. *The Computer Journal*, 59(5):630–649, 2016.
- [Sta17] Standard Performance Evaluation Corporation. SPEC CPU 2017 benchmark suite. <https://www.spec.org/cpu2017/>, 2017.
- [Sto81] Michael Stonebraker. Operating System Support for Database Management. *Communications of the ACM*, 24(7):412–418, 1981.
- [Str12] Bjarne Stroustrup. Foundations of C++. In *Programming Languages and Systems (ESOP 2012), Lecture Notes in Computer Science 7211*, pages 1–25, Berlin, Heidelberg, 2012. Springer.
- [SVS16] Mike Spertus, Faisal Vali, and Richard Smith. Template argument deduction for class templates (rev. 6). ISO/IEC JTC1/SC22/WG21 document P0091R3, 2016-06-24, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0091r3.html>, accessed 2026-09-19, 2016.
- [ȚCH02] Cristian Țăpuș, I-Hsin Chung, and Jeffrey K. Hollingsworth. Active harmony: Towards automated performance tuning. In *ACM/IEEE Conference on Supercomputing (SC)*, pages 1–11, 2002.
- [Ten12] Gil Tene. HdrHistogram: A high dynamic range (hdr) histogram. <https://github.com/HdrHistogram/HdrHistogram>, 2012.
- [TFB⁺11] Martin Thompson, Dave Farley, Michael Barker, Patricia Gee, and Andrew Stewart. Disruptor: High Performance Alternative to Bounded Queues for Exchanging Data Between Concurrent Threads. Technical paper, LMAX, May 2011. Version 1.0.
- [The18] The Open Group. dlopen — open a symbol table handle. the open group base specifications issue 7, 2018 edition (IEEE std 1003.1-2017). <https://pubs.opengroup.org/onlinepubs/9699919799/functions/dlopen.html>, accessed 2026-09-19, 2018.
- [TLH94] J. Torrellas, H. S. Lam, and J. L. Hennessy. False sharing and spatial locality in multiprocessor caches. *IEEE Transactions on Computers*, 43(6):651–663, 1994.
- [TMS⁺16] Ghizlane Tibba, Christoph Malz, Christoph Stoermer, Natarajan Nagarajan, Licong Zhang, and Samarjit Chakraborty. Testing automotive embedded systems under X-in-the-loop setups. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–8, 2016.
- [Tra24] Transaction Processing Performance Council. TPC benchmark C and TPC-DS specifications. Transaction Processing Performance Council, San Francisco, CA, USA, 2024.
- [TWHD16] Alan Talbot, Jonathan Wakely, Howard Hinnant, and James Dennett. Splicing Maps and Sets (revision 5). ISO/IEC JTC1/SC22/WG21 document P0083R3, 2016-06-24, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0083r3.pdf>, accessed 2026-09-19, 2016.

- [UHL⁺17] Sigmund Ungethuen, Dirk Habich, Wolfgang Lehner, et al. Hardware acceleration for database systems: A comprehensive survey. *ACM Computing Surveys*, 50(1):3:1–3:35, 2017.
- [vdKHA⁺19] Erik van der Kouwe, Gernot Heiser, Dennis Andriesse, Herbert Bos, and Cristiano Giuffrida. SoK: Benchmarking flaws in systems security. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 310–325, 2019.
- [vEB75] Peter van Emde Boas. Preserving order in a forest in less than logarithmic time. In *16th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 75–84, 1975.
- [VIA26] VIA Research. Pim-malloc: A fast and scalable dynamic memory allocator for processing-in-memory systems. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2026. Accepted to HPCA 2026; also available as arXiv:2505.13002 (2025).
- [VJG17] David Vandevorde, Nicolai M. Josuttis, and Douglas Gregor. *C++ Templates: The Complete Guide*. Addison-Wesley, 2nd edition, 2017.
- [vRLK⁺18] Alexander van Renen, Viktor Leis, Alfons Kemper, Thomas Neumann, Takushi Hashida, Kazuichi Oe, Yoshiyasu Doi, Lilian Harada, and Mitsuru Sato. Managing non-volatile memory in database systems. In *ACM SIGMOD International Conference on Management of Data*, pages 1541–1555, 2018.
- [VRM⁺18] Giuseppe Vietri, Liana V. Rodriguez, Wendy A. Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. Driving cache replacement with ML-based LeCaR. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2018.
- [Vyu11] Dmitry Vyukov. Bounded MPMC queue. <https://www.1024cores.net/home/lock-free-algorithms/queues/bounded-mpmc-queue>, 2011.
- [WF09] Leland Wilkinson and Michael Friendly. The history of the cluster heat map. *The American Statistician*, 63(2):179–184, 2009.
- [WG68] Martin B. Wilk and Ramanathan Gnanadesikan. Probability plotting methods for the analysis of data. *Biometrika*, 55(1):1–17, 1968.
- [Wie14] Roel J. Wieringa. *Design Science Methodology for Information Systems and Software Engineering*. Springer, Berlin, Heidelberg, 1 edition, 2014.
- [Wil64] J. W. J. Williams. Algorithm 232: Heapsort. *Communications of the ACM*, 7(6):347–348, 1964.
- [Wil83] Dan E. Willard. Log-logarithmic worst-case range queries are possible in space $\Theta(N)$. *Information Processing Letters*, 17(2):81–84, 1983.

- [WLLM13] Jonathan Wakely, Stephan T. Lavavej, and Joaquín M. López Muñoz. Adding heterogeneous comparison lookup to associative containers (rev 4). ISO/IEC JTC1/SC22/WG21 document N3657, 2013-03-19, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3657.htm>, accessed 2026-09-19, 2013.
- [WM95] William A. Wulf and Sally A. McKee. Hitting the memory wall: Implications of the obvious. *ACM SIGARCH Computer Architecture News*, 23(1):20–24, 1995.
- [WM08] Vincent M. Weaver and Sally A. McKee. Can hardware performance counters be trusted? In *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*, pages 141–150, 2008.
- [WNJ19] Xingbo Wu, Fan Ni, and Song Jiang. Wormhole: A fast ordered index for in-memory data management. In *14th ACM European Conference on Computer Systems (EuroSys)*, 2019.
- [WPD01] R. Clint Whaley, Antoine Petit, and Jack J. Dongarra. Automated empirical optimization of software and the ATLAS project. *Parallel Computing*, 27(1–2):3–35, 2001.
- [WRH⁺12] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg, 1 edition, 2012.
- [Xu24] Chuanqi Xu. Modules ABI Requirement. Technical Report P3092R0, ISO/IEC JTC1/SC22/WG21, January 2024. Online-Ressource.
- [YZZ⁺23] Hanmei Yang, Xin Zhao, Jin Zhou, Wei Wang, Sandip Kundu, Bo Wu, Hui Guan, and Tongping Liu. NUMAlloc: A faster NUMA memory allocator. In *2023 ACM SIGPLAN International Symposium on Memory Management (ISMM)*, 2023.
- [ZDL⁺13] Matei Zaharia, Tathagata Das, Haoyuan Li, Timothy Hunter, Scott Shenker, and Ion Stoica. Discretized Streams: Fault-Tolerant Streaming Computation at Scale. In *24th ACM Symposium on Operating Systems Principles (SOSP)*, pages 423–438, 2013.
- [Zen26a] Zentrum für Informationsdienste und Hochleistungsrechnen (ZIH), TU Dresden. HPC compendium: HPC resources (hardware overview) and CPU cluster barnard. https://compendium.hpc.tu-dresden.de/jobs_and_resources/hardware_overview/, accessed 2026-09-19, 2026.
- [Zen26b] Zentrum für Informationsdienste und Hochleistungsrechnen (ZIH), TU Dresden. HPC compendium: NVIDIA Grace Hopper superchip (test system). https://compendium.hpc.tu-dresden.de/jobs_and_resources/nvidia_grace_hopper/, accessed 2026-09-19, 2026.
- [ZGH⁺25] Tingji Zhang, Boris Grot, Wenjian He, Yashuai Lv, Peng Qu, Fang Su, Wenxin Wang, Guowei Zhang, Xuefeng Zhang, and Youhui Zhang. Hierarchical prefetching: A software-hardware instruction prefetcher for server applications. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 2*, pages 529–544. ACM, 2025.

- [ZGV⁺24] Zhuangzhuang Zhou, Vaibhav Gogte, Nilay Vaish, Chris Kennelly, Patrick Xia, Svilen Kanev, Tipp Moseley, Christina Delimitrou, and Parthasarathy Ranganathan. Characterizing a memory allocator at warehouse scale. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 192–206, 2024.
- [ZL77] Jacob Ziv and Abraham Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337–343, 1977.
- [ZLL⁺18] Huanchen Zhang, Hyeontaek Lim, Viktor Leis, David G. Andersen, Michael Kaminsky, Kimberly Keeton, and Andrew Pavlo. SuRF: Practical range query filtering with fast succinct tries. In *ACM SIGMOD International Conference on Management of Data*, pages 323–336, 2018.
- [ZSZ⁺24] Qian Zhang, Haoyun Song, Kaiyan Zhou, Jianhao Wei, and Chuqiao Xiao. A prefetching indexing scheme for in-memory database systems. *Future Generation Computer Systems*, 156:179–190, 2024.

List of Figures

1.1	The “cache wall”: typical orders of magnitude of the access latency per level of the memory hierarchy (literature values after [HP19, Dre07]); across the hierarchy the latency grows by about two orders of magnitude.	5
1.2	The separability problem: as a monolith it remains unclear which design decision takes effect; only the decomposition into orthogonal axes makes each contribution measurable in isolation.	6
2.1	The landscape of search structures: five classes with typical representatives.	13
2.2	Cache-line utilization: a cache line is always loaded in full; if only a few of its bytes are used, effective bandwidth drops — cache-aware layout raises utilization.	16
3.1	The design space: the axes span a multidimensional Cartesian product; a concrete algorithm is a point within it. For illustration, three of the eighteen organ axes are chosen — the real storage organ of node type (T4), memory layout (T5) and allocator (T6); the full design space has eighteen organ dimensions; to these is added an optional nineteenth organ axis for disk I/O, which does not describe an assignment but describes how other axes are described (Section 3.4.1).	26
3.2	Gallery of the most important reconstructed search structures: per method (with paper ID) its origin class, the characteristic structural idea, and the axes at which it makes its main contribution. Trie, hybrid, succinct, and cache-sensitive methods stand next to a non-tree hash counter-check (SwissTable) — all as points in the same axis design space. The complete axis dissection is given in Table 3.2.	35
3.3	The anatomy bridge: just as a living being consists of interacting organs, a search algorithm is a composition of axis-organs; the blood connects the organ axes, the habitat (the system axes) carries the animal, and the measurement system takes the role of the laboratory that measures each organ in isolation.	38
3.4	Synthesis state of the art → axes: the design building blocks of the known methods are decomposed into orthogonal axes; each axis collects the contributions of the entire corpus — the inverse of Figure 3.1 (there an algorithm is a point, here an axis is a collecting basin).	39
3.5	Axis = organ, anatomy = wiring: the storage organ (composition of node type, layout, and allocator) exports a generalized storage-organ interface (in the code <code>LayoutAwareChunkedStore</code>) over which the search runs (solid, <i>mandatory</i>); further organs such as the telemetry <i>can</i> use it via optional observer hooks (dashed, <i>optional</i>), but need not.	40

- 3.6 Inter-organ usage graph (“which axis uses which”): there is exactly *one* canonical hub — the storage organ `LayoutAwareChunkedStore`, which composes node type (T4), layout (T5), and allocator (T6). *Mandatorily* (solid) the store consumes these three axes, and the search (T0) runs over it. All remaining couplings are *optional* (dashed): prefetch (T7) and the telemetry (measurement instrumentation, Section 3.4.1) read the store only via `requires`-gated observer hooks — an axis *can* use foreign interfaces, but *need* not. The coupling of concurrency (T8) to the store is not connected (dotted). The graph is an example wiring of the storage organ, not a structure evaluated in the code. 40
- 3.7 The three-level model: family (level 1, outer interface, feature core composition) → genus (level 2, fixed axis set, feature special composition) → organs/axes (level 3). Four families: `Map` possesses as its sole genus `SearchAlgorithm`, the “mammal” with the eighteen organ axes; `Container` the genera `Set/Sequence/Adapter/View`, which are formed from the same axis pool; `Graph` is laid out as an interface, a genus is not implemented; `HeuristikAdapter` possesses the reroute genus `FunctionInterfaceReroute` of the hybrid tier. 41
- 3.8 The four families and the tier hierarchy down to the compiled binary 42
- 3.9 Carrier stages and interface surfaces: the three-valued admission matrix of the seven stamp interfaces over the four carrier stages planner, `CacheEngineBuilder` (CEB), hybrid tier binary (optional insertion), and tier binary (28 cells; green = mandatory, grey = forbidden; the cell organ line/CEB is the static barrier “the CEB carries no organ axes”) and beneath it the interface surfaces per stage — surface 1 genus interface (runtime), surface 2 stamp (compile time, ABI-stable), surface 3 measurement cut-through. The configuration travels from the XML via planner and CEB to the tier binary; the hybrid tier binary is a stage optionally inserted between the CEB probe dock and the tier binary (dashed: mux switch at the probe dock with dock slots for tier binaries; the column order of the matrix follows the processing topology with the hybrid at its insertion position, the arc is the standard route without hybrid); no stage passes through layers, each contains its layer compiled in. 43
- 3.10 The run chain of the four work modes build → measure → compare → release as a registered sub-axis of the measurement realm, cumulative in this order and repeated synchronously on the hybrid level; compare reads the measurement store and computes the best binary or recombination, release generates the fully optimized binary without measurement sensors and re-measures its wall clock. A debug state does not exist. The assignment to the operating target pictures measurement, evaluation, working, and hybrid mode is given by Section 3.6.2. 44

- 3.11 Design patterns as building blocks of the architecture: above the classical GoF patterns used throughout the code with their respective role, in the middle the two new compile-time patterns named in this thesis, below the code-dissected *three-stage concept chain* per axis (topic concept → axis concept → cache-engine permutation concept → CRTP base → `mp_list` registry) as a refinement of *Strategy/CRTP*; a literal `std::variant` occurs in no organ axis (forbidden) but only in a test stock isolated from the build. Their composition forms the design-pattern-driven, measurement-oriented architecture (Chapter 4). 45
- 3.12 The five interface layers of the experiment pipeline: from the declarative definition and evaluation layer E4 (user view) through the permutation B⁺-tree (E3) and the tier binaries (E2) to the runtime configuration (E1); the infrastructure (E0) supports everything; its completion forms the last step. The return channel leads the measurements back to the evaluation in E4; the measurement system permeates all layers as the cross-cutting concern M (laboratory equipment, Section 3.4). Each layer is completed individually, with a fixed interface contract to its neighbouring level. 46
- 3.13 The measurement trade-off triangle: every axis permutation spans against the three opposing measurement components latency, throughput, and memory consumption and is a point inside it (cf. the design space in Figure 3.1, where the same permutation is a point); the Pareto front of the non-dominated permutations is the bridge from the three quantities to the “best binary”. The fourth component, central for cache-aware search structures — the behaviour at the node, data, and index level via hardware performance counters — is mandatorily built into the measurement path; open are the Intel event catalogue, the last-level counter on Zen 5, and the only model-bound L2/coherence assignment. . . . 53
- 3.14 The store: the store tree is organized by family, genus, and realm; beneath lie four stocks under stamp keys — stock 1 the tier binaries (SHA-512 fingerprint; the binary with and the one without measurement sensors are two entries, because the measurement-gate state is a fingerprint member), stock 2 the measured values (fingerprint of the binary plus hardware identity of the machine), stock 3 the synthesis batches with the synthesized measurement curves as files, and stock 4 the solutions per user XML. The apparatus recognizes what exists via the fingerprint and re-measures only what is missing. 56
- 3.15 The M-model focused on the build and measurement phases. The outer `messung_`-driver loop (*What*, thesis code) feeds the configurations of the three measurement series A/B/C; the `CacheEngineBuilder` (*How*, library) splits into the two genuinely interesting phases: **build** (`BuildOrchestrator/ExperimentTree` builds a standalone tier binary per configuration, resumable and RAM-aware) and **measure** (`perm_runner` drives and measures each tier binary individually through the observer interface `IObservableTier`). The `CacheEngine` supplies the default axis building blocks, the candidate under test PRT-ART its own organs; their bidirectional relationship ((a) instantiates, (b) calls services) founds the multi-candidate capability. The `ExecutionEngine` remains the shared measurable root. 57
- 3.16 The builder’s three-stage evaluation: each stage produces non-redundant permutation binaries; their mapping onto the measurement series A/B/C is shown by Table 3.8. . . . 58

3.17	PRT-ART as a demonstrator in the M-model on three levels: the four candidate slots in its own repository (page type as build axis, prefetch, value handle, telemetry as measurement-tooling sub-axis, Section 3.4.1); the reference composition of the cache engine, which differs from ART in exactly one axis — the path compression as redirect organ —, while the optimistic lock coupling is inherited from ART; and the codegen-capable candidate composition, which takes sixteen of eighteen axes from the ART composition and replaces prefetch and value handle. The further own building blocks (4+1+2 pool allocator, MultiLevel layout) run in the measurement path via the cache-engine default (attachment: subsequent implementation step, not implemented). Thus a complete, measurable living being arises from a few own plus many default organs.	60
3.18	The heuristic loop: from the measurement series the best axis composition per load profile is learned, stored as an XML load profile, and fed back via the workload filter into the next measurement.	61
4.1	The carrier-stage chain in code	69
4.2	Gallery of the six own key points of PRT-ART, each at its axis: a 4+1+2 pool allocator (T6), optimistic lock coupling with reserved value blocks (T8), path-oriented bundle prefetch (T7), a MultiLevel layout (T5) and cache-coherence-aware telemetry together with the DensityTracker and the hypothesis metrics H1/H2/H3 — the telemetry axis is split in two, a sweep sub-axis of the measurement tooling and a compile-time quantity of the builder (Section 3.4.1). The tiles show the <i>building blocks of the candidate repo</i> ; overridden in the apparatus are the path-compression redirect (T3, engine-side reference composition), the prefetch (T7) and the ChainRef value handle (T10, slot composition of the candidate under test) as well as the build axis page type; the building blocks for T5/T6 run in the measurement path via the cache-engine default, OLC is inherited from ART, and the remaining organ axes the slot composition draws from the ART reference of the cache engine (sixteen of eighteen).	72
4.3	Gallery of the eighteen organ axes (T0–T17, canonical slot order): per axis the implementation concept — short name, sub-axis family and a representative selection of the interchangeable building blocks. Each axis is a <code>Concept</code> interface with several strategy building blocks (<i>Strategy</i> pattern); a concrete composition occupies exactly one per slot. The <i>default</i> (“Def.”) is a derived comparison baseline, not a code flag. The numbers in parentheses (T0, T6, T15) are the <i>catalogues</i> of the complete type lists (<code>All*</code>); the <i>enabled inventory</i> of a build configuration is the subset formed by the enable filter, which the generated registry XML reports per axis (for instance four search algorithms and three allocators in the in-house default). The T numbers are slots of the composition, not directory numbers of the implementation; the abbreviations io1–io3 and IO1–IO3 belong to different axes (index organization and I/O dispatch). Next to the eighteen slots stands the organ meta-meta <code>disk_io</code> ; the persistence target is pinned to its in-memory representative in the canonical reference catalogue (Section 3.4.1). The complete matrix with origin sources is given in Appendix D.	75

- 4.4 The *one* architecture as a single hierarchy without parallel trees. At the root stands `IExecutionEngine` as the *driver of measurement* — the universal measurable contract of measurement lifecycle (`warm_up/run/reset/shutdown`) and reports (`engine_name/engine_kind/lifecycle_state`) across all three engine kinds Anatomy, Virus and Hybrid; beneath it branch the anatomy-bearing *families* (level 1: Map, Container, Graph, HeuristikAdapter) with their *genera* (level 2) and the axis-less *viruses*. `SearchAlgorithm` is the focal living being (mammal); its anatomy is its *body* — the fixed composition *and wiring* of the 18 organs —, and “SearchEngine” only its ABI runtime view. The mammal branch is drawn out down to the individual: a concrete axis composition (level 4) is compiled per permutation as a standalone tier binary (level 5). Container possesses an organ set of its own with the genera Set, Sequence, Adapter and View, whose arities are carried by the genus build admission; Graph is declared and possesses no genus; the family HeuristikAdapter carries the reroute genus of the hybrid stage. 76
- 4.5 The ABI-stable C module boundary (seven mandatory symbols; the hybrid `.so` exports the same set and reports its target genus via `genus`) and the variadic 1/2/N API as the view of the candidate under test. Complex keys are reduced by a fingerprint component to 16-byte binary strings of fixed length; the identity of a binary is carried separately from this by the `permutation_fingerprint` field of the ABI POD, the SHA-512 tier fingerprint and the SHA-256 planner fingerprint. 77
- 4.6 UML map of the code interfaces: on the left the engine/anatomy hierarchy — the measurable root `IExecutionEngine` as the *driver of measurement* (`Anatomy ∪ Virus ∪ Hybrid`) over `IAatomyBase`, the four families and six genera and the axis-less `IVirusExecutionEngine`, beneath it the *body* `SearchAlgorithm-Anatomy<C>` (composition *and wiring* of the eighteen axes) and its ABI runtime view `SearchAlgorithmAbiAdapter<A>` (“SearchEngine”), which is measured via the drive/observer contracts `IDriveableTier/IObservableTier` and whose measurement crosses as surface 3 through the visitor `IMessVisitor`; on the right the live path repeated per axis — a two-layer Concept, a requires-secured CRTP base, the complete `mp_list` building-block family, the compile-time filter over the enable flag and the composition slot, which contains exactly one type. The variant-based resolution `resolve_baustein` is a rejected alternative isolated from the build (dashed). . . . 78

4.7	The measurement pipeline in two forms. The in-process path (top) is the seven-phase pipeline of the <code>ExperimentDriver</code> per measurement series (Enumerate → Generate → Compile → Link+Load → Execute → Measure → Persist; two opt-in phases compile missing modules in the running experiment and run a functional test per module with ABI version check). Highlighted are the substantial phases: <code>COMPILE</code> (the <i>build</i> of the tier binaries) and <code>EXECUTE/MEASURE</code> (the <i>measurement</i>); <code>Generate/Link+Load/Persist</code> are wiring. The out-of-process path (bottom) is a design decision for process isolation: the <code>BuildOrchestrator</code> <i>first</i> builds <i>all</i> tier binaries (resumable, RAM-aware), after which <code>perm_runner</code> loads and measures each binary individually via the observer interface <code>IObservableTier</code> — one DLL per process, because the DLL contains no <code>main</code> and the runner is the executable that loads it; no state is shared between permutations, and a cluster task runs exactly one runner with exactly one DLL.	80
4.8	The downstream evaluation toolchain	81
4.9	The levels E4–E0 in code	81
4.10	The four abstraction levels M0–M3	85
4.11	The two shut-off levels of the measurement	87
4.12	The register of design patterns	92
4.13	The hybrid stage in code	93
5.1	The automated evaluation chain from the user XML to the manuscript. Upper row: The XML declares what, where, and when; the validator checks it against the registries; the planner emits the deterministic plan; the CEB builds the tier binaries per system permutation; the measurement collects the three measurement levels and the counters with one measurement thread. Lower row: The workbook is the trunk (one data sheet under a constant sheet key; the fan-out into one sheet per sub-axis permutation is planned but not implemented), the CSV form its ordered tree of CSV files generated from it (one folder per sheet; not a single child document; implemented is a flat file), the generators render per language into the <code>tabellen</code> directory, the manuscript includes the files — every number carries its run. Dashed: parts not implemented (XML creation mode, <code><publish></code> section, third factory).	97
5.2	The store cycle in measurement operation: The CEB builds in batches of 4096 binaries each; every binary lies under its SHA-512 fingerprint in stock 1; the recognition criterion skips what is inventoried with a passed test attestation and otherwise lets build or measure; the measurement runs at the probe dock with one measurement thread, while the CEB produces the parallel requests; the measured values lie under fingerprint and hardware identity in stock 2 and flow into the workbook (the CSV tree form with one folder per sheet is planned; implemented is a flat file). The structure of the store (tree, four stocks, key schema) is shown by Figure 3.14; dashed the full store build-out.	103

5.3	The measurement-curve type system in the heuristic counter-proof. Left: per axis one curve of the metric over the axis parameter, separated by measurement level (wallclock, macro, micro); the intersection of two variants is the break-even point from which the recombination is estimated. Middle and right: The CEB compiles the estimated recombination via the hybrid module emitter; the hybrid tier binary is the mux stage optionally inserted between the CEB probe dock and the tier binary, docks at the probe dock, carries dock slots for tier binaries, and itself produces no code; the comparison is made on the wall clock, primarily against the paper reconstructions, secondarily against the best single binary. Bottom: the three build variants of the measurement-error test plan, kept apart via the gate fields of the fingerprint. Dashed: router and eviction (not implemented).	105
A.1	Cycles per organism permutation (v5_pipeline_demo)	162
A.2	Cycles per organism permutation (cartesian_smoke43)	164
A.3	3D raw-data view: overall latency (ns/op) (search algorithm x workload)	166
A.4	3D raw-data view: lookup latency p50 (ns/op) (search algorithm x workload)	166
A.5	working-set sweep: overall latency (ns/op)	167
A.6	working-set sweep: lookup latency p50 (ns/op)	168
A.7	stacked per-axis latency attribution	169
A.8	latency spread p50-p99 per search algorithm x operation; point=p50, whisker=p99. . .	172
A.9	ECDF of overall latency – distribution over configurations	173
A.10	Pareto scatter: median vs tail latency (one point per configuration and operation kind; diagonal = no tail surcharge)	174

List of Tables

2.1	Cache-near structure designs and the hardware parameters they explicitly bind into the design.	20
3.1	Dialectical adoption of the state-of-the-art instances per composition axis (in canonical slot order T0–T17; telemetry and target ISA belong to the system sphere and to the measurement tooling, respectively, Section 3.4.1. Cluster codes: A trie/node, B hybrid/layout, C allocation, D prefetching, E synchronization, F hardware/measurement/engineering)	27
3.2	Axis dissection of the search methods: eighteen organ axes in canonical slot order (T0–T17), the quantities of the system and measurement sphere, the candidate build axis, and the optional organ meta-meta axis; per axis the sources contributing an organ.	29
3.3	Allocator profiles (23, all with <code>expected_workload</code> tag) with licence, workload affinity, and attachment state	31
3.4	Target hardware and scheduling profiles assigned in this thesis per reconstructed SOTA method (selection; the profiles are reconstruction targets of this thesis, not necessarily original-paper properties; the hardware-strategy column follows the vocabulary of the consumer-less axis draft <code>hardware_strategy.hpp</code> conserved in the code, the target ISA of each reconstruction stands in the <code><isa></code> field of its SOTA profile (system sphere, in the code a build constant; P03 and P31: <code>X86_AVX2</code> , P27 and P32: <code>X86_AVX512</code>); the axis catalogue HW1–HW4 with the platform profiles follows in Appendix D, Section “Build-HW”; there is no paper-by-paper list).	32
3.5	Benchmark frameworks and the publications that use them	33
3.6	Load-profile catalogue (14 profiles, derived from the 33-paper analysis)	33
3.7	SOTA profiles (33: 30 full profiles, 3 abstract) with workload affinity	36
3.8	Mapping of the three-stage evaluation onto the mandatory measurement series	58
4.1	The four work modes of a permutation as a registered sub-axis of the measurement realm (<code>WorkMode</code> , four entries; a fifth breaks the compilation). No mode measures in parallel; only <code>measure</code> switches the measurement on.	70
4.2	Axis typology per category (<code>AxisCategory</code> with the order <code>mess < system < organ</code> ; family discriminator <code>AxisKind</code> with six values). The numbers are structural numbers of the source tree (registry headers and generated registry XML), not measured values. . .	74
4.3	The stamp contract as a matrix of seven interfaces and four carriers (28 cells, three-valued admission M = mandatory, P = permitted, F = forbidden; P is unoccupied in the existing code). The forbidden cell of the organ line at the CEB is the static barrier “the CEB carries no organ axes”; the cells are checked per heir via <code>static_assert</code> . The column order follows, like Figure 3.9, the processing topology with the hybrid tier binary as the optional insertion between CEB and tier binary.	82

4.4	The four sub-axis registries of the measurement tooling: one <code>constexpr</code> file each with index-equals-enum check; the main axis itself holds the three toolings wallclock, macro and micro.	87
4.5	The four store stocks and their key components according to the compile-time-static key schema; the version lock binds the axis sources to digest and stamped version.	94
5.1	Profile-aware workload routing (fallback heuristic)	98
5.2	Real datasets, their characteristics, and their reproducibility record	98
5.3	The counters of the measurement chain and their sources: four generic perf events (portable), two model-bound RAW events from the catalogue for Zen 5 (cross-checked by measurement), the RAPL energy as best-effort reading, and IPC/CPI as a declared, not wired-up category. Per counter the counter bundle carries a <code>*_source_available</code> flag; a number means “source has delivered”, n/a means “source is missing”. The Intel status names what is collected on prod2 without a catalogue entry.	100
5.4	The three measurement column lists of the cache engine and their authority. <code>total_cycles</code> of the pipeline view carries the representative p50 latency in nanoseconds (legacy name), not cycles. F15-LEGACY comprises twelve columns and, appended at the end, <code>degeneriert</code> (12 + 1).	107
5.5	The merge tokens of the user XML and their merge strategies in the merge plan; the stage names are aliases, the full join is the alias of <code>union</code> . The series column follows the assignment from Table 3.8.	108
5.6	Validity of the evaluation according to the four classes of threats (construct, internal, external, conclusion): per threat the countermeasure in the apparatus and its implementation status — implemented, declared and not measured (n/a), or not implemented. No row names a number without a run.	110
6.1	The chain of evidence per main and sub-question: status in three grades (answered, structurally answered, open), the supporting sections with their kind of evidence (structure, test, measurement; for RQ2.d the literature evidence), and the open remainder without a number.	118
A.1	Artefacts of the measurement appendix: metric, measurement level (w/ma/mi) and the data basis contained in the present corpus; numbers only from the embedded artefacts .	160
A.2	Measurement series <code>v5_pipeline_demo</code> : pre-configured organism permutations (in-process path, <code>run_workload</code>)	161
A.3	Measurement series <code>cartesian_smoke43</code> : pre-configured organism permutations (in-process path, <code>run_workload</code>)	163
A.4	Bias-break matrix (median ns/op, search method x load profile)	163
A.5	Non-collected metrics of the heatmap family <code>lc_surface_*</code> (search algorithm x load profile): six notes instead of surfaces — two metrics measured but not drawable as a 1x1 matrix, four not collected	165

A.6	Non-collected metrics of the 3D family <code>lc_surface3d_*</code> (search algorithm $\times$ load profile): four notes instead of views; the two measured metrics (overall latency, lookup latency p50) are shown by the two views of this section with one support point each . . .	165
A.7	Non-collected metrics of the sweep family <code>ld_sweep_*</code> (latency over the working-set size n): one note instead of a curve; the two measured metrics (overall latency, lookup latency p50) are shown by the two curves of this section with one support point each . . .	167
A.8	Per-axis observer detail values per tier binary (measured counters only)	169
A.9	Non-collected metrics of the ratio family <code>lc_surface_ratio_*</code> (quotient median/reference median): six notes instead of matrices, because the reference axis value <code>search_algo/linear_scan</code> is not measured alongside in the corpus	174
A.10	Honest limitations: unresolved caveats (no finding silently lapses)	176
E.1	Overview of the twenty architecture decisions: short title, evidencing section and implementation status — <i>implemented</i> (implemented and evidenced), <i>partial</i> (implemented, with a declared open item) or <i>design</i> (designed only, not implemented); the justifications are in the records, not in the table	253
F.1	Complete <code>std::map</code> interface (C++23)	264
F.2	Complete <code>std::vector</code> interface (C++23)	267
F.3	Composite functions (internal composition of primitive operations)	270

A Complete Measurement Series

This appendix documents the measurement series and evaluation artefacts of the measurement chain (Section 3.4.6). Table A.2 lists, per pre-configured living-being permutation, the metrics of the 16-column schema: the operation count (`op_count`), the representative p50 latency in nanoseconds — carried in the schema under the legacy column name `total_cycles` and headed “cycles” in the table; these are not clock cycles¹ — as well as the columns of the cache misses L1–L3, which are not collected in this series (`pmc_available` = 0) and are therefore tabulated as 0: they are to be read as explicitly coded non-observation (principle of explicitly coded non-observation, Section 3.4.1), not as a measured zero; an output of these cells as n/a is not implemented. With the switch `COMDARE_ENABLE_PMC` set (build option, default off so that the unit tests build without hardware access; the CI sets it in the PMC paths), the measurement chain collects L1D, last-level, dTLB and branch misses generically via `perf_event_open`² [dM10] as well as L2 demand misses and fills from the cache of a foreign CCX³ model-bound via the RAW event catalogue — cross-checked by measurement for the AMD Zen 5 microarchitecture (CPUID family 26) [Lin25, WM08]; models without a catalogue entry (all Intel models) carry n/a (limitation 1 in Section A.10; Section 5.1.5). Figure A.1 compares the same metric graphically; its axis label “Cycles” follows the legacy column name. The series documented here are smoke proofs of the complete chain run with axis variation; they do not replace the mandatory measurement series A–C (Section 5.2.4, Table 3.8), whose complete runs are not available (n/a) because the full build has not been carried out.

The two permutation tables (Tables A.2 and A.3) originate from the in-process path (`run_workload`) of a chain run on the development machine (Section 5.2.4). All other artefacts of this appendix stem from *one* smoke run of the measurement pipeline on the AMD Zen 5 machine — stage `measure:smoke` of the first end-to-end run of the chain⁴, system permutation optimization level O3 with SIMD option `no_extension` —, and this run contains exactly *one* cell: one search method (`k_ary`), one load profile (`ycsb_c`), one tier binary, one working set $n = 4096$, $N = 1$. The measured composition is identified by its `binary_id` with 18 segments (header row of Table A.8); stamp version line, fingerprint, host, date and run identifier are not carried by the generator outputs (n/a in the artefact) — a provenance line per artefact that would make the identity stamps of Section 3.4.4 readable in tables and diagrams is not implemented in the generator; the supporting record is the chain of evidence in this paragraph. PMC metrics appear in this appendix only as table columns (not collected); a PMC figure type does not exist in

¹`total_cycles`: the field of the snapshot schema holds the representative latency of the measurement in nanoseconds (the merged p50 value), not counted processor cycles; the column name is a legacy of the first schema version; the column is therefore headed “cycles” and denotes the p50 latency in nanoseconds.

²`perf_event_open(2)`: the Linux system call that opens a hardware or software counter as a file descriptor; generic events are mapped by the kernel per microarchitecture, model-specific ones require the raw encoding of the event (`PERF_TYPE_RAW`: event number and mask as the hardware counts them).

³CCX (core complex): a core complex with its own L3 cache; demand fills from the cache of another CCX are the coherence observable of the core PMU, a literal invalidation counter does not exist in the event set.

⁴Provenance of the smoke run: pipeline 15800, measurement job 377503, host prod1 (AMD Zen 5); these details come from the run log of the pipeline, the artefacts themselves contain none of them.

the generator. Every number in this appendix comes from these artefacts; no other measurement is cited. The metrics of this appendix are distributed over the three measurement levels of the measurement-tooling main axis — wallclock (w), macro (ma) and micro (mi) in exactly this order (Section 3.4): $ns_per_op = total_ns / timed_ops$ is the host wall-clock of the measurement across the load framework (w); $op_<kind>_p50$ and $op_<kind>_p99$ are the latencies of the family interface operations insert, lookup, erase, scan and rmw of the tier binary (ma); $seg_<axis>_ns$ and $stat_<axis>_<field>$ are the per-axis times and observer counters across the axis interfaces (mi); the PMC columns belong to the micro instrumentation (mi). This appendix does not redefine the levels, it assigns them: Table A.1 names, per artefact, the metric, its level and the data basis that the present corpus contains. In every section the design statement (what a representation delivers) is separated from the corpus statement (what the present run contains).

Table A.1: Artefacts of the measurement appendix: metric, measurement level (w/ma/mi) and the data basis contained in the present corpus; numbers only from the embedded artefacts

Artefact	Metric	Level	Data basis in the corpus
Table A.2, Figure A.1	op_count ; p50 latency (ns) under “cycles”; HW columns L1–L3	w (HW columns: mi)	6 permutations, 2000 operations per row; HW columns 0 = not collected
Table A.3, Figure A.2	as above	w (HW columns: mi)	43 permutations (27 + 16), 1000 operations per row; HW columns 0 = not collected
Bias-break matrix (Table A.4)	median ns_per_op (nearest-rank), only two_phase_valid	w	1×1 ($k_ary \times ycsb_c$): 672 ns/op
Heatmaps $lc_surface_*$ (6)	ns_per_op ; op_*_p50	w; ma	ns_per_op and lookup measured, but 1×1 below the 2×2 minimum; insert, erase, scan, rmw not executed — six notes, no surface (represented by Table A.5 until the complete measurement series)
3D views $lc_surface3d_*$ (6)	as above	w; ma	two views with one support point each (671.5 ns/op; 750 ns), four notes (not executed) (represented by Table A.6 until the complete measurement series)
Sweep curves ld_sweep_* (3)	ns_per_op , op_insert_p50 , op_lookup_p50 over n	w; ma	one measured n (4096): one data point each for ns_per_op and lookup; insert not executed (note) (represented by Table A.7 until the complete measurement series)
Per-axis attribution $seg_attribution$	$seg_<axis>_ns$ against $seg_run_total_ns$	mi	one bar (k_ary); 19 segments = 18 organ axes + framework remainder
Observer detail (Table A.8)	$stat_<axis>_<field>$	mi	one tier binary ($binary_id$ with 18 segments)
Range bars $latency_range$	op_*_p50 and op_*_p99	ma	one category ($k_ary/lookup$): p50 750 ns, p99 1170 ns

continued on next page

Table A.1 — continued

Artefact	Metric	Level	Data basis in the corpus
ECDF latency_ecdf	ns_per_op across the configurations	w	$N = 1$: single-step staircase at 671.5 ns/op
Pareto scatter latency_tradeoff	op_*_p50 against op_*_p99	ma	one point (750/1170 ns); no front
Exchangeability ld_exchange_* (4 longtables)	median rel. Δ ns/op per sibling pair, IQR	w; ma	0 of 480/480/480/640 sibling pairs: empty shells (represented by a substitute line until the complete measurement series)
Forest plots exchange_forest, exchange_forest_vs_reference	as above, aggregated	w	no file (0 pairs; reference not measured): substitute text
Ratio matrices lc_surface_ratio_* (6)	quotient median/reference median	w; ma	reference linear_scan not measured: six notes (represented by Table A.9 until the complete measurement series)
Normalized bars lc_normbar_* (6)	as above, per search algorithm	w; ma	no file (reference not measured): substitute texts
Limitations (Table A.10)	list of caveats, no measured number	—	16 rows

Permutation	ops	cycles	L1	L2	L3
ArtComposition_0	2000	825053	0	0	0
HotComposition_1	2000	740176	0	0	0
StartComposition_2	2000	805750	0	0	0
SurfComposition_3	2000	873976	0	0	0
WormholeComposition_4	2000	696350	0	0	0
MasstreeComposition_5	2000	770871	0	0	0

Table A.2: Measurement series v5_pipeline_demo: pre-configured organism permutations (in-process path, run_workload)

A.1 Cartesian Smoke Series (43 Permutations, DLL Measurement)

The following measurement series arose from the complete chain run: 43 built permutation DLLs in two sub-blocks — 27 cartesian combinations $\text{SIMD} \times \text{memory layout} \times \text{allocator}$ ($\text{avx2}, \text{scalar}, \text{sse4} \times \text{aos}, \text{hybrid}, \text{soa} \times \text{jemalloc}, \text{mimalloc}, \text{std}$) and 16 combinations over four two-valued quantities (node width compact/wide, path compression lazy/none, internal search binary/linear, telemetry leaf_only/sampling; ²⁴) — were measured at the probe dock, persisted as binary measurement records, converted via pipeline stage 03 into the 16-column schema and embedded automatically. The variant names are the build matrix of this series; in the axis model of the code, SIMD is a system axis (option avx2 of the registry of the system axes), memory layout and allocator are organ axes (memory_layout, allocator), and telemetry is not a binary_id axis (limitation 2 in Table A.10); the remaining names do not coincide in name with registry building blocks and are not equated with them. The series thus varies seven quantities; the complete measurement series varies all 18 composition axes, plus the optional organ meta-meta axis disk_io without a binary_id share. Table A.3 and

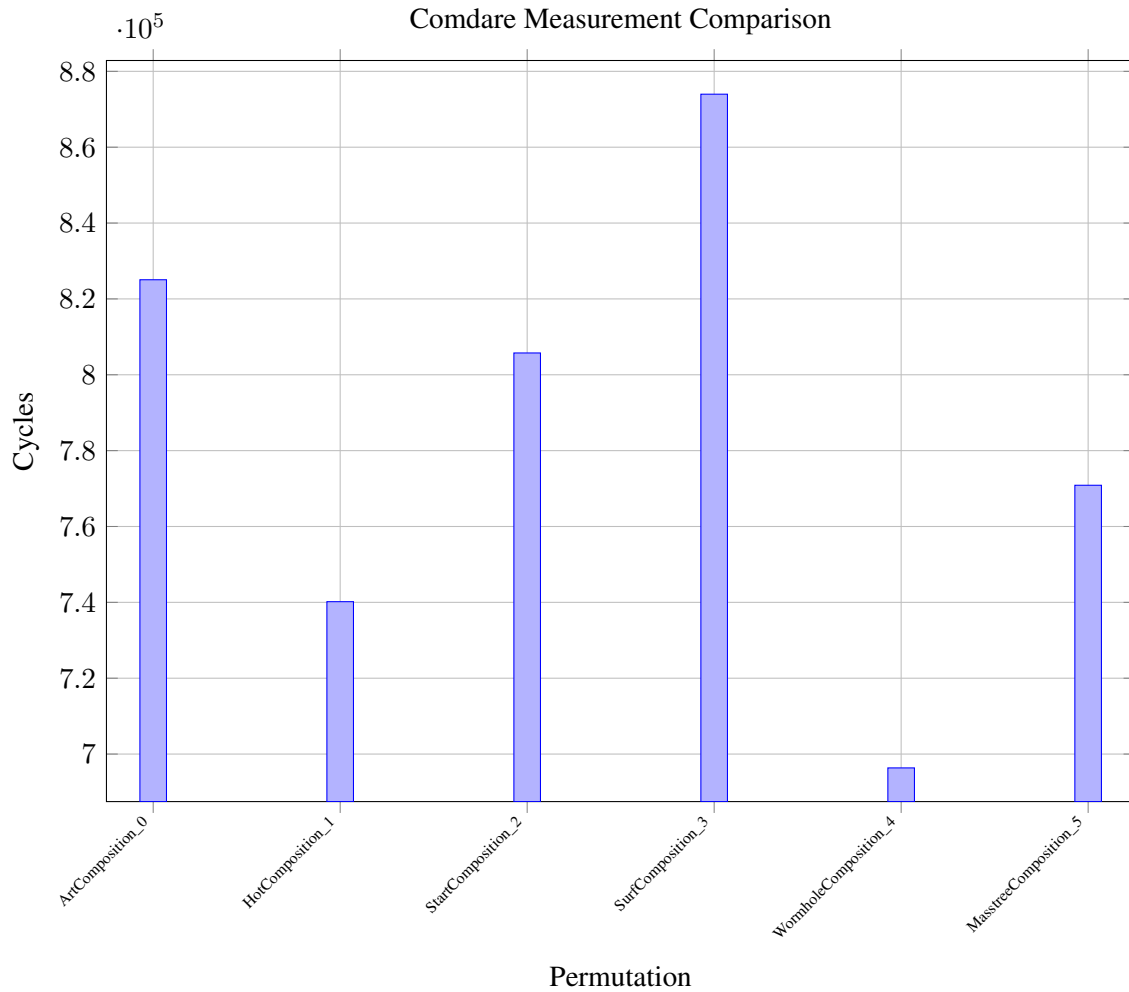


Figure A.1: Cycles per organism permutation (v5_pipeline_demo)

Figure A.2 show the results; the HW columns stand at 0 = not collected, and the axis label “Cycles” of the figure follows the legacy column name (see above).

A.2 Bias-Break Matrix

Table A.4 summarizes the median latency (ns/op, nearest-rank⁵) measured per search method and load profile across all dynamic settings and repetitions; only the runs marked `two_phase_valid`⁶ are taken into account. The matrix makes visible under which load profiles a given method breaks its systematic bias. In the present corpus the matrix contains one cell (`k_ary × ycsb_c`, 672 ns/op — formatted as an integer; the same cell stands in the diagrams as 671.5 ns/op); a break pattern across load profiles cannot be read from a 1×1 matrix — any explanatory power presupposes the complete measurement series (n/a).

⁵Nearest-rank: the q -quantile is the smallest sample value below which at least the share q of the sample lies (Hyndman and Fan, type 1); there is no interpolation, every percentile is an actually measured value [HF96].

⁶`two_phase_valid`: the validity flag of the two-phase operation loop — only a measurement whose second (warm) phase completed validly enters the evaluation (Section 3.4).

Permutation	ops	cycles	L1	L2	L3
avx2_aos_jemalloc	1000	8025	0	0	0
avx2_aos_mimalloc	1000	7995	0	0	0
avx2_aos_std	1000	7995	0	0	0
avx2_hybrid_jemalloc	1000	315164	0	0	0
avx2_hybrid_mimalloc	1000	243930	0	0	0
avx2_hybrid_std	1000	230759	0	0	0
avx2_soa_jemalloc	1000	423930	0	0	0
avx2_soa_mimalloc	1000	400634	0	0	0
avx2_soa_std	1000	398265	0	0	0
scalar_aos_jemalloc	1000	6975	0	0	0
scalar_aos_mimalloc	1000	7005	0	0	0
scalar_aos_std	1000	6975	0	0	0
scalar_hybrid_jemalloc	1000	1366319	0	0	0
scalar_hybrid_mimalloc	1000	1279365	0	0	0
scalar_hybrid_std	1000	1356329	0	0	0
scalar_soa_jemalloc	1000	440835	0	0	0
scalar_soa_mimalloc	1000	394200	0	0	0
scalar_soa_std	1000	438960	0	0	0
sse4_aos_jemalloc	1000	16845	0	0	0
sse4_aos_mimalloc	1000	14160	0	0	0
sse4_aos_std	1000	14324	0	0	0
sse4_hybrid_jemalloc	1000	1281180	0	0	0
sse4_hybrid_mimalloc	1000	1275404	0	0	0
sse4_hybrid_std	1000	1279350	0	0	0
sse4_soa_jemalloc	1000	397095	0	0	0
sse4_soa_mimalloc	1000	392294	0	0	0
sse4_soa_std	1000	396780	0	0	0
pa_compact_lazy_binary_leaf_only	1000	348105	0	0	0
pa_compact_lazy_binary_sampling	1000	375600	0	0	0
pa_compact_lazy_linear_leaf_only	1000	445815	0	0	0
pa_compact_lazy_linear_sampling	1000	423090	0	0	0
pa_compact_none_binary_leaf_only	1000	360765	0	0	0
pa_compact_none_binary_sampling	1000	346980	0	0	0
pa_compact_none_linear_leaf_only	1000	398939	0	0	0
pa_compact_none_linear_sampling	1000	391875	0	0	0
pa_wide_lazy_binary_leaf_only	1000	309300	0	0	0
pa_wide_lazy_binary_sampling	1000	311805	0	0	0
pa_wide_lazy_linear_leaf_only	1000	445605	0	0	0
pa_wide_lazy_linear_sampling	1000	492495	0	0	0
pa_wide_none_binary_leaf_only	1000	307110	0	0	0
pa_wide_none_binary_sampling	1000	311925	0	0	0
pa_wide_none_linear_leaf_only	1000	445140	0	0	0
pa_wide_none_linear_sampling	1000	478635	0	0	0

Table A.3: Measurement series cartesian_smoke43: pre-configured organism permutations (in-process path, run_workload)

Search method	y_{c3b_c}
k_ary	672

Table A.4: Bias-break matrix (median ns/op, search method x load profile)

Metric	Status in the corpus	Reason (note of the artefact)
Overall latency (ns_per_op)	measured, not drawable	1×1 matrix (k_ary × ycsb_c) below the 2×2 minimum of the pgfplots surface form; measured value present in the corpus (Table A.4)
Insert latency p50 (op_insert_p50)	not collected	operation not executed in the corpus (op_insert_n = 0)
Lookup latency p50 (op_lookup_p50)	measured, not drawable	1×1 matrix below the 2×2 minimum of the pgfplots surface form; measured value present in the corpus (Table A.1)
Erase latency p50 (op_erase_p50)	not collected	operation not executed in the corpus (op_erase_n = 0)
Scan latency p50 (op_scan_p50)	not collected	operation not executed in the corpus (op_scan_n = 0)
RMW latency p50 (op_rmw_p50)	not collected	operation not executed in the corpus (op_rmw_n = 0)

Table A.5: Non-collected metrics of the heatmap family `lc_surface_*` (search algorithm × load profile): six notes instead of surfaces — two metrics measured but not drawable as a 1×1 matrix, four not collected

A.4 Three-Dimensional Raw-Data View

The same six metrics are additionally provided as a 3D raw-data view over the plane (search algorithm × load profile). In the corpus two views contain one support point each (ns_per_op: 671.5 ns/op; lookup p50: 750 ns) and four the note of the operation not executed; the corpus does not span a surface. Both surface axes are *nominal*: the spanned surface deliberately carries *no* interpolation or trend claim and is to be read solely as a raw-data and inspection view alongside the heatmap.

Table A.6 summarizes the four notes; the note artefacts of the generator are retained and appear at this position as 3D views after the complete measurement series.

Metric	Status in the corpus	Reason (note of the artefact)
Insert latency p50 (op_insert_p50)	not collected	operation not executed in the corpus (op_insert_n = 0)
Erase latency p50 (op_erase_p50)	not collected	operation not executed in the corpus (op_erase_n = 0)
Scan latency p50 (op_scan_p50)	not collected	operation not executed in the corpus (op_scan_n = 0)
RMW latency p50 (op_rmw_p50)	not collected	operation not executed in the corpus (op_rmw_n = 0)

Table A.6: Non-collected metrics of the 3D family `lc_surface3d_*` (search algorithm × load profile): four notes instead of views; the two measured metrics (overall latency, lookup latency p50) are shown by the two views of this section with one support point each

A.5 Working-Set Sweep

The following curves plot the latency over the working-set size n (keys) — one curve per swept axis value, namely for the overall latency (ns_per_op) as well as the p50 latencies of insert and lookup. In the corpus one measured value of n is present (4096): ns_per_op and lookup p50 contain one support

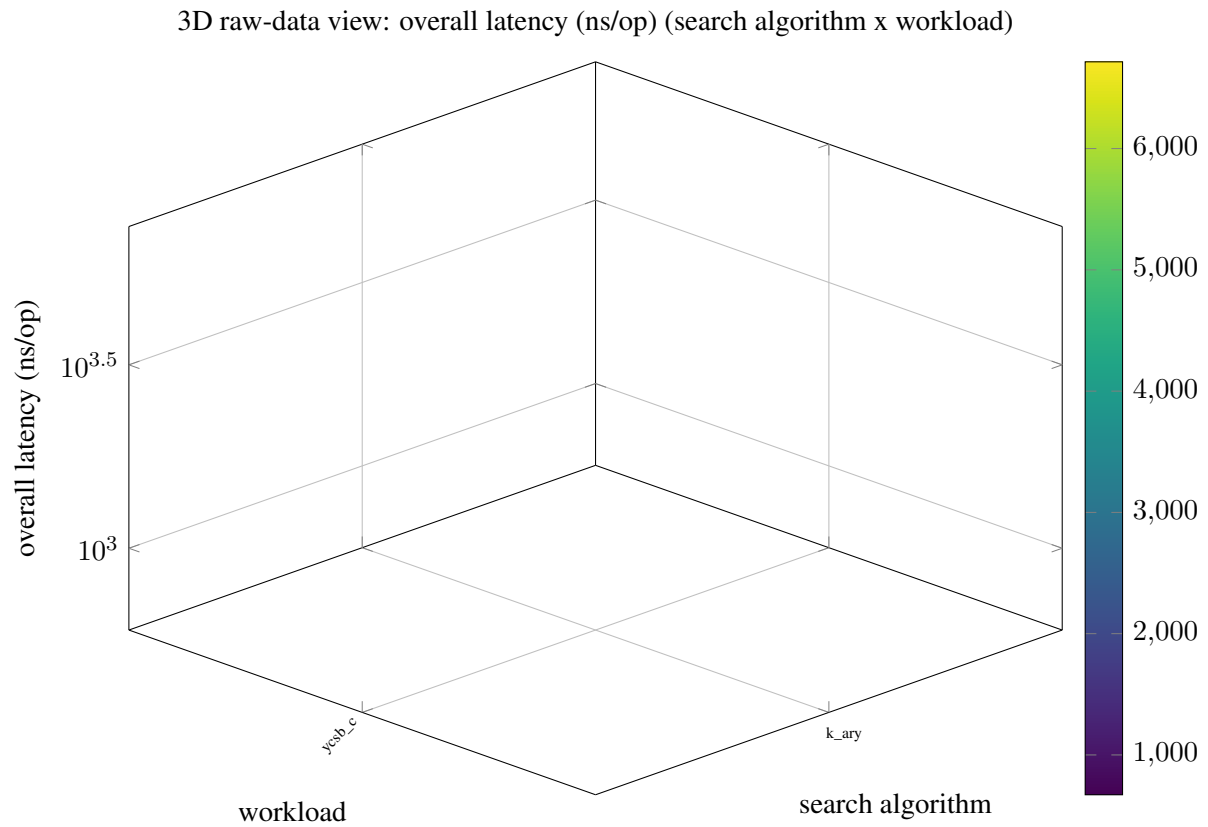


Figure A.3: 3D raw-data view: overall latency (ns/op) (search algorithm x workload)

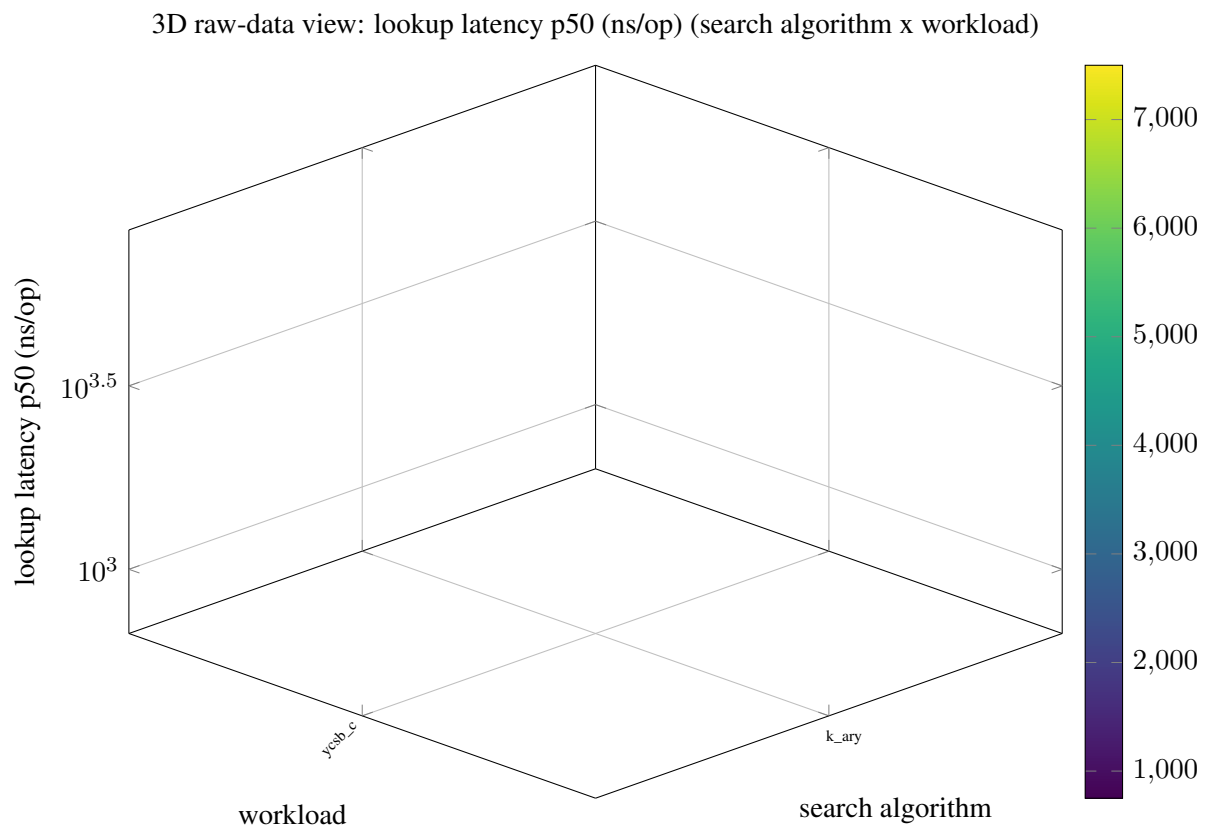


Figure A.4: 3D raw-data view: lookup latency p50 (ns/op) (search algorithm x workload)

point each (“1 sample”), `insert` was not executed and carries the note instead of a curve. Without a measured value no support point arises: a measurement series without a `working_set_n` column deliberately produces *no* curve, an operation not executed (`op_<kind>_n = 0`) no point.

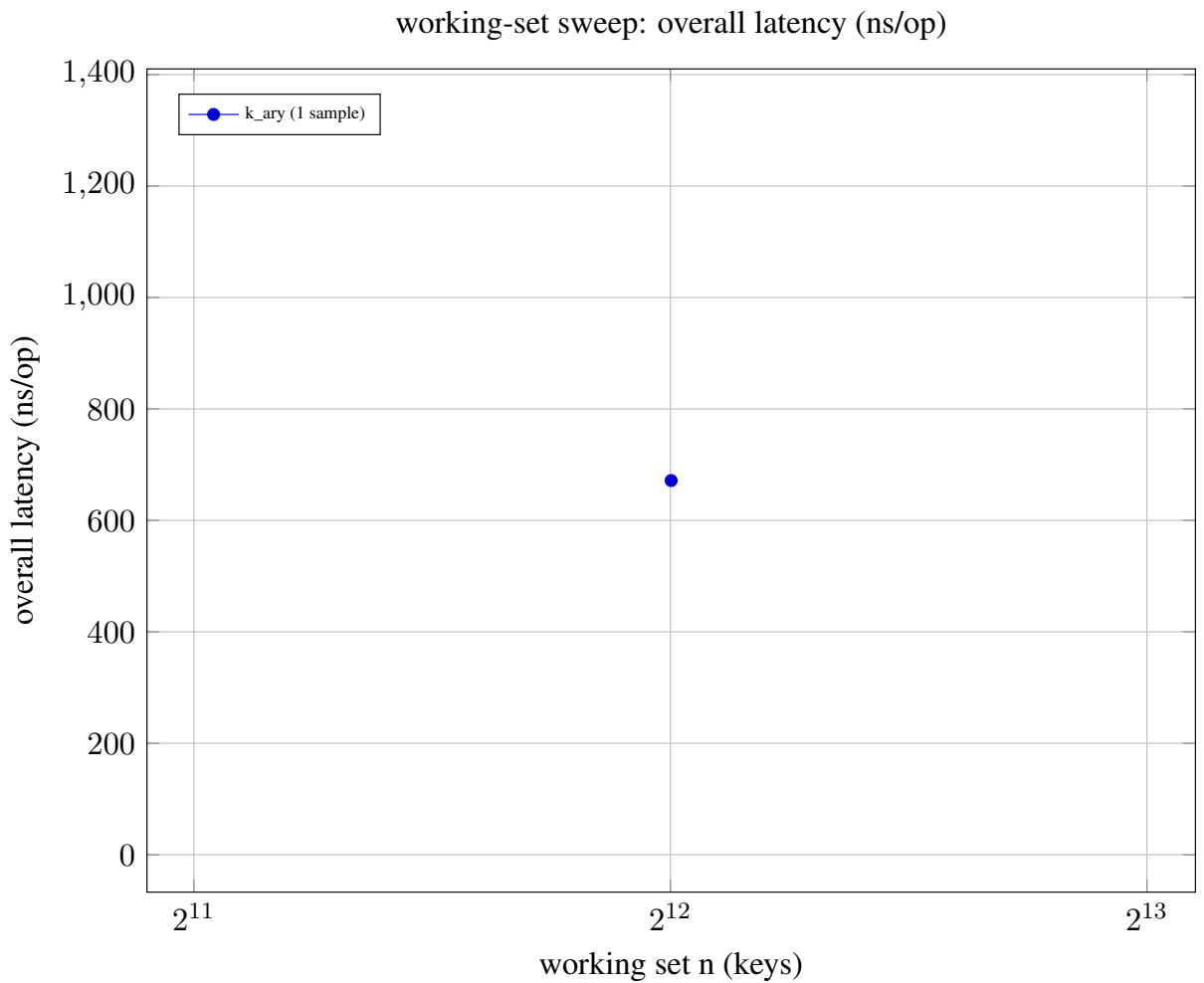


Figure A.5: working-set sweep: overall latency (ns/op)

Table A.7 names the note of the sweep metric not collected; the note artefact of the generator is retained and appears at this position as a curve after the complete measurement series.

Metric	Status in the corpus	Reason (note of the artefact)
Insert latency p50 over n (<code>op_insert_p50</code>)	not collected	operation not executed in the corpus (<code>op_insert_n = 0</code>): no support point, no curve

Table A.7: Non-collected metrics of the sweep family `ld_sweep_*` (latency over the working-set size n): one note instead of a curve; the two measured metrics (overall latency, lookup latency p50) are shown by the two curves of this section with one support point each

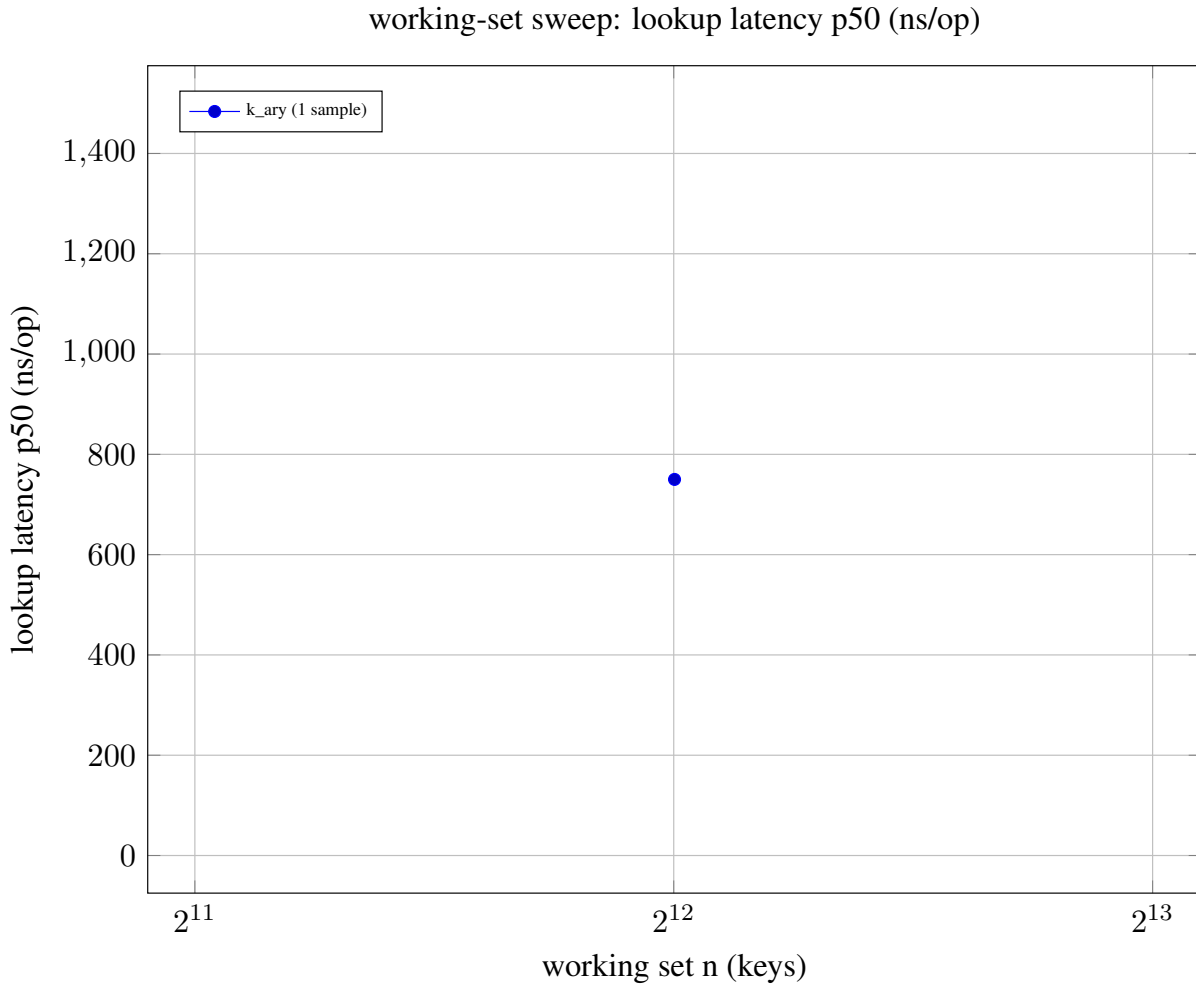


Figure A.6: working-set sweep: lookup latency p50 (ns/op)

A.6 Per-Axis Latency Attribution

The following figure decomposes the segment runtime per search method into the contributions of the individual organ axes (stacked bars). The 100 % whole per bar is the wall-clock of the separate segment run (`seg_run_total_ns`), *not* the real-workload latency (`total_ns`); the 19 segments — the 18 organ axes of the canonical composition order plus the framework remainder — sum exactly to this base (coverage ≈ 1.0). The corpus contains one bar (`k_ary`).

A.6.1 Per-Axis Observer Detail

The following longtable lists, per tier binary, the measured observer counters (`stat_*` columns from the DLL-side axis schema) — per axis and observer field the measured uint64 counter value; fields not collected (n/a) are omitted (no synthetically added 0 value). Zeros in the table are measured counter readings of inactive sub-functions of pinned strategies (for instance `budget_reject`, `fail`, `register`; limitation 4 in Table A.10), not fields not collected — the latter are absent from the table. The axis schema is single-source (18 axes times 8 field slots, emitted DLL-side, read header-driven host-side); the corpus contains one tier binary whose `binary_id` the header row of Table A.8 names.

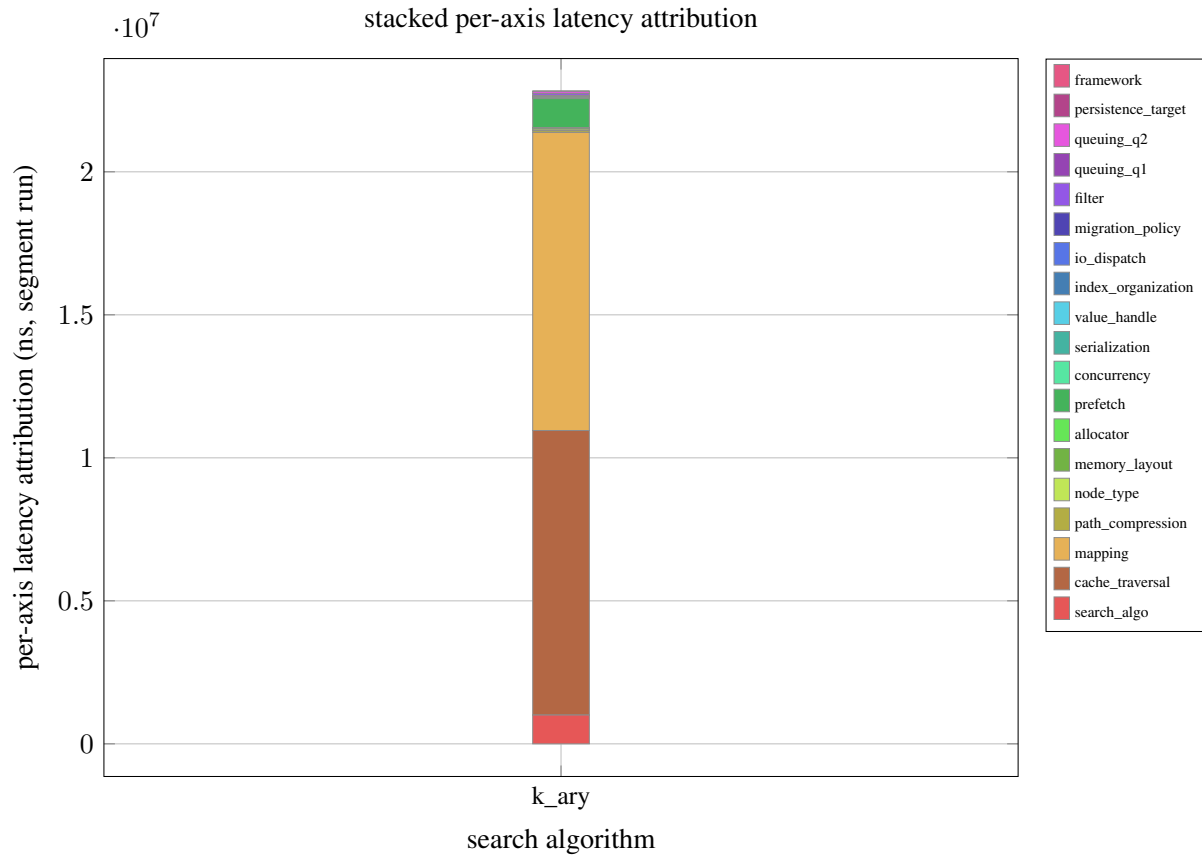


Figure A.7: stacked per-axis latency attribution

Table A.8: Per-axis observer detail values per tier binary (measured counters only)

axis	observer field	value
search_algo=k_ary/cache_traversal=linear_fanout/mapping=direct_placement/path_compression=path_compression_none/ node_type=node4/memory_layout=memory_layout_cache_line_aligned/allocator=std_malloc/prefetch=prefetch_none/ concurrency=concurrency_none/serialization=serialization_raw_binary/value_handle=value_handle_inline/ index_organization=index_org_heap/io_dispatch=io_in_memory_only/migration_policy=migration_none/filter=filter_bloom/ queuing_q1=no_buffer/queuing_q2=eager_flush/persistence_target=persistence_memory_only [ycsb_c]		
allocator	alloc_cnt	2099211
	budget_reject	0
	bytes_alloc	537444328
	bytes_in_use	286720
	dealloc_cnt	2098186
cache_traversal	fail	0
	batch_size	4096
	batch_visited	32768
	peak_tracked	0
	register	0
	resolve	20000
	resolve_hit	20000
	resolve_miss	0
concurrency	unregister	0
	acquire	20000
	contention	0
	pattern_id	0
	release	20000

continued on next page

Table A.8 – continued

axis	observer field	value
filter	validation_fail	0
	checksum	18432
	hash_probes	16384
	neg	0
	pos	4096
index_organization	probe	1
	checksum	16378
	indirect_lookups	0
	predicate_evals	4096
	records	4096
io_dispatch	scan	1024
	align_adjusts	0
	bytes	262144
	checksum	16378
	dispatch_cnt	4096
mapping	rounds	1024
	indirect_steps	20000
	peak_mapped	0
	register	0
	resolve	20000
memory_layout	resolve_hit	20000
	resolve_miss	0
	reverse_lookup	0
	cache_lines	4096
	checksum	16378
migration_policy	field_bytes	32768
	line_bytes	64
	records	4096
	scan	1024
	cold_votes	0
node_type	decisions	4096
	hot_votes	0
	migrations	0
	tier_moves	0
	checksum	40496
path_compression	find	1
	keys_stored	4
	queries	4096
	bytes_saved	70000
	checksum	7
persistence_target	compress	10000
	cuts	10000
	prefix_len	70000
	bytes_staged	0
	checksum	0
prefetch	device_flushes	0
	records_staged	36864
	rounds	9
	addrs_enqueued	0
	hot_path_hints	0
queuing_q1	max_queue_depth	0
	suggestions	0
	trigger	0
	get	0
	overflow	0
	peak_size	0

continued on next page

Table A.8 – continued

axis	observer field	value
queuing_q2	put	0
	underflow	20000
	decisions	0
	flush_complete	0
	full_flush	0
	no_flush	0
search_algo	partial_flush	0
	erase	0
	hit	10000
	insert	0
	lookup	10000
	miss	0
serialization	peak	0
	bytes	262144
	checksum	16378
	records	4096
	serialize	1024
	value_handle	4096
	indirect_deref	0
	peak_chain_depth	1
	version_strips	0

A.7 Latency Distribution

The range bars show the p50–p99 span per search method and operation (dot at p50, whisker to p99); since per operation only the percentiles p50, p99 and p999 are measured and the schema carries no quartile bounds, deliberately *no* box plot⁸ with interpolated quartiles is drawn. The ECDF⁹ shows the distribution of the overall latency *across the configurations* (share of permutations with $ns_per_op \leq x$) — not a per-operation CDF, as the raw samples per operation are not available. In the corpus one category is present (k_ary/lookup: p50 750 ns, p99 1170 ns) and one configuration ($N = 1$): the ECDF is a single-step staircase at 671.5 ns/op.

Finally, the Pareto scatter plots, per configuration and operation kind, the median against the tail latency (p50 against p99). The lower left front would be the set of configurations that are not surpassed by any other in both metrics at once (Pareto dominance¹⁰). In the corpus one point is present (k_ary/lookup: 750/1170 ns), so a front cannot be stated. The three latency numbers of this appendix are three metrics of the same cell: the overall latency per operation across the run (671.5 ns/op), the p50 latency of the lookup operation (750 ns) and its p99 (1170 ns, as 750 + 420 in the range bar).

⁸Box plot: the distribution display of median, quartiles and whiskers after Tukey; it presupposes the quartile bounds that the schema does not carry [MTL78].

⁹ECDF (empirical cumulative distribution function): the share of observations less than or equal to x , as a step function over x ; the probability display of raw data without a distributional assumption [WG68].

¹⁰Pareto dominance: a solution dominates another if it is worse in no objective and better in at least one; the non-dominated solutions form the Pareto front [ED18, Ehr05].

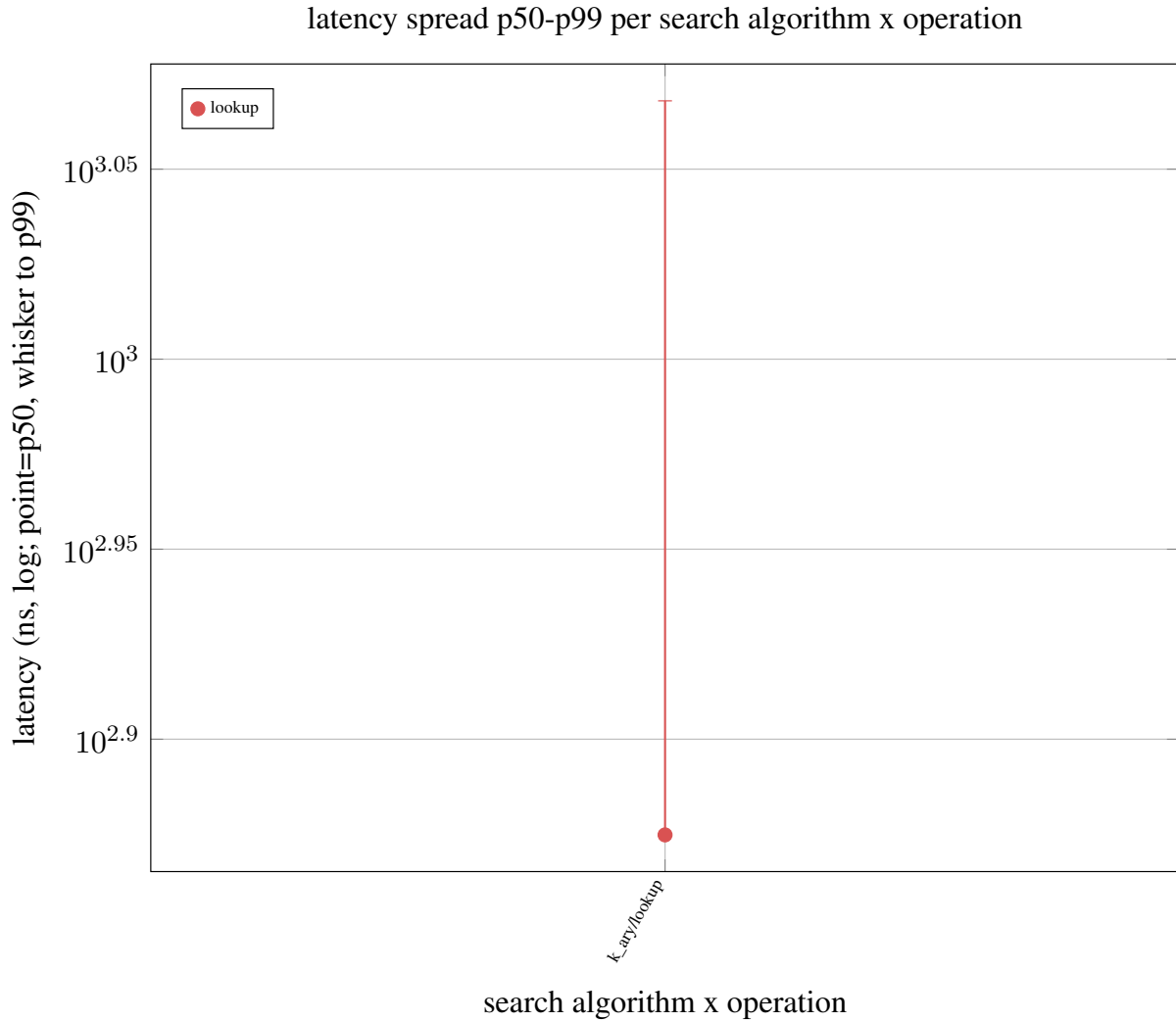


Figure A.8: latency spread p50-p99 per search algorithm x operation; point=p50, whisker=p99.

A.8 Reference-Relative Evaluation

The following representations relate all measured values to *one* reference. That reference is a measured axis value of the corpus itself (default `search_algo/linear_scan`) and expressly *not* an external library baseline: a measured `std::map` performance series does not exist, because every `std::map` in the corpus is the conformance oracle of the probe dock and thus a correctness, not a performance data point. If the reference axis value does not occur in a measurement series, the representations of this section carry the note of the missing reference or its substitute text; in the present corpus `linear_scan` is not measured alongside, all ratio representations are therefore without a denominator (n/a).

A.8.1 Ratio Matrices

Per interface function, one matrix shows the quotient of the measured median and the reference median. The diverging colour scale is centred on equality (ratio 1), so that advantage and disadvantage relative to the reference are distinguishable without comparing numbers. In the analysis role this matrix supersedes the heatmap of Section A.3; the raw heatmap remains alongside it as the raw-value view. In the present

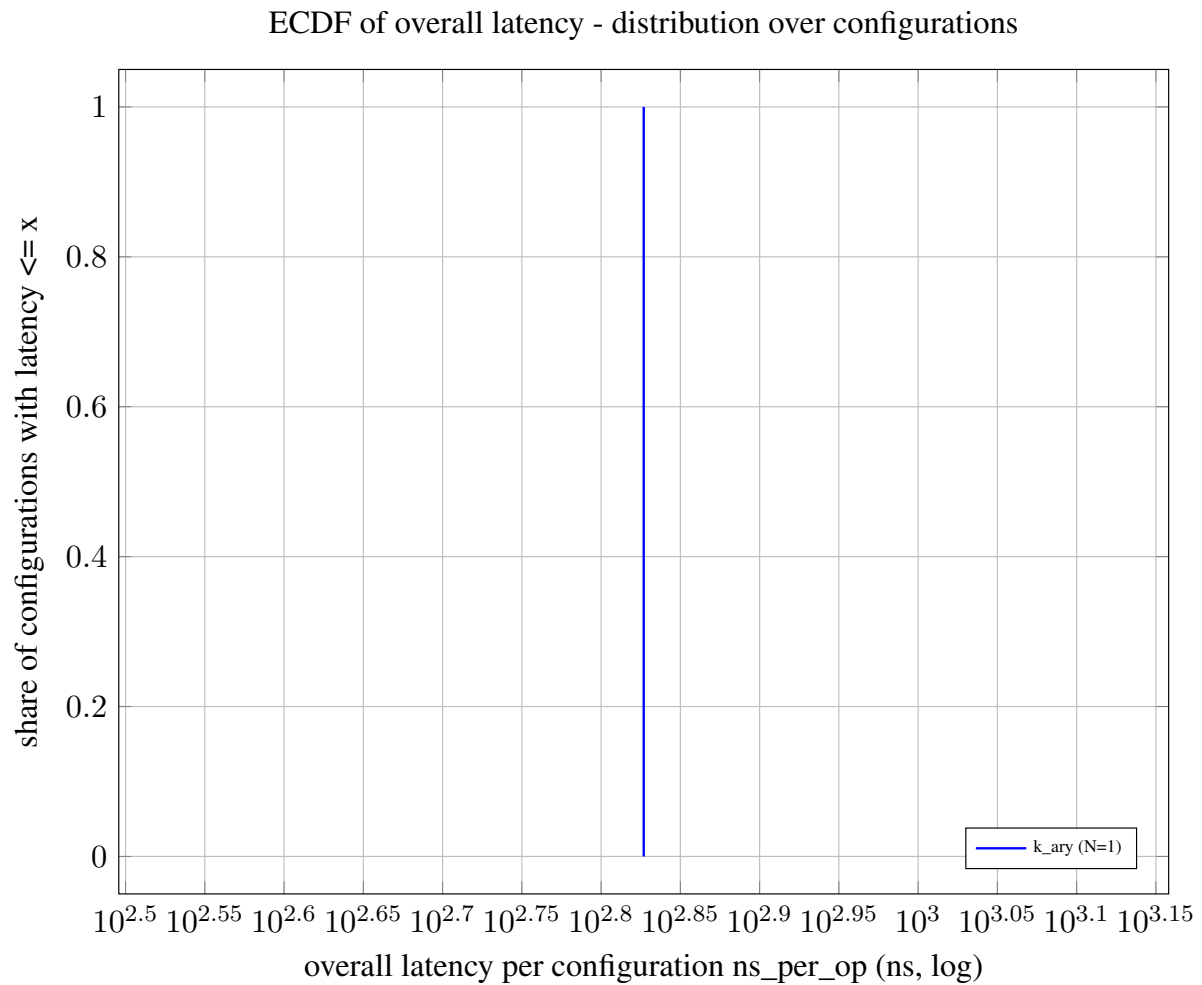


Figure A.9: ECDF of overall latency – distribution over configurations

corpus neither the ratio matrix nor the heatmap carries data (reference not measured; surface below 2×2). Where ratios are aggregated, this is done geometrically, never as an arithmetic mean of quotients¹¹ — this also holds for the normalized bars of the next subsection.

Table A.9 summarizes the six notes; the note artefacts of the generator are retained and appear at this position as ratio matrices after the complete measurement series with a measured reference.

A.8.2 Normalized Bars

The same statement condensed across the load profiles: one bar per search algorithm, with the reference line at 1. Where the ratio matrix shows the load-profile pattern, the bar gives the overall statement per search algorithm.

*(This is where the normalized bars of the **overall latency** (ns_per_op) would stand — one bar per search algorithm, reference line at 1. They are not present: they share the denominator of the ratio matrices, and the reference axis value was not measured alongside in the series available so far.) (This is where the same bars for the **p50 latency of insert** would stand; they too lack the reference axis value*

¹¹The arithmetic mean of normalized ratios depends on the choice of reference and can reverse the ranking; the geometric mean, by contrast, is reference-invariant [FW86].

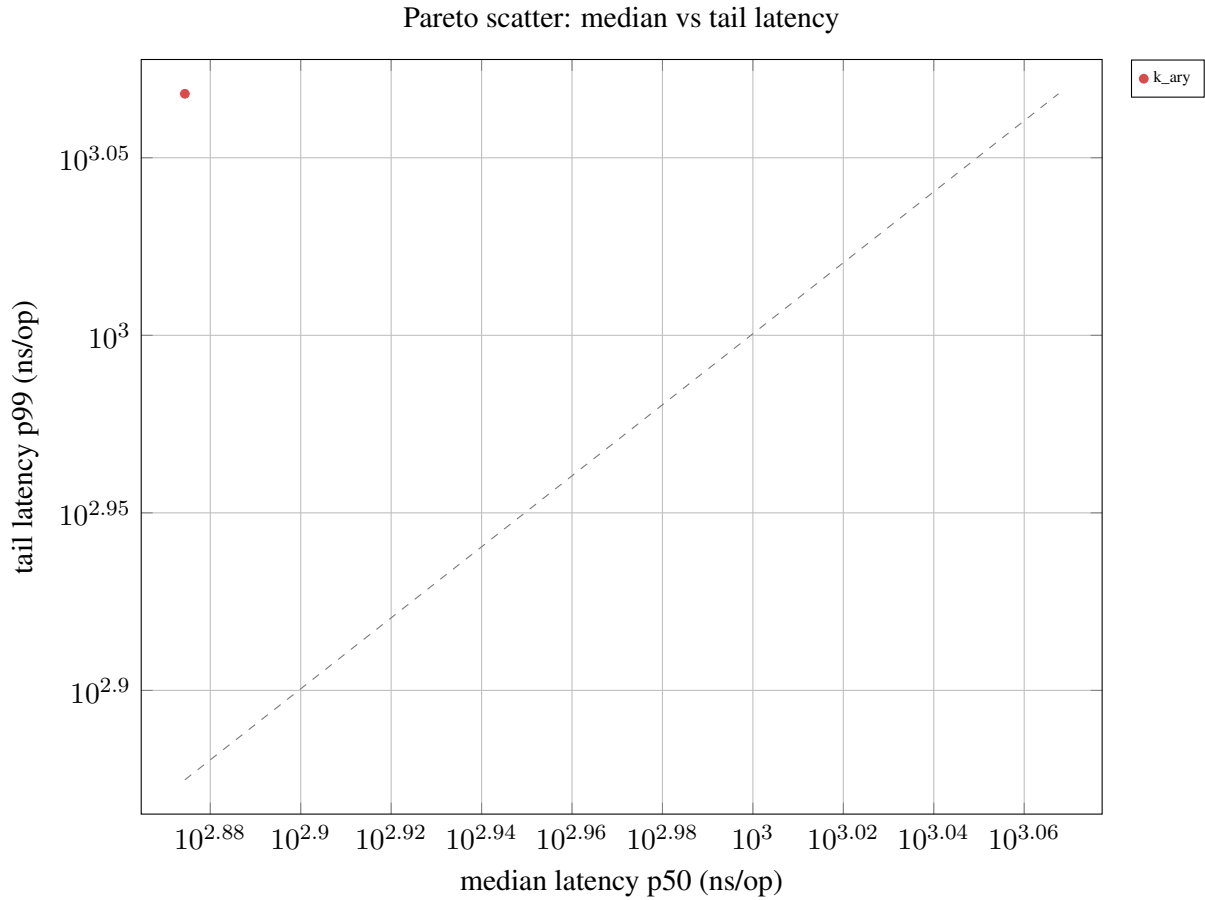


Figure A.10: Pareto scatter: median vs tail latency (one point per configuration and operation kind; diagonal = no tail surcharge)

Metric	Status in the corpus	Reason (note of the artefact)
Overall latency (ns_per_op)	n/a (no denominator)	reference <code>linear_scan</code> not measured
Insert latency p50 (op_insert_p50)	n/a (no denominator)	reference <code>linear_scan</code> not measured; operation moreover not executed
Lookup latency p50 (op_lookup_p50)	n/a (no denominator)	reference <code>linear_scan</code> not measured
Erase latency p50 (op_erase_p50)	n/a (no denominator)	reference <code>linear_scan</code> not measured; operation moreover not executed
Scan latency p50 (op_scan_p50)	n/a (no denominator)	reference <code>linear_scan</code> not measured; operation moreover not executed
RMW latency p50 (op_rmw_p50)	n/a (no denominator)	reference <code>linear_scan</code> not measured; operation moreover not executed

Table A.9: Non-collected metrics of the ratio family `lc_surface_ratio_*` (quotient median/reference median): six notes instead of matrices, because the reference axis value `search_algo/linear_scan` is not measured alongside in the corpus

*measured alongside.) (This is where the same bars for the p50 latency of **lookup** would stand; they too lack the reference axis value measured alongside.) (This is where the same bars for the p50 latency of **erase** would stand; they too lack the reference axis value measured alongside.) (This is where the same bars for the p50 latency of **scan** would stand; they too lack the reference axis value measured alongside.) (This is where the same bars for the p50 latency of **rmw** would stand; they too lack the reference axis value measured alongside.)*

A.9 Axis Exchangeability

The four longtables of the generator family `ld_exchange_*` are the carriers of the exchangeability evidence per variable axis from the sibling-pair differences (median rel. Δ ns/op with respect to v): `search_algo`, `node_type`, `memory_layout` and `prefetch`; the aggregate per value pair and interface function is the nearest-rank median of the relative delta with its IQR¹² across all sibling pairs and valid load profiles. Evidence presupposes sibling pairs — the generator expects 480/480/480/640 (`search_algo`, `node_type`, `prefetch`, `memory_layout`); the present corpus delivers 0, the four tables are therefore empty and prove nothing; the same generator run fills them as soon as the complete measurement series is available. That `search_algo` and `prefetch` are the least confounded is a caveat of the measurement setup (limitation 10 in Table A.10), not a measurement result. Until the complete measurement series, the following line stands in place of the four tables.

(0 sibling pairs in the corpus (state of the measurement series); the four longtables `ld_exchange_*` appear at this position after the generator re-run of the complete measurement series.)

The following forest plot¹³ summarizes the exchangeability of the `ns_per_op` metric (dot = median rel. Δ , whisker = IQR, zero reference line; small- n rows greyed out). The whisker lies symmetrically around the median (via the IQR width), as the aggregate carries only the IQR width, not the quartile bounds.

(This is where the forest plot of the `ns_per_op` exchangeability would stand. It is not present: without sibling pairs (0 of 480/480/480/640) the generator writes no file — no empty figure, no synthetically produced aggregate.)

The second forest plot shows the same aggregates, but turns all pairs onto *one* fixed reference axis value in the sense of Section A.8, so that the yardstick of the comparison is named unambiguously instead of remaining open. The turn converts the relative delta exactly — d becomes $d' = -d/(1 + d)$, because the denominator changes together with the reference value; a mere sign inversion would be an arithmetic error here. Points that cannot be mapped this way are omitted, not substituted.

(The reference-relative forest plot is not present: it presupposes sibling pairs and a reference axis value measured alongside; in the present corpus both are missing.)

A.10 Limitations

Table A.10 lists the caveats of the measurement artefacts that are maintained in the generator and not resolved (16 rows), so that every caveat remains documented; the limitations of the thesis from the user's perspective are stated in the conclusion (Section 6.2).

¹²IQR (interquartile range): the width between the 25 % and the 75 % percentile, here as $p75 - p25$ of the relative delta.

¹³Forest plot: the display of several effect estimates as dots with uncertainty whiskers stacked at a common reference line, adopted from meta-analysis [LC01].

Table A.10: Honest limitations: unresolved caveats (no finding silently lapses)

#	Caveat	Status / consequence
1	Cache misses (core metric): L2 + coherence model-bound real (Zen 5: <code>PERF_TYPE_RAW 0x964/0x1443</code>), otherwise n/a; L1/L3/dTLB only collected with the PMC switch set; energy best-effort	Without <code>COMDARE_ENABLE_PMC</code> (build default OFF, so that <code>test:unit</code> builds without HW access) <code>NullPmcSource</code> reports <code>available=false</code> and all HW columns stay 0 — that is how the tabulated zeros arose. With the switch, <code>LinuxPerfPmcSource</code> collects L1, L3 (last-level) and dTLB misses for real via <code>perf_event_open(2)</code> (kernel syscall, no vendor library, no PAPI required — PAPI remains an optional secondary path). L2 and coherence invalidations have no portable generic counter; they are collected model-bound via the RAW event catalogue (<code>pmc_raw_event_katalog.hpp</code> ; Zen 5, family 26: <code>l2_cache_req_stat.ic_dc_miss_in_l2 = PERF_TYPE_RAW 0x964</code> = demand IC+DC misses in L2 without L2 prefetch; <code>ls_dmnd_fills_from_sys.remote_cache = PERF_TYPE_RAW 0x1443</code> = demand fills from the cache of another CCX, the core-PMU coherence observable — a literal invalidation counter does not exist in the amdzen5 set); models without a catalogue entry honestly report n/a (per-field flag), never a guessed encoding. L3 can additionally fail to open on individual AMD platforms; energy runs best-effort via RAPL sysfs and stays empty without read access to the zone. The Intel PCM Windows driver is still pending for the Windows lane (#26/P4). On models without a catalogue entry (today only Zen 5 carries one) the L2/coherence gap remains observable only indirectly via the wall-clock proxy (<code>seg_memory_layout_ns/ns_per_op</code>).
2	14 pinned axes = 0 exchangeability evidence	Only 4 of the 18 composition axes vary (<code>search_algo</code> , <code>node_type</code> , <code>memory_layout</code> , <code>prefetch</code>). Pinned (1 value each): <code>cache_traversal</code> , <code>mapping</code> , <code>path_compression</code> , <code>allocator</code> , <code>concurrency</code> , <code>serialization</code> , <code>value_handle</code> , <code>index_organization</code> , <code>io_dispatch</code> , <code>migration_policy</code> , <code>filter</code> , <code>queuing_q1</code> , <code>queuing_q2</code> , <code>persistence_target</code> . (Since INC-2d, <code>isa</code> is the system axis <code>target_isa</code> , and since INC-2c telemetry is no longer a <code>binary_id</code> composition axis — neither therefore counts as a pinned organ axis any more.)
3	Observer probe counters (<code>*_find/*_probe/*_get</code>) by design ≈ 1	Expected behaviour, no phantom: one probe per lookup. Says nothing about internal iterations.
4	Honest-0 of inactive sub-features of pinned strategies	<code>concurrency/migration/io/value_handle</code> report 0 (feature inactive), not a measurement error.
5	Wall-clock not bit-reproducible	Seed drives keys (deterministic), NOT CPU timing; wall-clock varies run-to-run.
6	RC dimension only nominal	K1: $\times 18 \rightarrow \times 3$ degenerated; RC axis does not provide full value breadth.
7	2/21 scan profiles = no-op (excluded)	<code>yicsb_e</code> and <code>lp_range_scan</code> perform no real scan; excluded for scan diffs (19 valid workloads).
8	Insert profiles measure upserts; <code>stat_*</code> columns include load phase	The insert path is effectively an upsert; the <code>stat_*</code> counters also accumulate the load phase.
9	prefetch measures key values as pseudo-addresses (K9)	The prefetch axis interprets key values as addresses; no real memory prefetch.
10	<code>node_type</code> + <code>memory_layout</code> : Q2 step-4 shadowing	Apparatus artefact possible (<code>search_organ_shadowing</code>); <code>search_algo</code> + <code>prefetch</code> are the least confounded.
11	<code>container_mirror</code> (path B): O(n) flatten+rebuild as extra apparatus	Path-B living beings (SOTA trees/tries/hash) keep a sorted <code>container_mirror</code> alongside the real search organ; its O(n) flatten+rebuild (on new inserts + successful erases; #211, unresolved as of M3) is EXTRA apparatus work the real tree does not incur $\rightarrow$ the measured <code>ns_per_op</code> of path-B tiers is inflated by this mirror share. (For the path-A array family the same O(n) rebuild is the GENUINE structural cost, not an extra artefact.) Resolved with #188/#211 + successor rebuild (#215).
12	<code>memory_layout</code> effect partly sub-noise	Part of the layout differences lies below the measurement noise $\rightarrow$ PMC externally gated (#26).

continued on next page

Table A.10 – continued

#	Caveat	Status / consequence
13	Secondary indirection organs (<code>cache_traversal/mapping</code>): uint16 key saturation	The uint64 PRIMARY key is narrowed (<code>static_cast</code>) into the smaller <code>slot_index/key_type</code> fields of the PINNED <code>cache_traversal/mapping</code> organs when auto-coupling (low bits); for records > type width their observer entries collide. Does NOT affect the primary lookup (uint64); observer-only limitation of pinned axes (cf. rows 2/4).
14	Capsule-internal (swallowed) OOM in a measured op	An OOM propagated outward now hard-invalidates the measurement (<code>two_phase_valid=false</code>). An OOM caught INTERNALLY by tier code via <code>catch(...)</code> (e.g. defensive memento skip) is not observable from outside → documented as a conservative residual caveat, not forced.
15	Timer overhead (<code>clock::now()</code>) not calibrated	The wall-clock segment timings include the non-subtracted intrinsic overhead of the <code>std::chrono::steady_clock::now()</code> calls (an additive, run-stable ns component per measured segment). NO empty-loop calibration is performed → documented caveat; affects the absolute ns origin, not the relative per-axis comparisons (G5 audit, MESS-MINOR-7).
16	Design labels = naming convention, no measurement effect	A few pattern labels in code/docs are naming/design caveats with no measurement effect: “hybrid visitor” (in reality a memento, no <code>accept()/host visit</code>), plus <code>interpreter/template/decorator</code> labels and <code>reserve/if constexpr</code> hygiene. Pure naming convention → documented as a caveat, no influence on measured values (G5 audit, PATTERN-MINOR-1..7).

Row 1 names the situation of the core metric cache misses: L1D, last-level and dTLB misses only with the PMC switch set; L2 demand misses and foreign-CCX fills model-bound via the RAW event catalogue — cross-checked by measurement for the AMD Zen 5 microarchitecture (CUID family 26) (`PERF_TYPE_RAW 0x964` and `0x1443`) [Lin25], for models without a catalogue entry n/a (all Intel models); energy best-effort via RAPL [DGH⁺10]. A re-layout of the table with a three-valued status per row and a revision of the terms of its rows 4 and 10 are not implemented in the generator.

B Code Structure

This appendix documents the source-code structure of the four repositories from Section 4.1 verifiably at directory level (two to three levels deep). Authoritative for each repository is the branch `development`: for the two code repositories in the commit that the super-repository carries as submodule pointer (`gitlink`¹), for the super-repository its own state and for the manuscript repository the commit from which the CI builds the PDF: `comdare-cache-engine eb69abc1`, `comdare-prt-art b539d6ee`, `probst-Diplomarbeit-cache-engine d989e008` and `20260931-overleaf-diplomarbeit d32cc492`. Build artefacts (`build*`), IDE directories and LaTeX build artefacts are hidden; the vendored third-party code under `ext/` and the preliminary documents of the thesis appear as collective entries. All paths named exist in the named state; paths are given relative to the respective repository root. The licence basis of each repository (`LICENSE`, `NOTICE`, `LICENSES/`, `REUSE.toml`²) is part of the structure; the licence statement of the thesis itself is made in the front matter.

comdare-cache-engine — building-block library, planner and builder

The repository follows the Pitchfork layout³: `apps/` contains the executable programs, `libs/` the libraries, `libs/common` the cross-cutting code and `libs/traeger` the division into the four carrier stages; the vendored third-party code lies collected under `ext/` and is not expanded here. A GNU build path precedes the CMake build: `configure.sh` and a static `Makefile` with the GNU standard targets⁴ hand over to the building CMake script.

The axes reside in three realms (axis homes; Appendix C), one per category: `organ_axes/` (22 directories: the 18 main composition axes, `organ_meta_meta/` as well as `cacheline/`, `simd/` and `telemetry_axis/`), `system_axes/` (`target_isa`, `operating_system` and `external_utils` together with their sub-axes as well as `compiler`, `scheduling` and `extension_hardware`; sub-axes include `optimization_level`, `compiler_atomic` and `simd`) and `mess_axes/` (measurement tooling registry of the three measurement levels `wallclock`, `macro` and `micro`). Each realm owns a generated registry XML as a machine-readable image of its contents: `algorithm_profiles/cache_engine_axis_registry.xml`, `include/`

¹A submodule is a Git repository embedded in a superordinate repository; `.gitmodules` declares path and url per submodule, optionally `branch` and `update`. The `gitlink` is the commit of the submodule recorded in the superordinate repository [Git26].

²REUSE is a specification of the Free Software Foundation Europe for machine-readable licence and copyright information: the licence texts reside under `LICENSES/` with the SPDX identifier as file name, and `REUSE.toml` maps files to their licences [Fre24].

³Convention for the directory structure of native C/C++ projects (Pike, *cxx-pf/R1*; specification in the repository `vector-of-bool/pitchfork`): `apps/` for programs, `libs/` for libraries, besides `tools/`, `tests/`, `docs/` and `external/` as named root directories [Pik18].

⁴`all`, `install`, `check`, `installcheck`, `uninstall`, `clean`, `distclean`, `mostlyclean` and `maintainer-clean` according to the GNU Coding Standards, section *Standard Targets for Users* [Fre26]; the test target there is called `check`, not `test`.

```

| | |-- mess_axes/      measurement realm (measurement tooling registry
| | |                  wallclock/macro/micro)
| | |-- mess/          measurement/control seam (MessVisitor, genus
| | |                  cascade, steering docks planner -> CEB)
| | |-- hybrid/        hybrid tier stage (dock contracts, dock array,
| | |                  dock factory, XML parser, module ABI v1,
| | |                  stamp chain, binary proxy, heuristics adapter)
| | |-- topics/, anatomy/, compositions/, profile_facade/
| | |                  axis roof (AxisKind), anatomy core,
| | |                  composition and catalogue layer;
| | |                  profile_facade/planner/ = planner substance
| | |                  (ExperimentPlanDirector, simulation, version,
| | |                  legend)
| | |-- execution_engine/ ExecutionEngine root over AnatomyBase
| | |                  (measurement layer)
| | |-- builder/       codegen/, permutation_loop/, module_loader/,
| | |                  experiment_driver/ (ExperimentDriver,
| | |                  phases 1-7), experiment_tree/ (axis-path
| | |                  serialization, genus binding, CEB generator),
| | |                  build_orchestrator/, pruef_dock/, bestandslog/,
| | |                  lager_ablage/, measure_storage/, curve_fit/,
| | |                  overlay_source_set.hpp (overlay source set)
| | |                  et al.
| | |-- heuristik/, subsystems/, concurrency_manager/, reclamation/,
| | |                  harness/, anatomy_drive/, src/, virus/
| | |                  curves/spline/break-even, twelve subsystem
| | |                  engines (c01..c12) + platform profiler,
| | |                  concurrency manager, RCU reclamation,
| | |                  measurement harness, drives, SHA cores,
| | |                  graph algorithms
| | |-- include/cache_engine/abi/ ABI headers (stempel_basis.hpp,
| | |                  baustein_variants.hpp, resolve_baustein.hpp,
| | |                  module_abi_v1.hpp, anatomy_version_stamp.hpp,
| | |                  anatomy_fingerprint.hpp,
| | |                  anatomy_stamp_entries.hpp et al.)
| | |-- include/cache_engine/measurement/ realm roots + registries
| | |                  (system_axis.hpp, measurement_*_registry.hpp,
| | |                  system_axis_registry.xml,
| | |                  measurement_axis_registry.xml,
| | |                  ram_frequency_reading.hpp, spd_ddr5_parser.hpp,
| | |                  ram_probe_chain.hpp, hardware_probe_factory.hpp)
| | |-- include/cache_engine/lager/, naming/
| | |                  store key schema, term alias registry
| | |-- include/cache_engine/{api,concepts,allocators,platform,
| | |                  platform_probe,fingerprint,indexes,hbm}/
| | |                  remaining public headers (collective entry)
| | |-- algorithm_profiles/ sota/ (33 profile files *.profile.xml +
| | |                  sota_h2_scores.xml), cache_engine_axis_registry.xml
| | |                  (generated), paper_experiments/, allocators/,
| | |                  load_profiles/, thesis_profiles/,
| | |                  permutation_axes.xml
| | |-- traeger/       carrier stages planner/ -> ceb/ -> tier/ -> hybrid/

```

```

| | (build order; CMake targets;
| | substance in libs/cache_engine/)
| |-- common/ cross-cutting: config/, measurement/, platform/,
| | serialization/ (xml_config_parser/), succinct/
| |-- execution_engine/ result aggregation (result_aggregator,
| | experiment_demo)
| |-- search_engine/ declared skeleton: node/, page/, traversal/,
| | operations/, memory_layout/, value_handle/ et al.
| | (CMake entries only, no source files)
| `-- test_infra/ benchmark_suite/, test_data/, workload_generator/
|-- ext/ vendored third-party repositories of the A/P series,
| arranged by allocator/, prefetch/, queuing/, traversal/;
| besides io/ (liburing, libxlsxwriter, zlib) and
| A05-jemalloc (collective entry)
|-- tests/ unit/, integration/, generic_module_tests/,
| module_specific_tests/, fixtures/,
| pruefling_fixture/, deprecated/ (archive of parked
| tests)
|-- tools/ registry generators axis_registry_gen/,
| system_axis_registry_gen/, measurement_axis_registry_gen/;
| axis_version_lock/ (version lock); overlay_source_hash_gen/;
| vor_push_gate/; abi_stability_test/, gen_golden_fullpilot/,
| best_binary_selector/, p27_bundle_finder/, ycsb_cli/,
| mess_report/, latex_anhang/, latex_toolchain/,
| compiler_provisioning/, prerequisites/, gitlab_ci_zih_push/,
| socks5_zih_delivery/
|-- benchmarks/, scripts/ micro/platform/YCSB benchmarks; CI checks,
| lint and landing-gate scripts (shell)
|-- modules/, prerequisites/, *.py README only resp. four workbench helper
| scripts (Python, outside the build chain)
`-- docs/ architecture, plan, status, ledger, audit,
licence and working documentation

```

Component	Path	Purpose
CacheEngineBuilder	apps/cache_engine_builder/	builder executable (configuration to permutation modules)
ExperimentDriver	libs/cache_engine/builder/ experiment_driver/	phase-1-to-7 pipeline (host side): phase1_enumerate to phase7_export, two opt-in steps
Code generation	libs/cache_engine/builder/codegen/	generate_module_ from_profile incl. templates
Permutation loop	libs/cache_engine/builder/ permutation_loop/	enumeration of the axis permutations
Module loader	libs/cache_engine/builder/ module_loader/	loading of the generated modules

Component	Path	Purpose
ABI headers	libs/cache_engine/include/ cache_engine/abi/	incl. stempel_basis.hpp (CRTP contract of all stamp carriers), baustein_ variants.hpp, resolve_baustein.hpp, anatomy_ fingerprint.hpp, anatomy_stamp_ entries.hpp
XML config parser	libs/common/serialization/ xml_config_parser/	parser of the profile and experiment XMLs
Axis realms	libs/cache_engine/{organ_axes, system_axes,mess_axes}/	one realm per category; organ_axes/ one subdirectory per main composition axis (18) as well as organ_meta_meta/ 33 profile files
SOTA profiles	libs/cache_engine/ algorithm_profiles/sota/	(*.profile.xml): 30 full profiles of standalone search designs (Section 3.2) and three abstract profiles olc, louds and vampir, plus sota_ h2_scores.xml
Adapters A/P series	adapters/	ABI adapters over the ext/ third-party code (10 A, 12 P entries)
Planner	apps/experiment_planner/	planner role as its own binary (comdare-experiment- planner); subcommand CLI incl. validate, plan, simulate, check-size, status, version, help
Plan director	libs/cache_engine/profile_facade/ planner/	deterministic director walk (experiment_plan_ director.hpp), planning simulation (planner_ simulation.hpp), planner version and fingerprint (planner_version.hpp)

Component	Path	Purpose
Carrier stages	libs/traeger/	stage chain planner, ceb, tier, hybrid (in this build order; in the processing chain the hybrid is the optional insertion between the CEB probe dock and the tier) as CMake targets; the substance resides in libs/
Axis registries	libs/cache_engine/ algorithm_profiles/ cache_engine_axis_registry.xml, libs/cache_engine/include/ cache_engine/measurement/{system, measurement}_axis_registry.xml	cache_engine/ generated images of the three realms (tools/ *_registry_gen)
Version lock	tools/axis_version_lock/	axis_version.lock over the overlay cut (719 records in the named state), checked by the CI gate contract : axis-version-lock
Overlay source set	libs/cache_engine/builder/ overlay_source_set.hpp, tools/ overlay_source_hash_gen/	file set and hash of fingerprint member [7]
Hybrid stage	libs/cache_engine/hybrid/	dock contracts, dock array, dock factory, XML parser of the <hybrid_tier> section, module ABI v1, stamp chain, binary proxy, heuristics adapter; router and eviction not implemented
Measurement/ control seam	libs/cache_engine/mess/	MessVisitor seam (mess_naht.hpp), genus cascade (genus_kaskade.hpp), steering docks from the planner to the CEB (steuer_dock.hpp)
Reproduction path	configure.sh, Makefile	GNU build path before CMake: configure, make, make check, make install; Makefile static with the GNU standard targets, hands over to CMake

Component	Path	Purpose
Licence basis	LICENSE, NOTICE, LICENSES/, REUSE.toml	Comdare Research License 1.0 (SPDX LicenseRef-Comdare-Research-1.0), Change Date 2031-08-10, thereafter Apache-2.0; REUSE mapping per third-party package

comdare-prt-art — candidate under test

The candidate under test is organized as a header library⁵ under `prt_art/include/prt_art/`; each building block resides in its own thematic subdirectory (14), `prt_art/src/` is an empty implementation directory (header-only; the entire library code resides under `include/`). The cache engine loads it as a plugin: `comdare_pruefling.cmake` registers the code path via `COMDARE_CE_PRUEFLING`, the generated `prt_art_axis_registry.xml` is the registry path for the planner (`COMDARE_PRT_ART_DIR`); a cross-check in the candidate contract of the cache engine reports it when only one of the two paths is set, and aborts the configuration when both point to different locations (Section 4.2). In the stamp the candidate under test carries its axes under its own namespace (prefix `prt-art.`).

```
comdare-prt-art/                                branch development @ b539d6ee
|-- CMakeLists.txt, comdare_pruefling.cmake      build entry point (CMake) + plugin
|                                                  registration snippet for the ce
|-- LICENSE, NOTICE, LICENSES/, REUSE.toml      licence basis (Comdare Research
|                                                  License 1.0)
|-- README.md, CITATION.cff, CONTRIBUTING.md, SECURITY.md, CODE_OF_CONDUCT.md,
| CHANGELOG.md, PROJECT_LAYER_MAP.md, CMakePresets.json  (collective entry)
|-- prt_art/
|   |-- include/prt_art/      candidate building blocks (headers, 14 directories)
|   |   |-- identity/         prt_art_search_engine.hpp (PrtArtSearchEngine),
|   |   |                     prt_art_execution_engine_adapter.hpp,
|   |   |                     prt_art_pruefling_factory.hpp,
|   |   |                     prt_art_identity.hpp, status.hpp
|   |   |-- allocator/        pool_descriptor.hpp, pool_router.hpp,
|   |   |                     pool_set.hpp
|   |   |-- concurrency/      olc_with_reserved_blocks.hpp
|   |   |-- prefetch/         path_oriented_prefetch.hpp,
|   |   |                     redirect_prefetch.hpp, distance_estimator.hpp
|   |   |-- memory_layout/    multi_level_layout.hpp et al.
|   |   |-- measurement/     density_tracker.hpp, hypothesis_metrics.hpp
|   |   |-- telemetry/        leaf_only_counter.hpp
|   |   |-- slots/           slot wrappers of the four axis contributions
|   |   |                     (axis_01_page_type, axis_07_prefetch,
|   |   |                     axis_11_telemetry, axis_14_value_handle)
```

⁵Header-only: a library whose definitions reside entirely in header files and are compiled upon inclusion; there is no separately compiled library file.

```

| | | + composition demo
| | '-- ... nodes/, default_lookup/, value_handle/,
| | value_buffer/, internal_search/, serialization/
| |-- legacy_reimpl/ 14 re-implementations of the papers P11-P27
| | (without P15, P20, P25; P11-CSS-tree ...
| | P27-Zhang-ASPLOS-Hierarchical)
| |-- algorithm_profiles/ prtart_pruefling.profile.xml,
| | permutation_axes_extension.xml,
| | prt_art_axis_registry.xml (generated by
| | registry_gen/, own namespace engine=prt_art)
| |-- codegen/ + permutations_codegen/ candidate codegen templates
| |-- registry_gen/ registry generator + round-trip check
| | (registry_roundtrip.cmake)
| |-- src/ empty implementation directory (header-only)
| '-- tests/unit/ module-level tests
|-- tests/unit/ repository-wide unit tests
|-- cmake/ build helper files
'-- docs/ architektur/, ledger-sections/ (repo-local
ledger), sessions/

```

Component	Path	Purpose
PrtArtSearchEngine	prt_art/include/prt_art/identity/ prt_art_search_engine.hpp	hybrid engine of the candidate under test: vector API for one, map API for two, map-tuple API for more than two parameters
ExecutionEngine adapter	prt_art/include/prt_art/identity/ prt_art_execution_engine_adapter.hpp	ABI coupling to the measurement infrastructure
Pool allocator	prt_art/include/prt_art/allocator/	descriptors of the seven pools: four page pools A-D, one rest pool R, two value pools (4+1+2)
OLC	prt_art/include/prt_art/ concurrency/	OLC with reserved blocks
Prefetch	prt_art/include/prt_art/prefetch/	path-oriented prefetch, redirect prefetch
MultiLevel layout	prt_art/include/prt_art/ memory_layout/	multi_level_layout.hpp et al.
DensityTracker, H1-H3	prt_art/include/prt_art/ measurement/	density_tracker.hpp, hypothesis_metrics.hpp
legacy_reimpl	prt_art/legacy_reimpl/	14 comparison re-implementations (P11-P27, without P15, P20, P25)
Candidate profile	prt_art/algorithm_profiles/	profile and axis-extension XML

Component	Path	Purpose
Registry extract	<code>prt_art/algorithm_profiles/ prt_art_axis_registry.xml,prt_art/ registry_gen/</code>	generated axis registry of the candidate under test (engine=prt_art) with round-trip check; also present as a fixture in the cache engine (tests/unit/thesis_tiere/)
Slot contribution	<code>prt_art/include/prt_art/slots/</code>	four slot wrappers and composition demo (slot merge into the axes 01, 07, 11, 14)
Plugin integration	<code>comdare_pruefling.cmake</code>	registration snippet for the plugin controller of the cache engine (COMDARE_CE_TRUEFLING; contract <code>cmake/pruefling_kontrakt.cmake</code> of the cache engine)
Repo ledger	<code>docs/ledger-sections/</code>	repo-local architecture and open-items ledger (four-ledger structure)
Licence basis	<code>LICENSE, NOTICE, LICENSES/, REUSE.toml</code>	Comdare Research License 1.0 (SPDX LicenseRef-Comdare-Research-1.0)

probst-Diplomarbeit-cache-engine — super-repository

The super-repository bundles the measurement driver, the experiment configurations, the nine-stage evaluation pipeline (01 to 09), CI checks and the embedding of the three other repositories as sub-modules [Git26]. Under `Code/external/` reside three entries: the two code repositories and the manuscript repository as the static location of the LaTeX attachment generated from the measurement, next to the rest of the source code, a location that also covers operation without a network. The same manuscript repository is additionally embedded as the build path `thesis/diplomarbeit`, in which the CI builds the PDF; `.gitmodules` thus lists four submodules.

```

probst-Diplomarbeit-cache-engine/    branch development @ d989e008
|-- .gitmodules                      four submodules (two of them pointers to
|                                   the manuscript)
|-- configure.sh, Makefile           GNU build path before CMake (all,
|                                   install, check, installcheck, uninstall,
|                                   clean, ...)
|-- LICENSE, NOTICE, LICENSES/, REUSE.toml
|                                   licence basis (Apache-2.0 for the own
|                                   code; LICENSES/ carries the licence
|                                   texts of the third-party components)
|-- README.md, MANUAL_RUN.md, CITATION.cff, CONTRIBUTING.md, SECURITY.md,

```

```

| CODE_OF_CONDUCT.md, CHANGELOG.md, PROJECT_LAYER_MAP.md    project documents
|-- Code/
|   |-- 01_sample_data_generator/    generation of sample data
|   |-- 02_messung_driver/           measurement driver (main.cpp,
|   |                               v32_orchestrator.hpp, plugin_loader.hpp,
|   |                               measurement_writer.hpp, ...)
|   |-- 03_binary_to_csv/            evaluation stage 1: binary measurement
|   |                               data -> CSV
|   |-- 04_csv_to_latex/             evaluation stage 2: CSV -> LaTeX tables
|   |-- 05_diagram_generator/        evaluation stage 3: diagram generation
|   |                               (TikZ)
|   |-- 06_latex_to_pdf/             manuscript build (build_thesis.sh/.bat)
|   |-- 07_tier_binary_report/       report on the tier measurement binaries
|   |-- 08_appendix_generator/       in-process generator of the measurement
|   |                               appendix (links csv_to_latex and
|   |                               diagram_generator directly)
|   |-- 09_tex_formatter/            typesetting-neutral LaTeX source formatter
|   |-- common/                     shared percentile canon
|   |                               (percentile_canon.hpp)
|   |-- ci_wachen/                  CI checks in C++ (CI invariants,
|   |                               YAML scanner, gitlink parity,
|   |                               licence consistency, mutation run,
|   |                               XML well-formedness, WIDE aggregate)
|   |-- experiment_config/           messreihen.xml (catalogue of measurement
|   |                               series: A_defined, A_full,
|   |                               B_CacheEngine_Permis, C_Merge_Alt_Neu),
|   |                               thesis_profiles/ (f1_durchstich),
|   |                               sample_data/
|   |-- test_data_xml/              XML schemas experiment_schema.xsd and
|   |                               messreihe_v32_schema.xsd, golden XML,
|   |                               test-data XML
|   |-- external/                   submodules: comdare-cache-engine,
|   |                               comdare-prt-art,
|   |                               20260931-overleaf-diplomarbeit (static
|   |                               location)
|   |-- tests/, cmake/              repo-wide tests, build helper files
|   `-- CMakeLists.txt, CMakePresets.json, USAGE.md, MANUAL_RUN.md, README.md
|-- thesis/diplomarbeit/            manuscript repository as submodule
|                                   (build path; structure in its own section);
|                                   besides ZIHLatexVorlage/
|                                   (document-class template)
|-- ci/, scripts/                  CI checks (shell) and hygiene scripts
|                                   (diff/ASCII check, worktree check,
|                                   test-data procurement)
|-- docs/                          ledger (full and dense version),
|                                   architecture and plan documents, meeting
|                                   records, glossary, measurement planning
|                                   with primary records
|                                   (docs/architektur/measurement/),
|                                   working documentation
|-- Forschungsarbeiten/            literature collection per axis (collective
|                                   entry)

```

```

|-- measurement/          thesis_compiles/: persisted PDF builds
|                          of the CI write-back
'-- <preliminary documents> exposes, sketches, registration papers and
                             preparatory work of the thesis (collective
                             entry)

```

Component	Path	Purpose
messung_driver	Code/02_messung_driver/	measurement driver: loads permutation modules, orchestrates the measurement series
Experiment configurations	Code/experiment_config/	messreihen.xml (catalogue of the driver's measurement series: A_defined, A_full, B_CacheEngine_Perms, C_Merge_Alt_Neu), thesis_profiles/ (profile of the first end-to-end run of the chain), sample_data/
binary_to_csv	Code/03_binary_to_csv/	evaluation pipeline, stage 1
csv_to_latex	Code/04_csv_to_latex/	evaluation pipeline, stage 2, with shared percentile canon (Code/common/)
diagram_generator	Code/05_diagram_generator/	diagram generation from CSV data
latex_to_pdf	Code/06_latex_to_pdf/	manuscript build scripts
tier_binary_report	Code/07_tier_binary_report/	report on the tier measurement binaries
appendix_generator	Code/08_appendix_generator/	evaluation stage 4: in-process generation of the measurement appendix (per language a core and a presentation .tex from one WIDE matrix)
tex_formatter	Code/09_tex_formatter/	typesetting-neutral LaTeX source formatter
XML schemas	Code/test_data_xml/	experiment_schema.xsd (experiment profile), messreihe_v32_schema.xsd (measurement series), golden XML and test-data XML

Component	Path	Purpose
CI checks	Code/ci_wachen/, ci/, scripts/	checks in C++ (gitlink parity, licence consistency, CI invariants) and shell (appendix forward channel, freshness, stock-log, diff/ASCII checks)
Submodules	Code/external/	embedding of the two code repositories and of the manuscript repository (static location); second pointer thesis/diplomarbeit as build path
Submodule declaration	.gitmodules	four submodules; build path thesis/diplomarbeit with branch and update, static location deliberately without
Manuscript	thesis/diplomarbeit/	manuscript repository as submodule (build path of the PDF CI)
Measurement write-back	measurement/thesis_compiles/, docs/architektur/measurement/	persisted PDF builds of the CI write-back; measurement planning with the primary records of the raw data
Reproduction path	configure.sh, Makefile	GNU build path before CMake (as in the cache engine)
Licence basis	LICENSE, NOTICE, LICENSES/, REUSE.toml	Apache-2.0 for the own code; submodules and third-party components each under their own licence (21 licence texts under LICENSES/)

20260931-overleaf-diplomarbeit — manuscript repository

The manuscript repository contains the LaTeX sources of both language versions (switch `\thesislang` in `diplomarbeit.tex`), the generator output of the measurement under `anhang/{de,en}/tabellen/` and its own CI with the jobs `lint:secrets`, `lint:latex` and `thesis:pdf`. The measurement CI of the super-repository deposits the generator output by commit in this repository and triggers the PDF build; this write-back channel is the reason why the manuscript is its own versioned repository (Section 4.1). In the super-repository it is embedded twice: as the build path `thesis/diplomarbeit` and as the static location `Code/external/20260931-overleaf-diplomarbeit`. The assignment sheet resides as its own chapter under `aufgabenstellung/`; the licence basis is CC BY 4.0 (`LICENSES/CC-BY-4.0.txt`) [Cre13], the licence statement is in the front matter.

```

20260931-overleaf-diplomarbeit/      branch development @ d32cc492
|-- diplomarbeit.tex                 root document: language switch, front
|                                   matter, licence paragraph, inclusion of
|                                   chapters and appendices
|-- aufgabenstellung/               de.tex, en.tex (official assignment sheet
|                                   as a chapter)
|-- kapitel/{de,en}/                six included chapters 01_einleitung ..
|                                   06_fazit as well as non-included source
|                                   files
|-- anhang/{de,en}/                 appendices A .. F
|   '-- tabellen/                   generator output of the measurement (*.tex,
|                                   incl. le_limitierung.tex)
|-- config/                          language.tex (language switching)
|-- literatur.bib                   bibliography database (BibTeX)
|-- zihpub.cls, alphadin.bst, plaindin.bst, zih_logo_de_sw.{eps,pdf}
|                                   document class, bibliography styles, logo
|-- Makefile, build.ps1, build_all.ps1
|                                   local build entry points (pdflatex/bibtex;
|                                   PowerShell)
|-- .gitlab-ci.yml                  CI: lint:secrets, lint:latex, thesis:pdf
|-- LICENSE, NOTICE, LICENSES/, REUSE.toml  licence basis (CC BY 4.0)
|-- docs/, sessions/, tools/        repo-local ledger, source notes,
|                                   working documentation, TeX formatter
'-- <PDF version>                   archived PDF version of the thesis
                                   (collective line)

```

Component	Path	Purpose
Root document	diplomarbeit.tex	front matter, licence paragraph, \thesislang switch, include list of the chapters and appendices
Chapters	kapitel/de/ (kapitel/en/)	six included chapters; the English version follows the German one as a translation
Appendices	anhang/de/ (anhang/en/)	A measured values, B code structure, C glossary, D building-block matrix, E architecture decisions, F comparison interfaces
Generator output	anhang/{de,en}/tabellen/	.tex generated by 04_csv_to_latex, 05_diagram_generator and 08_appendix_generator (tables, diagrams, limitations); location of the measured values (Appendix A)
Assignment sheet	aufgabenstellung/	de.tex, en.tex
Bibliography	literatur.bib	BibTeX database of the thesis

Component	Path	Purpose
Class and styles	zihpub.cls, alphadin.bst, plaindin.bst	document class and bibliography styles
CI	.gitlab-ci.yml	lint:secrets, lint:latex, thesis:pdf (DE and EN with warning gates)
Licence basis	LICENSE, NOTICE, LICENSES/, REUSE.toml	CC BY 4.0 (SPDX CC-BY-4.0), copyright held by the author
Repo ledger	docs/ledger-sections/	repo-local ledger (four-ledger structure)

C Glossary

This glossary summarizes the abbreviations and central terms used in the thesis. Each technical-term entry first names the German source term (the term of the code identifiers and of the German edition), then the definition and finally the aliases used synonymously in this thesis; registry and code identifiers are set in typewriter type. The references point to the section that introduces the term.

ABI Application Binary Interface — the binary-stable interface through which the builder loads the permutation modules (living beings).

Adaptive vs. dynamic vs. measurement-driven Distinction of terms for adapting the algorithm behaviour. *Adaptive*: adaptation based on run-time-observed data characteristics (e.g. adaptive node sizes in ART). *Dynamic*: in general usage any decision taken at run time; in this thesis, more narrowly, the run-time selection of the fully compiled permutation *module* via the binary identity `binary_id` (module loading) — all eighteen organ axes are fixed per binary at compile time; in addition there are run-time-*variable* parameters of dynamic sub-axes (*Unter-Achsen*: prefetch distance, batch size, inline threshold, pool budget; cf. Section 3.4), no run-time choice of individual axis strategies. *Measurement-driven*: selection of the configuration based on empirically collected measurement series — the leitmotif of this thesis; the corresponding selection mechanism is implemented in a first increment (`best_binary_selector`: ranking and delivery of the best permutation per metric from the measurement CSVs), the fully automatic, load-profile-spanning selection is the subject of the outlook (cf. Section 6.3).

Alder Lake Intel microarchitecture with heterogeneous performance and efficiency cores; the second measurement platform of this thesis (prod2, Intel Core i9-12900K) belongs to it. In the system registry it is the recombination `prod2_alder_lake` (CPU fabrication GenuineIntel/6/151/2, RAM nominal rate 4800 MT/s declared, CAS latency not collected).

Anatomy (*Anatomie*) The *body* of a living being: the fixed composition of its eighteen axis organs and, above all, their *wiring* — the generalized interfaces through which one organ uses the services of the others (for instance a container genus as the node substrate of the axis `node_type` or as index organization). Not a second model beside the search algorithm but its internal structure (cf. Section 3.3.1).

Answer status (*Antwort-Status*) The level of answering reported in the conclusion per main and sub research question: *answered* (a test or a measurement substantiates the answer), *structurally answered* (the apparatus that delivers the answer is implemented and tested, the measurement is not available) or *open* (neither structure nor measurement is available). Without measured values and concrete evidence a question counts as not answered, even if its structure is sound (cf. Section 6.1).

AoSoA Array of Structures of Arrays — memory layout that splits records into blocks of fixed width and, within a block, lays them out field by field (SoA); it combines the cache-line density of the SoA layout with the locality of the AoS layout. In this thesis one value of the organ axis `memory_layout`.

Array-indexed page, vector page (*Punktseite, Kompaktseite*) The two page forms of the PRT-ART B⁺ node by occupancy density: the array-indexed page addresses entries directly via an array (8- or 16-bit index, density up to 50 %), the vector page holds sorted key-value vectors (density above 50 %); the boundary at 50 % is the page-type boundary, the three switching rules of the candidate under test are called *density thresholds*.

ART Adaptive Radix Tree [LKN13]; reference design with adaptive node sizes (Node4/16/48/256).

Autonomous profile-driven heuristic evaluation (*autonome Heuristik-Profil-Auswertung*) The intended mechanism that automatically derives, per load profile, the best axis configuration from the measurement series and stores it as a reusable heuristic policy (target mechanism: ML classification of load-profile/axis features onto configurations). First building blocks are implemented — the automatic best-binary selection from measurement series (`best_binary_selector`) and the extraction/storage as an XML load-profile result —; the ML classification itself is outlook (cf. Section 6.3).

AVX, AVX-512 x86 SIMD extensions (256- and 512-bit, respectively).

Axis (*Achse*) An orthogonal partial decision (“organ”) of a search algorithm. The framework assigns its axes to three realms: eighteen organ main axes (composition axes in fixed slot order T0–T17, registry label T00–T17) and one optional organ meta-meta axis `disk_io`; three system main axes (`target_isa`, `operating_system`, `external_utils`) with their complex wrapper; and the measurement realm with the measurement-tooling main axis (cf. Section 3.4.1). Every axis is a main, sub- or meta-meta axis (see the respective entries); only the eighteen organ main axes enter the `binary_id`.

Axis success parameter (*Achsen-Erfolgs-Parameter*) The characteristic quantities collected per axis interface, by which an interface call is measured from beginning to end; the measuring instrument is the measurement checkpoint `checkpoint_measure` (micro level), and from the macro level upwards all parameters including the PMC counters are merged (cf. Section 3.4). The catalogue of the parameters per axis is not available (subject of future work).

binary_id The identity path of a tier binary in the permutation space: the assignment of the eighteen organ main axes in canonical order (eighteen segments, the last one `persistence_target`). System and measurement axes never appear in the `binary_id` (registry attribute `binary_id` with the value `never`); they act via build version, stamp lines and CSV columns.

built_stem Field of the textual plan (revision v1.1) that states, per build step, the actually generated file stem — produced by the very function the build orchestrator calls during the build; evidence of the *one* main channel (cf. Section 4.3.2).

CacheEngineBuilder (CEB) The autonomous platform-measurement system of the chain and the second of the four carrier stages; root of the system realm. The CEB receives from the planner the released system-axis space, permutes system and organ axes into tier binaries, builds and stamps them per platform and runs the seven-phase pipeline via the `ExperimentDriver`. Its stamp contains the measurement and the system line but no organ line (the static barrier); its SHA-512 fingerprint is provenance, not a reuse criterion (cf. Sections 3.4.6 and 4.3.2).

Candidate under test (*Prüfling*) The new search algorithm to be integrated on trial (here PRT-ART); it is compiled by the builder against the core constituents of the cache engine. Attributive short form for constituents of the candidate under test: candidate (candidate slots, candidate build axis, candidate variants). This role is distinct from the bare noun: in the ranking, the Pareto front, and the selection filter chain, a candidate is a permutation under selection (measured in the ranking and the Pareto front, measured or still to be built in the filter chain), and the comparison candidates of the workload variation are the compared methods. The measurement instrumentation is called *measurement sensor* or, by its literature alias, *probe* (see *Measurement sensor*); the fixed names *probe dock* and *probe effect* keep their meaning (see *Probe dock*, *Instrumentation artefact*).

Carrier stage (*Träger-Stufe*) One of the four stages that carry the anatomy, in build order: planner, CacheEngineBuilder (CEB), tier binary, hybrid tier binary (each stage generates the next, no hybrid without a tier); in the processing chain the hybrid tier binary sits as the optional mux insertion between the CEB probe dock and the tier binary. No stage passes through layers — each contains its layer firmly compiled in; only the configuration travels from the user XML to the finished tier binary. Every stage carries a stamp and offers three interface surfaces (Figure 3.9).

CEB see *CacheEngineBuilder*.

checkpoint_measure The central measuring instrument of the system: a measurement point that collects the input parameters and the time at one instant. On the wall-clock level it is the parameter-empty minimum; if the macro or the micro level is active, it additionally performs the parameter bracket — time and observer are united in the checkpoint, the responsibility is separated per measurement level. The rows are deposited in the measurement arena, the nesting is kept by the stack arena. Alias: measurement checkpoint.

CLU Cache-line utilization — the fraction of used bytes per loaded cache line.

Complex axis (*Komplex-Achse*) Bracket over a fixed recombination of several main axes that jointly act as *one* axis: the outer bracket over target ISA, operating system and extension hardware, and the inner complex axis `target_isa` (RAM frequency, CAS latency, CPU fabrication); in the identity stamp a complex axis occupies exactly one field (cf. Section 3.4.2). An undeclared member is stamped as undeclared, never guessed; the CAS latency is not collected for any measurement machine (n/a: the SMBIOS structure type 17 does not carry it).

Consumer (*Konsument*) The role of the thesis code towards the engine: a consumer links exclusively against the cache engine and uses one door (`get_cache_engine()`); the measurement driver is at the same time its driver. To be distinguished from this is the operator — the human who controls planner and installation. Alias: formal consumer and driver.

CPI, IPC Cycles per instruction and instructions per cycle, respectively.

CRTP Curiously Recurring Template Pattern — C++ idiom for static polymorphism: a class derives from a class template that receives this very class as its template argument, so that the base accesses the derived class at compile time [Cop95]. In the framework the construction form of the one stamp contract of all carrier stages (the template `StempelBasis` with the derived class and the carrier stage as parameters) and of the axis concepts.

CSV tree (*CSV-Baum*) The CSV form of the result workbook: not a single child document but an ordered tree of CSV files — one overall folder per workbook, beneath it one folder per sheet, therein the CSV files of the individual sub-axis sweeps —, always generated from the `xlsx` workbook (`xlsx` to `csv`, as a strategy of the same factory) and, as a representation, its analogously mirrored counterpart; via XML the workbook, the CSV tree or both are deposited at the same target. Implementation status: the seam writes the CSV projection as one flat file; the tree form is not implemented (Section 5.2.2).

Density thresholds (*Dichte-Schwellen*) The three fixed switching rules of the candidate under test PRT-ART between its four page representations by occupancy density, in the order `Array256` (up to 25 %) → `Array65535` (25 to 50 %) → `Vector<u8,u8>` (50 to 75 %) → `Vector<u16,u16>` (above 75 %); the candidate under test switches by these fixed thresholds; the platform-calibrated rules are not available, their determination presupposes the measurement runs (sub-question RQ3.a, Section 1.3; Section 3.5). The threshold at 50 % is at the same time the page-type boundary between array-indexed and vector page (see *Array-indexed page*, *vector page*), not a further switching rule.

dTLB Data translation lookaside buffer; caches data address translations.

ECDF Empirical cumulative distribution function — step function that assigns to each value the fraction of observations not exceeding it [WG68]; in the measurement appendix the representation of the latency distribution across the configurations (a single-step staircase for a single configuration).

Explicitly coded non-observation (*explizit kodierte Nicht-Erhebung*) Principle of the measurement chain: a value not collected is a state of its own (NaN according to IEEE 754 or a validity flag [IEE19]) and is never carried as zero or as a phantom value — neither in the schema nor in CSV, `xlsx`, table or diagram; a measurement series without a valid row produces no output instead of an empty one. The missing value thus remains evaluable as a category of its own [Gra09]. Alias: principle of explicitly coded non-observation; in the code the vocabulary `honest-0/honest-empty` of the comments and probe-dock messages, in the diagram generator of the tool repository (`diagram_generator`) the placeholder routine `write_honest_empty_placeholder` (cf. Section 3.4.1).

Eytzinger layout Arrangement of a sorted array in the level order of an implicit complete binary tree (children of index i at $2i$ and $2i + 1$), so that binary search proceeds in a cache-line- and prefetch-friendly manner [KM17]. In this thesis one value of the memory-layout axis.

Family (*Gattung*; code: `AnatomyGattung`) An externally visible interface — Map, Container, Graph, HeuristikAdapter —, the topmost level (level 1) of the framework; the family is derived from the genus (one maintained source) and is determined via a family-independent C symbol (the

symbol bears the same name for every family). In the product-line reading the family is the feature core composition, the genus the feature special composition. Reading note: “Adapter” without prefix always means the container genus, the hybrid family is always called HeuristikAdapter (see *Three-level model, Genus*).

Family subclass (*Gattungs-Unterklasse*) see *Genus* (alias).

Fingerprint The cryptographic identity hash per carrier stage: the planner carries a SHA-256 over version, ISA and operating system (64 hex characters); the tier binary a SHA-512 over eleven members in fixed order (format 6: format identifier, measurement, system and organ line, sub-axis value set, toolchain, enabled-set signature of the build axes, overlay source set, measurement gates, hybrid composite, build-version base; 128 hex characters); the CEB a SHA-512 as provenance. The fingerprint is the store key of stock 1 and the recognition criterion of a permutation.

First-touch policy NUMA placement rule of the operating system: a memory page is assigned to the node whose processor writes it first; therefore the initializing thread decides on the memory locality [Lam13].

Fit stamp (*Passungs-Stempel*) Design of a check that validates the compiled-in platform *class* of a tier binary against the run-time verdict of the hardware detection on every measurement run; not implemented (cf. Section 6.1).

Forest plot Representation of several effect estimates with their uncertainty intervals as stacked point-and-bar rows [LC01]; in the measurement appendix the view of the axis exchange per configuration against a reference.

Full profile, abstract profile (*Vollprofil, abstraktes Profil*) The two types of state-of-the-art profiles: a full profile describes a self-contained search design completely; an abstract profile provides only individual organs (OLC as concurrency organ [LABN16], LOUDS as memory-layout organ [Jac89], VAMPIR as measurement technology [BH23]) and leaves the remaining ones to the standard construction kit — the same type that the candidate under test PRT-ART corresponds to. Of the 33 profiles, 30 are full profiles and 3 abstract.

Genus The second level beneath a family, with a fixed axis set (code: *AnatomyGenus*). Six genera: SearchAlgorithm under Map — in the metaphor canon of this thesis the “mammal” with the full eighteen-axis anatomy —, Set, Sequence, Adapter and View under Container (in the metaphor canon the animal classes “bird”, “reptile”, “invertebrate” and “plant” as living beings, each with its own compile-time-statically fixed axis set; see *Metaphor canon*) as well as FunctionInterfaceReroute under HeuristikAdapter. The latter classifies the hybrid artefact; *genus()* of a hybrid binary, by contrast, reports the inherited target genus, the rerouting remains transparent to the outside. The genus is the only maintained source of the classification; per docking genus there is exactly one probe dock (five docks for SearchAlgorithm, Set, Sequence, Adapter and View), whereas the classifying genus FunctionInterfaceReroute has none and its binaries dock at the dock of the reported target genus. Alias: family subclass, tier subclass.

Gitlink The commit of a submodule recorded in the superordinate repository (entry mode 160000); `.gitmodules` declares per submodule the path, the URL and optionally the branch [Git26]. The super-repository thus carries the authoritative state of engine, candidate under test and manuscript.

GoF “Gang of Four” — the canonical design patterns after Gamma et al. [GHJV94].

golden reference catalogue (*golden-Referenzkatalog*) The fixed regression detector of the whole system: seventeen organ axes with two values each, `persistence_target` fixed to the in-memory representative, yield $2^{17} = 131072$ identities, anchored via a CRC-64 checksum instead of via the materialized list. It is to be kept apart from the cardinality of the real permutation space, which the planner computes dynamically per profile and platform from the activated catalogues (cf. Section 3.4.6).

H2 dossier (*H2-Akte*) The tool-computed data source of the evaluation hypothesis H2 (code quality per source): a weighted cppcheck finding density per kLOC over the vendored original sources of the state-of-the-art designs; without a vendored original source the dossier carries `n/a` with a reason. Not to be confused with the PRT-ART telemetry metrics H1–H3.

HBM High-Bandwidth Memory.

HDR histogram High dynamic range histogram with logarithmically graded bins and guaranteed relative precision across many orders of magnitude [Ten12]; in the evaluation the distribution view of the latencies, whereas the percentiles are determined nearest-rank from the raw values (see *nearest-rank median*).

header-only Library form whose entire code lies in header files and is compiled upon inclusion; no separate library artefact arises. The candidate under test PRT-ART and the hybrid stage exist in this form.

HOT Height Optimized Trie [BZP⁺18].

HybridAware The sixth core mode of the measurement model and value `HybridAware` of the scheduling dimension `hetero_core_dispatch` (beside `None`, `PCoresOnly` and `ECoresOnly`): the run knows the heterogeneous core topology, pins its threads at run time via the affinity interface of the operating system onto selected cores of both types and measures there with the counter set belonging to the core type; precondition of the cache-line-awareness measurement and therefore a mandatory part of the measurement model (cf. Section 3.4.1).

Hybrid tier binary The fourth carrier stage in the build order, in the processing chain the optional mux insertion between the CEB probe dock and the tier binary: a module of the family `HeuristikAdapter` (genus `FunctionInterfaceReroute`) that inherits at compile time the interfaces of one family and one genus and puts calls through to attached tier binaries according to heuristic criteria; it exports the same mandatory symbols as a tier, reports the target genus to the outside and is built exclusively by means of the CEB. Implementation status: classification, dock layer, XML parser and binary proxy are implemented; the module export as a separate compiled artefact, the reroute router, the displacement, the state synchronization (memento sidecar) and the live ranking are not implemented (cf. Section 4.5.1).

Instrumentation artefact (*Instrumentierungs-Artefakt*; literature term: probe effect) The alteration of the observed behaviour by the measuring instrument itself [Gai86]; in this thesis the difference between the tier binary with and without measurement sensors, which is checked via the plausibility invariant (slower with measurement sensors than without) and factored out via the measurement-error interpolation.

Instrumentation shut-off level (*Abschaltungsstufe*) Graduation in which the measurement instrumentation of a tier binary is withdrawn: level 1 switches off the collection of the axis parameters in the macro and the micro level, so that the wall-clock measurement remains for `measure` and `compare`; level 2 (permutative, `release`) also removes the wall-clock measurement and with it the measurement surfaces and interfaces from the compiled artefact. Implementation status: the code contains the measurement gates G2 and G3, whose state is a fingerprint member, and the release mode without instrumentation; the two-level shut-off as a permutation quantity is intended but not implemented (Figure 4.11, Section 4.5).

Interface surface (*Interface-Fläche*) One of the three contract surfaces that every carrier stage offers: surface 1 the genus interface (run time, Abstract Factory), surface 2 the stamp (compile time, ABI-stable), surface 3 the measurement cut-through (`IMessVisitor`), which crosses the measurement as a seam of its own so that family and genus interfaces remain unchanged (Figure 3.9). Alias: contract surface, carrier surface.

IQR Interquartile range — difference between the third and the first quartile ($p_{75} - p_{25}$); in the measurement appendix the dispersion measure of the sibling-pair differences in the exchangeability long tables and the forest plots (whisker = IQR); a box plot of the latencies is deliberately not drawn there, because the schema carries per operation only the percentiles p_{50} , p_{99} and p_{999} but no quartile boundaries (p_{25}/p_{75}) [MTL78].

Living being (*Lebewesen*) A completely composed tier module of a family — all families are living beings; the `SearchAlgorithm` is the case with the full eighteen-axis anatomy, and only the ABI adapter view narrows the term to it (technical identifier there: `SearchAlgorithm` instance). The user XML carries a living being with one or more tiers. Delimitation: axis-less Graph modules are viruses (see *Virus*).

LOUDS Level-Ordered Unary Degree Sequence — succinct (space-saving close to the information-theoretic bound) tree encoding that stores the node degrees level by level in unary [Jac89].

M0–M3 (meta-levels M0–M3) The four abstraction levels by which this thesis orders its axis terms, following the four-level meta-level architecture of the Object Management Group (MOF) [Obj16]: M3 the meta-meta axes as the indirection of a description; M2 the meta axes as the literal classification of the main and sub-axes carried under their term; M1 conception and run-time implementation; M0 the binary code. The assignment of M3 is fixed, the cut between M2 and M1 is a reading (Figure 4.10, Section 4.5).

Main axis (*Haupt-Achse*) A compile-time-static axis whose choice is compiled firmly into the CEB or the tier binary and appears in the build legend: the eighteen organ main axes (identity-forming), the

three system main axes and the measurement-tooling main axis. Its sub-axes are permuted by the planner at run time. In the stamp entry the main axis is the bracket-free level 0.

Measurement level (*Mess-Ebene*) One of the three levels of the measurement-tooling main axis, in exactly this order: *wallclock* (w) — the wall clock of the overall runs in the CacheEngineBuilder across the load frameworks; *macro* (ma) — the time of the genus-interface functions of the tier binary; *micro* (mi) — time and success parameters per axis interface. Subsets are permitted, any other order is syntactically wrong (cf. Section 3.4). Alias: w/ma/mi; wallclock, macro, micro level.

Measurement realm (*Mess-Realm*) The third axis realm beside organ and system axes: the concerns of measuring itself, rooted at the experiment *planner*. Its main axis is the measurement tooling (wallclock/macro/micro), the fan-out of the CEB type; its sub-axes are the sixteen measurement categories (CSV columns), the work mode, workloads and datasets as well as the write-back methods; the load framework is its meta-meta main axis. All measurement-realm axes are strictly binary-id-neutral (cf. Section 3.4.1).

Measurement sensor (*Messfühler*) The measurement instrumentation compiled into a tier binary (measurement cut-through, observers, PMC counters); its assignment is encoded in the measurement-gates member of the fingerprint, so that the binary with and the binary without measurement sensors possess different identities and both lie in the store. The difference of their run times is the instrumentation artefact. The word *probe* is used in this thesis as an alias of the measurement sensor; the candidate under test is never called *probe* (see *Candidate under test*).

Merge strategy (*Verbund-Strategie*) The manner, declared per axis, in which a candidate slot is joined with the engine axis: *Verbund1_CeOnly* (only the engine, no candidate directive), *Verbund2_Replace* (the candidate axis replaces the engine axis, with fallback; the default), *Verbund2_Hybrid* (engine and candidate under test hybrid per candidate under test) and *Verbund3_Union* (the non-redundant union); the XML tokens are *replace*, *merge* and *union*. Alias: stage 1 to 3 (old name), full join (for union).

MESI, MOESI, MESIF Cache coherence protocols (base protocol and AMD/Intel variants, respectively).

Meta-meta axis (*Meta-Meta-Achse*) An axis of level M3: it describes not an assignment but the indication of a description, and stamps as the bracket appendage of its realm line; the meta-meta level of a stamp entry is its bracket depth. Each realm has its own discriminator: *organ_meta_meta* (*disk_io*), *system_meta_meta* (the hardware-family axes beneath the hub of the extension hardware, such as the SIMD family) and *measurement_meta_meta* (*load_framework*); the PMC collection counts among the meta-meta measurement axes.

Metaphor canon (*Metapher-Kanon*) The biological reading of the architecture, used uniformly in this thesis: organ = axis; anatomy = the body of a living being; animal (*Tier*) = a living being of a family (the animal classes as examples per genus: “mammal” = SearchAlgorithm, “bird”, “reptile”, “invertebrate” and “plant” = the four container genera Set, Sequence, Adapter and View); virus = axis-less Graph module; blood = the liquid organ that connects all organ axes and is condensed in the planner into one compile-time configuration; habitat = the system axes; laboratory = the

measurement axes (cf. Section 3.3.1). Where the text means the measurement cross-section of the host, it says so without metaphor.

MLP Memory-level parallelism — number of simultaneously outstanding memory accesses that a core can overlap; bounded by the miss status holding registers (see *MSHR*).

MPKI Misses per kilo-instructions — cache or TLB misses per thousand executed instructions; normalizes miss counts across programs of different length.

MSHR Miss Status Holding Register — buffer of a cache that manages outstanding misses and thereby bounds the maximum number of simultaneously open accesses (MLP).

nearest-rank median Percentile definition without interpolation: the p -quantile is the value at rank $\lceil p \cdot n \rceil$ of the sorted sample (type 1 according to Hyndman and Fan [HF96]); all percentiles of the evaluation are determined this way from the raw values, the HDR distribution complements the view.

NINE Non-Inclusive-Non-Exclusive — inclusion policy of a shared last-level cache that guarantees neither inclusion nor exclusion with respect to the private caches.

NUMA Non-Uniform Memory Access.

OLC Optimistic Lock Coupling — optimistic concurrency technique (ARTSync, P08) [LABN16].

Organ Metaphorical term for an *axis* (level 3 of the three-level model); a search algorithm possesses eighteen organ axes. Beside the eighteen organ main axes the organ realm carries an optional organ meta-meta axis (*disk_io*), which takes effect only with a disk write-back target (see *Axis*, *Meta-meta axis*).

Overlay source set (*Overlay-Quellmenge*) The one set of source files over which the overlay member of the tier fingerprint hashes: per axis the files of its own implementation plus the shared tier substance, concatenated in fixed order per axis category and hashed once with SHA-512; the same set is the population of the version lock.

Page type (*Seitentyp*) The candidate slot `page_type` (prt-art axis_01): the node form of the PRT-ART B⁺ tree; in the canon of the engine it corresponds to the organ main axis `node_type` (slot T04). The slot numbers of the candidate under test do not follow the slot order T00–T17 of the engine.

Pareto front Set of the configurations that cannot be improved in any metric without deteriorating in another (Pareto dominance) [ED18]; in the measurement appendix the view of the latency-throughput-memory trade-off of the permutations.

P-core, E-core (*performance core, efficiency core*) The two core types of hybrid processors (on Intel since Alder Lake); the measurement pins threads per core type and collects the PMC counters separately. The core dispatch is the dimension `hetero_core_dispatch` of the scheduling sub-axis at `target_isa`, the core class the run-time sub-axis `core_class`; the measurement model carries six core modes (four pinning modes per core type, the unpinned reference run

and HybridAware as the sixth mode), the connection to the run profile is not implemented (cf. Section 3.4.1).

Pitchfork layout Convention for the directory structure of native C/C++ projects (`apps/`, `libs/`, `tools/`, `tests/`, `docs/`, `external/` — in the engine `ext/`), which the engine follows [Pik18].

Planner (*Planer*) The experiment planner (`ExperimentPlanDirector`): resolves the user XML against the three offering catalogues, selects the measurement axes and steers the builder orders of the chain; root of the measurement realm and first of the four carrier stages, as a tool a subcommand program (`validate`, `plan`, `cache-key`, `fingerprint`, `status`, `check-size`, `simulate`, `version`, `help`); its stamp is the only one to carry the version line and a SHA-256 fingerprint (cf. Section 3.4.6).

PMC (Hardware) Performance Monitoring Counter. In the measurement realm the counter path is a building block of the collector axis (regime `PmcCounter`); the model-bound raw events are supplied by a catalogue per microarchitecture, and in the axis typology PMC counts among the meta-meta measurement axes.

Probe dock (*Priif-Dock*) The builder-side measurement orchestration per genus: exactly one dock per genus (`SearchAlgorithm`, `Set`, `Sequence`, `Adapter`, `View`) loads a living being, drives it through the interface operations and measures it; hybrid docks bind attached tiers. The dock is no ABI boundary — the ABI boundary remains the genus-specific drive sub-interface including the POD snapshot; the persistence of the results is taken over by the result intake into the experiment tree and the write-back (cf. Section 3.3.2).

Provenance level (*Provenienz-Stufe*) Trust level of a collected hardware value — configured-measured, SPD-JEDEC base value or declared-not-measured — carried by every result together with its collection timestamp; a lower level displaces a higher one only with the label travelling along (cf. Section 3.4.3).

PRT-ART Probst Redirect Tree / ART hybrid — the candidate under test of this thesis.

Rank (*Rang*) Centrality classification of the state-of-the-art publications (rank-1/2/3); explained in Section 1.3. Not to be confused with the ranking of the measurement (see *Ranking*).

Ranking (*Rangbildung*) The ordering of the measured permutations by a metric across the three benchmarking granularities (thesis assignment, sub-task 7); the first increment is the ranking per metric from the measurement CSVs with delivery of the best binary (`best_binary_selector`).

RCU Read-Copy-Update — reclamation scheme (McKenney, P29) [McK01].

Realm One of the three domains of the axes, each with its own registry, registry XML and stamp line: the organ realm (root: the anatomy of the tier), the system realm (root: the CEB) and the measurement realm (root: the planner); the canonical order is measurement, system, organ axes. Alias: axis home, axis realm.

Registry XML The machine-readable image of the axis inventory (axes, building blocks, sub-axes, machine signatures) generated per realm by compile-time reflection; it is never edited by hand and is the offer against which the planner resolves the user XML.

REUSE, SPDX REUSE specification (Free Software Foundation Europe): convention of assigning every file of a repository to a licence via SPDX licence identifier and `REUSE.toml`; SPDX is the standardized identifier catalogue for licences. The assignment re-licenses nothing — the licence files remain authoritative [Fre24].

RRIP, DRRIP (Dynamic) Re-Reference Interval Prediction — cache replacement heuristic [JTJE10].

SIMD Single Instruction, Multiple Data.

SOSD Search On Sorted Data — index benchmark [KMvR⁺19].

SOTA State of the Art.

SPD, JEDEC Serial Presence Detect — the identification memory of a memory module standardized by JEDEC (the standards body of the memory industry), which carries, among other things, the JEDEC base frequency [JED25]; in the hardware detection the middle provenance level (SPD-JEDEC base value) between configured-measured and declared-not-measured (see *Provenance level*).

Stamp (*Stempel*) The compiled-in, ABI-stable identity inscription of a carrier: per realm one line of entries of the form `achse=algo@X.Y.Z` with optional flags and recursive bracket groups (`c{p.e}` is `c` with the sub-members `p` and `e`, never a flat abbreviation), plus version line and fingerprint. Four carrier stages and seven stamp interfaces form a three-valued admission matrix of 28 cells (mandatory, permitted, forbidden); the CEB carries no organ line, only the planner a version line, only the hybrid attached tiers. The stamp is at once identity, store key and recognition criterion; the version expression of the driver, by contrast, serves only classification (Figure 3.9). Alias: identity stamp, stamp system.

Stock (*Bestand*) One of the four deposit classes of the store, fixed compile-time-statically in the key schema: stock 1 the built tier binaries, keyed by their SHA-512 fingerprint, the measurement cell bracketed beside it; stock 2 the measured values, keyed by the fingerprint of the measured binary and the hardware identity of the measuring machine, the platform cell bracketed beside it and never fused into the digest; stock 3 the synthesis batches with the synthesized measurement curves as files; stock 4 the solutions per user XML (cf. Section 3.4.5).

Store (*Lager*) The persistent holdings of the apparatus, in which what is built and what is measured lies under stamp keys (see *Stock*): the store tree is organized by family, genus and realm, a permutation is recognized via its fingerprint instead of being built anew, measurement is always switched on and re-measures in the store only what is missing there. Every machine configuration has its own measurement stock; an algorithm version change triggers the rebuild of the affected store stack together with re-measurement. Implementation status: stocks 1 and 2 as well as the key schema of all four stocks are implemented; the mechanics of stocks 3 and 4, the curve facet, the storage path of the hybrid tier and the target selection via the XML form the full store build-out and are not implemented (Figure 3.14, Section 3.4.5).

Sub-axis (*Unter-Achse*) An axis beneath a main axis, as a rule dynamic and permuted by the planner at run time: its values travel as settings via the probe dock into the tier binary, appear in no build legend and never in the `binary_id`. Examples: the run-time parameters prefetch distance, batch size, inline threshold, pool budget and thread count as well as the sixteen measurement categories, the workload and the work mode (measurement realm); optimization level, atomics capability, SIMD family, NUMA node, page size and core class (system realm). Individual sub-axes are compile-time-static tuples of their main axis (for instance scheduling and compiler) and act via the build version. Registry token `sub_axis`. Alias: `subaxis`. Sub-axis and permutation axis are not mutually exclusive.

Subset (*Teilmenge*) In this thesis in two fixed meanings: the permissible subset of the measurement levels (any subset of wallclock/macro/micro in canonical order) and the conformance subset of the interface operations, which the comparison gate checks before every measurement (Appendix F).

Subsystem In this thesis the four carrier subsystems of the chain (measurement driver, CacheEngineBuilder, CacheEngine, candidate under test); in the code, by contrast, the twelve `ISubEngine` stages of the recommendation pipeline behind the facade `ICacheEngine`. The two meanings are not mixed in the text.

SuRF Succinct Range Filter [ZLL⁺18].

SVE, NEON, RVV ARM (SVE, NEON) and RISC-V (RVV) SIMD extensions, respectively.

Three-level model (*Drei-Ebenen-Modell*) The layering of the one architecture: *family* (level 1, the outer algorithm interface — Map, Container, Graph, HeuristikAdapter), *genus* (level 2, family subclass with a fixed axis set — SearchAlgorithm, Set, Sequence, Adapter, View, FunctionInterfaceReroute) and *organs/axes* (level 3). See Section 3.3.2.

Tier binary The third carrier stage: the dynamically loadable module of a tier, built by the CEB per permutation, which contains exactly one organ assignment (`binary_id`) together with the system and measurement choice firmly compiled in, is loaded via letter-identical C symbols (the four primal symbols, the family and genus query and the version lines) and lies in the store as stock 1. Alias: tier, permutation module. The word *Tier* is German for *animal* and names, in the metaphor canon of this thesis, the fully composed module — the living being with all its organs —, not a level or layer.

TLB Translation Lookaside Buffer.

Two-gate protocol (*Zwei-Gate-Protokoll*) The acceptance protocol per implementation package: gate 1 is the double-run gate of the engine repository — the full test suite runs in four cells (gcc and clang, each as release and debug build) without change skip, and only a run without failed tests is accepted, without an exception list; gate 2 is the integration gate of the project repository (super-repository), whose pipeline builds error-free without an exception set (cf. Section 4.4).

two_phase_valid Validity flag of a measurement row: only a measurement whose workload was run through the two-phase load profile (build-up, then measurement phase) counts as valid; rows with `two_phase_valid` equal to `false` do not enter median, matrix or diagram.

Version lock (*Versions-Lock*) The checked-in lock file that records category, axis version and SHA-256 per source file of the overlay source set; every change to a recorded file demands the deliberate regeneration commit, a deviation against the discovery is a failure, and a CI gate checks it.

Virus An axis-less Graph or pure-mathematics module at the same root as the living beings: without axis system, without composition and outside the permutation tree, but measurable with its own flat measurement POD (latency, throughput, module-specific counters). Plural: viruses.

working_set_n The column of the working-set size (number of keys) of a measurement series; it is the abscissa of the sweep curves of the curve loader, and the cell tokens `n/a` and `failed` are not numbers — without a measured value no support point arises.

Workload selection bias (*Lastprofil-Selektionsverzerrung*) The distortion of a comparison by choosing the load profiles such that one design performs preferentially well under them [BDH⁺16]; the thesis counters it with the global standard workload as a permutation over all available workloads and with profiles reproduced per publication.

Work mode (*Arbeitsmodus*) The four modes of the run chain, in this order: `build` (building the tier binaries), `measure` (the deterministic measurement run), `compare` (comparison of the measurement levels and permutations from the measurement store) and `release` (delivery without measurement sensors). The work mode is a sub-axis of the measurement realm (`work_mode`), not a property of the binary identity; the mode token `compare` is to be distinguished from the identically named alias of the wallclock level in the measurement configuration. Alias: mode, run chain.

XML load-profile result (*XML-Lastprofil-Ergebnis*) The persistence format demanded by the thesis assignment (sub-task 8, automation of system detection and binary selection) in which the heuristic policy extracted for an overall algorithm — reusable per architecture focus — is stored under reference to the applied load profiles; it serves as input to the autonomous profile-driven heuristic evaluation. Import parser and export writer including the extractor are implemented (round-trip-capable schema `comdare_load_profile`); outlook is the autonomous heuristic evaluation built on top (ML classification).

YCSB Yahoo! Cloud Serving Benchmark [CST⁺10].

Zen 5 AMD microarchitecture (CUID family 26); the first measurement platform of this thesis (`prod1`, AMD Ryzen 9 9950X3D, Granite Ridge, two core complexes) belongs to it, and its raw-event catalogue is cross-checked by measurement. In the system registry it is the recombination `prod1_zen5`. In this thesis the term denotes exclusively the microarchitecture.

D Building-Block Matrix

This appendix contains the complete offering matrix of the building blocks — the `All*` lists of the code — referenced by the state of the art (Section 3.2); the inventory enabled per build configuration is named separately by each section. Each axis is an *organ* of the living being: a concrete search algorithm (technically `SearchAlgorithm`, metaphorically a living being) is the fixed composition of its eighteen organ axes (T0–T17), each of which corresponds to an organ; the building-block variants listed per axis are the interchangeable forms of that organ, and their permutation is, in the metaphor canon of this thesis, the exchange of organs on the living being. The *anatomy* of this living being is the wiring *between* the organs: axis algorithms use the interfaces of other axes (the allocation axis, for instance, provides a shared allocation interface for the remaining organs). For each axis it lists the purpose, the *static* sub-axes (compile-time-selectable variant families) and the *dynamic* sub-axes (run-time parameters), as well as the concrete building-block variants with their source (paper identifier or code/baseline). Beyond the eighteen composition slots the organ registry carries an optional organ meta-meta axis¹ `disk_io`, which forms no nineteenth composition slot and never enters the `binary_id` (Section D.19). The entries are taken from the code of the `comdare-cache-engine` and verified against the axis-algorithm-paper-code map of the `ce` repository (file `docs/architecture/18_achsen_algorithmus_paper_code_map.md`).

Beyond the eighteen organ slots this appendix documents the two remaining axis categories of the triad (cf. Section 3.4.1). The *system realm* comprises exactly three main axes in canonical, code-guarded order — target ISA (`target_isa`), operating system (`operating_system`) and extension hardware (`external_utils`) — plus the outer complex bracket `build_target_complex`, which is explicitly not a fourth main axis; the order is not an editorial decision but follows the single ordering source of the code. The *measurement realm* comprises the measurement categories, the measurement tooling and the load framework as meta-meta axis at the end of the realm. In the anatomy metaphor, system and measurement axes are neither a further organ nor the blood (the fluid organ that connects the organ axes), but the *habitat* of the animal (system axes) and the equipment of the *laboratory* (measurement axes; Section 3.3.1); both are not part of the permutation tree of the `binary_id` (`binary_id=never`) and strictly neutral towards the binary identity; their sub-axes are nevertheless permuted by the planner (`stage=runtime`). The entries of both spheres are additionally verified against the three axis registries of the code (`cache_engine_axis_registry.xml` for the organ realm, `system_axis_registry.xml` and `measurement_axis_registry.xml`). Three conserved offering catalogues outside the build-driving offering — the SIMD catalogue Build-SIMD under the extension hardware as well as the page-type catalogue Build-PG and the platform hardware strategy Build-HW — are marked as such and collected in Section D.30 (Conserved designs) at the end of this appendix.

¹Meta-meta in the sense of the M3 level of the four-level layering of the Meta Object Facility [Obj16]: the indirection of a description. Meta axes (M2) classify main and sub-axes, M1 is the conception or the run-time implementation, M0 the binary code. The assignment of M3 is fixed; the cut between M2 and M1 is a reading of this thesis.

D.1 T0 Search Algorithm Axis T0 (Search Structures and Search Methods)

This axis is the *search organ* (T0) of the living being *SearchAlgorithm* (cf. Section 3.2).

Systematic permutation of 22 concrete search algorithms and search structures (S01–S22): 17 base building blocks — array-based, sorted vectors, original paper wrappers, search methods like k-ary/interpolation, and ordered structures like skip-list/B-tree — plus five opt-in building blocks disabled by default, namely the four per-K k-ary wrappers K2/K4/K8/K16 (S18–S21) and the Swiss-table hash structure (S22), to measurably quantify the impact of search paradigm choice on performance characteristics within a uniform `std::map` interface. *Registry state*: the enabled inventory of this build configuration comprises four building blocks (`k_ary`, `interpolation`, `eytzipinger`, `linear_scan`); the catalogue S01–S22 is the offering. Sub-axes classify by access density (SA1 dense, SA2 sparse, SA3/SA4 multilevel), enable compliance proofs through paper mixins (original-code proof, supervisor requirement: `is_original` marking per function/wrapper), and identify SIMD-capable and store-traversable variants.

Static sub-axes

ID	Name	Description
SA1	Direct-Addressed High Density (Dense Access)	Array family: direct-addressed ($O(1)$ lookup), high fill density $\geq 50\%$. Includes <code>Array256SearchAlgo</code> (256-slot compact array, ART Node256 re-impl) and <code>Array65535SearchAlgo</code> (uint16-based, mid-density 25-50% direct-addressed).
SA2	Sparse/Ordered (Sparse Access)	Sorted vectors, ordered search methods, and tree/hash structures: low fill density, range-scan support, variable $O(\log n)$ – $O(1)$ lookup. Includes <code>VectorU8U8SearchAlgo</code> , original paper wrappers (ART/HOT/START/Wormhole/SuRF), search methods (k-ary, interpolation, Eytzipinger, hash, Swiss table), and structures (skip-list, BST, B-tree).
SA3	Multi-Byte Discriminators (Multilevel Access)	Multi-byte search structures with indexed sub-discriminators (e.g., START with cost-DP multi-level). <code>VectorU16U16SearchAlgo</code> (u16-keys, sorted <code>lower_bound</code>) and <code>OriginalStartSearchAlgo</code> (publication P05) belong here.
SA4	Direct-Addressed Medium Density (Direct Multibyte)	Shares the direct-addressing access pattern (like SA1) with multi-byte granularity (uint16 size) and medium fill density 25–50% (vs. SA1 $\geq 50\%$). Example: prt-art <code>Array65535</code> . This sub-axis captures the density variant without breaking the SA1 abstraction.

Building blocks

Variant	Description	Source
<code>Array256SearchAlgo</code> (S01)	ART Node256 direct-addressed re-implementation: <code>std::array<std::optional<u64>, 256></code> with $O(1)$ lookup/insert/erase. Cache-line-aligned, SIMD-capable, no heap allocation. Baseline for dense accesses.	Code/Baseline (Leis ICDE 2013 ART, publication P01)
<code>VectorU8U8SearchAlgo</code> (S02)	Sparse sorted key-value vector pair: <code>std::vector<u8> keys + std::vector<u64> values</code> with <code>std::lower_bound</code> . $O(\log n)$ lookup, $O(n)$ insert/erase. Concept-equivalent to HOT-k-constrained. SIMD-capable (bitmask scan).	Code/Baseline (Binna SIGMOD 2018 HOT, publication P02)

Variant	Description	Source
VectorU16U16SearchAlgo (S03)	Sparse sorted u16 vector pair: <code>std::vector<u16> keys + std::vector<u64> values</code> . Simplified START concept (multi-byte discriminators without cost-DP). $O(\log n)$ lookup, $O(n)$ insert/erase.	Code/Baseline (Fent ICDEW 2020 START, publication P05)
Array65535SearchAlgo (S09)	Mid-density direct-addressed uint16-based (25–50% fill density). Migrated from prt-art. $O(1)$ lookup but with memory overhead vs. SA1 denser arrays.	Code/Baseline (migrated from prt-art)
OriginalArtSearchAlgo (S04)	ART paper binding (<code>unodb::db: is_original_module()=true</code> (4/4 functions original). Paper mixin via SHA256-validated tool code generation. Original-code proof (supervisor requirement): <code>get_compiler()=gcc-9.5</code> , all insert/lookup/erase/clear marked original.	P01 (Leis/Kemper/Neumann ICDE 2013, Apache-2.0 unodb)
OriginalHotSearchAlgo (S05)	HOT paper binding (HOTRowex): <code>is_original_module()=false</code> (2/4 original). insert/lookup original, but erase/clear gaps (HotRowex is append-only; re-impl fallback). Publication P02.	P02 (Binna et al. SIGMOD 2018, ISC)
OriginalStartSearchAlgo (S06)	START paper binding (sosd adapter): <code>is_original_module()=false</code> (2/4 original). insert/lookup original, erase/clear gaps (no remove in the sosd adapter). Publication P05, multi-byte discriminators.	P05 (Fent/Jungmair/Kipf/Neumann ICDEW 2020, MIT)
OriginalWormholeSearchAlgo (S07)	Wormhole paper binding: <code>is_original_module()=false</code> (3/4 original). put/get/del original, clear gap. Hybrid hash+trie+B+ index. GPL-3.0 licensed (blocks original-code linking for the thesis).	P07 (Wu/Ni/Jiang EuroSys 2019, GPL-3.0)
OriginalSurfSearchAlgo (S08)	SuRF paper binding (FST with LOUDS bitmap): <code>is_original_module()=false</code> (1/4 original, lookup only). Succinct range filter, multi-header library. Publication P10.	P10 (Zhang/Lim/Andersen SIGMOD 2018, Apache-2.0)
KArySearchAlgo (S10)	k-ary search, a search <i>method</i> (Schlegel/Gemulla/Lehner DaMoN 2009): K-way partition instead of halving in binary search. K is <code>iterable_aspect</code> ($K \in \{2,4,8,16\}$). $O(\lceil \log_{K+1} n \rceil)$ iterations, parallel ILP comparisons. Array family but no faithful dedicated traversal organ (SA2 classification). C++23 re-impl, <code>is_original=false</code> .	Schlegel et al. DaMoN 2009 [SGL09]
InterpolationSearchAlgo (S11)	Interpolation search, a search <i>method</i> (Perl/Itai/Avni CACM 1978): distribution-aware position estimation instead of blind halving. $O(\log \log N)$ avg with uniform distribution, $O(N)$ worst-case. Not SIMD-vectorizable (data-dependent index). C++23 re-impl.	Perl/Itai/Avni CACM 1978 [PIA78]
EytzingerSearchAlgo (S12)	Eytzinger/BFS-layout branch-free search (Khuong/Morin JEA 2017): cache-layout search <i>method</i> with branch-free kernel. Keys in BFS order of the implicit search tree. For large arrays, per the publication, the fastest general-purpose layout. C++23 re-impl (experiment harness without standard licence).	Khuong/Morin JEA 2017 [KM17]
SkipListSearchAlgo (S13)	Skip-list, a probabilistic ordered <i>structure</i> (Pugh CACM 1990): $O(\log n)$ expected search/insert/erase without explicit rebalancing. Each node has <code>1..kMaxLevel</code> forward pointers (coin-flip probability 0.5). Index-based (<code>std::vector<Node></code> with uint32 indices, value-semantic). Textbook algorithm, C++23 re-impl.	Pugh CACM 1990 [Pug90]
HashSearchAlgo (S14)	Open-addressing hash table, <i>unordered</i> (Knuth TAOCP 3 §6.4): Fibonacci hash, linear probing, tombstone-erase on resize. $O(1)$ expected lookup/insert at <code>load_factor < 0.7</code> . No range-scan support (distinct classification). Correct tombstone marking for probe-chain integrity. C++23 re-impl.	Knuth TAOCP 3 §6.4 [Knu98]

Variant	Description	Source
LinearScanSearchAlgo (S15)	Unsorted linear scan baseline (Leis ICDE 2013 ART Node4): $O(n)$ lookup on an unsorted KV array. Simplest comparison-zero-point structure: insert $O(1)$ -amortized, erase $O(1)$ via swap-and-pop. Store-traversable (array family). No range-scan support.	Code/Baseline (Leis ICDE 2013 ART, publication P01, Node4)
BinarySearchTreeSearchAlgo (S16)	Unbalanced binary search tree (Knuth TAOCP 3 §6.2.2 Hibbard deletion): deterministic comparison-tree baseline (vs. skip-list probabilistic). Index-based (<code>std::vector<Node></code> with <code>uint32</code> indices, free-list). Hibbard erase (3 cases: leaf/1 child/2 children via in-order successor). $O(n)$ worst-case with sorted input.	Knuth TAOCP 3 §6.2.2 [Knu98]
BTreeSearchAlgo (S17)	Balanced block-oriented multi-way B-tree (Bayer/McCreight Acta Inf. 1972 / CLRS Ch. 18): $t=4$, hence [3..7] keys per node, up to 8 children. Always balanced ($O(\log n)$ guaranteed). Index-based, <code>alignas(64)</code> nodes (cache-line alignment). Top-down splitting on insert, borrow/merge on erase. The standard structure for database indexes.	Bayer/McCreight Acta Inf. 1972 / CLRS [BM72]

Building blocks (opt-in, disabled by default)

The following five building blocks are registered in the catalogue and conformant, but disabled by default in the build configuration; the registration builds nothing, it makes the variants selectable.

Variant	Description	Source
KArySearchAlgoK2 (S18)	Compile-time K wrapper of the k-ary search with $K=2$ (binary-search baseline): one animal binary per K (<code>KArySearchAlgoK<N> → KAryTraversal<K></code>) with its own enable flag (default OFF) and its own name <code>k_ary_k2</code> (separation in the <code>binary_id</code>).	Schlegel et al. DaMoN 2009 [SGL09] (Code/Baseline)
KArySearchAlgoK4 (S19)	$K=4$ (paper default, 5-way partition); name <code>k_ary_k4</code> , otherwise as S18.	Schlegel et al. DaMoN 2009 [SGL09] (Code/Baseline)
KArySearchAlgoK8 (S20)	$K=8$; name <code>k_ary_k8</code> , otherwise as S18.	Schlegel et al. DaMoN 2009 [SGL09] (Code/Baseline)
KArySearchAlgoK16 (S21)	$K=16$; name <code>k_ary_k16</code> , otherwise as S18.	Schlegel et al. DaMoN 2009 [SGL09] (Code/Baseline)
SwissTableSearchAlgo (S22)	CE-native re-implementation of the Swiss-table principle: open addressing with one control byte per slot (empty, deleted, 7-bit H2 fingerprint), groups of 16 slots, H1/H2 split and tombstones for erase; unordered, no range scan (SA2). Allocation exclusively through the allocator axis (four buffers in one allocation ledger). Disabled by default and, without organ backing, not to be read as a Swiss-table measurement; <code>is_original=false</code> .	Google Abseil Swiss Tables [Goo18] (talk CppCon 2017); C++23 re-impl

Contributing sources: P01, P02, P05, P07, P10, k-ary (Schlegel et al. 2009), interpolation (Perl et al. 1978), Eytzinger (Khuong/Morin 2017), skip list (Pugh 1990), hashing/BST (Knuth TAOCP 3), B-tree (Bayer/McCreight 1972), Swiss Tables (Google/Abseil 2018) [LKN13, BZP⁺18, FJKN20, WNJ19, ZLL⁺18, SGL09, PIA78, KM17, Pug90, Knu98, BM72, Goo18].

D.2 T1 Cache Traversal (Fanout Lookup)

This axis is the *cache-traversal organ* (T1) of the living being `SearchAlgorithm` (cf. Section 3.2).

Axis `axis_03b` defines three distinct strategies for key resolution in small fanout arrays (typically B+ tree or trie inner nodes). It spans a spectrum from linear scan ($O(N)$, cache-friendly, small), via multiplicative hashing with open addressing ($O(1)$ amortized, Knuth-Fibonacci), to sorted binary search ($O(\log N)$, ordered traversal, B+ tree standard). All three are textbook baselines without original code linking, implemented as pure C++23 re-implementations.

Static sub-axes

ID	Name	Description
CT1	Linear Access	Linear-scan-based lookup methods on small, unsorted fanout arrays. Characterized by $O(N)$ resolution with simple comparisons, typical for nodes with fanout < 32 in B+ trees. Sub-axis for <code>LinearFanout</code> and <code>BinarySearchFanout</code> (both: <code>axis_tag = linear_access_tag</code>).
CT2	Hash-Based Access	Hash lookup with open addressing and linear probing. $O(1)$ amortized resolution, characterized by Fibonacci multiplier ($11400714819323198485 \approx 2^{64}/\varphi$), power-of-2 bucket count, dynamic rehashing when load factor > 0.7 . Only <code>HashLookup</code> wrapper. Sub-axis: <code>axis_tag = hash_access_tag</code> .

Building blocks

Variant	Description	Source
<code>LinearFanout</code>	Unsorted fanout traversal via <code>std::find_if</code> and linear scan. $O(N)$ lookup, no hash overhead, cache-friendly for small N (< 32). Classic B-tree baseline (Bayer/McCreight 1972). Belongs to sub-axis CT1.	Code/Baseline — Bayer/McCreight 1972 (Acta Informatica)
<code>HashLookup</code>	Fibonacci-hash open addressing with linear probing (Knuth TAOCP Vol. 3, §6.4). $O(1)$ amortized, dynamic rehashing (doubles capacity when load ≥ 0.7), iterable runtime capacity (<code>kIterableInitialCapacities: {8, 16, 64, 256, 1024}</code>). Standalone C++23 re-implementation, no original code binding. Belongs to sub-axis CT2.	Code/Baseline — Knuth 1998 (The Art of Computer Programming Vol.3)
<code>BinarySearchFanout</code>	Sorted fanout traversal via <code>std::lower_bound</code> . $O(\log N)$ lookup with sorted ordering, preserves key sequencing for range-capable routing layers. Classic B+ tree inner-node search (Bayer/McCreight 1972), third distinct strategy of the axis. C++23 re-implementation, <code>is_original=false</code> . Belongs to sub-axis CT1.	Code/Baseline — Bayer/McCreight 1972 (Acta Informatica)

Contributing sources: Bayer/McCreight 1972, Knuth TAOCP Vol.3 [BM72, Knu98].

Related literature: P17, P21, P32 [BDFC05, CGM01, SKK⁺25].

D.3 T2 Slot-to-Offset Mapping (Traversal Sublayer)

This axis is the *mapping organ* (T2) of the living being `SearchAlgorithm` (cf. Section 3.2). Mediation between logical slot indices and physical memory offsets in trie inner nodes and memory pools. The axis provides two fundamentally different mapping strategies: direct absolute offsets (linear packing) or pool-relative offsets with automatic rebase mechanism supporting memory reallocation and persistent data structures.

Static sub-axes

ID	Name	Description
MP1	Direct-Access Mapping (MP1)	Direct slot-to-absolute-offset mapping without indirection. Wrapper family for classic array-of-pointers layout as in B-tree inner nodes (Bayer/McCreight 1972). Simple cache locality, $O(N)$ lookup via <code>std::find_if</code> .
MP2	Pool-Relative Mapping (MP2)	Slot-to-pool-relative-offset mapping using based-pointer arena pattern. All offsets stored relative to dynamic <code>pool_base_address</code> . Enables $O(1)$ rebase on memory reallocation — core property for persistent data structures (Driscoll et al. 1989) and <code>CustomAlignedStructure</code> pattern in prt-art.

Building blocks

Variant	Description	Source
DirectPlacement	Wrapper for direct slot-to-absolute-offset mapping (linear packing). <code>std::vector<pair<slot_index_type, offset_type>;</code> <code>resolve_offset/register_slot</code> via linear <code>std::find_if</code> . Usage: prt-art <code>INode::placement_page_</code> + mapping table. Storage: 16-bit slot index + <code>size_t</code> offset per entry. No pool base dependency.	CE-Baseline
PoolRelative	Wrapper for pool-relative-offset mapping with $O(1)$ rebase capability. <code>std::vector<pair<slot_index_type, relative_offset></code> + mutable <code>pool_base_address</code> . On pool reallocation: <code>rebase(new_pool_base)</code> updates only <code>pool_base_</code> — all relative offsets remain valid. Used with prt-art <code>CustomAlignedStructure</code> + versioning scenarios (persistent data structures per Driscoll/Sarnak/Sleator/Tarjan 1989).	CE-Baseline

Contributing sources: — (CE baseline, own wrappers).

Related literature: Bayer/McCreight 1972, Driscoll et al. 1989, P03, P07 [BM72, DSST89, MKM12, WNJ19].

D.4 T3 Path Compression

This axis is the *path-compression organ* (T3) of the living being `SearchAlgorithm` (cf. Section 3.2). Defines the compression granularity and strategy for key paths in trie structures. Controls how inner nodes store and navigate redundant byte sequences or significant bits of common prefixes to optimize memory and cache efficiency.

Static sub-axes

ID	Name	Description
PC1	Compression Granularity	Defines the minimal unit of path compression: bit-wise (Patricia), byte-wise (ART), or no compression (raw path). Determines trie height and optimal code blocks for traversal. Code token: <code>granularity_tag</code> .
PC2	Skip Strategy	Defines the navigation pattern at inner nodes: linear over 8 bytes (ByteWise), single-bit comparison (Patricia), or direct jump without intermediate steps. Influences branch density and prefetch behaviour. Code token: <code>skip_strategy_tag</code> .

ID	Name	Description
PC3	Decode Complexity	Classifies the runtime complexity when extracting prefix information: $O(1)$ for direct extraction, $O(\log n)$ for structure-based search, $O(n)$ for sequential check. Important for operations like <code>common_prefix_len()</code> and key comparisons. Code token: <code>decode_complexity_tag</code> .

Building blocks

Variant	Description	Source
PathCompression None	No path compression; raw key paths are stored completely and uncompressed in nodes. Serves as measurement baseline with <code>compression_ratio=1.0</code> . Used as control case to quantify memory and height savings of compressing strategies.	Code/Baseline
ByteWisePath Compression	Byte-by-byte compression (ART-style); stores up to 7 prefix bytes per node in a compact <code>u64</code> word with length tag in high byte. Typical 2x reduction in trie height (<code>compression_ratio=0.5</code>). Origin: Leis et al., ART ICDE 2013 (P01). Employs goldstandard operations <code>common_prefix_len()</code> and <code>cut()</code> for navigation optimization.	P01
PatriciaPath Compression	Single-bit-split path compression (Patricia trie); navigates bit-wise instead of byte-wise, reduces trie height by typically factor 3 (<code>compression_ratio=0.3</code>). Origin: Morrison PATRICIA JACM 1968; modernly implemented in HOT (Binna SIGMOD 2018, P02) and Wormhole (Wu Euro Sys 2019, P07). F15-operative primitives: <code>key_split_bit()</code> for MSB-first bit extraction, <code>path_descend_scan()</code> for goldstandard-compliant <code>_scan</code> signature with key-dependent variable depth.	P02

Contributing sources: P01, P02, P07, Morrison 1968 [LKN13, BZP⁺18, WNJ19, Mor68].

Related literature: P04, P05, P09 [BFTV24, FJKN20, Jac89].

D.5 T4 Node Types (Node-Type Axis)

This axis is the *node-type organ* (T4) of the living being `SearchAlgorithm` (cf. Section 3.2).

The axis `axis_04_node_type` defines the four node formats of the Adaptive Radix Tree (ART) as exchangeable variants: `Node4`, `Node16`, `Node48`, and `Node256`. Each node type offers a different balance between memory, cache efficiency, and lookup speed. This axis forms the foundation for adaptive tree reorganization and is central to the design of search algorithms.

Static sub-axes

ID	Name	Description
NT1	Capacity Class (NT1)	Classification by fanout size and slot count: small (<code>Node4</code> with 4 slots, linear search), medium (<code>Node16</code> with 16 slots, SIMD binary search) to maximal (<code>Node256</code> with 256 slots, direct-addressed).
NT2	Access Pattern (NT2)	Method of key lookup within a node: linear scan (<code>Node4</code>), SIMD binary search (<code>Node16</code>), indirect indexing (<code>Node48</code>), or direct addressing (<code>Node256</code>).

ID	Name	Description
NT3	Compactness (NT3)	Memory layout and density: sparse and cache-aligned (Node4), dense with indirection (Node48), or maximally direct (Node256), with impacts on memory usage and cache-line misses.

Building blocks

The table lists three kinds of entries from the same axis directory; only the first block is the value set of the axis.

Variant	Description	Source
<i>Axis variants (AllNodeTypes, four)</i>		
Node4NodeType	Small ART node with 4 slots and linear search. Ideal for sparsely populated upper tree levels, fits completely in a cache line (64 bytes). Implemented as CE re-implementation of the ART specification with <code>node_find_scan()</code> as linear scan over sorted keys.	P01
Node16NodeType	Medium ART node with 16 slots, optimized for SIMD-based binary search (SSE2 PCMPSTRM / AVX2 VPCMPSTRM). Balance between memory and search speed; largest capacity without indirection. CE re-implementation of the Leis ICDE 2013 specification.	P01
Node48NodeType	Large ART node with 48 slots, using indirection: 256-byte ChildIndex (byte→slot mapping, 0=empty) plus 48-slot ChildArray. Trade-off between memory (vs Node256 direct) and cache misses (vs Node4 linear). Double-indirection access pattern (index + child).	P01
Node256NodeType	Maximal ART node with 256 slots, directly addressed without index. O(1) lookup (no scan, no index touch). High memory overhead (256 pointers), but optimal search speed for dense nodes. Default for lower tree levels.	P01
<i>Paper node formats (reconstruction targets of this thesis, no CE baseline)</i>		
Internal/Border	Node-type pair of Masstree (trie of B ⁺ trees): internal nodes navigate the 8-byte key slices of the B ⁺ level, border nodes form the leaf level with the values or the transition to the next trie layer. Paper node format (reconstruction target of this thesis, no CE baseline).	P03 (Mao et al. 2012) [MKM12]
Decision/Span	Node-type pair of the B ² -tree (B ⁺ with embedded binary inner nodes, cache-aware): decision nodes carry the binary branching decision within the page, span nodes group contiguous key ranges. Paper node format (reconstruction target of this thesis, no CE baseline).	P06 (Schmeisser et al. 2022) [SSL ⁺ 21]
LOUDS-Dense/Sparse	Node-type pair of the succinct range filter SuRF: LOUDS-Dense encodes the upper, densely populated trie levels bitmap-based with direct addressing, LOUDS-Sparse the lower, sparsely populated levels byte-sequentially succinct. Paper node format (reconstruction target of this thesis, no CE baseline).	P10 (Zhang et al. 2018) [ZLL ⁺ 18]
<i>Storage organs of the composition (storage organs, no node_type values)</i>		
NodeTypeSlotStore	Storage organ: node-type-driven bounded slot storage. Spans <code>std::array</code> with capacity <code>N: :max_capacity()</code> over a shared uint64 key. Fulfills the StorageOrgan concept and is drop-in substrate for ComposedSearch. Baseline variant without layout/allocator integration.	Code/Baseline
ComposedStore	3-axis storage organ: <code>node_type(N) ⊕ layout(L) ⊕ allocator(A)</code> . Unbounded vector over allocator A; layout flows in as static <code>cache_line_size</code> and via optional <code>organ_scan_field_sum()</code> . Measurable (requirement F15).	Code/Baseline
NodeChunkedStore	3-axis storage organ with runtime impact of the <code>node_type</code> axis: slots lie in chunks of capacity <code>N: :max_capacity()</code> . Chunk count = number of node-sized allocations; thus the <code>node_type</code> axis is measurable (Node4 many chunks, Node256 few). Proof of delegation.	Code/Baseline

Variant	Description	Source
LayoutAware ChunkedStore	Layout-honoring variant of NodeChunkedStore: stores records at layout-driven stride (<code>eff_stride</code>). Each record occupies <code>eff_stride</code> bytes (key 0..8, value 8..16, rest padding). Makes the <code>memory_layout</code> axis measurable, OOB-safe for scan methods.	Code/Baseline
ObservableNode Type	ObservableAxis wrapper around a <code>node_type</code> strategy. Passes through static API (<code>max_capacity/name/topic_tag</code>) + offers <code>observe_node_find()</code> as instance driver. Tracks <code>find_count</code> , <code>keys_stored</code> , <code>queries_run</code> , <code>last_checksum</code> . Format-divergent lookup latency measurable (out-of-process path).	Code/Baseline

Contributing sources: P01 [LKN13], P03 [MKM12], P06 [SSL⁺21], P10 [ZLL⁺18].

D.6 T5 Memory Layout

This axis is the *memory-layout organ* (T5) of the living being `SearchAlgorithm` (cf. Section 3.2). This axis classifies the spatial organization of records in memory, directly impacting cache efficiency, SIMD vectorization potential, and false-sharing characteristics. It captures four orthogonal design dimensions, three of which exist as sub-axis tags in the code (HM1–HM3): (1) alignment strategies (cache-line padding vs. strict packing), (2) data organization (row-oriented AoS vs. column-oriented SoA vs. hybrid AoSoA), (3) packing density (dense vs. succinct bit-packed); the fourth, the stride pattern (HM4, sequential vs. interleaved), is a planned dimension without a carrying layout building block, for which the code carries no tag (a permutable stride value would break the `AllLayouts` list of the 320 catalogue, which is fixed at five layout building blocks). Layout choice is a primary driver of cache locality and SIMD capability during index traversal.

Static sub-axes

ID	Name	Description
HM1	Alignment Strategy	Choice between cache-line-padded layout (false-sharing avoidance) and strict packing (memory density). Determines stride overhead during sequential traversal.
HM2	Data Organization	Primary memory ordering: Array-of-Structs (AoS, row-oriented), Struct-of-Arrays (SoA, column-oriented), or hybrid AoSoA (block-mixed). Critical for cache-line utilization during field-selective scans.
HM3	Packing Density	Storage density level: from standard records (8–64 bytes) to compact bit-packing (e.g., LOUDS bitmaps with ~1 bit per slot). Affects decode latency and L1/L2 utilization.
HM4	Stride Pattern (planned, no code tag)	Addressing pattern during traversal: sequential strides (consecutive records), interleaved (e.g., to separate hot/cold data), or cache-aware layouts (e.g., Eytzinger BFS). Optimizes prefetch efficiency. In the code without a carrying layout building block and without a tag: a permutable stride value would break the <code>AllLayouts</code> list of the 320 catalogue, which is fixed at five layout building blocks.

Dynamic sub-axes

ID	Parameter	Description
CacheLineSize	Cache-line size (line-size dimension of the cacheline sub-axis)	Planner parameter for the target platform cache line: 32 bytes (embedded), 64 bytes (standard x86-64/ARM64), 128 bytes (among others some Power ISAs) or 256 bytes (for example s390x). The value space of the cacheline sub-axis comprises 4 line sizes $\times$ 3 alignment values $\times$ 5 prefetch-hint values = 60 configurations; the choice is applied per build as the <code>NTTP CacheLineConfig</code> and affects alignment/padding overhead for all aligned layouts.

Building blocks

Variant	Description	Source
AoSStrictMemoryLayout	Array-of-Structs, tightly packed without cache-line alignment. Advantage: minimal memory footprint (record_size strides, dense). Disadvantage: false-sharing under concurrent writes, poor cache-hit ratio for narrow field scans. Typical for Wormhole index.	Code/Baseline
CacheLineAlignedMemoryLayout	AoS with 64-byte per-record alignment. Avoids false-sharing via padding (stride = round_up(record_size, 64)). Standard for ART/HOT/Masstree/START. Build constant: at record_size=48 (measurement build), 16 bytes of padding arise per record; a measurably higher latency than with aos_strict is the expected cache effect (measurement evidence in the corpus: n/a, the complete measurement series is not available).	Code/Baseline
SoAMemoryLayout	Struct-of-Arrays: all fields stored column-wise sequentially. Advantage: contiguous field scans (e.g., keys-only without values) maximize cache-line utilization (16 $\times$ fewer cache-lines than AoS for single-field scan). Ideal for SIMD vectorization and OLAP indexes. Benefit described in Abadi et al. SIGMOD 2008.	Abadi et al. 2008 [AMH08]
PackedBitmapMemoryLayout	Bit-packed succinct layout (LOUDS / SuRF FST). Stores n entries in $\sim n \cdot \lg(\sigma)$ bits instead of bytes. Extreme compactness (32 $\times$ fewer cache-lines than AoS), but higher decode cost per lookup. Foundation: Jacobson LOUDS FOCS 1989 (P09). Modern impl: SuRF Apache-2.0.	P09 (Jacobson LOUDS FOCS 1989)
AoSoAMemoryLayout	Array-of-Structures-of-Arrays: hybrid layout with block width W=8 (AVX2 lanes). Within block: SoA (fields contiguous). Across blocks: AoS (blocks strided). Unites SIMD vectorization (SoA within) with spatial locality (AoS across). Standard in HPC/SIMD engines. Convention; no single originating paper, but an idiom of the SPMD compiler ispc [PM12].	Code/Baseline (HPC/SIMD convention; ispc [PM12])

Contributing sources: P09, Abadi et al. 2008, ispc (Pharr/Mark 2012) [Jac89, AMH08, PM12].

Related literature: P06, P11, P12, P32 [SSL⁺21, RR99, RR00, SKK⁺25].

D.7 T6 Allocator Axis (T6)

This axis is the *allocation organ* (T6) of the living being `SearchAlgorithm` (cf. Section 3.2).

This axis provides the memory management strategy for the cache engine. It defines 26 concrete allocator wrappers for various algorithms and techniques (from classical baselines like Buddy and Slab through modern research allocators like mimalloc, jemalloc, smalloc to specialized variants for NUMA, PIM

and formally verified systems as well as a memory-type virtualization following the VAMPIR design, A24). All wrappers satisfy standardized concepts and are orthogonally structurable by 7 compile-time variant families (freelist topology, size-class schema, thread locality, synchronization, allocation policy, reclamation, fragmentation strategy).

Static sub-axes

ID	Name	Description
AA1	Free-List Topology	How are free memory blocks organized? Examples: per-page sharding (mimalloc A04), per-thread caching (snmalloc A07), buddy trees (A19), single-global free-list (dlmalloc A20), spans (scalloc A08), hybrid strategies (hmalloc A15).
AA2	Size-Class Schema	How are block sizes classified? Examples: slab per object size (A02), dlmalloc bins (A20), jemalloc size-classes (A05), scalloc spans (A08), buddy power-of-2 (A19), formally verified bins (StarMalloc A13), std::pmr pools (A22).
AA3	Thread Locality	How thread/core-local is allocation? Examples: tcmalloc thread-local caches (A06), rpmalloc thread-heaps (A10), snmalloc message-passing (A07), tcmalloc-warehouse hyperscale (A14), Hoard per-heap locks (A01).
AA4	Synchronization	What concurrency mechanism? Examples: lock-free CAS (Michael A03, LRMalloc A11), per-heap locks (Hoard A01), single-threaded (Exgen A18), wait-free (Crystalline A17).
AA5	Allocation Policy	How are allocation requests routed? Examples: NUMA-origin awareness (NUMALoc A09), cache-set awareness (CAMA A12), NUCA awareness (TCMalloc-Warehouse A14), PIM awareness (PIM-malloc A16), polymorphic memory resource (std::pmr A22), NFP memory-type virtualization (VampirNfp A24).
AA6	Reclamation	How are deleted blocks reclaimed? Examples: magazine cache (Slab A02), page-heap release (TCMalloc A06), lock-free reclamation (Crystalline A17), deferred-free (mimalloc A04), Vmem magazine extension (A23).
AA7	Fragmentation Strategy	How is fragmentation prevented? Examples: dlmalloc coalescing (A20), object colouring (Slab A02), buddy splitting (A19), page-local sharding (mimalloc A04), span pooling (scalloc A08).

Building blocks

Variant	Description	Source
HoardAllocator (A01)	Per-heap free-lists with SMP scalability. Berger et al. (ASPLOS 2000). Synchronization concept via per-heap locks + local heap caches per thread.	P— (OSS, Apache-2.0)
SlabAllocator (A02)	Object-caching kernel allocator with magazine-cache strategy. Bonwick (USENIX 1994, 2001). Classic textbook paradigm without standalone OSS code.	P— (Baseline/Textbook)
MichaelLockFree Allocator (A03)	Lock-free CAS allocator with hazard pointers. Maged Michael (PLDI 2004). Re-implementation with IBM patent background.	P— (LGPL Re-Impl)
MimallocAllocator (A04)	Free-list sharding with deferred-free and page-local management. Leijen/Zorn/de Moura (MSR-TR-2019-18, APLAS 2019). Original code linking (is_original=2/2).	P— (OSS, MIT, is_ original)
JemallocAllocator (A05)	Arena-based size-classes with per-CPU arenas and tcache layer. Evans (BSDCan 2006). Standard allocator of Free BSD and Facebook. Original code linking (is_original=2/2).	P— (OSS, BSD-2-Clause, is_ original)

Variant	Description	Source
TCMallocAllocator (A06)	Thread-caching malloc with central page-heap. Ghemawat/Menage (Google ca. 2005) + TEMERAIRE (OSDI 2021). Google production standard.	P— (OSS, Apache-2.0)
SnmallocAllocator (A07)	Message-passing allocator with asymmetric heap operations. Paul Liétar et al. (ISMM 2019). Original code linking (is_original=2/2).	P— (OSS, MIT, is_original)
ScallocAllocator (A08)	Span-based free-list with virtual spans for multi-core scalability. Aigner/Kirsch/Lippautz/Sokolova (OOPSLA 2015). Low fragmentation via span pooling.	P— (OSS, BSD-3-Clause)
NUMAAllocator (A09)	NUMA-origin awareness with faster memory allocation. UTSASRG (ISMM 2023). Optimized for multi-socket architectures.	P— (OSS, licence not determined)
RPMallocAllocator (A10)	Per-thread span caching without dependencies on other allocator libraries. Mattias Jansson (2017). Original code linking (is_original=2/2).	P— (OSS, Public-Domain/MIT, is_original)
LRMallocAllocator (A11)	Lock-free allocator with hazard pointers for multi-core scalability. Leite/Rocha (VECPAR 2018, LNCS 11333). Original code linking (is_original=2/2).	P— (OSS, MIT, is_original)
CAMAAAllocator (A12)	Cache-Aware Memory Allocator with predictable behaviour. Herter/Backes/Haupenthal/Reineke (ECRTS 2011, Saarland). No standalone OSS implementation.	P— (Baseline/Research)
StarMallocAllocator (A13)	Formally verified hardened memory allocator with F*/Steel. Inria-Prosecco: Reitz/Fromherz/Protzenko et al. (OOPSLA 2024). Mathematically verified allocator.	P— (OSS, Apache-2.0)
TCMallocWarehouse Allocator (A14)	TCMalloc hyperscale extension with HugePage support for warehouse-scale computing. TEMERAIRE (Hunter OSDI 2021) + warehouse characterization (Zhou ASPLOS 2024). NUCA-aware TCMalloc variant.	P— (OSS, Apache-2.0)
HMallocAllocator (A15)	Hybrid lock-free allocator with scalable memory management. Li/Yao/Tang/Lin (IEEE ICPADS 2019). No standalone OSS implementation.	P— (Research Baseline)
PIMMallocAllocator (A16)	Processing-in-memory allocator for heterogeneous hardware with fast local memory. VIA-Research (HPCA 2026, arXiv:2505.13002). Specialized for PIM hardware.	P— (OSS, MIT)
CrystallineReclamation (A17)	Wait-free safe-memory-reclamation family (Nikolaev/Ravindran DISC 2021 / PLDI 2024). Reclamation policy, not a standalone allocator.	P— (Research)
ExgenAllocator (A18)	Single-threaded specialized allocator with optimizations for single-threaded workloads. UT Austin (IEEE CAL 2025, arXiv:2510.10219). “Old is Gold” principle.	P— (Research, licence not determined)
BuddyAllocator (A19)	Buddy system with power-of-2 splitting for simple fragmentation control. Knuth (TAoCP Vol.1, 1968). Classic textbook paradigm.	P— (Baseline/Textbook)
DlmallocAllocator (A20)	Doug Lea malloc with bins-based classification. Lea (1987–2012). Original code linking (is_original=2/2).	P— (OSS, Public-Domain/CC0, is_original)
PtMalloc2Allocator (A21)	ptmalloc2 (glibc malloc) with explicit wrapper control. Wolfram Gloger / Ulrich Drepper (1990s–2000s). Licence-blocked (LGPL-2.1+).	P— (OSS, LGPL-2.1+, blocks linking)
StdMalloc (A22a)	Standard libc malloc (proof of concept) — portable fallback allocator. Baseline for all standard library implementations.	P— (Standard-Library)
PmrResourceAllocator (A22b)	std::pmr::memory_resource (polymorphic memory resource). Halpern (N3916, C++17). Abstract interface without behaviourally distinct default implementation.	P— (Standard-Library, C++17)
PoolResourceAllocator (A22c)	std::pmr::unsynchronized_pool_resource with own size-class pool. Halpern (N3916, C++17). Behaviourally distinct without vendor linking (F15-operative).	P— (Standard-Library, C++17, F15-operative)

Variant	Description	Source
VmemMagazinesAllocator (A23)	Vmem + magazines as extension of the slab allocator for multi-core scaling. Bonwick (USENIX ATC 2001). No standalone OSS implementation.	P— (Baseline/Textbook)
VampirNfpAllocator (A24)	Allocator front following the VAMPIR design (virtualization of non-functional memory properties): allocations pass through an NFP-labelled memory tier (target tier DRAM), the NFP descriptor (<code>bandwidth_class</code> , <code>latency_class</code> , <code>capacity</code>) is exposed. Structural reconstruction of the buildable memory-type virtualization part only: poster and review provide neither code nor a cost formula; energy is deliberately not modelled (no counter, otherwise an empty metric); compensation, transparent OS migration and global pipeline scheduling are out of scope. <code>is_original=false</code> , disabled by default.	P33 (VAMPIR poster SOSp 2023 [BSB ⁺ 23], TU Dresden; reconstruction, <code>is_original=false</code>)

Registry state: the generated organ registry carries three enabled building blocks for the allocator axis — `std_malloc`, `pmr_resource` and `pool_resource`; the catalogue A01–A24 is retained as the offering catalogue, the remaining wrappers can be switched on per vendor availability and enable flag.

Contributing reclamation sources: P29, P30 [McK01, Mic04a]; for the full allocator corpus A01–A24 of the code (the literature corpus A01–A23 of Section 3.2 plus the allocator front A24 reconstructed from the SOTA profile P33 without an allocator profile of its own) see section “A-Korpus”.

D.8 T7 Prefetch Strategy

This axis is the *prefetch organ* (T7) of the living being `SearchAlgorithm` (cf. Section 3.2).

The prefetch axis determines the strategy for anticipatory loading of nodes and addresses along the expected search path in a trie data structure. It supports four concrete strategies with different trigger mechanisms and heuristics: no prefetch (measurement baseline), distance-estimated software prefetch based on node density and cache latency (ART standard), explicit hardware prefetch instructions (Worm-hole standard), and path-oriented prefetch with bundle granularity (prt-art, this thesis).

Static sub-axes

ID	Name	Description
PF1	Trigger Mechanism	Type of control over the prefetch timing: whether no mechanism (baseline), through compiler hints via distance estimator, explicit hardware prefetch instructions, or path-oriented prediction
PF2	Distance Heuristic	Calculation method for prefetch distance (how far ahead to load): linear, adaptive based on node density and tier latency, or no heuristic
PF3	Granularity	Prefetch size per trigger: cache-line (64 bytes, individual), page (4K, OS window), or bundle (multiple cache-lines, trie-path-oriented)

Building blocks

Variant	Description	Source
NonePrefetch	No prefetch strategy (baseline). Serves as comparison reference for measurement series. Trigger: none; heuristic: none; granularity: N/A. <code>is_active()</code> returns false.	Code/Baseline
DistanceEstimator Prefetch	Distance-estimated software prefetch (ART standard per Leis ICDE 2013). Estimates cache distance to next node based on node density [%] and tier latency [cycles] with heuristic: <code>estimate = clamp(1 + sparseness*8 + latency_factor, [1,16])</code> . Trigger: compiler hints via <code>__builtin_prefetch</code> ; heuristic: adaptive (density + latency); granularity: cache-line. Applies ART pattern (unodb/libart).	P01 (ART/unodb, Leis ICDE 2013)
HardwarePrefetch	Explicit hardware prefetch instructions (Wormhole standard per Wu EuroSys 2019). Uses PREFETCHT0/T1/T2/NTA on hash-anchor-chain without software heuristic; CPU estimates distance autonomously. Trigger: explicit hardware instruction; heuristic: hardware-managed; granularity: hardware-determined. License GPL-3.0 blocks <code>is_original</code> linking.	P07 (Wormhole, Wu EuroSys 2019)
PathOriented Prefetch	Path-oriented prefetch heuristic (prt-art, this thesis). Pre-loads all nodes along the predicted trie path before the search starts. Tracker follows the search path and recommends the next address via linear step extrapolation. Hot-path hints: interprets the first 8 bytes of a key as uint64 address. Trigger: path-oriented prediction; heuristic: linear (step extrapolation); granularity: bundle (multiple cache-lines). Related literature: Chen/Gibbons/Mowry SIGMOD 2001 (pB+-Trees Prefetching) + Mowry 1994.	P21 (related: Chen/Gibbons/Mowry SIGMOD 2001); this thesis

Contributing sources: P01, P07, P21 [LKN13, WNJ19, CGM01].

Related literature: P23, P25, P26, P27 [Kha11, MWR25, ZSZ⁺24, ZGH⁺25].

D.9 T8 Concurrency Synchronization Axis (Concurrency Axis T8)

This axis is the *concurrency organ* (T8) of the living being `SearchAlgorithm` (cf. Section 3.2).

This axis controls synchronization patterns and safe memory reclamation for concurrent access to tree/index structures. The axis provides compile-time selectable synchronization patterns (None/Blocking/Reader-Writer/Optimistic/Lock-Free/Wait-Free) as well as orthogonal reclamation schemes (RCU/Hazard-Pointer) for safe memory deallocation in lock-free structures.

Static sub-axes

ID	Name	Description
CC1	Synchronization Pattern (CC1)	Compile-time variant family of synchronization patterns: None (single-threaded baseline), Blocking (pessimistic mutex), ReaderWriter (shared_mutex), Optimistic (lock-free version validation), LockFree (CAS-loops), WaitFree (bounded steps). Each family has distinct runtime characteristics (0-overhead to mutex-overhead).
CC2	Reclamation Scheme (CC2)	Orthogonal compile-time variant family for safe memory deallocation in lock-free structures: RCU (deferred grace-period via userspace-rcu), Hazard-Pointer (per-thread in-use marks). Composable with lock-free patterns, optional for Blocking/Optimistic.

Building blocks

Variant	Description	Source
NoneConcurrency	No synchronization, single-threaded baseline. Both acquire() and release() are no-ops (volatile dummy). Comparison zero point for concurrency-overhead measurement (F15).	Code/Baseline
Blocking Concurrency	Pessimistic mutex locking with std::mutex. Global coarse-grained lock. Serializes all accesses. Upper bound of the locking overhead. Publication: Dijkstra CACM 1965.	Code/Baseline
ReaderWriter Concurrency	Single-writer/multi-reader via std::shared_mutex. Lock-sharing for read-heavy workloads. Middleweight between pessimistic and optimistic patterns. Publication: Courtois et al. CACM 1971.	Code/Baseline
OlcOptimistic Concurrency	Optimistic Lock-Coupling (ART-Sync): version counter per node, lock-free readers with retry-on-conflict. Cheap under low contention. Publication: Leis DaMoN 2016 / DEBULL 2019 (P08, is_original=true, Apache-2.0, unodb).	P08
LockFree Concurrency	Lock-free via Compare-And-Swap (std::atomic CAS-loops). System-wide progress. Generic CAS pattern without blocking. Publication: Michael/Scott PODC 1996 (reference paper).	Code/Baseline
WaitFree Concurrency	Wait-free: each thread completes in a bounded number of steps (strongest guarantee). Without retry loop like LockFree. Realized via sequence lock / fetch-add. Publication: Herlihy TOPLAS 1991 (theory reference).	Code/Baseline
RcuConcurrency	Read-Copy-Update (McKenney OLS 2001): lock-free readers via per-thread nesting counter (relaxed order, <i>no</i> memory barrier on the read side). Reclamation scheme orthogonal to pattern. Local re-impl (P29 userspace-rcu LGPL-2.1, is_original=false).	P29
HazardPointer Concurrency	Hazard Pointers (Michael TPDS 2004, P30): per-thread marked in-use pointers prevent premature deallocation (ABA-safe). Reclamation scheme. Local re-impl (P30 NO LICENSE, is_original=false). SEQ_CST store on acquire().	P30
OlcReservedBlocks Concurrency	OLC + cache-line-reserved value blocks (multi-writer cache-coherence-storm avoidance). Specialization of the Optimistic pattern with additional atomic block-ID reservation (fetch_add). prt-art option (is_original=false).	Code/Baseline

Contributing sources: Dijkstra 1965, Courtois et al. 1971, P08, Michael/Scott 1996, Herlihy 1991, P29, P30 [Dij65, CHP71, LABN16, MS96, Her91, McK01, Mic04a].

D.10 T9 Serialization (Encoding and Compression)

This axis is the *serialization organ* (T9) of the living being `SearchAlgorithm` (cf. Section 3.2).

Axis `axis_10` (organ slot T9) determines how data is encoded and compressed within the cache-engine system: from raw-binary storage layouts via variable-length encoding to succinct bit-packing. The choice of serialization strategy directly impacts CPU cost (delta encoding, bit manipulation, LEB128 variant), memory overhead and IO-bandwidth utilization — critical for LSM-trees and read-heavy workloads with approximate filters (SuRF/LOUDS).

Static sub-axes

ID	Name	Description
SR1	Byte-Order and Word Format	How are bytes arranged within a record? Raw (memcpy raw-layout), Varlen (LEB128-varint with signalling bits), or Packed (bit-by-bit packing)?
SR2	Density (Memory Utilization)	Memory overhead per record: Sparse (complete records), Dense (with padding), or Succinct (information-theoretically dense, $n \cdot H + o(n)$ bits)?
SR3	Compression and Preprocessing	Block compression (LZ4/Snappy, LZ77-based) or delta-encoding (zigzag-transform for negative deltas)? Or no compression?

Building blocks

Variant	Description	Source
RawBinary Serialization	Raw byte storage layout (memcpy). Baseline without transformation. Minimal CPU cost. Typical for in-memory workloads without compression. Variant: <code>serialize_scan()</code> reads 32-bit fields + order-sensitive checksum.	Code/Baseline
VarLen Serialization	Variable-length encoding with signalling bits (LEB128-VarInt). Standard for ART (Leis ICDE 2013 P01). Small values = few bytes. Signalling bits mark byte-length. Data-dependent loop (1-5 bytes). FNV-1a-mix of varint bytes.	P01
Succinct Serialization	Bit-by-bit packing with minimal bit-width (LOUDS/SuRF). Achieves information-theoretic minimum $n \cdot H + o(n)$ bits. Read-only after build. Optimal for approx-filters + read-heavy. Highest per-element CPU cost (bit manipulation). Variant: <code>serialize_scan()</code> with 64-bit accumulator.	P10, P09
Compressed Serialization	Block compression (LZ4/Snappy) layered on raw-binary. Trade-off: CPU cost vs IO/memory bandwidth. Variant: delta-encoding against predecessor + zigzag-transform (negative delta $\rightarrow$ small unsigned). Typical for LSM-trees. Medium CPU cost.	Code/LZ77 (Ziv/Lempel 1977) [ZL77]

Contributing sources: P01, P09, P10, Ziv/Lempel 1977 [LKN⁺13, Jac89, ZLL⁺18, ZL77].

D.11 T10 Value Handle (Value-Handle)

This axis is the *value-handle organ* (T10) of the living being `SearchAlgorithm` (cf. Section 3.2). The value-handle axis defines how data values (Values) are stored and accessed in the cache index: inline within node slots, externally in a pool, via immutable-shared references (RCU-style), versioned pointers (MVCC), or chained external references (multi-value). This axis characterizes the storage-layout strategy for values and thus determines access latency (direct vs. indirect) and concurrency semantics (snapshot vs. versioning).

Static sub-axes

ID	Name	Description
VH1	Storage Location	Compile-time variants for the physical placement of the value: inline in the node slot (combines key/value in one slot without indirection) or external in a separate pool (the node holds offset/pointer only).
VH2	Ownership Semantics	Compile-time variants for ownership and sharing strategy: owned (exclusive per key), shared (immutable reference-counted via RCU-style), weak (borrowing without increment).
VH3	Versioning	Compile-time variants for concurrency control: no versioning (simple ownership), version tags in pointer (MVCC for reader snapshot isolation).

Building blocks

Variant	Description	Source
InlineValueHandle	Value embedded directly in the node slot (no indirection). Standard for small values like <code>uint64_t</code> . Minimal access latency (1 direct read), but node memory is consumed by larger values. Source: ART (Leis ICDE 2013, P01).	P01
ExternalPoolValueHandle	Value stored externally in a pool; the node holds the pool offset only. Enables variable value sizes and more compact nodes, but costs 1 additional cache miss per lookup (pointer chasing). Source: Wormhole (Wu EuroSys 2019, P07).	P07
ImmutableSharedRefValueHandle	Value via reference-counted pointer (RCU-style snapshot sharing). Optimal for reader concurrency: readers see stable snapshots, writers create new immutable snapshots. Requires <code>RefCount</code> acquire/release per access. Source: RCU (McKenney OLS 2001, P29).	P29
VersionedPointerValueHandle	Pointer with version tag in the upper byte (MVCC). Enables multi-version concurrency control: readers get a snapshot version for visibility control, writers create a new version. Tombstone detection via version bit. Source: Masstree (Mao EuroSys 2012, P03) and SMART (OSDI 2023).	P03
ChainRefValueHandle	Multi-value chained external reference (linked list). The node holds the offset to the chain head in the pool; each chain node holds (<code>value_offset</code> , <code>next_offset</code>). Most expensive variant with 2 dependent dereferences per value access. Source: prt-art (local re-implementation).	Code/Baseline

Contributing sources: P01, P03, P07, P29, SMART (OSDI 2023) [LKN13, MKM12, WNJ19, McK01, LZS⁺23].

D.12 T11 Index Organization

This axis is the *index-organization organ* (T11) of the living being `SearchAlgorithm` (cf. Section 3.2). Classification of data storage and index structuring across three orthogonal dimensions: storage order (heap/clustered/unordered), index count (none/single/multiple) and data embedding (separate storage vs. embedded in leaf pages). This axis models fundamental DBMS design decisions that decisively influence cache behaviour, access patterns and I/O characteristics based on the alignment of index order with storage order. The three sub-axes carry the identifiers IO1–IO3 (index organization); the I/O dispatch axis (T12) uses the same identifiers for different subjects.

Static sub-axes

ID	Name	Description
IO1	Storage Order	Arrangement of records in storage: unordered (insertion order in heap), sorted by primary key (clustered), or with pointer indirection (NonClustered). Defines the sequential or random access patterns during range scans. Code token: <code>storage_order_tag</code> .
IO2	Index Multiplicity	Number of supported indexes per table: none (heap baseline), single index (clustered, MySQL InnoDB primary key), or multiple secondary indexes (NonClustered, SQL Server NONCLUSTERED INDEX). Code token: <code>index_count_tag</code> .

ID	Name	Description
IO3	Data Embedding	Storage placement of the data relative to the index: separate in the row store (Clustered/NonClustered with pointer hop), or directly embedded in index leaf pages (IOT/trie-style like ART/HOT). Code token: <code>data_</code> <code>embedding_tag</code> .

Building blocks

Variant	Description	Source
ClusteredIndex Organization	Storage order corresponds to index order (1 primary key). Optimized for range scans along the primary key with sequential disk reads (cache-friendly). Standard: MySQL InnoDB, PostgreSQL CLUSTER, SQL Server CLUSTERED INDEX. Access pattern: sequential forward iteration without predicate evaluation, $O(n)$ aggregation scan.	Bayer & McCreight, Acta Informatica 1972 (10.1007/BF00288683)
HeapIndex Organization	Storage order = insertion order, <i>no</i> index (baseline). Lookup via $O(n)$ full scan with predicate evaluation per record (data-dependent branch, no early exit). Standard: PostgreSQL heap tables, HBase. Access pattern: unordered sequential scan with comparison load and branch misprediction.	Garcia-Molina, Ullman & Widom, Database Systems (Textbook Baseline)
IotIndexOrganization	Data directly embedded in index leaf pages (Oracle IOT, SQL Server CLUSTERED INDEX style). Saves the pointer indirection vs. NonClustered. Equivalent to B+tree leaves with embedded data or trie nodes (ART/HOT/Wormhole). Access pattern: sequential like Clustered, but with additional load from the data in the leaf (two fields per record).	Srinivasan et al., VLDB 2000 (Oracle8i IOT) [SDF ⁺ 00]
NonClusteredIndex Organization	Index order $\neq$ storage order, multiple secondary indexes per table. Lookup via index + pointer hop to the data row (random stride, cache misses). Standard: SQL Server NONCLUSTERED INDEX, PostgreSQL CREATE INDEX. Access pattern: index navigation followed by random storage hops (LCG-determined, actual cache-miss latency from TLB/L2 invalidation), then a residual-predicate comparison per fetched record (data-dependent branch on the fetched data row after the pointer hop).	Comer, ACM Computing Surveys 1979 (The Ubiquitous B-Tree, 10.1145/356770.356776)

Contributing sources: Bayer & McCreight 1972, Comer 1979, Srinivasan et al. 2000, Garcia-Molina, Ullman & Widom (Textbook) [BM72, Com79, SDF⁺00, GMUW09].

D.13 T12 I/O Dispatch Axis

This axis is the *I/O-dispatch organ* (T12) of the living being `SearchAlgorithm` (cf. Section 3.2).

Modelling and selection of I/O strategies for persistence and memory access in the index: RAM-only storage (baseline), buffered via the OS page cache (standard), direct I/O with the `O_DIRECT` flag (NVMe optimization), or memory-mapped file I/O (persistent memory). This axis decouples the search and tree structures from the persistence implementation and permits A/B measurements of the dispatch overhead profiles under identical workloads. The three sub-axes carry the identifiers IO1–IO3 (I/O); the index-organization axis (T11) uses the same identifiers for different subjects.

Static sub-axes

ID	Name	Description
IO1	Persistence Model	Classifies the storage type: RAM-only (no persistence, pure measurement baseline), or disk/persistent memory (file-backed indexes with various caching strategies).
IO2	Caching Strategy	Characterizes OS buffer usage: Buffered (OS page cache, read-ahead + write-back, standard), Direct I/O (<code>O_DIRECT</code> , page-cache bypass, 512B/4 KB alignment), or Memory-Mapped (file-backed <code>mmap</code> with automatic OS paging and page-fault-driven access).
IO3	Write Durability	Defines the consistency guarantees: no <code>fsync</code> (highest performance, no safeguard), <code>fsync</code> after batch (compromise, batch commits), or atomic writes (highest safety, lowest performance). In this implementation primarily present as a conceptual sub-axis; operational realization occurs through the caching strategy.

Building blocks

Variant	Description	Source
<code>InMemoryOnly</code>	Purely RAM-based index without persistence. Direct, sequential record access without dispatch overhead. Fastest path and comparison zero point of the axis. Used for the measurement baseline and performance referencing under conditions without I/O latency.	Code/Baseline
<code>BufferedIo</code>	Standard <code>read()/write()</code> via the OS page cache with automatic read-ahead and write-back. Default for general-purpose DBMS. Trade-off: convenient, but double buffering (app cache + OS cache leads to higher memory usage). Reference publication: Stonebraker CACM 1981 (concept/position paper Operating System Support for Database Management, DOI 10.1145/358699.358703). Classification: engineering pattern, no algorithm code of the reference.	P-Stonebraker-1981
<code>DirectIo</code>	Bypass of the OS page cache via the <code>O_DIRECT</code> flag (<code>open(2)</code>), NVMe-SSD-optimal. 512B/4KB alignment required. Enables DBMS-owned cache strategies (RocksDB, MySQL). Higher CPU cost, but no cache pollution from background activity. No dedicated publication as source (Linux API since 2.4.10); treated as a deliberate engineering baseline.	Code/Baseline
<code>MmapIo</code>	Memory-mapped file I/O (<code>mmap</code>) with persistent memory or read-heavy workloads. The OS pages automatically, no copy between user and kernel space. Trade-off: page-fault latency spikes vs. reuse-friendliness. Reference publication: Crotty et al. CIDR 2022 (anti-pattern baseline Are You Sure You Want to Use MMAP in Your DBMS?, db.cs.cmu.edu/.../cidr2022-p13-crotty.pdf). Deliberately cited as anti-pattern reference to document limits and overhead. OSS reference MIT-licensed.	P-Crotty-2022

Contributing sources: Stonebraker-1981, Crotty-2022 [Sto81, CLP22].

D.14 T13 Migration Strategy Axis

This axis is the *migration organ* (T13) of the living being `SearchAlgorithm` (cf. Section 3.2).

The migration strategy axis (T13, `axis_migration`) encapsulates decision logic for data placement across multi-tier storage hierarchies (RAM/SSD/HDD, hot/cold tiers). It models when and how frequently accessed data is promoted to faster storage or rarely accessed data is demoted, without performing

an actual tier migration (decision-logic measurement only). Supports four concrete strategies: static (baseline), heat/cold separation, multi-tier hierarchy, ML-driven adaptive.

Static sub-axes

ID	Name	Description
MG1	Trigger Mechanism (MG1)	Determines when a migration decision is triggered: none (static), threshold-based (Threshold), observation-driven (Observation). Compile-time variant per strategy.
MG2	Migration Direction (MG2)	Determines the direction of data movement: none (static), upward (RAM→SSD), downward (SSD→RAM), bidirectional. Compile-time variant.
MG3	Migration Granularity (MG3)	Specifies the scaling unit for migration decisions: none (static), Page, Block, Object (single item); the size of the page and block unit is not fixed by the code (pure tag types <code>granularity_tag</code> ; the migration building blocks measure the decision logic without a real block transfer). Compile-time category.

Dynamic sub-axes

The three quantities are design parameters of the axis; the code carries them as compile-time constants inside the decision scans (`w_freq = 3` and `w_rec = 1` in `axis_migration_adaptive.hpp`, `kTiers = 3` in `axis_migration_tier_based.hpp`; the hot/cold scan decides by recency comparison without a threshold), a run-time or user parameter is implemented for none of the three, and the generated registry carries no run-time sub-axis for T13.

ID	Parameter	Description
D1	Decision Threshold	Run-time parameter (plan; no code parameter): hot/cold classification boundary (e.g., recency counter limit for LRU-based promotions/demotions), user-configurable; the hot/cold scan in the code decides per record by recency comparison with the last seen field value, without a boundary.
D2	Feature Scoring Weights	Run-time parameter (plan; a constant in the code) for AdaptiveMigration: weights (<code>w_freq</code> , <code>w_rec</code>) for frequency and recency features in the ML linear combination, fixed in the code as <code>w_freq = 3</code> and <code>w_rec = 1</code> . Self-adaptation of the weights foreseen but not implemented; implemented is a running score accumulator with fixed weights.
D3	Tier Configuration	Run-time (plan; a constant in the code): number of storage tiers (e.g., <code>kTiers=3</code> for RAM/SSD/HDD in TierBasedMigration; in the code the compile-time constant <code>kTiers = 3</code>), capacities per tier and promotion/demotion thresholds (plan; without a code object).

Building blocks

Variant	Description	Source
NoMigration	Static placement without migration (baseline). Data is positioned once, no movement between tiers. Benchmark zero point for comparison with active strategies.	Code/Baseline
HotColdMigration	Heat/cold separation via LRU + probabilistic recency counting. Frequently accessed data (hot) is promoted to faster cache, rarely accessed data (cold) is demoted to slow storage. Standard for LRU-K and LFU cache systems.	Anti-Caching (PVLDB 2013) [DPT ⁺ 13]

Variant	Description	Source
TierBased Migration	Multi-level migration across the tiers RAM→SSD→HDD (k Tiers = 3 in the code; the paper-code map of the ce repository names the tier sequence RAM→NVM→SSD→HDD, an NVM tier is not implemented in the code) with modulo-based tier mapping. Each data item is mapped modulo-wise to its target tier by a field value. Standard for RocksDB, LSM-trees and tiered cache systems with configurable thresholds; the building block itself has no threshold parameter (table of the dynamic sub-axes).	Managing NVM in DBMSs (SIGMOD 2018) [vRLK ⁺ 18]
AdaptiveMigration	ML-driven adaptive migration with online learning (Cachelib/ LeCaR-style). Weighted linear combination of frequency and recency features computes an adaptive score; a running learning accumulator models online learning. Higher overhead vs. optimal hit rate.	ML-based LeCaR (USENIX HotStorage 2018) [VRM ⁺ 18]

Contributing sources: Anti-Caching (PVLDB 2013), LeCaR (HotStorage 2018), Managing NVM (SIGMOD 2018) [DPT⁺13, VRM⁺18, vRLK⁺18].

D.15 T14 Filter Axis

This axis is the *filter organ* (T14) of the living being `SearchAlgorithm` (cf. Section 3.2).

The filter axis T14 offers selectable probabilistic membership-query filters (point and range queries; all four building blocks answer approximately, their construction is deterministic). It covers classic Bloom filters with k hash functions, the deletable cuckoo filter with bucket hashing and fingerprints, the succinct range filter SuRF with LOUDS-trie navigation, and the space-saving xor filter with constant 3-way-XOR latency. Purpose: make the trade-offs between space, query latency, mutability and false-positive rate directly measurable in the cache-engine experiment. As an exact comparison zero point the pass-through variant `NoneFilter` is additionally registered (E14), disabled by default in the build configuration.

Static sub-axes

ID	Name	Description
FT1	Query Type	Distinction between point queries (membership: key present?) and range queries (range: keys in [a,b]?). Point filters are Bloom/Cuckoo/Xor; only <code>RangeSurfFilter</code> supports range semantics.
FT2	Mutability	Division into immutable filters (Bloom, <code>RangeSurf</code> , Xor immutable after build) vs. mutable (Cuckoo with delete operation). Determines wrapper design and caching possibilities during runtime.
FT3	Error Profile	Characterization of the error-tolerant semantics: false-positive-only (Bloom, Cuckoo, Xor — filter answers “yes” incorrectly) vs. lossless (theoretically deterministic answer). Bloom: approx. $k \cdot n$ hash tests; Xor: exactly 3 XOR operations (branch-free).

Building blocks

Variant	Description	Source
BloomFilter	Classical Bloom filter per Bloom 1970: m-bit bitmap, k independent hash functions (double hashing per Kirsch/Mitzenmacher 2006). Point queries only; false positives possible, no deletes. Implementation with an actual persistent m-bit bitmap (8 KiB, m=65536, k=4 double hashing), which is probed in the out-of-process measurement path over the actually inserted keys; the pseudo-bitmap probe declared as such (k=4 hash tests per query, characteristic early-exit latency on mismatch) remains only in the in-process segment-timing scan.	Bloom 1970 (CACM 13:422-426)
CuckooFilter	Cuckoo filter per Fan et al. CoNEXT 2014: bucketed cuckoo hashing with 1-byte fingerprints. Supports deletes and lookup counting (advantage vs Bloom). Probe characteristic: 2 bucket lookups + fingerprint comparison (partial-key cuckoo per CoNEXT 2014 §3: i2 = i1 XOR hash(fp)). Higher insert cost, but flexible mutability.	Fan et al. CoNEXT 2014 (DOI:10.1145/2674005.2674994)
RangeSurfFilter	SuRF (Succinct Range Filter) per Zhang et al. SIGMOD 2018: LOUDS-bitmap trie for range queries. Only filter with range semantics (lower_bound/upper_bound at byte-prefix level). Probe characteristic: multi-level trie descent with succinct-LOUDS navigation (up to kMaxDepth=6), not exactly 1 hash test, hence latency variable per prefix length. Practical in RocksDB SST optimizations.	P10 (Zhang et al. SIGMOD 2018)
XorFilter	Xor filter per Graf+Lemire 2020 (JEA 25): XOR-based filter with approx. 9 bits/key (vs. 10 bits/key Bloom at the same FP rate). Immutable after build. Probe characteristic: exactly 3 independent table accesses (3 disjoint table thirds h0, h1, h2) + 3-way XOR, branch-free, constant latency without early exit. Safe against optimization-away, smallest memory footprint of all filters.	Graf+Lemire 2020 (JEA 25:Article 1.5, DOI:10.1145/3376122)
NoneFilter	Pass-through variant without filtering (E14): every probe passes unconditionally (input equals output), 0 bit structure memory and 0 hash tests per probe — the comparison zero point for the space/time trade-off of the four filters (family 5, end-append, golden-320-neutral). Registered in the catalogue and conformant, but disabled by default in the build configuration (option COMDARE_AXIS_FILTER_ENABLE_NONE, default OFF), so that the enabled inventory of the axis comprises four building blocks.	Code/Baseline

Contributing sources: Bloom 1970, Fan et al. CoNEXT 2014, Zhang et al. SIGMOD 2018, Graf+Lemire JEA 2020 [Blo70, FAKM14, ZLL⁺18, GL20].

D.16 T15 Buffering Q1 (Operations Buffer)

This axis is the *buffering organ* (*Q1*) (T15) of the living being `SearchAlgorithm` (cf. Section 3.2). The axis `axis_ql_queueing` realizes various buffering strategies for operations in the cache-engine context. It encompasses textbook ADTs (FIFO, LIFO, Append-Only, Priority-Heap), specialized versioning techniques (Delta-Chain, Copy-on-Write, Epoch, Tombstone) and modern lock-free queues (SPSC/MPMC following Lamport and Vyukov). The strategies are selected at compile time via the axis registry (the `mp_list` of the axis registry, filtered by enable flags) per permutation binary (no runtime switch) and are classifiable by access pattern (sequential, ordered, cyclic, versioned, batched, lock-free).

Static sub-axes

ID	Name	Description
QS1	Sequential Access	FIFO/LIFO/Append-Only: access at head or tail, typical for write-coalescing + MemTable buffering
QS2	Ordered Access	Skip-List/Priority-Heap: sorted drain, typical for LSM compaction and hot-key eviction
QS3	Cyclic Access	Bounded ring buffer: fixed capacity, Disruptor pattern, wrap-around with overflow handling
QS4	Versioned Access	Delta-Chain/Tombstone/CoW/Epoch: multi-version concurrency control, snapshot isolation, reclamation windows
QS5	Batched Operations	Batched-Insert-Buffer: accumulation of operations into sub-batches, SIMD vectorization, cache-locality optimization
QS6	Lock-Free Concurrent	SPSC/MPMC: CAS loop or wait-free Lamport model, lock-freedom without mutex

Building blocks

Variant	Description	Source
NoBuffer	Passthrough/identity (baseline): <code>put()</code> is a no-op, <code>get()</code> always returns <code>std::nullopt</code> . Useful as comparison point and in classic B+/ART scenarios with direct leaf access. Family Q01, QS1 <code>sequential_access</code> .	Code/Baseline
AppendOnlyBuffer	Append-only linear buffer (LSM MemTable, Bw-Tree delta): monotonically growing <code>std::vector</code> without deletion in the middle. Optimized for pure append + bulk drain. Family Q02, QS1 <code>sequential_access</code> . Literature: Levandoski/Lomet/Sengupta Bw-Tree ICDE 2013, O'Neil LSM 1996.	Code/Baseline
FIFOQueueBuffer	FIFO queue (<code>std::deque</code>): first-in-first-out via <code>deque::push_back / deque::pop_front</code> . Unbounded, can grow efficiently in the middle. Textbook ADT, typical for write-coalescing. Family Q03, QS1 <code>sequential_access</code> .	Code/Baseline
LIFOStackBuffer	LIFO stack (<code>std::vector</code>): last-in-first-out via <code>vector::push_back / vector::pop_back</code> . Unbounded, optimized for hot-path reuse and version tombstones. Family Q04, QS1 <code>sequential_access</code> .	Code/Baseline
BoundedRingBuffer	Bounded ring buffer (Disruptor pattern): fixed capacity, cyclic access with wrap-around. Overflow: <code>drop_oldest</code> (default). <code>iterable_aspect_t</code> : capacity as runtime loop. Family Q05, QS3 <code>cyclic_access</code> . Literature: LMAX Disruptor 2011.	LMAX 2011
PriorityHeapBuffer	Priority heap (<code>std::priority_queue</code>): max-heap, <code>get()</code> returns the highest priority first. First strategy with <code>supports_priority_ordering()=true</code> . Unbounded, textbook ADT (Williams 1964). Application: LRU approximation, hot-key promotion, eviction queues. Family Q06, QS2 <code>ordered_access</code> .	Code/Baseline
DeltaChainBuffer	Delta-chain buffer (Bw-Tree, Levandoski 2013): first versioned strategy (<code>is_versioned()=true</code>). Each <code>put()</code> appends a delta record with ascending version ID to the top of the chain. <code>get()</code> LIFO drain with version ID. Append-versioning paradigm. Family Q07, QS4 <code>versioned_access</code> .	Bw-Tree (Levandowski et al. ICDE 2013) [LLS13]
SkiplistBuffer	Skip-list buffer (sorted buffer, RocksDB-style): <code>put()</code> inserts in sorted order via <code>std::set</code> . <code>get()</code> ASCENDING drain (minimum first), typical for LSM-compact output. Difference from <code>PriorityHeapBuffer</code> : <code>skiplist=ascending</code> , <code>heap=descending</code> . Literature: Pugh CACM 1990. Family Q08, QS2 <code>ordered_access</code> .	Pugh 1990, RocksDB

Variant	Description	Source
TombstoneBuffer	Tombstone-marker buffer (LSM + MVCC): <code>put()</code> creates a live entry, deleted entries remain as tombstones (<code>std::nullopt</code>) until the compaction run. <code>get()</code> skips tombstones FIFO-like, <code>is_versioned()</code> = true (erase-versioning, unlike delta-chain append-versioning). <code>size()</code> counts live slots only. Family Q09, QS4 <code>versioned_access</code> . Literature: O’Neil LSM-tree 1996.	O’Neil 1996
CopyOnWriteBuffer	Copy-on-write snapshot buffer (persistent data structures): <code>put()</code> / <code>get()</code> increments the snapshot version, earlier snapshots remain logically preserved via <code>shared_ptr</code> . Snapshot-versioning paradigm (unlike delta-chain append, tombstone erase). <code>is_versioned()</code> = true. Family Q10, QS4 <code>versioned_access</code> . Literature: Driscoll/Sarnak/Sleator/Tarjan JCSS 1989.	Driscoll et al. 1989
EpochBuffer	Epoch-based buffer / QSBR (quiescent-state-based reclamation): entries accumulated in the current epoch. After the <code>epoch_threshold</code> transition => the earlier epoch can be reclaimed. <code>iterable_aspect_t</code> : <code>epoch_threshold</code> as runtime parameter. <code>is_versioned()</code> = true (reclamation-window versioning). Family Q11, QS4 <code>versioned_access</code> . Literature: McKenney RCU OLS 2001, Masstree EuroSys 2012, SMART OSDI 2023.	P29 (RCU OLS 2001), Masstree 2012
BatchedInsertBuffer	Batched-insert buffer (OLAP indexing, ART bulk insert): <code>put()</code> buffers into sub-batches of size <code>batch_size</code> , complete batches are handed over (bulk drain). Goal: amortized insert cost through bulk operations, SIMD vectorization, cache locality. First QS5 strategy. Family Q12, QS5 <code>batched_access</code> . Literature: Leis ART ICDE 2013 §6.	P01 (Leis ART ICDE 2013)
LockFreeSPSCBuffer	Lock-free SPSC ring queue (Lamport 1977, 1983): single-producer/single-consumer. <code>put()</code> wait-free (1 atomic.store + 1 atomic.load), <code>get()</code> wait-free. No CAS needed thanks to disjoint modified sets. <code>ProgressGuarantee</code> : :LockFree (even wait-free). <code>iterable_aspect_t</code> : capacity. Overflow: dropping. Mandatory contract: exactly 1 producer + 1 consumer. Family Q13a, QS6 <code>lock_free_access</code> .	Lamport 1977, 1983
LockFreeMPMCBuffer	Lock-free bounded MPMC queue (Vyukov 2010–11): multi-producer/multi-consumer via per-cell atomic sequence number and CAS loop. <code>Cell.sequence == pos</code> (producer writes), then <code>pos+1</code> (consumer reads). Retries on contention (wait-free per op <i>not</i> guaranteed). Capacity <i>must</i> be a power of 2. <code>iterable_aspect_t</code> . Family Q13b, QS6 <code>lock_free_access</code> . <code>is_original_eligible</code> =true (Vyukov BSD-2).	Vyukov 1024cores 2010–11, Michael/Scott PODC 1996
OriginalLockFreeMpmcConcurrentQueue	<code>moodycamel::ConcurrentQueue</code> (Cameron Desrochers, block-based MPMC): proof wrapper for original code with original-code linking (2/6 methods original: <code>put</code> , <code>get</code> ; 4/6 gaps: <code>emplace</code> , <code>peek_front</code> , <code>peek_back</code> , <code>clear</code> are re-impl). <code>is_original_module()</code> = false because of the gap fillings. Header-only, BSD-2/BSL-1.0 dual. Analogous to <code>LockFreeMPMCBuffer</code> , but with an actual submodule binding. Family Q15, QS6 <code>lock_free_access</code> . <code>is_original_eligible</code> =true (moodycamel BSD-2).	moodycamel 2014, BSD-2/BSL-1.0

The family numbers run Q01–Q13b and Q15; Q14 is not assigned in the catalogue, the two lock-free queues share family Q13 (Q13a SPSC, Q13b MPMC), the original wrapper carries Q15.

Contributing sources: LMAX 2011, P01 (Leis ART ICDE 2013), Bw-Tree (Levandoski et al. ICDE 2013), P29 (RCU OLS 2001), Lamport 1977, 1983, Vyukov 1024cores 2010–11, Michael/Scott PODC 1996, Pugh CACM 1990, O’Neil LSM-tree 1996, Driscoll et al. 1989, Masstree EuroSys 2012, SMART OSDI 2023, Williams 1964, moodycamel 2014 [TFB⁺11, LKN13, LLS13, McK01, Lam77, Lam83, Vyu11, MS96, Pug90, OCGO96, DSST89, MKM12, LZS⁺23, Wil64, Des14].

D.17 T16 Buffering Q2 (Flush Policy)

This axis is the *buffering organ* (*Q2*) (T16) of the living being `SearchAlgorithm` (cf. Section 3.2). Axis *Q2* defines automatic flush strategies for write buffers. It decides *when* a filled buffer propagates its output to the next stage — after each operation (Eager), never (Lazy), on threshold excess (Watermark), after a time window (Timed) or adaptively by workload frequency (EWMA-controlled LSM style). This is central to the memory/throughput trade-off and the batching efficiency in cache engines.

Static sub-axes

ID	Name	Description
FS1	Event-Triggered	Flush is triggered by explicit events (put/get operation or eviction). Encompasses EagerFlush (after every operation) and LazyFlush (only on eviction).
FS2	Threshold-Triggered	Flush is triggered when the buffer fill exceeds a threshold (default 75%). Reduces unnecessary flushes under sparse traffic.
FS3	Time-Triggered	Flush is triggered periodically after a time window (micro-batching), independent of the buffer fill. Classic Hadoop/Spark/Kafka pattern for bounded-latency guarantees.
FS4	Adaptive-Triggered	Flush threshold is learned adaptively from the workload frequency via EWMA. High write rate → flush earlier (60% threshold); low rate → flush later (95% threshold). First self-adaptive Q2 strategy.

Building blocks

Variant	Description	Source
EagerFlush	Pessimistic write-through policy: flushes completely (Full Flush) after <i>every</i> <code>put()</code> operation. Lowest latency, but maximum flush frequency and highest throughput overhead — classic synchronous-write pattern.	Textbook Write-Through Policy / Code Baseline
LazyFlush	Optimistic write-back policy: <i>never</i> flushes automatically, only on explicit eviction from the buffer. Maximum batching, minimum flush frequency — classic deferred-write pattern. <code>should_flush()</code> <i>always</i> returns <code>NoFlush</code> .	Textbook Write-Back Policy / Code Baseline
WatermarkFlush	Threshold-controlled flush policy (default 75% capacity). Iterable aspect with 5 variants (50%, 65%, 75%, 85%, 95%) for runtime permutation. Related to the LSM-tree memtable trigger (O’Neil 1996).	LSM-Tree O’Neil 1996 (related) / Code Implementation
TimedFlush	Time-window-based micro-batching policy (iterable aspect: 10/100/1000/10000 ms). Inspired by Kafka (NetDB 2011) and Spark D-Streams (SOSP 2013) for bounded-latency guarantees. Independent of the buffer fill.	Kafka NetDB 2011 / Spark SOSP 2013 / Code Implementation
AdaptiveLsmFlush	Self-adaptive watermark policy with EWMA over the burst rate. Threshold interpolates between 60% (high write rate) and 95% (low rate). Conceptually RocksDB Dynamic Leveled Compaction + O’Neil LSM 1996; the EWMA logic is CE-specific. First Q2 strategy with <code>is_adaptive=true</code> .	RocksDB Dynamic Leveled Compaction / O’Neil LSM-Tree 1996 / Code Implementation

Contributing sources: O’Neil et al. 1996 (LSM-Tree), Kafka (NetDB 2011), Spark D-Streams (SOSP 2013) [OCGO96, KNR11, ZDL⁺13].

D.18 T17 Persistence Target (Persistence-Target)

This axis is the *persistence-target organ* (T17) of the living being `SearchAlgorithm` (cf. Section 3.2); it is the eighteenth organ slot (T17).

The axis makes the storage target of the structure explicit as an organ slot of its own instead of implicitly carrying an in-memory assumption. In the golden reference catalogue it is pinned to its in-memory representative: seventeen axes with two expressions each and `persistence_target` with one yield $2^{17} = 131072$ reference identities per system permutation; every `binary_id` nevertheless carries the eighteenth segment `persistence_target=persistence_memory_only`. The cardinality of a complete measurement series is different from that: it is not a static number but is counted per profile from the Director walk (Section 3.4.6).

Building blocks

Variant	Description	Source
<code>MemoryOnlyTarget</code>	In-memory storage as the default target; the only enabled representative of the canonical measurement catalogue. No device write-back path (<code>has_device_writeback_path()=false</code>).	Code/Baseline
<code>DiskWritebackTarget</code>	Disk write-back target; fully present as a building block (type, registry, sub-axes PT1–PT2), but switched off per option in the canonical catalogue, so that the axis remains single-element there. It is at the same time the carrier of the organ meta-meta axis <code>disk_io</code> (Section D.19), whose stamp suffix attaches only to compositions with this target.	Code/Baseline

Contributing sources: — (CE baseline, own building blocks).

D.19 Organ Meta-Meta Axis: Disk I/O (`disk_io`)

This quantity is the only organ meta-meta axis of the organ realm (meta-meta in the sense of the M3 level, cf. the footnote at the beginning of this appendix). It is no nineteenth composition slot: the composition stays at eighteen organ axes, and the axis never enters the `binary_id` (`binary_id=never`, `stage=ct`, `baustein_count=1`); the organ registry carries it as category `organ_meta_meta` beside the eighteen slots.

The axis carries the stamping identity of the member `disk_io`: its axis code version (`1.0.0.c`) attaches as a bracket appendage to the end of the organ line (member [3] of the animal fingerprint), and exclusively for compositions with the persistence target `DiskWritebackTarget` (carrier axis `persistence_target`, carrier value `persistence_disk_writeback`); compositions with `MemoryOnlyTarget` remain byte-identical, their suffix is empty and produces no empty bracket pair. The form follows the system model: the organ line of an I/O carrier ends in `; [disk_io=code@1.0.0.c]`. The binding strategy type $\rightarrow$ meta-meta lives in the ABI layer, not in the axis header; carriers of the selection are the eighteen existing slot aliases of the composition. For the channel machinery, `disk_io` is declared as a genus channel of its own; the channel itself is declared as a concept but not implemented. Thus the stamp mechanics of the axis are implemented, the channel construction is not; also not implemented is the profile connection: the profile parser does not read the registry block of the axis, and a reference to it in organ position is rejected by the profile validation, so that the axis is not

selectable via experiment profile and is switched off; in the golden reference catalogue its suffix remains empty (Section 3.4.1).

Dynamic sub-axes

ID	Parameter	Description
<code>disk_io</code>	Provisioning stage of the disk I/O	<code>stage=runtime</code> , <code>value_type=token</code> , <code>option_count=1</code> . Only option staging: <code>DiskWritebackTarget</code> measures exclusively the write-back preparation (no device write path, <code>has_device_writeback_path()=false</code>); a value device follows only with the actual device path — the registry contains only occupied options. The family spans its sub-axis of the same name (sub-axis label = axis label).

Building blocks

Variant	Description	Source
<code>DiskIoOrganMeta</code> <code>Meta</code>	Building block <code>disk_io</code> , <code>enabled(enabled=true)</code> ; satisfies the concept <code>OrganMetaMetaAxis</code> (category directory <code>organ_axes/organ_meta_meta/</code> , root model is the SIMD family of the system hub); axis kind <code>organ_meta_meta</code> , explicitly no organ main axis (<code>static_assert</code>).	Code/Baseline

Contributing sources: — (CE baseline, own building block).

D.20 System Sphere: Target ISA (`target_isa`)

This quantity is the system main axis `target_isa` — itself a complex axis (cf. Sections 3.4.1 and 3.4.2). The axis specifies the target CPU instruction set architecture as a compile-time constant and thereby enables cross-platform optimizations for four dominant server architectures: x86-64 (Intel/AMD), AArch64 (ARM), Power-ISA (IBM) and RISC-V (open source). Each variant defines a hardware-specific SIMD baseline (SSE2, NEON, VSX, scalar) and thus characterizes the vector acceleration potential for cache-engine operations such as `simd_field_sum` (accumulation of 32-bit word fields).

Static sub-axes

ID	Name	Description
IS1	Word Size	CPU data-path width: 32-bit, 64-bit or 128-bit. In the cache engine standardized to 64-bit for all four ISA profiles.
IS2	CPU Vendor/Family	Instruction set family: <code>intel_amd</code> (x86-64), <code>arm</code> (AArch64), <code>ibm</code> (Power-ISA), <code>open</code> (RISC-V). Determines SIMD extension compatibility: SSE2/AVX2/AVX-512 (x86-64), NEON/SVE/SVE2 (AArch64), VSX (Power), RVV (RISC-V).

ID	Name	Description
IS3	ZIH Cluster Affinity	Production machines of the ZIH of TU Dresden [Zen26a]: Barnard (Intel Xeon Platinum 8470, Sapphire Rapids, x86-64), Capella (AMD EPYC 9334, x86-64), the Grace Hopper Superchip test system (72 Arm Neoverse V2 cores, AArch64; test system with limited availability [Zen26b]), N/A (RISC-V and Power: not among the current HPC resources of the ZIH compendium). Enables hardware-specific experiments with production cluster profiles.

Beyond these three design dimensions, the build-driving system registry carries five sub-axes on the complex wrapper: two compile-time ones (`target_isa_complex` and `scheduling`, each `stage=ct` with `kind=fixed_enum_tuple`) and three dynamic ones (`numa_node`, `page` and `core_class`). The first bundles the named recombinations of the target machines (`complex_count=2`) with three fixed members each.

Recombination	Member	Declaration
<code>prod1_zen5</code> (<code>isa=x86_64</code>)	<code>ram_frequency_mhz</code>	<code>declared=false</code> — no declared figure.
<code>prod1_zen5</code>	<code>cas_latency_cl</code>	<code>declared=false</code> — no declared figure.
<code>prod1_zen5</code>	<code>cpu_fabrication</code>	<code>declared=true</code> , <code>value=AuthenticAMD/26/68/0</code> (vendor/family/model/stepping).
<code>prod2_alder_lake</code> (<code>isa=x86_64</code>)	<code>ram_frequency_mhz</code>	<code>declared=true</code> , <code>value=4800</code> , <code>declaration_source=dmidecode 2026-07</code> .
<code>prod2_alder_lake</code>	<code>cas_latency_cl</code>	<code>declared=false</code> — no declared figure.
<code>prod2_alder_lake</code>	<code>cpu_fabrication</code>	<code>declared=true</code> , <code>value=GenuineIntel/6/151/2</code> .

Note: the values originate from `kDeclaredMachines`, which in turn mirrors the `<machines>` block of the user XML. `declaration_source` preserves the fixed origin of a declared figure as a note, not as a provenance level; run-time levels never appear in this XML.

The second compile-time sub-axis `scheduling` carries five fixed members. It hangs as a `sub_axis` on the `target_isa` complex wrapper and is, by the design decision of this thesis, at the same time a permutation axis of the planner — sub-axis and permutation axis are not mutually exclusive. In the build offering it is `stage=ct` with a fixed five-tuple; the permutation through the planner is fixed as a design but not implemented.

Member	Registry value	Description
<code>worker_pool_layout</code>	<code>value_ordinal=0</code>	Distribution of the worker pool across the cores.
<code>simd_worker_count_limit</code>	<code>value=2</code>	Upper bound of concurrent SIMD workers.
<code>hetero_core_dispatch</code>	<code>value_ordinal=0</code>	Dispatch across heterogeneous cores (P and E cores).
<code>co_routine_strategy</code>	<code>value_ordinal=0</code>	Coroutine strategy of job processing.
<code>batch_granularity</code>	<code>value_ordinal=0</code>	Granularity of batch processing.

Note: the ordinals are raw enum values without a compile-time label template; the identifier strings are compile-coupled to the accessor calls of `concepts::scheduling_strategy.hpp`.

Dynamic sub-axes

ID	Parameter	Description
numa_node	NUMA node	stage=runtime, value_type=token, option_source=machine_resolved. Mirror of the organ member alloc_hw.numa_node; the admissible values are the node identifiers of the concrete machine and are not knowable at build time — hence a declared option source instead of an option catalogue.
page	Page-size release	stage=runtime, value_type=token, option_source=machine_resolved. The release of page sizes (4k/2m/1g) as a system capability; enforcement lives on the organ side in the AllocPage Hint NTPP, this axis is the system side of the release.
core_class	Core class	stage=runtime, value_type=token, option_source=machine_resolved. The class of the executing core (P or E core on hybrid processors) as the execution locality of the measurement — a sibling of numa_node and page, which capture the memory locality; distinct from scheduling, which contains the policy fixed at compile time: core_class says whether and which core classes the machine carries. Admissible values are machine-resolved; the separate measurement on P and E cores is thus named in the catalogue as the system side of core pinning, the measurement itself is performed by the measurement system (Section 3.4).

Name collision (guarded at compile time): the sub-axis is called `page` and is not `page_type`. `axis_01_page_type` is the tree-node `PageKind` of the organ world and an entirely different axis; the `static_assert` in `target_isa_sub_axes.hpp` records this. The building-block catalogue of the page type is in the conserved offering catalogue “Build-PG”.

Building blocks

Variant	Description	Source
Amd64Isa	x86-64 / Intel 64 ISA (AMD/Intel server+desktop). Dominant server platform with SSE2 SIMD baseline (4 uint32 lanes per 128-bit register). Implements SSE2 vectorization via <code>_mm_loadu_si128</code> + <code>_mm_add_epi32</code> on native x86-64 hosts; scalar fallback on non-x86 build hosts. ZIH presence [Zen26a]: Barnard (Intel Xeon Platinum 8470, Sapphire Rapids), Capella (AMD EPYC 9334), Alpha Centauri (AMD EPYC 7352), Romeo (AMD EPYC 7702).	Vendor spec (AMD 2000 / Intel ongoing)
Aarch64Isa	ARMv8-A 64-bit ISA (ARM Holdings). Growing server relevance (Apple M-Series, AWS Graviton, Ampere Altra). SIMD baseline: NEON (128-bit, 4 uint32 lanes). Cache-engine implementation: 4-way unrolled scalar fallback (models NEON width without ARM intrinsics on x86 build hosts); native NEON vectorization on ARMv8 hosts. ZIH presence: Grace Hopper Superchip test system (72 Arm Neoverse V2 cores; test system with limited availability [Zen26b]).	Vendor spec (ARM Ltd., since 2011/2013)
PowerPcIsa	Power ISA (POWER9/10, ppc64le little-endian). IBM Power Systems (AC922, IC922, S1022). SIMD baseline: VSX (Vector-Scalar Extension, 128-bit, 4 uint32 lanes). Cache-engine implementation: 4-way unrolled scalar fallback (models VSX width without Power intrinsics on x86 build hosts). Relevant for HPC workloads with NVLink GPU connectivity (AC922). ZIH presence: carried in the ZIH compendium only as a decommissioned archive system (Power9), not among the current HPC resources.	Vendor/standards spec (IBM/OpenPOWER, v3.0 2017, v3.1 2020)

Variant	Description	Source
RiscVIsa	RISC-V RV64GC open-source ISA (RV64 General + Compressed). Growing relevance (SiFive, Alibaba Xuantie, Andes, ETH Zurich cores). SIMD baseline: <i>none</i> (RVV is optional, not part of the RV64 GC baseline). Cache-engine implementation: deliberately plain scalar path (not unrolled) – models the missing vector unit of the base ISA and thus produces a real, slower runtime versus x86-64 and ARM. ZIH presence: not listed in the ZIH compendium.	Standards spec (RISC-V Intl.; UCB TR since 2011)

Registry state: the build-driving system registry carries two enabled building blocks — `x86_64` (native, without triple and march flags) and `aarch64` (cross compilation with `triple=-target aarch64-linux-gnu` and `march=-march=armv8-a`). The four-ISA catalogue listed above is retained as an *offering* catalogue: Power-ISA and RISC-V are described as building blocks but are not enabled in the build offering.

Contributing sources: —

D.21 System Sphere: Operating System (`operating_system`)

This quantity is the second system main axis and thus part of the *habitat* and of no organ (cf. Section 3.4.1); in the canonical, code-guarded order it occupies position two of three.

The main axis carries the *class* identity only: exactly three operating-system families (`baustein_count=3`), together with `stage=ct` and `binary_id=never`. The *instance* — distribution, release, kernel and patch level — resides entirely in the three sub-axes; entering a distribution as a fourth expression of the main axis would miss that cut. The existing machines (Windows 11 and Windows Server 2022, the Linux Docker matrix, two macOS machines) fall entirely into the three families.

Dynamic sub-axes

ID	Name	Description
<code>os_version</code>	Distribution/release identity	The release identity of the operating-system instance within its family: on Linux <code>ID</code> and <code>VERSION_ID</code> from <code>/etc/os-release</code> , on Windows the product line, on macOS the product version.
<code>kernel</code>	Kernel release	The kernel release of the instance: on Linux and macOS the <code>uname</code> release, on Windows the NT kernel triple.
<code>build</code>	Build and patch level	The build and patch level of the instance <i>including</i> the update state (design decision of this thesis: the update state is a constituent of the build identifier). A separate <code>update_zustand</code> is therefore explicitly <i>not</i> a fourth sub-axis but a constituent of this one.

Note: all three are `stage=runtime` with `value_type=token` and `option_source=machine_resolved`: they have no compile-time option catalogue because their admissible values depend on the concrete machine and are not knowable at build time. The run-time acquisition is implemented in probe files of their own (`operating_system_probe_{linux,macos,windows}.hpp`); the attachment to the registry declares only the options. The three names are part of the generated registry and thus fixed.

Building blocks

Variant	Description	Source
linux	Linux family (<code>LinuxOperatingSystem, posix=true</code>). Covers the entire Docker distribution matrix; the individual distribution is a matter for the sub-axis <code>os_version</code> , not for this expression.	Code/Baseline
windows	Windows family (<code>WindowsOperatingSystem, posix=false</code>). Covers desktop and server product lines (Windows 11, Windows Server 2022).	Code/Baseline
macos	macOS family (<code>MacosOperatingSystem, posix=true</code>). Covers the existing Apple Silicon and Intel machines.	Code/Baseline

Contributing sources: — (CE baseline, own building blocks).

Stamp neutrality: the three sub-axes are run-time sub-axes and therefore never appear in the binary stamp. Their attachment changes neither the stamp lines nor the build-version suffix, and it must not raise the axis code level of `operating_system` (check against a version jump): such a jump would shift the SHA512 of all new builds against the existing stock and would force exactly the full rebuild of all legacy binaries that the design decision of this thesis rules out.

D.22 System Sphere: Extension Hardware (`external_utils`)

This quantity is the third system main axis; it is exclusively the collecting hub of the hardware meta-meta axes — SIMD and vector extensions, GPU, FPGA and NPU as well as external accelerators (cf. Section 3.4.1).

The hub carries *only* system meta-metas; the load framework explicitly does not belong to it. Because the meta-meta axes receive no stamp line of their own but appear as a bracket appendage at the end of the respective realm line, this hub closes the system realm: the appendage is folded type-driven from the member type list of the hub (`ExternalUtilsHub` → `SimdExternalUtilsFamily`), the level resides in the bracket depth and not in the name. In the build-driving offering the axis carries `baustein_count=1, stage=ct` and `binary_id=never`.

Dynamic sub-axes

ID	Name	Description
simd	Vector extension level	<code>stage=runtime, value_type=token, option_count=3</code> . The family identifier is at the same time the sub-axis label (notation <code>external_utils.simd=...</code>); the per-dialect reflected compiler flags come from the option, not from the family.

The three options of the sub-axis with their flags per compiler dialect:

Option	Flag (gpp and clang)	Flag (msvc)
<code>no_extension</code>	— (empty)	— (empty)
<code>avx2</code>	<code>-mavx2</code>	<code>/arch:AVX2</code>
<code>avx512</code>	<code>-mavx512f</code>	<code>/arch:AVX512</code>

Note: `simd` materializes as a flag of the compilation seam and therefore appears in the build-version suffix (cf. Section 3.4.4) — never as a run-time type switch and never in the `binary_id`.

The sub-axis is accompanied by a fine-grained feature catalogue `simd_feature_catalog` with `count=23` features, tiered by `tier`: fourteen features of tier `avx512`, four of tier `avx256`, three companion features (`companion`) and two scalar ones (`scalar`). The flags are identical throughout for `gpp` and `clang`.

Feature (cpuinfo)	Flag (gpp and clang)	Tier (tier)
<code>avx512f</code>	<code>-mavx512f</code>	<code>avx512</code>
<code>avx512cd</code>	<code>-mavx512cd</code>	<code>avx512</code>
<code>avx512vl</code>	<code>-mavx512vl</code>	<code>avx512</code>
<code>avx512dq</code>	<code>-mavx512dq</code>	<code>avx512</code>
<code>avx512bw</code>	<code>-mavx512bw</code>	<code>avx512</code>
<code>avx512ifma</code>	<code>-mavx512ifma</code>	<code>avx512</code>
<code>avx512vbmi</code>	<code>-mavx512vbmi</code>	<code>avx512</code>
<code>avx512_vbmi2</code>	<code>-mavx512vbmi2</code>	<code>avx512</code>
<code>avx512_vnni</code>	<code>-mavx512vnni</code>	<code>avx512</code>
<code>avx512_bitalg</code>	<code>-mavx512bitalg</code>	<code>avx512</code>
<code>avx512_vpopcntdq</code>	<code>-mavx512vpopcntdq</code>	<code>avx512</code>
<code>avx512_vp2intersect</code>	<code>-mavx512vp2intersect</code>	<code>avx512</code>
<code>avx512_bf16</code>	<code>-mavx512bf16</code>	<code>avx512</code>
<code>avx512_fp16</code>	<code>-mavx512fp16</code>	<code>avx512</code>
<code>avx2</code>	<code>-mavx2</code>	<code>avx256</code>
<code>fma</code>	<code>-mfma</code>	<code>avx256</code>
<code>f16c</code>	<code>-mf16c</code>	<code>avx256</code>
<code>avx_vnni</code>	<code>-mavxvnni</code>	<code>avx256</code>
<code>gfni</code>	<code>-mgfni</code>	<code>companion</code>
<code>vaes</code>	<code>-mvaes</code>	<code>companion</code>
<code>vpclmulqdq</code>	<code>-mvpclmulqdq</code>	<code>companion</code>
<code>bmi2</code>	<code>-mbmi2</code>	<code>scalar</code>
<code>popcnt</code>	<code>-mpopcnt</code>	<code>scalar</code>

Machine signatures (`count=3`): the registry records for each target machine the subset of the features actually present — `prod1_zen5` (`host_isa=x86_64`) with 22 features, `prod2_alder_lake` with 9 and `odroid_gracemont` with 9. Only the Zen 5 machine has the `avx512` tier; the other two end at `avx256` together with the companion and scalar features.

Building blocks

Variant	Description	Source
<code>simd</code>	Vector extension family (<code>SimdExternalUtilsFamily</code> , <code>family_id=simd</code> , enabled). The only enabled member of the hub; the families for GPU, FPGA and NPU are foreseen in the design but are not part of the build-driving offering.	Code/Baseline

Contributing sources: — (CE baseline, own building blocks).

D.23 System Sphere: Outer Complex Bracket (`build_target_complex`) and Build Toolchain (`build_toolchain`)

This quantity is the command-bracket wrapper over the fixed recombination of the three system main axes (`member_count=3`: `target_isa`, `operating_system`, `external_utils`) and thus closes the system realm block (cf. Section 3.4.2).

As a bracket it is explicitly *not* a fourth main axis: the canonical system axis order stays at three. Three static exclusion checks (`static_assert`) record that `compiler`, `scheduling` and `load_framework` are no system main axes. The load framework is not a *member* of this bracket either: it belongs to the measurement realm, whereas the system meta-metas are carried under the hub `external_utils`. The axis carries `stage=ct` and `binary_id=never`.

Static sub-axes

The sub-axis group `build_toolchain` (`sub_axis_count=3`) hangs on the bracket. The registry names the three members flat; the nesting underneath shows the actual parent chain.

Member	Parent reference	Description
<code>compiler</code>	<code>build_target_complex</code>	<code>stage=ct</code> , <code>baustein_count=2</code> . The compiler choice itself; it is a member of the sub-axis group <code>build_toolchain</code> of the outer bracket.
<code>opt_level</code>	<code>compiler</code>	<code>stage=runtime</code> , <code>value_type=token</code> , <code>option_count=5</code> . Optimization level per compiler dialect.
<code>atomic128</code>	<code>compiler</code>	<code>stage=runtime</code> , <code>value_type=token</code> , <code>option_count=2</code> . Release of the 128-bit compare-and-swap instruction.

Note: the group hangs on `build_target_complex` and not on `target_isa` because it describes the *build* arising from the recombination of all three main axes and not a property of the target ISA alone.

Building blocks

Variant	Description	Source
<code>gcc</code>	GNU compiler (<code>GccCompilerAxis</code> , driver <code>g++-16</code> , <code>supports_fno_gnu_unique=true</code>).	Code/Baseline
<code>clang</code>	LLVM compiler (<code>ClangCompilerAxis</code> , driver <code>clang++-22</code> , <code>supports_fno_gnu_unique=false</code>).	Code/Baseline

Offering and build minimum: the driver identifiers `g++-16` and `clang++-22` are the default drivers of the offering (`driver_default` of the compiler building blocks); the build itself requires `g++ ≥ 15.3` as the minimum and standard of the measurement machines and deliberately pins no version (newer `g++` admitted); the CI test cells prefer `g++-15` where available, and the clang full build runs on clang 22. Clang and MSVC have no version minimum.

The five options of the sub-axis `opt_level` with their flags per dialect; the column `ieee754_deterministic` states whether the level leaves the floating-point semantics untouched:

Option	Flag (gpp and clang)	Flag (msvc)	ieee754_deterministic
O0	-O0	/Od	true
O1	-O1	/O1	true
O2	-O2	/O2	true
O3	-O3	/O2	true
Ofast	-Ofast	/O2	false

The default of the build is O2; O3 is selectable via the XML, and the choice enters the build-version suffix as a field.

The two options of the sub-axis `atomic128`:

Option	Flag (gpp and clang)	Flag (msvc)
no_cx16	— (empty)	— (empty)
cx16	-mcx16	— (empty)

Build version identifier: `opt_level` and `simd` materialize as flags of the compilation seam and appear as segments (`+opt=`, `+ext=`) in the build-version suffix of the binary — an ordered eight-segment sequence in which an empty member simply produces no segment (cf. Section 3.4.4); `atomic128` likewise materializes as a flag of the compilation seam, but stands not in the suffix but as a member-owned field of the toolchain member in the fingerprint preimage (beside `vendoropt`, ADR-17 in Appendix E). They never appear in the `binary_id`.

Version flag grammar: the version of every axis algorithm is uniformly three-part; a `v` prefix is excluded, short forms are forbidden. Compile-time checks enforce the form:

```

version := UINT '.' UINT '.' UINT [ '.' flag ]*
flag    := basis [ '{' sub [ '.' sub ]* '}' ] | companion
basis   := 'c' | 'g' | 'f' | 'n' | 'x128' | 'x256' | 'x512'
          | 'm64'
sub     := token [ '{' sub [ '.' sub ]* '}' ] (recursive)

```

A dot precedes every flag, including the first (`1.0.0.c`, never `1.0.0c`); the basis attaches directly to its opening brace, with no dot of its own in between (`c{p.e}`, never `c.{p.e}`). The suffix `e` denotes an *efficiency core* and is a sub-flag like any other under the basis `c`; the basis `c` without a brace is semantically `c{p}`; a sub-flag for experimentally introduced candidate algorithms does not exist. The bases `c`, `g`, `f` and `n` name the target hardware class (CPU, GPU, FPGA and NPU, of which only `c` is produced and the other three are reserved), `x128`, `x256` and `x512` the three views onto the same vector register file (XMM, YMM, ZMM), and `m64` the media basis of the MMX/3DNow! family on the x87-aliased MM registers, whose sub-flags the catalogue check passes only under this basis (`1.0.0.c.m64{mmx.mmxext.3dnow}`, never as bare tokens without a basis). Example form: `memory_layout=SoaMemoryLayout@1.0.0.c{p.e}.x512{f.v1.bw.dq}.gfni`.

D.24 Measurement Sphere: Measurement Categories

(measurement_category)

This quantity belongs to the third realm — the measurement axes, the equipment of the *laboratory* (cf. Section 3.4.1).

It carries `stage=ct`, `binary_id=never` and `baustein_count=16`. Its values manifest as columns of the result tables and as the setting label of a run, never in the binary identity. In the chapter model the sixteen categories are the *sub-axis* of the measurement tooling; the measurement registry additionally carries them as an offering block of their own, because the tooling does not invent its categories but selects them from this catalogue. The column `regime_ordinal` separates the time- and observer-based regime (0) from the counter-based PMC regime (1). The sixteen categories are the offering catalogue of the category axis; the number of measurement sources acquired per axis interface and parameter set is different from that and not a subject of this catalogue (Section 3.4).

Building blocks

Category	category_ordinal	Regime (regime_ordinal)
CLU	0	0 — time/observer
CACHE_MISS_L1	1	1 — PMC
CACHE_MISS_L2	2	1 — PMC
CACHE_MISS_L3	3	1 — PMC
DTLB_MISS	4	1 — PMC
MEMORY_FOOTPRINT	5	0 — time/observer
BRANCH_MISS	6	1 — PMC
IPC_CPI	7	1 — PMC
LATENCY_MEAN	8	0 — time/observer
LATENCY_P50	9	0 — time/observer
LATENCY_P95	10	0 — time/observer
LATENCY_P99	11	0 — time/observer
LATENCY_P999	12	0 — time/observer
THROUGHPUT	13	0 — time/observer
ENERGY_J	14	1 — PMC
FILL_BUFFER_OCCUPANCY	15	0 — time/observer

Regime split: nine categories lie in the time and observer regime (CLU, MEMORY_FOOTPRINT, the five latency categories, THROUGHPUT, FILL_BUFFER_OCCUPANCY), seven in the PMC regime (the three cache-miss levels, DTLB_MISS, BRANCH_MISS, IPC_CPI, ENERGY_J). The split coincides with the “nine/seven” division of Section 3.4.1.

Contributing sources: — (CE baseline, own building blocks).

D.25 Measurement Sphere: Measurement Tooling/Collector

(collector)

This quantity is the instrument axis of the measurement realm: it records with which tool a category is acquired (cf. Section 3.4.1).

It carries `stage=ct`, `binary_id=never` and `baustein_count=3`. Every building block carries its regime and the set of categories it serves; the union of the three sets covers ten of the sixteen categories. Of the remaining six, the four latency-percentile categories (`LATENCY_P50` through `LATENCY_P999`) are served not by an acquisition tool but by the downstream evaluation (nearest-rank over the raw values, complemented by the HDR histogram, Section 4.5); `MEMORY_FOOTPRINT` and `FILL_BUFFER_OCCUPANCY` are described in the offering catalogue but assigned to no acquisition tool.

Building blocks

Variant	Regime	Categories served
<code>WallClockSystemAxis</code>	0	<code>LATENCY_MEAN</code> , <code>THROUGHPUT</code>
<code>ObserverSnapshot</code>	0	<code>CLU</code>
<code>SystemAxis</code>		
<code>PmcSystemAxis</code>	1	<code>CACHE_MISS_L1</code> , <code>CACHE_MISS_L2</code> , <code>CACHE_MISS_L3</code> , <code>DTLB_MISS</code> , <code>BRANCH_MISS</code> , <code>IPC_CPI</code> , <code>ENERGY_J</code>

Contributing sources: — (CE baseline, own building blocks).

D.26 Measurement Sphere: Telemetry Building Blocks (Conserved Catalogue)

This quantity is a *two-way split* main quantity — sweep sub-axis of the measurement tooling in the planner *and* compile-time main quantity in the builder (cf. Section 3.4.1). Its building-block catalogue is retained here as a conserved catalogue.

Defines strategies for measuring index operations (insert, lookup, erase) with different overhead profiles. Supports density tracking for adaptive node-type selection, global insert counters for throughput measurements, HDR histograms for latency quantiles, and leaf-only counters to avoid cache-line false sharing in the measurement cycles.

Static sub-axes

ID	Name	Description
TM1	Scope	Validity scope of the counter management: leaf-only (leaf nodes only), per-node (all nodes), or global (single accumulator for the entire structure). Compile-time choice, impacts overhead through contention on shared cache lines.
TM2	Metric Type	Type of the measured metric: counter (event counter), histogram (distribution with quantiles), or density (slot occupancy per node). Impacts the memory footprint and the measurement granularity.
TM3	Overhead Level	Magnitude of the measurement overhead: low (atomic operations), medium (per-node data structures), high (histogram logging). Determines the suitability for different workload scenarios (benchmarking vs. production).

Building blocks

Variant	Description	Source
LeafOnly Counter	Counter <i>only</i> in leaf nodes (avoids cache-line ping-pong by dispensing with updates of inner nodes). Low overhead with leaf-only scope. Corresponds to the coherence-friendly counter concept from Kuehn DaMoN 2023, although the original paper is not a leaf-counter algorithm — CE baseline for false-sharing avoidance.	P28 (Kuehn DaMoN 2023, context; Code/Baseline)
DensityTracker	Density/slot-occupancy tracking per node for adaptive node-type selection (4→16→48→256). Contextual telemetry tied to the ART structure (Leis ICDE 2013), measures the slot occupancy of all nodes. Higher overhead than LeafOnly, but provides the adaptation signal for dynamic rebalancing decisions.	P01 (ART/Leis ICDE 2013, context; Code/Baseline)
InsertCounter	Global atomic insert counter with minimal overhead (1 atomic increment per insert). Contextual telemetry tied to the HOT structure (Binna SIGMOD 2018), global scope. Sufficient for throughput measurement series, no latency granularity.	P02 (HOT/Binna SIGMOD 2018, context; Code/Baseline)
Latency Histogram	HDR histogram for the lookup latency distribution (p50/p95/p99 quantiles). Based on HdrHistogram (Gil Tene), CC0-1.0 licence. Highest overhead (per lookup), but provides complete latency statistics for SLA-oriented measurement series and measurement series with scientific ambition. Only is_original candidate of the axis.	HdrHistogram (github.com/HdrHistogram/HdrHistogram_c, CC0-1.0, is_original)
Probability-Hints	Probability hints of the cache-oblivious Bender line (P16/P17): distribution- and probability-related occupancy hints that serve as input to the layout decisions. Paper building block of the axis dissection (reconstruction target of this thesis, no CE baseline).	P16/P17 (Bender 2000/2005) [BDFC00, BDFC05]

The CRTP organ registry of the telemetry (`AllTelemetries`) carries exactly the four strategies of the first four rows; beside it the code contains the vtable family `ITelemetryStrategy` with six kinds (per-node counter, leaf-only counter, sampled leaf-only counter, retroactive aggregation, path-read counter P26 and the probability hints of this thesis), which is no organ registry and is consumed by the telemetry engine (`c05_telemetry_engine`) and its tests. Two further telemetry targets with ZIH relation — an OTF2 trace connection to the performance analysis tool Vampir [KBD⁺08] and a connection to the centre-wide job monitor PIKA [DWKN20] — are foreseen as marker types of a variant catalogue consumed only by tests and isolated from the build, and are not part of the build-driving offering; the reconstruction of the VAMPIR memory-type design (A24) is independent of that.

Contributing sources: HdrHistogram (Gil Tene) [Ten12], P16/P17 (Bender) [BDFC00, BDFC05].

Related literature: P01, P02, P28 [LKN13, BZP⁺18, Kue23].

D.27 Measurement Sphere: Load Framework (`load_framework`) — Meta-Meta Axis

This quantity is the meta-meta main axis of the measurement realm and therefore stands at its end (cf. Section 3.4.1).

It carries `category=measurement_meta_meta`, `stage=ct`, `binary_id=never` and `baustein_count=1`. It is the meta-meta main axis of the planner in the measurement realm; it explicitly does not belong under the hub of the system meta-metas. In the stamp it receives *no* meta-meta line of its own: its identifier appears as a dynamic bracket appendage at the *end* of the existing measurement realm line. The feeding path here is entry-driven — the chosen identifier together with its

version comes from the framework registry of the measurement realm and therefore does not reside in the type, unlike the type-driven system hub.

Dynamic sub-axes

The registry carries seven sweep dimensions (`resource_control_version=1`); six of them are `stage=runtime`, the seventh (`work_mode`) is `stage=plan`.

Dimension	Origin (source)	Value type and meaning
<code>workload</code>	<code>measurement:load_framework</code>	token — the load profile; the single source is the sub-axis label of the load framework itself.
<code>thread_count</code>	<code>resource_control_pod</code>	uint — number of worker threads.
<code>prefetch_distance</code>	<code>resource_control_pod</code>	uint — prefetch distance.
<code>pool_budget_bytes</code>	<code>resource_control_pod</code>	uint — memory budget of the pool.
<code>batch_size</code>	<code>resource_control_pod</code>	uint — batch size per operation.
<code>inline_threshold_bytes</code>	<code>resource_control_pod</code>	uint — threshold of inline storage.
<code>work_mode</code>	<code>measurement:run_methodology</code>	token — the run method of a run (build, measure, compare, release), <code>stage=plan</code> ; the single source is the compile-time registry of the run methodology, no hand-written value.

Note on non-observation: the repetition count of a run is deliberately *not* in this list: it is a pure label dimension without a compile-time counterpart and becomes reflectable only after its typing; a manual entry would be an unsubstantiated value — the registry instead carries an explicitly coded non-observation. The run methods, by contrast, are typed: there are *four* — build, measurement, comparison and release execution (build, measure, compare, release) —, backed by a compile-time registry without run-time branching; the comparison execution possesses the selectability and reads the measurement store without acquiring reference values itself. They stand in the list as the dimension `work_mode`.

Building blocks

Variant	Description	Source
<code>ycsb</code>	The YCSB load-profile framework; enabled, with the sub-axis label <code>workload</code> . The only enabled building block of the axis.	Code/Baseline

Contributing sources: — (CE baseline, own building blocks).

D.28 A-Korpus Allocator Corpus (A01–A23 and A24)

The allocator axis documents 26 memory management strategies (identifiers A01–A21, the three standard-library baselines A22a–A22c, A23 and A24) that affect dynamic memory management in search workloads. It covers classic textbook allocators (Buddy, Slab), industrial standards (dlmalloc, jemalloc, TCMalloc, mimalloc, snmalloc), specialized variants (NUMA-aware, PIM-Malloc, cache-aware), modern hardened implementations (StarMalloc) and a memory-type virtualization following the VAMPIR design (A24). Here A01–A23 form the literature corpus of the 23 works of Section 3.2 with one allocator profile each

(Table 3.3), whereas A24 is the allocator front reconstructed from the SOTA profile P33 without an allocator profile of its own. All 26 wrappers satisfy the `AllocatorStrategy` concept and enable systematic measurement of the influence of memory management on search-tree performance (F15 operativity). The enabled inventory of this build configuration comprises three wrappers (see the registry-state note in section T6).

Static sub-axes

The sub-axes AA1–AA7 are defined in section T6; the following table assigns the corpus examples to them.

ID	Name	Description
AA1	Free-List Topology	Structure of the free memory blocks: per-page sharding (mimalloc), per-thread caching (snmalloc, rpmalloc, TCMalloc), buddy tree (Buddy allocator), single global free-list (dlmalloc), per-heap locks (Hoard).
AA2	Size-Class Schema	Classification of block sizes: slab-per-object-size (Slab), dlmalloc bins, jemalloc size-classes, scallo spans, Buddy power-of-2.
AA3	Thread Locality	Degree of thread isolation: tcmalloc thread-local caches, rpmalloc per-thread spanning, snmalloc message-passing, Hoard per-heap locks.
AA4	Synchronization	Concurrency mechanism: lock-free CAS (Michael, LRMalloc), per-heap locks (Hoard), wait-free (Crystalline), mutex-based.
AA5	Allocation Policy	Routing of allocation requests: NUMA-origin-aware (NUMAalloc), cache-set-aware (CAMA), NUCA-aware (TCMalloc-Warehouse), PIM-aware (PIM-Malloc), NFP memory-type virtualization (VampirNfp).
AA6	Reclamation	Release of deleted blocks: magazine cache (Slab), page-heap release (TCMalloc), lock-free reclamation (LRMalloc), deferred free (mimalloc).
AA7	Fragmentation Strategy	Avoidance of fragmentation: dlmalloc coalescing, object colouring (Slab), buddy splitting, page-local sharding (mimalloc), span pooling (scallop).

Building blocks

Variant	Description	Source
<code>HoardAllocator</code> (A01)	Per-heap free-lists with SMP scaling. Three-tier structure: thread-local heaps, global heap for cross-thread migration, false-sharing reduction via segregated allocation requests.	A01 [BMBW00] (Berger/McKinley/Blumofe/Wilson, ASPLOS 2000, 10.1145/378993.379232) + Code/Baseline
<code>SlabAllocator</code> (A02)	Object-caching kernel allocator. Slab-per-object-size with per-CPU magazine cache (Bonwick 1994/2001). Classic Solaris kernel implementation for pre-allocated object pools.	A02 [Bon94] (Bonwick, USENIX 1994) + Code/Baseline
<code>MichaelLockFreeAllocator</code> (A03)	Lock-free CAS allocator per Michael 2004. Hazard-pointer-protected lock-free data structures, LGPL re-implementation with IBM patent background.	A03 [Mic04b] (Michael, PLDI 2004, 10.1145/996841.996848) + Code/Baseline
<code>MimallocAllocator</code> (A04)	Free-list sharding (3 free-lists per page). Microsoft Research 2019: deferred_free hooks, temporal cadence, page-local sharding. is_original: MIT direct binding.	A04 [LZdM19] (Leijen/Zorn/de Moura, APLAS 2019, 10.1007/978-3-030-34175-6_13) + is_original (github.com/microsoft/mimalloc, MIT)
<code>JemallocAllocator</code> (A05)	Arena-based size-class allocator. Evans 2006 / Free BSD. Per-CPU arenas with tcache layer for thread-local caches. Facebook standard. is_original: BSD-2-Clause direct binding.	A05 [Eva06] (Evans, BSDCan 2006, papers.freebsd.org/.../jemalloc) + is_original (github.com/jemalloc/jemalloc, BSD-2-Clause)

Variant	Description	Source
TCMallocAllocator (A06)	Thread-cache + central-cache + page-heap. Google gperftools 2005. Later TEMERAIRE (OSDI 2021) with warehouse-scale optimizations.	A06 [HKT ⁺ 21] (TCMalloc 2005; TEMERAIRE, OSDI 2021) + Code/Baseline
SnmallocAllocator (A07)	Message-passing allocator. Paul Liétar et al. ISMM 2019: free operations via lock-free queue to the allocating thread, cross-thread lock-contention avoidance. <code>is_original</code> : MIT direct binding.	A07 [LDPB19] (Liétar, ISMM 2019, 10.1145/3315573.3329980) + <code>is_original</code> (github.com/microsoft/snmalloc, MIT)
ScallicAllocator (A08)	Span-based, virtual-spans technique. Aigner/Kirsch/Lippautz/Sokolova OOPSLA 2015. Multicore-scalable with low fragmentation.	A08 [AKLS15] (Aigner et al., OOPSLA 2015) + Code/Baseline
NUMAAllocator (A09)	NUMA-origin-aware allocator. UTSASRG ISMM 2023. Node hint for local NUMA allocation, multi-node systems optimized.	A09 [YZZ ⁺ 23] (Yang et al., ISMM 2023) + Code/Baseline
RPMallocAllocator (A10)	Per-thread span caching. Mattias Jansson 2017, open-source implementation without a publication. Special case: mandatory init (<code>rpmalloc_initialize</code> , <code>rpmalloc_thread_initialize</code>). <code>is_original</code> : Public-Domain/MIT direct binding.	Code/Baseline (github.com/mjansson/rpmalloc, Public-Domain/MIT) + <code>is_original</code>
LRMallocAllocator (A11)	Lock-free + hazard pointers. Leite/Rocha VECPAR 2018 (LNCS 11333). Modern lock-free allocator with ABA protection. <code>is_original</code> : MIT direct binding.	A11 [LR18] (Leite/Rocha, VECPAR 2018, 10.1007/978-3-030-15996-2_17) + <code>is_original</code> (github.com/ricleite/lrmalloc, MIT)
CAMAAAllocator (A12)	Cache-aware memory allocator. Herter/Backes/Haupenthal/Reineke ECRTS 2011 (Saarland). Predictable cache behaviour, no public code reference.	A12 [HBHR11] (Herter et al., ECRTS 2011) + Code/Baseline
StarMalloc Allocator (A13)	Formally verified hardened allocator. Inria-Prosecco (Reitz/Fromherz/Protzenko et al.) OOPSLA 2024. F*/Steel formal verification, Apache-2.0 licence.	A13 [RFP24] (Reitz et al., OOPSLA 2024) + Code/Baseline
TCMallocWarehouse Allocator (A14)	TCMalloc-Warehouse / Hyperscale. Google Cloud OSDI 2021 / ASPLOS 2024. Hugepage-aware variant, product variant without a separate artefact.	A14 [HKT ⁺ 21, ZGV ⁺ 24] (Hunter OSDI 2021 / Zhou ASPLOS 2024) + Code/Baseline
HMallocAllocator (A15)	Hybrid, scalable, lock-free memory allocator. Li/Yao/Tang/Lin ICPADS 2019. Hybrid approach for scalability, no public code reference.	A15 [LYTL19] (Li/Yao/Tang/Lin, ICPADS 2019, 10.1109/ICPADS47876.2019.00064) + Code/Baseline
PIMMalloc Allocator (A16)	Processing-in-memory allocator. VIA-Research HPCA 2026 (arXiv:2505.13002). Memory distribution across compute nodes, PIM-hardware-optimized.	A16 [VIA26] (VIA-Research, HPCA 2026, arxiv.org/abs/2505.13002) + Code/Baseline
Crystalline Reclamation (A17)	Wait-free or lock-/wait-free safe-memory-reclamation family for non-blocking data structures. Not a standalone malloc/free allocator, but a reclamation policy.	A17 [NR21, NR24] (Nikolaev/Ravindran, DISC 2021 / PLDI 2024) + Code/Baseline
ExgenAllocator (A18)	Single-threaded specialized allocator. UT Austin IEEE CAL 2025 (arXiv:2510.10219). “Old is Gold” optimizations for single-thread workloads, no public code reference.	A18 [LJY25] (UT Austin, IEEE CAL 2025, arxiv.org/abs/2510.10219) + Code/Baseline
BuddyAllocator (A19)	Buddy system (power-of-2 splitting). Knuth TAoCP Vol.1 (1968), CACM 1965. Classic textbook allocator, binary-tree splitting without OSS code reference.	A19 [Kno65] (Knowlton, CACM 1965, 10.1145/365628.365655) + Code/Baseline
DlmallocAllocator (A20)	Doug Lea malloc (bins-based). Lea 1987–2012, public domain (CC0). Classic since 1987, bins size-class schema, oldest production implementation. <code>is_original</code> : public-domain direct binding.	A20 [Lea96] (Lea, gee.cs.oswego.edu/.../malloc.html, 1996) + <code>is_original</code> (github.com/ennorehling/dlmalloc, Public-Domain/CC0)
PtMalloc2 Allocator (A21)	<code>ptmalloc2</code> (glibc malloc). Wolfram Gloger 1990s-2000s. LGPL-2.1+ copyleft, linking blocked (no <code>is_original</code>). Standard basis for glibc-based systems.	Code/Baseline (malloc.de/en/) + License restriction

Variant	Description	Source
StdMalloc (A22a)	Standard libc malloc (proof of concept), portable fallback allocator. Baseline for all standard-library implementations without vendor linking.	Code/Baseline (C++ Standard Library) + Baseline
PmrResource Allocator (A22b)	std::pmr::memory_resource. WG21 N3916 (2014 → C++17). Polymorphic Memory Resources standard library, fallback to the new/delete allocator.	Code/Baseline (C++17 Standard Library, open-std.org/.../n3916.pdf) + Baseline
PoolResource Allocator (A22c)	std::pmr::unsynchronized_pool_resource. WG21 N3916 (C++17). Own size-class pool, F15-operative (first wrapper with its own memory-retention behaviour without vendor linking).	Code/Baseline (C++17 Standard Library, open-std.org/.../n3916.pdf) + is_operative
VmemMagazines Allocator (A23)	Vmem + magazines (slab extension). Bonwick 2001 USENIX ATC. Per-CPU magazines for scaling, kernel classic without public code reference.	A23 [BA01] (Bonwick, USENIX ATC 2001, usenix.org/.../bonwick.html) + Code/Baseline
VampirNfp Allocator (A24)	Memory-type virtualization following the VAMPIR design (Virtualized Non-Functional Memory Properties for Data-Pipeline Scheduling): allocator front over an NFP-labelled memory tier (target DRAM) with an exposed NFP descriptor (bandwidth_class, latency_class, capacity); structural reconstruction of the buildable part, no cost formula, no scheduling, no energy metric; disabled by default.	A24 [BSB ⁺ 23] (VAMPIR poster, SOSp 2023; TU Dresden) + Code/Baseline (is_original=false)

Contributing sources: cited per allocator A01–A24 individually in the source column of the building-block table.

D.29 Axis Inventory from the Generated Registries

The following table is not maintained by hand: the appendix generator (Code/08_appendix_generator) derives it from the three generated axis registries of the code — organ, system and measurement realm — and writes it, provided the run is invoked with at least one of the three registry options (`--organ-registry`, `--system-registry`, `--measurement-registry`), per language to `anhang/<lang>/tabellen/`. It is therefore a derivation from the compile-time reflection and not a second, hand-kept state beside the registries. If the registries contain no axis, no file is produced on purpose; an empty table is not synthesized.

(The axis inventory is not available for this state; a generator run with registry options produces it.)

D.30 Conserved Designs

The three following offering catalogues are conserved designs: living code with registries of its own that stands outside the build-driving offering of the three axis registries — the SIMD catalogue Build-SIMD under the extension hardware, the page-type catalogue Build-PG outside the eighteen composition slots and the platform hardware strategy Build-HW, which the system registry explicitly excludes. They remain listed here in full — with purpose, sub-axes, building blocks and sources — and are to be distinguished from the offering of the registries, which the preceding sections document (cf. Section 3.4.1).

D.30.1 Build-SIMD: Vector/Accelerator Extension Axis (Hardware Accelerators)

Conserved offering catalogue outside the build-driving offering: the axis `axis_09b_simd_extension` is an offering catalogue under the main axis `external_utils`, whose sub-axis `simd` carries the build-driving choice; its eight building blocks and the sub-axes SE1–SE3 remain listed in full. The axis `axis_09b_simd_extension` models the available SIMD/accelerator extensions of the CPU instruction set architecture as a sub-axis to `axis_09` (main ISA). It enables compile-time selection among various vector extensions (SSE2/AVX2/AVX-512 for x86; NEON/SVE2 for ARM; RVV for RISC-V) plus baseline (scalar) and GPU acceleration (CUDA), with backward-cumulative layer hierarchy and hardware-specific topology (SIMD units per socket, P/E core access).

Static sub-axes

ID	Name	Description
SE1	Vector Width	Compile-time-known maximum vector width of the SIMD extension in bits: 0 (none), 128, 256, 512, or -1 (scalable). Determines lane width and parallelization degree.
SE2	Accelerator Type	Categorizes the accelerator class: none (baseline), simd (vector SIMD), matrix (matrix engine), gpu (GPU), npu (neural processing unit). Influences the composition with the hardware topology (<code>axis_12</code>).
SE3	Compatibility Family	Constraint to the main ISA (<code>axis_09</code>): none (universal), x86, arm, riscv, cuda. Ensures compile-time validation: e.g., <code>Sse2SimdExtension</code> compatible only with <code>Amd64Isa</code> .

Building blocks

Variant	Description	Source
<code>NoSimdExtension</code>	Baseline variant without SIMD acceleration. Scalar fallback strategy for all permutations without accelerator. Universally compatible with all main ISAs (x86/ARM/RISC-V/PowerPC). Mandatory comparison point.	Code/Baseline
<code>Sse2SimdExtension</code>	Intel SSE2 128-bit vector extension (x86_64 ABI mandatory since 2003). Compatible only with <code>Amd64Isa</code> . Baseline on x86_64; provides 4× uint32 lanes/iteration. Standard on the cluster nodes Barnard/Capella.	Intel ISA extensions reference 319433 [Int26]
<code>Avx2SimdExtension</code>	Intel AVX2 256-bit vector extension (Haswell+ 2013). Only x86_64. Backward-cumulative: includes all SSE+AVX. Default optimization level for server DBMS. 2 AVX2 units/socket. ZIH standard on Barnard (Intel Sapphire Rapids) and Capella (AMD EPYC 9334).	Intel ISA extensions reference 319433 [Int26]
<code>Avx512SimdExtension</code>	Intel AVX-512 512-bit vector extension (Skylake-X+, Ice Lake Server, Zen 4+). Backward-cumulative: all SSE/AVX/AVX2 + 15 separate flags (VNNI for vector indexes, VPOPCNTDQ for bitmap indexes). Not on all x86_64 CPUs (for instance Haswell without AVX-512; on Intel's hybrid Alder Lake, Raptor Lake and Meteor Lake processors AVX-512 is switched off in the BIOS because of the E cores — so also on prod2, whose signature contains no avx512 feature). 1 AVX-512 unit/socket; E cores no access (Intel Hybrid).	Intel ISA extensions reference 319433 [Int26]

Variant	Description	Source
NeonSimdExtension	ARM NEON / Advanced SIMD 128-bit (AArch64 ABI mandatory ARMv8+). Compatible only with AArch64Isa. Baseline on all AArch64 CPUs (Apple M-Series, AWS Graviton 2/3/4, NVIDIA Grace). 1 NEON unit/socket; all cores accessible (big.LITTLE architecture).	ARM ARM DDI 0487 / Cortex-A8 TRM
Sve2SimdExtension	ARM SVE2 (Scalable Vector Extension 2) ARMv9 mandatory (scalable 128–2048 bit). Compatible only with AArch64Isa. ZIH test system NVIDIA Grace Hopper Superchip (Grace: 72 Arm Neoverse V2 cores with SVE2; test system with limited availability [Zen26b]). Scalable lane width; 1 unit/socket; all cores.	SVE [SBB ⁺ 17] (IEEE Micro 2017); SVE2/ Armv9 per ARM ARM [Arm24]
RvvSimdExtension	RISC-V Vector Extension (RVV v1.0, scalable VLEN 128–65536 bit). Compatible only with RiscVIsa. Optional (not baseline like SSE2/NEON). Growing relevance: SiFive Performance P870 (128-bit), T-Head Xuantie C908 (128-bit). ZIH presence: not listed in the ZIH compendium.	RVV-v1.0-Spec [RIS21]; Vitruvius+ (TACO 2023) [MPU ⁺ 23]
CudaGh200SimdExtension	NVIDIA CUDA on Hopper GH200 (Grace Hopper Superchip GPU accelerator). Hopper GPUs at the ZIH are located in Capella (4 × H100-SXM5 per node on AMD EPYC 9334 hosts [Zen26a]) and in the Grace Hopper Superchip test system (Hopper GPU with 96 GB HBM3, NVLink-C2C 900 GB/s [Zen26b]). Compatible with AArch64Isa (Grace CPU) and Amd64Isa (Hopper host). Massively parallel: 16 896 FP32 CUDA cores per H100-SXM5 GPU [APK ⁺ 22]. <code>units_per_socket()</code> = -1 (GPU is a separate device).	NVIDIA Tesla [LNOM08] (IEEE Micro 2008); NVIDIA Hopper Architecture In-Depth [APK ⁺ 22]

Sources: ISA specifications and whitepapers (Intel, ARM, IEEE Micro, ACM TACO, NVIDIA); individual references in the source column of the building blocks.

D.30.2 Build-PG Page Type Axis (PAGE-TYPE)

Delimitation and conservation: this section describes the `PageKind` of the organ world (`axis_01_page_type`) and is distinct from the system sub-axis `page` of the target ISA (page-size release 4k/2m/1g) — both quantities carry similar names but share neither building blocks nor semantics. The catalogue is a conserved offering catalogue outside the eighteen composition slots and remains listed in full.

The page type axis defines the structural and indexing strategies for branch pages in the trie index: from dense direct-addressed byte indexing (DenseByte), through multi-level density variants (Extended Dense, SparsePatricia), to path-collapse optimizations (Redirect, CustomCache) and B+-style sorted pages (BPlus). All page types are cache-line aware and implement the `PageTypeStrategy` concept interface with guaranteed interchangeability (F15 permutation).

Static sub-axes

ID	Name	Description
PG1	Structure Role (Branching)	Classification of the branch page architecture: branch page with full branching, collapsed single-path node (Redirect), or special cache-optimized variant.
PG2	Density Class (Density)	Degree of slot utilization and the resulting indexing strategy: densely populated (0-256 slots), extended (>256 slots), or sparse with Patricia compression.

ID	Name	Description
PG3	Path Collapse Strategy	Handling of unique remainder paths: no compression (standard), or collapse via Redirect/CoCo technique for space minimization.

Building blocks

Variant	Description	Source
DenseBytePageType	Dense byte-indexed branch page with direct 256-slot addressing (direct-addressed). Page type 1: direct mapping from key byte to slot index without redirection. Source: Adaptive Radix Tree (ART), Leis et al. 2013.	P01
ExtendedDensePageType	Extended dense page for >256 entries with multi-level indexing scheme (e.g., 2-level lookup or variable k-byte span). Page type 2: density class for more complex slot arrangements. Source: Adaptive Radix Tree extensions, PRT-ART.	P01 (extended)
SparsePatriciaPageType	Sparsely populated page with Patricia path compression (single-bit split). Page type 3: density class with bit-vector traversal for low branchingness. Source: HOT (Height Optimized Trie, Binna et al. 2018, based on the Patricia trie, Morrison 1968).	P02
RedirectPageType	Collapsed unique remainder path (single-path node). Page type 4: path-collapse strategy for memory optimization under monopoly-unique successor sequences. Neither classical branch nor leaf, stores only the suffix up to the next actual branching or value node. Source: CoCo-Trie (Boffa et al. 2024).	P04
CustomCachePageType	Cache-line-oriented custom page with explicit layout design for 64-byte alignment. Page type 5: structure role branch with hardware-aware geometry. Source: PRT-ART own development with TLB offset + cache-line-aligned memory layout (axis 5).	PRT-ART
BPlusPageType	B+-style sorted branch page with range-walk traversal. Page type 6: structure role branch, but with sorted key-array organization instead of byte indexing. Source: Masstree (Mao et al. 2012), B+-tree family.	P03

Contributing sources: P01, P02, P03, P04, Morrison 1968, PRT-ART [LKN13, BZP⁺18, MKM12, BFTV24, Mor68].

Related literature: P05, P06, P10, P11, P12, P13, P20 [FJKN20, SSL⁺21, ZLL⁺18, RR99, RR00, HP03, MBL25].

D.30.3 Build-HW — Platform Hardware Strategy

Status (conserved offering catalogue): this axis is excluded from the build-driving offering of the system registry. The registry records the exclusion verbatim: `extension_hardware_system_axis.hpp` is marked as deprecated, `hardware_isa_system_axis.hpp` a pure host descriptor that does not drive the build — neither belongs to the build-driving offering. The static platform profiles with their fixed constants for cache-line size, page size and SIMD bit width are conceptually superseded by the run-time hardware detection of the `CacheEngineBuilder` — a factory over ISA and operating system that acquires hardware values instead of setting them statically (cf. Section 3.4.3). The four sub-axes HW1–HW4 and the three hardware profiles remain listed in full as an offering catalogue.

Defines the platform hardware family (CPU architecture, SIMD capabilities, cache topology, memory paging) as compile-time constants for all cache-engine algorithms. The axis captures which platforms

are configurable at build time and provides static properties (cache-line size, page size, SIMD bit width, NUMA capability, HugePage capability) for algorithm-appropriate data structures.

Static sub-axes

The four sub-axes are orthogonal to one another; a platform family assigns its primary characteristic to exactly one tag, and the `CacheEngineBuilder` can group platforms by sub-axes (for instance `HW1` $\times$ `HW2` across all platforms).

ID	Name	Description
HW1	CPU Architecture Family (<code>cpu_family</code>)	x86_64 (Intel/AMD), aarch64 (ARM/Apple), riscv64 (RISC-V), generic (platform-independent).
HW2	SIMD Instruction Set Family (<code>simd_capability</code>)	SSE/AVX/AVX2/AVX-512 (x86), NEON/SVE (ARM), RVV (RISC-V), scalar (no SIMD).
HW3	NUMA and Cache Hierarchy (<code>memory_</code> <code>topology</code>)	UMA-single-socket, NUMA-multi-socket, Apple-unified-memory, HBM-on-package (Sapphire Rapids HBM, Grace Hopper), CXL-attached.
HW4	Page Configuration (<code>page_</code> <code>topology</code>)	Standard and HugePage configuration: 4K-standard (POSIX), 16K-apple-silicon, 2M-hugepage (x86), 64K-arm-server, 1G-gigantic-hugepage.

Building blocks

Variant	Description	Source
X86_64Hardware Profile	Modern x86-64 desktop/server platform (Intel/AMD). Build-time constants: <code>cache_line_size=64</code> , <code>memory_page_size=4096</code> , <code>simd_width_bits=256</code> (AVX2 conservative), <code>numa_capable=true</code> , <code>huge_page_capable=true</code> . Basis for desktop- and server-side experiments.	Code/Baseline
Aarch64Hardware Profile	ARM64/AArch64 server and mobile platforms (Ampere Altra, Apple Silicon M-series, Linux ARM). Build-time constants: <code>cache_line_size=64</code> , <code>memory_page_size=4096</code> , <code>simd_width_bits=128</code> (NEON standard), <code>numa_capable=true</code> , <code>huge_page_capable=true</code> . Enables cross-platform comparisons.	Code/Baseline
GenericHardware Profile	Conservative fallback profile for platform-independent configurations. Build-time constants: <code>cache_line_size=64</code> , <code>memory_page_size=4096</code> , <code>simd_width_bits=0</code> (scalar-only), <code>numa_capable=false</code> , <code>huge_page_capable=false</code> . Lowest common denominator when no specialized adapter matches.	Code/Baseline

Sources: platform profiles as Code/Baseline (no external publications).

E Architecture Decisions

This appendix collects the load-bearing design decisions of this thesis as compact *Architecture Decision Records*¹ (ADRs): for each decision the context in which it was made, the decision itself and its most important consequence. The goal is quick traceability — *why* the apparatus is built the way the chapters describe it; the detailed derivation and justification resides in the section of the main text referenced under *Evidence* and is not repeated here. Where design and implementation diverge, the ADR states the implementation status at exactly one place as a fourth field. The collection comprises twenty decisions: the twelve decisions of the conception (ADR-1 to ADR-12) and eight decisions of identity, interfaces, operation and build (ADR-13 to ADR-20); Table E.1 gives the overview.

Table E.1: Overview of the twenty architecture decisions: short title, evidencing section and implementation status — *implemented* (implemented and evidenced), *partial* (implemented, with a declared open item) or *design* (designed only, not implemented); the justifications are in the records, not in the table

ADR	Decision (short title)	Evidence (section)	Implementation status
ADR-1	C-stable module boundary instead of a C++ module ABI	Sect. 4.3.1	implemented
ADR-2	Compile-time permutation via type lists	Sect. 4.3	partial (activation of the eighteenth axis)
ADR-3	Three-level model as the one architecture	Sect. 3.3.2	implemented
ADR-4	Pass-through rule and enforced full assignment	Sect. 4.3	partial (pass-through building block of the node type)
ADR-5	Layer model E4–E0 with fixed contracts	Sect. 3.3.4	implemented
ADR-6	Explicitly coded non-observation instead of a phantom value	Sect. 3.4	implemented
ADR-7	Four-repository architecture with orthogonal carrier stages	Sect. 4.1	implemented
ADR-8	Family-wise adapter fallbacks	Sect. 4.3	implemented
ADR-9	Measurement gating in three layers and four work modes	Sect. 4.3.2	partial (replay run compare/release)
ADR-10	<code>std::map</code> contract with conformance gate	Sect. 3.4	partial (full <code>std::map</code> interface, container hull)
ADR-11	Design-pattern register with evidenced roles	Sect. 3.3.3	implemented
ADR-12	Dual measurement regime Talos and root Linux	Sect. 5.1	partial (Talos regime)
ADR-13	Four carrier stages and one stamp contract	Sect. 3.4.4	partial (stamp export hybrid, SOTA branch)
ADR-14	Identity cut: only organ axes form the <code>binary_id</code>	Sect. 3.4.1	implemented

continued on the next page

¹Architecture Decision Record (ADR): a short, numbered record of a design decision with context, decision and consequences — introduced by Nygard [Nyg11] as a short text file in a format in the manner of an Alexander pattern and described as a tool format by Kopp, Armbruster and Zimmermann [KAZ18].

Table E.1 — continued

ADR	Decision (short title)	Evidence (section)	Implementation status
ADR-15	Measurement-visitor seam (surface 3) instead of a sidecar	Sect. 3.3.2	implemented
ADR-16	Store as stock with compile-time keys	Sect. 3.4.5	partial (full store build-out)
ADR-17	Optimization level as the user's choice, build world identity-effective	Sect. 5.1.6	partial (O0/O1 choice, flag pass-through)
ADR-18	One official build route as a GNU front end over CMake	Appendix B	partial (framework export)
ADR-19	Four CI cells without skip	Sect. 4.4	partial (mini-pipelines per carrier stage)
ADR-20	XML as the only control	Sect. 4.3.2	partial (creation mode, headless control path, schema drift)

ADR-1 — C-stable module boundary instead of a C++ module ABI

Context: Each permutation is compiled as a standalone tier binary and loaded at run time; C++23 modules structure the translation units but, by themselves, guarantee no portable ABI stability [Xu24]. **Decision:** A uniform C module boundary exports exactly one living being per permutation — an `IAnatomyBase` instance — via seven mandatory symbols named independently of the family: the four primal symbols ABI version, magic, factory (`comdare_create_anatomy()`) and destroy, the two identity symbols family and genus, whereby the family is not maintained as a second datum but derived statically at compile time from the genus, and the stamp-line symbol. The identity of names across all families is the stability argument: the loader can determine the family via a family-independent symbol without knowing it in advance. Behind it lie two engine concepts: the `CacheEngine` as the library of the axis building blocks and the `ExecutionEngine` as the execution and measurement infrastructure with the root `IExecutionEngine`, under which `IAnatomyBase` stands; the API facade is `ICacheEngine`; the name “SearchEngine” denotes the ABI runtime view of the genus `SearchAlgorithm`, not a third link in the chain; the same-named module intended to house it separately is declared but not wired in. **Consequence:** ABI stability is established by the C boundary, not by the module system (ABI major 9); the loader checks every module on loading, fail-closed, against the mandatory symbol set and the identity derivation — a module with contradictory identity or without a stamp symbol is rejected rather than wandering into stamp, store and measurement series as a wrong identity. **Evidence:** Sections 1.4, 3.3.3 and 4.3.1; Figure 4.4.

ADR-2 — Compile-time permutation via type lists instead of runtime adaptation

Context: Eighteen organ composition axes already count $2^{17} = 131072$ binary identities per system permutation in the canonical measurement catalogue — one of them, the persistence target, pinned to its in-memory representative —, and the raw variant product space over all axis catalogues including their sub-axes lies orders of magnitude above that; the hot path must carry neither `virtual` calls nor runtime switches. The pinning is a budget decision (according to the catalogue header about 224 GB and 24 h of complete build instead of about 448 GB and 48 h); the variation of all eighteen axes is provided for, and the cardinality of a concrete plan is computed by the planner per XML in the director walk, never statically. **Decision:** The permutation runs at compile time via `Boost.Mpl`

type lists²: per axis one full list, filtered via the activation predicate into the enabled list, bound to the composition slot, plus concept-checked axis descriptors with the inventory `variants` and a generic registry-to-tree reflection; the golden reference catalogue selects per slot the first representatives of the enabled list — no runtime tag, no `std::variant`. `std::variant`, `virtual` and runtime switches are categorically excluded in tier and hybrid binaries; a static check enforces this, and the variant-based building-block cluster (`baustein_variants`, `resolve_baustein`) is a test inventory isolated from the build, without library consumers. **Consequence:** The axis abstraction remains a zero-cost abstraction; only the eighteen organ axes enter the binary identity (ADR-14). **Implementation status:** The inclusion of the eighteenth axis in the reference catalogue is not implemented; it requires two steps at once — catalogue degree two for the persistence target and the write-back switch of the persistence axis. **Evidence:** Sections 1.4, 3.3.3, 4.3 and 3.4.6; Figure 3.11.

ADR-3 — The three-level model as the *one* architecture

Context: The anatomy metaphor and the technical engine view threaten to drift apart into parallel model trees. **Decision:** There is exactly one hierarchy without parallel trees — family (level 1, outer interface: the probe-dock families `Map`, `Container` and `Graph` as well as the fourth family `HeuristikAdapter`, which has no dock of its own because its binaries report the target genus via `genus()` and dock at the dock of the target) → genus (level 2, the tier subclass with a fixed axis set, alias family subclass; the six genera `SearchAlgorithm`, `Set`, `Sequence`, `Adapter`, `View` and `FunctionInterfaceReroute`) → axis organs (level 3) — under `IExecutionEngine` as the common measurable root; a family of its own would arise only if the base axis set differed fundamentally. **Consequence:** The genus `SearchAlgorithm` under the family `Map` and its ABI runtime view (“`SearchEngine`”) are two views of the same living being, not a second model. **Evidence:** Sections 3.3, 3.3.2 and 4.3.1; Figures 3.7 and 3.8.

ADR-4 — Pass-through rule and enforced full assignment

Context: For non-tree-shaped methods (such as the hash cross-check), tree- and trie-specific axes like path compression or node type cannot meaningfully be assigned. **Decision:** Such axes are not dropped but assigned as pass-through variants (`None`) — a pass-through value is an algorithm with an empty path, not an omission; implemented is the pass-through building block for path compression (`PathCompressionNone`), the node type possesses no `None` variant and is regularly assigned one of the four ART node formats, which a non-tree-shaped building block does not use (design; without a code building block); the mandatory concept `IsComposition` of the genus `SearchAlgorithm` enforces at compile time the assignment of all eighteen organs (eighteen type requirements and an organ counter of eighteen with a check for ABI break), so that no composition of this genus with an open axis compiles. **Consequence:** Non-tree-shaped methods require no family of their own but a restricted axis subset within the same genus. **Evidence:** Sections 3.2, 3.3.2 and 4.3.

ADR-5 — Layer model E4–E0 with fixed contracts and contract tests

Context: With a product space that rules out any complete materialization, changes across several

²`Boost.Mp11`: a C++11 metaprogramming library for the compile-time manipulation of data structures that contain types; its list type is `mp_list` [Dim19].

levels at once are practically not acceptable; therefore each level is completed individually. **Decision:** The experiment pipeline is organized into the five layers E4–E0 (E4 definition and evaluation, E3 permutation tree, E2 tier binaries, E1 runtime configuration, E0 infrastructure) and the cross-cutting concern M; the E identifiers are a conceptual order with one code location each. Each layer is completed individually, with an explicitly fixed interface contract towards the neighbouring level (the fixing of the contract of layer E1 in the ABI adapter: signatures, data layout and capabilities fixed, semantic changes only additive), and contract-tested against a compile-time substitute (fake) of the neighbouring level. **Consequence:** Every level can be accepted individually before the next one begins; the boundary E3→E2 is contract-tested without a module build against a fake of layer E2, checked is solely the static tree and view structure including its mixed-radix bijection. **Evidence:** Sections 3.3.4 and 4.4; Figures 3.12 and 4.9.

ADR-6 — Explicitly coded non-observation instead of a phantom value

Context: Values that cannot truly be collected (counters without hardware access, axes without a real runtime parameter) invite plausible phantom values that would systematically poison the heuristic derivation. **Decision:** The system reports explicitly invalid or empty instead of a phantom value and distinguishes three states per measured value: valid measured value, explicitly coded non-observation (not applicable or source not available) and measurement error³. A validity flag travels with every event, so that a measured zero remains a measurement result; the wall-clock source offers only mean and throughput and refuses to label its mean as a percentile, and a negative duration is a measurement error, not a zero value; a PMC catalogue without a model entry yields “not applicable”, never a guessed encoding; the generators produce no output for a measurement series without a valid row, rather than an empty one. **Consequence:** Gaps remain visible and can be closed deliberately; the principle is programmed into the building blocks rather than merely documented. **Evidence:** Sections 3.4, 4.5 and 3.4.1.

ADR-7 — Four-repository architecture with orthogonal carrier stages

Context: Measurement apparatus, candidate under test, experiment definition including evaluation, and the manuscript have separate responsibilities and life cycles; the measured values are built directly into the manuscript under version control, which is why the manuscript is a repository of its own with its own pipeline. **Decision:** Four repositories — `comdare-cache-engine` (*how* measuring is done; building-block library and builder in the Pitchfork source-tree layout, Appendix C), `comdare-prt-art` (the candidate under test), the project repository `probst-Diplomarbeit-cache-engine` (*what* is tested by the user: experiment definition in XML and XSD, measurement driver, evaluation toolchain) and the manuscript repository with its own CI, into which the forward channel deposits the generator output as table and diagram files per commit —; orthogonal to this division lie the four carrier stages in build order planner → CacheEngineBuilder → tier binary → hybrid tier binary, where stage $N + 1$ hangs only on stage N (processing chain: CEB probe dock → [optional hybrid mux] → tier binary); planner and CEB are two binaries — the planner `comdare-experiment-planner` and the measurement driver `comdare-messung-driver` —, and the two roles appear as planner line and CEB

³The code vocabulary `honest-0/honest-100/honest-empty` is the implementation name of this principle (Appendix C); methodically, non-observation is a declared class of missing values in the sense of the missing-data literature [Gra09], never an imputed value.

contract line in the self-stamp of the measurement driver (`-version`) in one driver binary; the measurement aggregator is run host-side by the `ExperimentDriver`. **Consequence:** Builder and CacheEngine share the engine repository — the builder as an executable above its builder library, the CacheEngine as a library —; the roles remain cleanly separated, and the candidate under test is not a stage but the transversal supplier of organ building blocks. **Evidence:** Section 4.1 and Appendix B; Figure 4.1.

ADR-8 — Family-wise adapter fallbacks for self-contained runnable builds

Context: Third-party code (allocator vendors, SOTA building blocks) is not available or activated on every platform and in every build. **Decision:** Compile-time flag families switch adapters on or off — per vendor one family switch, whose use follows from approval and availability, behind 23 vendor shim headers —, each with a safe, family-wise chosen fallback: the allocator adapters replace the missing vendor in `if constexpr` branches with the portable aligned allocation (`portable_aligned_alloc`), the vendor shim supplies only forward stubs for the never-executed branch, and the axis value `StdMalloc` is a building block of its own, not a stopgap; data-structure adapters of third-party search methods switch via a family switch and possess a self-contained re-implementation as their body, while the binding to the original code runs via the library build with the original compiler. **Consequence:** The build remains self-containedly runnable without any third-party activation. **Evidence:** Section 4.3.

ADR-9 — Measurement gating in three layers and four work modes

Context: Measurement and observer couplings cost time and must not burden the production path. **Decision:** The measurement couplings of the runtime adapter are switched via the master option `COMDARE_MEASUREMENT_MODE`, which defines `COMDARE_MEASUREMENT_ON` and which the release mode forcibly switches off; the switchable area falls into three separable layers — base time, observer counters and fine-grained segment timers per axis —, the shut-off levels of the measurement. The run methodology knows four work modes of the same permutation, as a registry and not as a label: `build` (build without measuring), `measure` (deterministic single-thread measurement run, the only mode with measurement), `compare` (reads the measurement store, the stage before `release`) and `release` (full optimization without measurement instrumentation); the work mode is a sub-axis of the measurement realm and does not enter the binary identity, a debug state does not exist (`-debug` is a flag of the planner shell), and the measurement aggregator is run host-side by the `ExperimentDriver`. **Consequence:** The production path stays measurement-free and causes no measurement overhead; four modes are selectable, in build semantics only `measure` differs — all four registry rows carry the release build type. **Implementation status:** The mode-specific replay run of `compare` and the optimal release binary generated from measured values are planned but not implemented; `compare` is a selectable, validatable label with release-equivalent build semantics. **Evidence:** Sections 4.1, 4.3.2 and 5.1; Figures 3.10 and 4.11.

ADR-10 — The `std::map` contract as common denominator with a conformance gate

Context: Structurally different search algorithms can only be compared fairly through a common, well-understood outer interface. **Decision:** All methods are mapped onto the C++ contract of the ordered map `std::map<Key, Value>` [ISO24]; the container genera are held against one standard oracle each — `Set` against `std::set`, `Sequence` against `std::deque`, `Adapter` against

`std::queue`, `std::stack` and `std::priority_queue`, View against `std::span` —, and `std::vector` serves as the inner substrate of the genus *Adapter*. A conformance gate checks this contract before every measurement in the binding order `import` → `gate` → `measure`, via the five core operations `insert`, `lookup`, `erase`, `clear` and `size`, with conformance ratios as the result form. **Consequence:** Only comparables are compared; the axes describe exclusively the inner behaviour behind the fixed interface. **Implementation status:** The extension of the gate to the full `std::map` interface and the genus iterator library are not implemented; the requirement to run the container genera through a complete `std::vector` hull is not met — the implemented Sequence oracle is `std::deque`. **Evidence:** Sections 3.4 and 4.4.

ADR-11 — Design-pattern register with evidenced roles

Command as bundling and command pattern.

Context: The apparatus is deliberately a composition of named design patterns; unclear pattern roles would dilute the architecture. **Decision:** Used are, each with a named code location: *Strategy* with CRTP [Cop95] per organ axis (the stamp contract of the carriers is a CRTP contract as well, ADR-13), *Abstract Factory* (hybrid docks, composition factory, candidate registry), *Director* with *Builder* (plan emission), *Iterator* (the lazily traversed experiment tree), *Adapter* (third-party code behind the ABI boundary), *Observer* (measurement snapshots per axis), *Visitor* (the measurement seam at the `dlopen` transition, ADR-15), *Chain of Responsibility* (hardware collection chain and selection filter chain), *Memento* (copy-on-write in the ABI adapter, state per tier), *Command* and *Facade* — the same roles as the pattern register in Section 4.5.1 (Figure 4.12), named after the catalogue of Gamma et al. [GHJV94] and composed as a tuple of axis policies [Ale01]. The patterns *Composite* and *Decorator* are not among them: the measurement hulls of the axes hold no component instance and no full interface and are expressly not a *Decorator*, and a *Composite* in the sense of the catalogue has no counterpart in the code. *Not* used in the hot path and in the organ axes are *Singleton*, *Flyweight* and *Bridge*; host-side registries use Meyers singletons⁴, the catalogue generation uses flyweight tables per axis, and “Bridge” is in the code only a colloquial designation for connecting pieces (ABI adapter, CI bridge jobs, Sandy Bridge), not a pattern. The *Command* pattern carries the compile-time bundling of axis recombinations (the system complex axis as command wrapper, the hub of the external tools as invoker, the command base of the axes in the cross-cutting concern M) and the typed control command channel; the command channel of the hybrid mode is one of these roles. **Consequence:** Every pattern carries named roles with code location in the wiring of the apparatus and is executed at compile time rather than runtime-polymorphically. **Evidence:** Sections 3.3.3, 3.6.2 and 4.5.1.

ADR-12 — Dual measurement regime: immutable Talos and root Linux

Context: Production fidelity and measurement depth are in conflict: a production-faithful system denies precisely the hardware-counter access that deep measurements require. **Decision:** On the production target machines, every measurement is collected under two operating-system regimes: an immutable operating system (Talos⁵) as the production mirror and a root Linux with full hardware-counter access (`perf` [dM10, Lin26b] and the model-specific registers, MSR). **Consequence:** The

⁴Meyers singleton: the one instance as a function-local static object that is initialized on the first call [Mey05].

⁵Talos Linux: the immutable, API-driven operating system for Kubernetes by Sidero Labs [Sid26].

time- and observer-based categories run identically under both regimes and secure comparability; the PMC categories are collected only under the privileged regime. **Implementation status:** The decision stands; implemented is the root-Linux half: the reported measurements all originate from the root-Linux regime of the two production machines — prod1 with the AMD microarchitecture Zen 5 and prod2 with the Intel Core i9-12900K (Alder Lake) —; the Talos regime is planned but not measured. For Intel, the PMC raw-event catalogue contains no assignment; for Zen 5 it is cross-checked by measurement. **Evidence:** Sections 5.1 and 1.4.

ADR-13 — Four carrier stages and one stamp contract

The stamp as identity, store key, recognition criterion and classification.

Context: The configuration travels through four carrier stages (planner, CacheEngineBuilder, tier binary, and the hybrid tier binary optionally inserted between the CEB probe dock and the tier binary); each stage needs an ABI-stable, compile-time identity into which no measured value enters. **Decision:** A single CRTP contract binds all four carriers: seven stamp interfaces (version, measurement line, system line, organ line, fingerprint, overall stamp, attached lines) times four carriers yield 28 three-valued admission cells (Table 4.3); the absence of the organ line at the CEB is the static barrier “the CEB carries no organ axes”, and the check runs as a `static_assert` under every heir. The planner carries a SHA-256 fingerprint from version, ISA and operating system, the tier binary a SHA-512 fingerprint over eleven members (format 6), among them the three realm lines, the toolchain member and the hash of the overlay source set (concatenation in fixed order, SHA-512 once); the CEB carries a SHA-512 provenance fingerprint, which is not a reuse criterion. The bracket grammar of the stamp entries is defined at one place (`c{p.e}` is `c` with the sub-members `p` and `e`, never flat), and the bracket depth is the meta-meta level of the entry. **Consequence:** The stamp fulfils five roles — identity, cache key, store key, recognition criterion and classification; the self-stamp of the driver binary (`-version`) carries only the classification and is neither cache key nor build input. **Implementation status:** The registry-wrapper stamp of the SOTA branch and the stamp export of the hybrid stage are not implemented; the stamp identity in every diagram and every table of the appendix is likewise not implemented. **Evidence:** Sections 3.4.4, 3.3.2, 3.4.6, 4.4 and 5.2.1; Figure 3.9.

ADR-14 — Identity cut: only organ axes form the `binary_id`

System and measurement axes are identity-neutral.

Context: Three axis realms (organ, system, measurement) each carry a registry of their own with generated XML; if measurement or system values entered the identity, duplicate binaries of the same permutation would arise, and a store that holds the same binary several times. **Decision:** The `binary_id` is the path of eighteen `axis=value` segments in canonical slot order (the organ count eighteen is defined at exactly one place); all measurement axes and all system axes carry `binary_id = never` in their registries; sub-axes are runtime permutation and appear in no build-job legend; the work mode is identity-neutral. The build world instead travels along as a fingerprint member — the system line and the toolchain member with `vendoropt` (ADR-17). **Consequence:** Store key and regression detector (reference catalogue 2¹⁷ with CRC-64 checksum) are organ-stable; the same organ permutation against two build worlds is a different artefact with a different fingerprint. **Evidence:** Sections 3.4.1, 3.4.4 and 4.5; Table 3.2.

ADR-15 — Measurement-visitor seam (surface 3) instead of a sidecar

Family and genus interfaces remain unchanged.

Context: An observer that fetches fields from a tier binary from the outside measures the neighbourhood and not the object: what is measured is then decided by the host through the choice of fields; measurement fields in the domain interfaces would alter the family and genus contracts. **Decision:** Each carrier stage offers three interface surfaces: surface 1 the genus interface (runtime, abstract factory), surface 2 the stamp (compile time, ABI-stable), surface 3 the measurement cut-through; the measurement crosses as a seam of its own (`tier_measure_accept` with `IMessVisitor`), a visitor at the `dlopen` transition with double dispatch [GHJV94]; depending on whether the measurement facility is activated in CEB and tier binary, a measurement visitor is handed over at the interface or not. **Consequence:** The domain interfaces contain no measurement fields; the gate combination CEB off, tier on is evidenced by a coverage test; the measurement-gate assignment travels along as a fingerprint member of its own with positions of its own for the hybrid level. **Evidence:** Sections 3.3.2, 4.3.2 and 3.4; Figure 3.9.

ADR-16 — Store as stock with compile-time keys

Measurement always on, in the store only re-measure what is missing.

Context: A reference space of 2^{17} compositions per system permutation (according to the catalogue header about 224 GB and 24 h of complete build) is manageable only with reuse of built binaries and measured values; the measurement is to be always switched on and, in the store, to re-measure only what cannot be found there. **Decision:** Stock 1 holds the binary under its SHA-512 fingerprint; stock 2 holds the measured values under the pair of fingerprint of the measured binary and hardware identity of the measuring machine, the measurement cell stands beside it and is never fused into the digest; a binary found is skipped if its passed test attestation is inventoried, otherwise built or measured; a version lock over the overlay source set turns unnoticed source changes into a CI error. **Consequence:** The identity travels unchanged from the store via the measurement line (`permutation_id`, `fingerprint`) into workbook and appendix. **Implementation status:** Stocks 1 and 2 and the key schema are implemented; stocks 3 and 4 (synthesis batches and solutions), the planner leaf function, the CEB full placement, the hybrid storage path, the commit skip, the rebuild flag per stage and the XML section `publish` form the full store build-out and are not implemented. **Evidence:** Sections 3.4.5, 5.1, 3.4.4 and 3.6.2; Figures 3.14 and 5.2.

ADR-17 — Optimization level as the user's choice

House-wide O2, O3 on XML request; the build world is identity-effective.

Context: Measurement series over differently optimized build worlds are not comparable; the maximum optimization must remain selectable for the user. **Decision:** The release flags apply house-wide with O2 — including vendor build, `FetchContent` and embedding — via a forced cache value; O3 is built alongside only on XML input request; the build-world level enters the toolchain member of the fingerprint as the field `vendoropt` (compile error instead of a silent default); a gate rejects self-supplied release optimization flags that bypass the axis; the sub-axis `opt_level` offers O0 to Ofast with the determinism flag according to IEEE 754 [IEE19], which only Ofast negates. **Consequence:** Two builds of the same permutation against an O2 and an O3 world are different artefacts with different fingerprints; a silent skip is ruled out. **Implementation status:**

O0 and O1 as the user's choice in the XML and the pass-through of further compiler flags from the XML are planned but not implemented (default O2, O3 selectable as override). **Evidence:** Sections 5.1.6, 4.5 and 3.4.2.

ADR-18 — One official build route as a GNU front end over CMake

configure.sh, make, make check and make install.

Context: The only procedure official under Linux is `configure.sh`, `make` and `make install` or `make check`, respectively, and these three must lie in the root folder; a configuration may have only one source. **Decision:** `configure.sh` is a POSIX-sh front end without Python, building continues to be done with CMake; the Makefile is checked in statically and only forwards to CMake, with the GNU standard targets [Fre26] `all`, `install`, `check`, `installcheck`, `uninstall`, `clean`, `distclean`, `mostlyclean` and `maintainer-clean` (the test target is called `check`); the configuration is stored exclusively in `config.status` and in the CMake cache, and the Makefile deliberately contains no default values — a generated Makefile would be a second configuration source without benefit. **Consequence:** There is no second build route beside CMake and no second configuration source; the front end is checkable with `sh -n`. **Implementation status:** `configure.sh` and Makefile lie in the repository; the framework export for consumers (`install(EXPORT)`, `find_package`) and the follow-up checks are not implemented. **Evidence:** Appendix B.

ADR-19 — Four CI cells without skip

gcc and clang each as release and debug; a job fails or passes.

Context: Two compilers find different errors; a silent job skip or a tolerated error feigns a passed run. **Decision:** The test suite runs in four cells — gcc and clang each as release and debug build — without change skip; the cell is the warning, the job fails; `allow_failure` carries only two declared exceptions — the arm64 smoke build without an available runner and the manual relock maintenance job —, no build or test cell; per compiler and build type there is one object-cache key with exactly one writer; the minimum compiler level is g++ 15.3; the golden reference catalogue 2¹⁷ with CRC-64 checksum is the regression detector with fixed cardinality. **Consequence:** Accepted is only a run without failed tests in all four cells without an exception list — a gate in the sense of continuous delivery [HF10] —, and a test count is named only together with the run that produced it. **Implementation status:** The mini-pipeline per carrier stage (four stages times gcc and clang times release and debug) is planned but not implemented. **Evidence:** Section 4.4.

ADR-20 — XML as the only control

The XML carries the what, where and when, never the how.

Context: The apparatus is operated from a declarative experiment profile; terms are not translated but must be conformant — were a translation necessary, that is a regression and a compile-time error. **Decision:** Two schemas describe profile and measurement series (`experiment_schema.xsd`, `messreihe_v32_schema.xsd`); the parse seam aborts with an error message, the validation is a building block of its own, and the CI checks well-formedness; the XML describes the what, where and when, never the how; the axis registries are generated as XML and are thus machine-readable images; the planner is a subcommand tool (`validate`, `plan`, `cache-key`, `fingerprint`, `status`, `check-size`, `simulate`, `version`, `help`) and the shell control path; as the second

control path, an API command line with a headless service is planned, which serves the planner as a cell on Kubernetes or on bare metal, as well as a command-line-guided creation mode for the user XML in which unavailable options are not selectable. **Consequence:** There is no side route beside the XML; the SOTA profiles are executable as XML. **Implementation status:** The headless control path and the creation mode are not implemented; the hybrid section is parsed but not declared in the schema, the organ meta-meta section declared but not read, and the `publish` section is missing. **Evidence:** Sections 3.4.6 and 4.3.2; Figure 4.7.

F Comparison Interfaces: `std::map` and `std::vector`

This appendix presents — complementing Section 3.4 — the complete listing of all standard interface functions of the two comparison hulls `std::map` (ordered associative) and `std::vector` (sequence) as of C++23, with their officially expected semantics and their complexity guarantee per the ISO C++ standard [ISO24]. It also serves as the central reference for the further development of the cache engine: because the search-algorithm hull exchanges its internals at compile time — each axis choice yields a compiled binary of its own —, while at run time only the parameters of the dynamic sub-axes are readjusted (the registered dynamic dimensions of the measurement realm, Section 3.4), it becomes necessary to explain, per standard function, which behaviour the outer contract guarantees.

Variadic mapping. The hybrid search engine of the candidate under test PRT-ART ties the number of parameters to the hull: one template parameter demands all function interfaces of a `std::vector`, two those of a `std::map`, and $N > 2$ those of a `std::map<K, std::tuple<...>>` (Section 4.3.1). The cache engine derives the types from the same rule — with one parameter the key is assigned implicitly as a monotonically counted 64-bit value, with $N > 2$ the value becomes the tuple —; its two contract hulls are implemented as separate family adapters: the Map family via the functional drive of the module boundary with the five operations `tier_insert`, `tier_lookup`, `tier_erase`, `tier_clear`, and `tier_size`, the Sequence family via the sequence drive with `tier_push_back`, `tier_at`, `tier_size`, and `tier_clear`; an adapter cutting across the family is ruled out by the type system. The case $N > 2$ is type derivation without an adapter of its own; the module boundary carries key and value throughout as 64-bit values. The shared functions (iteration, `size/empty/clear`, `swap`, `get_allocator`) have the same name and the same role in both hulls; of these, both hulls pass `size` and `clear` across the module boundary, `empty` is derived host-side, iteration and `swap` do not cross the boundary, and `get_allocator` is, on the map side, an optional proxy sub-interface outside the oracle (Section F.3).

Composite functions. Some functions arise internally as a composition of more primitive operations — for instance `operator[]` as `lower_bound` followed by default insertion on a missing key, or `at` as `find` with a thrown `std::out_of_range` exception on absence —; such cases are marked in the “Note” column. The “Since” column names the introducing standard version. Three terms of the tables stem from the more recent development of the standard: the node handle¹, the deduction guides (CTAD)², and the transparent key (Section F.1). The exact cache-engine-internal function hops (*function-handle hops*³) are documented in the developer document `docs/ENTWICKLER-FUNCTION-HANDLE-HOPS.md` of the cache engine; it records, per standard function, the hop path and the status at the module boundary.

¹Since C++17, a node of an associative container can be unlinked with `extract` as a node handle, without copy or move, and spliced into another container with `insert` or `merge` [TWH16].

²*Class template argument deduction*, since C++17: the template arguments of a class template are deduced from the constructor arguments, via guides formed implicitly from the constructors or stated explicitly [SVS16].

³Project-internal term of the cache engine for the sequence of internal function hops (handles) from the entry at the module boundary through the axis organs to the result; the developer document records them per standard function and marks functions that are not wired up as such.

Conformance subset. Before each measurement, the conformance gate (Section 3.4) checks, per family, a core subset of the hull operations against an oracle of the standard library — `std::map` for the Map family, `std::deque` for the Sequence family —, so that only comparable things are compared; the subset is marked with † in both tables.

F.1 `std::map` (Ordered Associative)

Table F.1 lists the complete interface of `std::map<Key, T, Compare, Allocator>` as of C++23. A † marks the core operations that the conformance gate (Section 3.4) checks before each measurement, so that only comparable things are compared. Complexities refer to $n = \text{size}()$; for range operations N is the number of input elements; version markers and complexities follow the standard text [ISO24] and are cross-checked against the online reference [cpp26a]. The seventeen marked operations are exactly the operation set of the gate’s twelve core check blocks: no marked operation remains unchecked, no checked operation is unmarked (`size` is checked as well within the blocks, but belongs to the randomized core block and is therefore not marked). Four of the marked operations correspond directly to operations of the module boundary (`insert_or_assign` to the insert drive, `find` to the exact lookup, `erase` and `clear` to their drives); `at`, `operator[]`, `count`, `contains`, `empty` as well as `insert`, `emplace`, and `try_emplace` are composed host-side — `empty` from the `size`, the others from exact lookup and insertion; the three non-overwriting forms run over one path and are set against three different oracle calls; `begin/end`, `lower_bound`, `upper_bound`, and `equal_range` are synthesized by the host in order via the additive scan capability of the module boundary, and if a loaded binary lacks it, the gate logs these blocks as skipped, not as passed; `key_comp` is established at the oracle (key order less-than on 64-bit values), because the hull carries no comparator across the boundary.

Table F.1: Complete `std::map` interface (C++23)

Function	Official semantics	Complexity	Since
Construction, assignment, allocator			
(constructor)	empty; from Compare/Allocator; range; copy/move (incl. allocator-extended); initializer_list; from_range since C++23	$O(1)$ to $O(N \log N)$	C++98
(destructor)	destroys all elements and nodes	$O(n)$	C++98
operator=	copy/move/initializer_list assignment with allocator propagation	$O(n)$	C++98
get_allocator	copy of the associated allocator	$O(1)$	C++98
Element access			
at †	reference to the mapped value; throws <code>std::out_of_range</code> on absence	$O(\log n)$	C++11

Function	Official semantics	Complexity	Since
<code>operator[]</code> †	reference; inserts a value-initialized element on absence (no <code>std::out_of_range</code> ; the insertion can throw <code>std::bad_alloc</code> or constructor exceptions)	$O(\log n)$	C++98
Iterators			
<code>begin/end</code> †	iterator to first element / end, in key order (cbegin/cend since C++11)	$O(1)$	C++98
<code>rbegin/rend</code>	reverse iterators (crbegin/crend since C++11)	$O(1)$	C++98
Capacity			
<code>empty</code> †	empty test (<code>begin() == end()</code>); <code>[[nodiscard]]</code> since C++20	$O(1)$	C++98
<code>size</code>	number of elements	$O(1)$	C++98
<code>max_size</code>	theoretical maximum size	$O(1)$	C++98
Modifiers			
<code>clear</code> †	remove all elements	$O(n)$	C++98
<code>insert</code> †	element(s) without overwriting (value, P&&, hint, range, <code>initializer_list</code> , node handle since C++17)	$O(\log n)$; hint amort. $O(1)$; range $O(N \log(n+N))$	C++98
<code>insert_range</code>	insert an input range, no overwrite	$O(N \log(n+N))$	C++23
<code>insert_or_assign</code> †	insert, or assign to an existing value; <code>bool</code> reports insertion	$O(\log n)$	C++17
<code>emplace</code> †	construct in place, only if the key is absent (otherwise discarded)	$O(\log n)$	C++11
<code>emplace_hint</code>	like <code>emplace</code> , near the hint	$O(\log n)$; amort. $O(1)$	C++11
<code>try_emplace</code> †	like <code>emplace</code> , leaves the arguments untouched on an existing key	$O(\log n)$	C++17
<code>erase</code> †	by iterator/range/key (key: 0 or 1 removed)	iter. amort. $O(1)$; key $O(\log n)$	C++98
<code>swap</code>	exchange contents and comparator; iterators stay valid	$O(1)$	C++98
<code>extract</code>	unlink an element as a node handle (no copy/move)	iter. amort. $O(1)$; key $O(\log n)$	C++17
<code>merge</code>	splice nodes from a source map; existing keys remain there	$O(N \log(n+N))$	C++17

Function	Official semantics	Complexity	Since
Lookup			
<code>find†</code>	iterator to the key, or <code>end()</code>	$O(\log n)$	C++98
<code>count†</code>	number (0 or 1)	$O(\log n)$	C++98
<code>contains†</code>	membership (bool)	$O(\log n)$	C++20
<code>lower_bound†</code>	first key $\geq$ key	$O(\log n)$	C++98
<code>upper_bound†</code>	first key $>$ key	$O(\log n)$	C++98
<code>equal_range†</code>	pair [lower_bound, upper_bound)	$O(\log n)$	C++98
Observers			
<code>key_comp†</code>	copy of the key comparator	$O(1)$	C++98
<code>value_comp</code>	<code>value_compare</code> (compares pairs by key)	$O(1)$	C++98
Non-member functions			
<code>operator==</code>	equal size and pairwise-equal elements	$O(n)$	C++98
<code>operator<=></code>	lexicographical three-way comparison; synthesizes <code>< <= > >=</code>	$O(n)$	C++20
<code>std::swap</code>	specialization; calls <code>lhs.swap(rhs)</code>	$O(1)$	C++98
<code>std::erase_if</code>	removes all elements with <code>pred==true</code> ; returns the count	$O(n)$	C++20
Deduction guides	CTAD from <code>range/</code> <code>initializer_list/</code> <code>from_range</code> (<code>from_range</code> since C++23)	— (compile time)	C++17

The comparison operators `<=>`, standalone before C++20, are synthesized from `operator<=>` since C++20, and `!=` is rewritten from `operator==`. The heterogeneous overloads of `operator[]`, `at`, `try_emplace`, and `insert_or_assign` (transparent key⁴) belong only to C++26 and remain excluded here; the C++23 additions are the `from_range` constructor, `insert_range`, the transparent-key overloads of `erase/extract`, and the associated `from_range` deduction guides [JNC22, BVA21]. The `constexpr` capability of `std::map` likewise belongs only to C++26 and remains excluded [cpp26a].

⁴Heterogeneous lookup since C++14: if the comparator carries the type `is_transparent`, the lookup functions accept arguments of a type other than `key_type` without forming a temporary key object [WLLM13]; C++23 extends this to `erase/extract` [BVA21], C++26 to `operator[]`, `at`, `try_emplace`, and `insert_or_assign` [BVA23].

F.2 `std::vector` (Sequence)

Table F.2 lists the complete interface of `std::vector<T, Allocator>` as of C++23 ($n = \text{size}()$, for range operations N the number of input elements; version markers and complexities follow the standard text [ISO24] and are cross-checked against the online reference [cpp26b]). All members are `constexpr` since C++20⁵. A † marks the core operations that the conformance gate of the Sequence family checks before each measurement against a `std::deque` oracle (append, index access, size, clear; for these four operations the deque semantics coincide with those of `std::vector`): the sequence hull of this work offers exactly these four functions across the module boundary, and index access is checked there rather than undefined (status return: hit flag and output value at the module boundary, `std::optional` in the organ). All other functions of the table — capacity control, position and range operations, `front/back/pop_back`, `data`, iterators, `swap`, and the non-members — do not cross the module boundary; growth at the front end (`push_front`), which the deque oracle would additionally offer, is envisaged as an extension of the module boundary but not implemented, and is logged in the gate as skipped, not as passed.

Table F.2: Complete `std::vector` interface (C++23)

Function	Official semantics	Complexity	Since
Construction, assignment, allocator			
(constructor)	<code>empty/Allocator; n× default/value; range; copy/move (incl. allocator-extended); initializer_list; from_range</code> since C++23	$O(1)$ or $O(n)$	C++98
(destructor)	destroys all elements, frees storage	$O(n)$	C++98
<code>operator=</code>	<code>copy/move/initializer_list</code>	$O(n)$	C++98
<code>assign</code>	<code>replace contents with n× value/ range/initializer_list</code>	$O(n)$	C++98
<code>assign_range</code>	<code>replace contents with an input range</code>	$O(n)$	C++23
<code>get_allocator</code>	<code>copy of the associated allocator</code>	$O(1)$	C++98
Element access			
<code>at</code> †	<code>bounds-checked access; throws std::out_of_range</code>	$O(1)$	C++98
<code>operator[]</code>	<code>unchecked access (UB out of range)</code>	$O(1)$	C++98
<code>front/back</code>	<code>reference to first/last element (UB if empty)</code>	$O(1)$	C++98
<code>data</code>	<code>pointer to the contiguous storage (not in vector<bool>)</code>	$O(1)$	C++11
Iterators			

⁵Since C++20, all member functions of `std::vector` are `constexpr`; a `constexpr` object itself usually fails on the dynamic storage [Dio19].

Function	Official semantics	Complexity	Since
<code>begin/end</code> , <code>rbegin/rend</code>	forward/reverse iterators in index order (<code>c...</code> variants since C++11)	$O(1)$	C++98
Capacity			
<code>empty/size</code> †/ <code>max_size</code>	empty test, element count, maximum size	$O(1)$	C++98
<code>reserve</code>	capacity $\geq n$; reallocates and invalidates; throws <code>length_error</code> if $n > \text{max_size}()$	$O(n)$	C++98
<code>capacity</code>	currently allocated capacity	$O(1)$	C++98
<code>shrink_to_fit</code>	non-binding reduction of capacity toward <code>size()</code>	$O(n)$	C++11
Modifiers			
<code>clear</code> †	remove all elements (capacity unchanged)	$O(n)$	C++98
<code>insert</code>	before a position: <code>value/n</code> / <code>range/initializer_list</code> ; shifts the tail	$O(n)$; range forms $O(n+N)$	C++98
<code>insert_range</code>	insert an input range before a position	$O(n+N)$	C++23
<code>emplace</code>	construct an element in place before a position	$O(n)$	C++11
<code>erase</code>	remove a position/range, shift the tail down	$O(n)$	C++98
<code>push_back</code> †	append an element (copy/move)	amort. $O(1)$	C++98
<code>emplace_back</code>	append in place; returns a reference since C++17	amort. $O(1)$	C++11
<code>append_range</code>	append an input range	amort. $O(N)$	C++23
<code>pop_back</code>	remove the last element (UB if empty)	$O(1)$	C++98
<code>resize</code>	truncate to n , or fill with default/value	$O(n)$	C++98
<code>swap</code>	exchange contents and capacity with another vector	$O(1)$	C++98
Non-member functions			
<code>operator==</code>	equal size and element-wise equality	$O(n)$	C++98
<code>operator<=></code>	lexicographical three-way comparison	$O(n)$	C++20
<code>std::swap</code>	specialization; calls <code>lhs.swap(rhs)</code>	$O(1)$	C++98

Function	Official semantics	Complexity	Since
<code>std::erase</code>	removes all elements equal to a value; returns the count	$O(n)$	C++20
<code>std::erase_if</code>	removes all elements with <code>pred==true</code> ; returns the count	$O(n)$	C++20
Deduction guides	CTAD from <code>range/from_range</code> (<code>from_range</code> since C++23)	— (compile time)	C++17

The specialization `std::vector<bool>` packs boolean values bitwise: it has no `data()` and no contiguity guarantee, returns via `operator[]/at/front/back` a proxy type⁶ reference instead of `bool&`, and additionally offers `flip()` (at the vector and at the reference level), a static `swap(reference, reference)`, and a `std::hash` specialization. The comparison operators behave as for `std::map`.

F.3 Composite Functions and the Conformance Subset

Several interface functions are internally compositions of more primitive operations of the same interface (Table F.3); the table describes the ISO standard contract. The measurement hull of this work deliberately deviates from it in two places, because the ABI conformance contract demands `noexcept` and a throwing hull counts as non-conformant: `at` delivers a status return instead of the exception (hit flag and output value at the module boundary, `std::optional` in the organ), and `operator[]` is not a call of its own at the module boundary but is composed host-side from exact lookup and default insertion — with the ISO semantics (insertion on absence, no `out_of_range`), but without a reference return across the binary boundary; the write path runs via `insert_or_assign`. In the sequence hull, `operator[]` is moreover checked rather than undefined (Section F.2). For this thesis that is the central observation: all map accessors and modifiers of the hull rest on a single query primitive of the module boundary — the exact lookup, flanked by insertion and deletion —; an axis implementation therefore only needs to provide a correct exact lookup together with node linking, and the rest follows as a fixed composition. The ordered operations — `begin/end`, `lower_bound`, `upper_bound`, `equal_range` — additionally require an ordered search and are therefore carried as a separate, additive capability of the module boundary; unordered axes satisfy the same hull without it and pay for the missing order with linear effort on ordered reading. Exactly this interchangeability is the object of the measurement (Section 3.4). The conformance gate checks, per family, the core subset marked with † in Tables F.1 and F.2.

⁶Stand-in object: `vector<bool>::reference` behaves in an expression like `bool&`, but denotes a single bit instead of an addressable object, which is why `data()` and the contiguity guarantee are dropped [ISO24].

Table F.3: Composite functions (internal composition of primitive operations)

Function	Composition	Note
<code>map::operator[]</code>	<code>lower_bound</code> + default insertion on absence $\rightarrow$ reference	mutates on absence; no <code>out_of_range</code> , the insertion can throw
<code>map::at</code>	<code>find</code> + throw <code>std::out_of_range</code> on absence	the only access that throws on absence; never mutates
<code>map::insert / emplace</code>	ordered search + link a node only if the key is absent	no overwrite; <code>emplace</code> may still consume arguments on a hit
<code>map::try_ emplace</code>	like <code>emplace</code> , but arguments untouched on a hit	preserves move-only arguments
<code>map::insert_ or_assign</code>	ordered search + insert or assign	overwrites (unlike <code>insert</code>)
<code>map::count / contains / equal_range</code>	reduce to <code>find</code> resp. <code>lower_bound</code> + <code>upper_bound</code>	always 0 or 1 for a unique-key map
<code>vector:: push_back / emplace_back</code>	possible growth (realloc + relocation) + construction at the end	hidden cost + invalidation on realloc
<code>vector::insert / emplace</code>	possible growth + tail shift + insertion	extra $O(n)$ shift
<code>vector::resize</code>	append (default/value) or truncate the tail	bidirectional; does not shrink capacity
<code>vector::assign</code>	clear + insert range/ $n \times$	capacity reused where possible

At the module boundary, the order of primitiveness of this table is reversed: the one insert drive of the Map family carries the semantics of `insert_or_assign` — insert or update, the return reports whether a new key arose —, so that `insert_or_assign` is the pass-through and the non-overwriting forms `insert`, `emplace`, and `try_emplace` are composed host-side as an exact lookup followed by insertion only on a missing key: one path for three standard functions, which the gate sets against three oracle calls. The ISO contract carries it the other way round (`insert` as the basic form, `insert_or_assign` as the composition); both views are correct and describe different levels — exactly the contrast that makes the interchangeability behind a fixed interface concrete.

Functions that both hulls *share* by name — `size`, `empty`, `clear`, the iterator families, `swap`,

`get_allocator`, and `insert/emplace/erase` — have the same role but different referents: key semantics in the map, position/index semantics in the vector; `operator[]`, for instance, inserts in the map, but is an unchecked index access in the vector — checked in the sequence hull of this work (Section F.2), composed host-side in the map hull of this work (see above). This separation is structurally enforced in the cache engine: the Map and Sequence families have separate drive interfaces and separate oracles, and an adapter foreign to its family is ruled out by the type system. Which of these functions the module boundary provides, which the host composes, and which are not wired up is recorded, per function with its hop path, by the developer document named at the outset.

Acknowledgments

My thanks go first to apl. Prof. Dr.-Ing. Dirk Habich — a wise, friendly, disciplined person — for the dedicated supervision of this thesis: despite persistently scarce time, he always made time for precise questions, clear priorities, and the freedom to develop this work largely independently, with targeted support where it was needed. I thank Dr.-Ing. Alexander Krause for accompanying this work as second advisor. Finally, I thank the team at UltiHash: the instructive years at this startup shaped many of the experiences and questions that this work now implements technically.

Copyright Information

Confidentiality notice (restricted)

This thesis itself is not subject to a confidentiality notice. A restricted confidentiality notice concerns the software core of the `comdare-cache-engine` and the candidate under test PRT-ART: both may be inspected and tested for assessment; any business use, private individual use or distribution beyond the assessment requires a separate licence from the rights holder, BEP Venture UG (haftungsbeschränkt), owner of the registered trademark Comdare. Research is exempt: for research purposes at an institution both may be used freely. The experiments described in this thesis use the cache engine as a client via freely selectable test parameters and are not affected by the restriction.

Licences and copyright

The code of the diploma-thesis repository (measurement driver and evaluation pipeline) is licensed under Apache-2.0; bundled submodules and third-party components keep their own licences. The software core of the `comdare-cache-engine` and the candidate under test PRT-ART are licensed under the Comdare Research License 1.0: the source code is readable by everyone; research at an institution may use it freely; business use and private individual use each require a separate written licence from the rights holder. For the `comdare-cache-engine` the change date 2031-08-10 applies: from that day it becomes available to everyone under the Apache License 2.0. Previously published revisions keep their original licence irrevocably (`comdare-cache-engine`: Apache-2.0 before 2026-08-02, a dual research/commercial model from 2026-08-02 to 2026-08-09; PRT-ART: Apache-2.0 from 2026-08-02 to 2026-08-09). The rights to both software cores rest with BEP Venture UG (haftungsbeschränkt), Comdare brand. The text of this thesis is copyright Benjamin-Elias Probst and is licensed under CC BY 4.0 (Creative Commons Attribution 4.0 International).

Third-party material

Verbatim and paraphrased passages taken from other works are marked as quotations and referenced in the bibliography; they remain the property of their respective authors and are not covered by the licence of this thesis. All figures and tables in this thesis were produced by the author from the project's own measurement data and design documents (TikZ/pgfplots or generator output of the evaluation pipeline); no third-party graphics are included. The logo on the title page is a mark of Technische Universität Dresden (ZIH), is used according to its rules for theses and is excluded from the CC BY licence. Third-party benchmark datasets (such as the datasets of the SOSD benchmark) are not part of this thesis; they are only cited or used as input through the dataset loader, and their use is governed by the terms of their publishers. Third-party code vendored in the `comdare-cache-engine` (directory `ext/`) is not part of

the software within the meaning of the Comdare Research License and keeps its respective original licence; the complete attribution is given in the files `NOTICE`, `REUSE.toml` and `LICENSE_AUDIT_EXT.md` of the repository. *Comdare* is a registered trademark of the rights holder; its mention in this thesis does not grant any trademark rights to third parties.